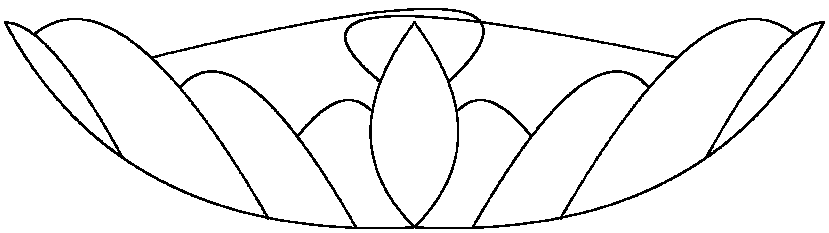
Foundations of information systems

Lecture notes

Patrick van Bommel

Radboud University Nijmegen



Contents

Voorwoord

Tijdens de ontwikkeling van een informatiesysteem worden verschillende modelleertechnieken gebruikt. Deze technieken richten zich met name op de specificatie van gegevensmodellen en procesmodellen. Gegevensmodellen beschrijven om welke gegevens het gaat en mogelijk ook hoe ze worden opgeslagen, terwijl procesmodellen beschrijven voor welke processen de gegevens nodig zijn. In dit boek spelen gegevensmodellen de hoofdrol.

Bij het ontwerpen van gegevensmodellen concentreert men zich eerst op de vraag *welke* gegevens in een systeem opgeslagen gaan worden, en dan pas op de vraag *hoe* dit op geschikte wijze kan gebeuren. De eerste vraag houdt zich bezig met zogenaamde *conceptuele* aspecten. De vraag *hoe* gegevens opgeslagen dienen te worden richt zich op zogenaamde *interne* aspecten.

Zoals gezegd, wordt, nadat is vastgesteld welke gegevens opgeslagen moeten worden, bepaald hoe dat op geschikte wijze kan gebeuren. De uitdrukking *geschikt* betekent in dit verband dat de vragen die aan het systeem gesteld gaan worden snel genoeg beantwoord kunnen worden, en tegelijkertijd dat de geheugenruimte die nodig is om alle gegevens op te slaan niet te groot wordt.

Hier is sprake van twee doelen die met elkaar in strijd zijn. Een wijze van opslaan die geschikt is met betrekking tot responstijd, is meestal ongeschikt met betrekking tot geheugenruimte. Omgekeerd leidt een wijze van opslag met ongeschikte responstijd vaak tot een geschikt geheugengebruik. We zeggen dan ook wel dat tijd en ruimte uitgewisseld worden.

Het vinden van een middenweg, waarin tijd en ruimte op zodanige wijze worden uitgewisseld dat rekening wordt gehouden met de specifieke wensen en mogelijkheden, wordt ook wel *optimalisatie van gegevensmodellen* genoemd. Helaas bestaat er geen algemene benadering om deze optimalisatie aan te pakken.

Daarom wordt in dit boek een mechanisme beschreven dat het mogelijk maakt om door de verzameling van alle mogelijke gegevensmodellen te navigeren. We noemen dit evolutie, omdat deze navigatie de geleidelijke ontwikkeling van een (initieel) gegevensmodel tot betere gegevensmodellen tot doel heeft. Uiteraard zijn we daarbij alleen geïntereseerd in gegevensmodellen die het conceptuele model op correcte wijze representeren.

De belangrijkste aspecten die bij een dergelijke evolutie een rol spelen worden hieronder toegelicht. Hierbij wordt achtereenvolgens gekeken naar (1) conceptuele gegevensmodellen en hun interne representaties, (2) optimalisatie van interne representaties en (3) evolutie van interne representaties.

1. Conceptuele gegevensmodellen zijn het resultaat van de informatie-analyse en dienen als uitgangspunt voor het transformatieproces. Om transformaties binnen een formeel raamwerk te kunnen beschouwen, moeten conceptuele gegevensmodellen ook op formele wijze beschreven worden.

   Het mechanisme om interne representaties van conceptuele modellen te beschrijven moet een duidelijke relatie laten zien tussen de interne representaties en het oorspronkelijke conceptuele model. Dit is belangrijk om de correctheid van interne representaties te garanderen.

   Als laatste aspect in deze categorie wordt vermeld dat de benadering voldoende algemeen moet zijn. Dat wil zeggen: het moet mogelijk zijn om verschillende typen gegevensbanksystemen als doelomgeving te kiezen, bijvoorbeeld relationele, hiërarchische en netwerk systemen.

2. Er moet een instrument zijn om gegevensmodellen te optimaliseren met betrekking tot verschillende criteria, zoals responstijd en geheugenruimte. Deze criteria zijn in strijd met elkaar (een betere responstijd komt meestal overeen met een slechter geheugengebruik, en andersom), en zijn bovendien niet direct vergelijkbaar (een byte is groter noch kleiner dan een seconde). Daarnaast kunnen ook andere criteria een rol spelen bij het transformatieproces, bijvoorbeeld het aantal tabellen of het aantal afhankelijkheden tussen gegevens.

   Er zijn twee extra zaken gerelateerd aan bovengenoemde transformaties. Ten eerste dient er een prototype te worden gemaakt dat de transformaties in werking kan stellen. Dan kunnen experimentele resultaten worden gebruikt om de benadering te illustreren.

Ten tweede moet er een formele basis gedefinieerd worden om het (dynamische) gedrag van transformatieprocessen te beschrijven en te voorspellen. Daarbij kan worden gedacht aan de waarschijnlijkheid dat bepaalde wenselijke of onwenselijke situaties zullen optreden, bijvoorbeeld de waarschijnlijkheid dat een optimum binnen een beperkt aantal stappen wordt gevonden.

3. Wat betreft de initialisatie van het transformatieproces, moet het mogelijk zijn om interne representaties te genereren op basis van parameters die het generatieproces sturen. Zo kunnen verschillende startpunten worden gekozen.

   Het transformatieproces zelf wordt gedefinieerd in termen van mutatie-operatoren, die een interne representatie kunnen omzetten in een andere. De geproduceerde interne representatie moet dan opnieuw een correcte representatie zijn van hetzelfde conceptuele gegevensmodel.

   Naast bovengenoemde correctheid moeten mutatie-operatoren compleet zijn, zodat alle mogelijke interne representaties ook inderdaad geproduceerd kunnen worden.

De basis voor dit boek is gelegd in [19]. De meeste publicaties waarop dit boek is gebaseerd zijn toegankelijk via http://www.cs.ru.nl/~pvb/reports.html. Lezers die meer willen weten over gegevensmodellen en transformaties vinden in [27] een mooi overzicht. Verder worden in [26] gegevensmodellen in de context van het Internet besproken.

Chapter 1

Introduction

1.1 Problem area

Different kinds of modelling techniques are involved in information system development, such as techniques for specifying data models and techniques for specifying process models. In these modelling techniques, a distinction can be made between implementation-oriented techniques and implementation-independent techniques (also called conceptual or problem-oriented techniques). Our focus of attention is data modelling, where conceptual data models are transformed into implementation-oriented data models.

1.1.1 Data modelling

Conventional implementation-oriented data modelling techniques are the relational, the hierarchical and the network data modelling technique ([157]). In each technique a model has particular structural characteristics (e.g. a table or a network) and has particular basic operators (data definition operators and data manipulation operators).

So-called normal forms may be used in order to guide the design process for these models. In general a normal form is defined as a data model having certain desirable structural properties, which may be expressed by integrity constraints (e.g. functional dependencies). Although normalization improves the process of database design significantly, the above-mentioned modelling techniques often are still implementation-oriented.

The advantage of conceptual modelling is (e.g. [66], [36], [124]), that it gives the designer the opportunity to separate the concern of constructing a correct model from that of finding an appropriate implementation. Once the conceptual model has been specified in a suitable language, a proper realisation can be (automatically) derived, using a specification language that is more implementation-oriented. This approach is a typical example of software development in phases (see e.g. [148]), and is in harmony with the three level architecture for information systems modelling ([66]) and transformational approaches to software development ([126]). Moreover, it is in line with iterative and rapid application development.

1.1.2 Transformation of data models

Conceptual data models specify *what kind of* data must be handled by the information system under development, rather than *in what way* these data will be actually organized (see [66]). The actual organization of the data is done according to the implementation-oriented representation chosen for the conceptual model at hand. Implementation-oriented representations for conceptual models will be called *internal representations* in the sequel.

There is one important property typical for internal representations: efficient usage of basic resources. More concretely, the internal representation chosen for a given conceptual data model should be attuned to the particular application environment in which it will operate, such that the usage of basic resources is in harmony with the needs and possibilities of that environment. This particularly becomes a difficult problem when databases are large, e.g. consisting of 500 to 2000 relational database tables.

Well-known basic resources are for example response time and storage space. Consider for example the situation where important database operations (queries as well as updates) do not have the desired response time. In order to solve the efficiency problem, implementation-oriented data models may be attuned to the specific requirements of their environment (e.g. query patterns). This is a highly complex matter for several reasons. As an example, an improvement for one class of operations often is a worsening for other classes. The fact that structural modifications also affect the storage space requirements further complicates the problem.

1.2 Intention

1.2.1 Requirements

Our focus is data modelling and transformation, including conversion of conceptual data models into internal representations. We aim at a general strategy, for *describing and examining transformations of data models*. We also consider the notion of trade-off in those transformations, for example the time/space and update/retrieval trade-off. Note that, if the actual implementation of a given conceptual data model must satisfy specific requirements of a particular application environment, such a strategy is essential.

We consider different kinds of transformations, driven by integrity constraints as well as by resource usage (e.g. time/space-driven transformations). We also include the Optimal Normal Form (e.g. [124], [166]).

We impose the following requirements on a framework for the transformation of data models:

formal description of conceptual models: Transformations are often based on conceptual data models. To be able to consider transformations within a formal framework, conceptual models have to be defined in a formal way.

correctness of internal representations: The mechanism used for describing internal representations of conceptual data models should present a clear relation between the underlying conceptual model and candidate internal representations. This is essential for guaranteeing the correctness of internal representations.

generality of approach: The approach should not be restrictive. Therefore it should be possible to choose different types of target environments, in order to support implementation in different Database Management Systems. This will guarantee the applicability of our approach. Note that this does not exclude other ways of implementing the internal representations, for example directly in a third generation programming language or even in assembler.

The requirements discussed above are general requirements with respect to conceptual models and their internal representations. In order to decide whether a given transformation is to be preferred over another transformation, we add the following requirements:

choice of transformation: There should be an instrument for choosing between different transformations. This instrument must be based on several criteria, for example integrity constraints, number of tables, response time, and storage space.

formalization of transformation dynamics: There should be a formal basis for describing and predicting the behaviour of aforementioned instrument for transformation. This will be based on Markov theory.

1.2.2 Approach

According to requirement *correctness*, there must be a clear relation between the conceptual model and candidate internal representations. Therefore an internal representation is defined *in terms of* the conceptual model, such that the underlying information structure of the original conceptual model is explicitly reflected in the internal representations.

According to requirement *generality*, the internal representations must be suitable for implementation in different types of Database Management Systems. Therefore these representations are defined in such a way that they can be easily interpreted in terms of conventional data modelling techniques, such as the relational model, the hierarchical model and the network model. The overall situation is shown in figure 1.1.

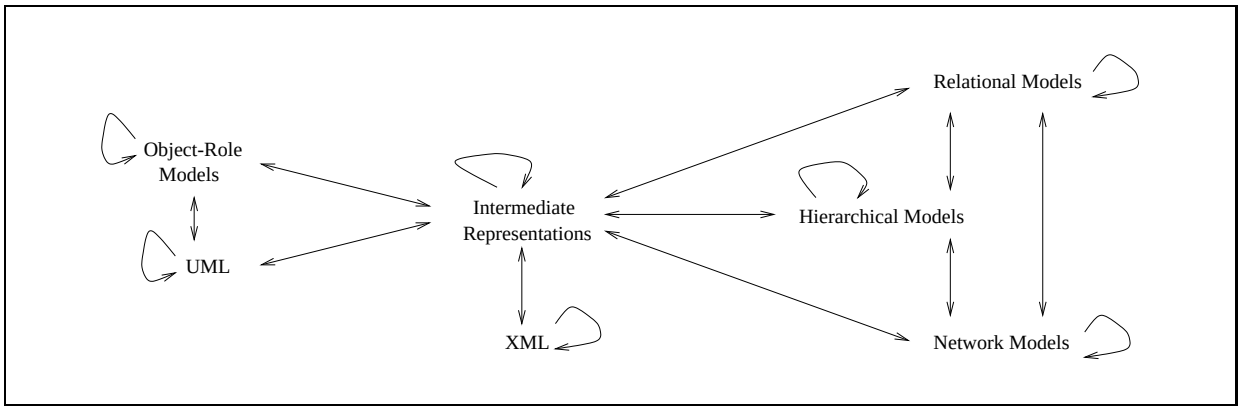


Figure 1.1: Overview of transformations

1.2.3 Additional requirements

In the context of transformation processes, several additional requirements can be recognized. The transformation of a given internal representation into another internal representation is also called a *mutation*. For this specific kind of transformation, we define the following additional requirements:

parameterized initialization: There should be a systematic generator, producing candidate internal representations for a given conceptual data model. This allows for opportunities to influence the generation process, such that it produces representations having certain desirable properties, for example based on absence of redundancy and optional database fields, or on other structural properties such as database table size.

correctness of mutations: In order to be able to 'search' there should be mutation operators for internal representations which yield a proper result, i.e. the result again specifies a correct internal representation of the same conceptual model.

completeness of mutations: These mutation operators must be complete in the sense that for each internal representation, any other internal representation can be produced by applying mutation operators. This will guarantee the connectedness of the solution space of internal representations.

1.3 An example application: The Project Case

In this section an example of data transformation is described ([22]). First the conceptual data model of this case is discussed in section 1.3.1. This model specifies *what kind of* data must be stored in the

database under development, rather than *in what way* these data will be actually organized (see [66]). Next we discuss internal representations of the conceptual model in section 1.3.2. Internal models focus on the question how databases may be organized. Finally, some criteria to be used in choosing between different transformations will be illustrated in section 1.3.3.

1.3.1 Underlying information structure

In this section we informally discuss a modelling technique for making conceptual descriptions of complex information structures (as encountered in administrative environments), viz. the object-role data modelling technique (see e.g. [124], [164] and [158]). We clarify this technique by an example.

Consider an organization where information is to be recorded about projects and persons being involved in those projects. In order to specify this more precisely, we describe an information structure containing the following (see also figure 1.2). A *Project* has a budget of a certain *Amount of dollars* (fact type *Budget*). A *Person* is involved in a *Project* for a certain *Duration* (fact type *Involvement*) and is coworker of a certain *Department* (fact type *Coworkership*). Finally, a *Project* is managed by a certain *Person* (fact type *Management*).

The information structure described above is graphically represented in figure 1.2. The circles represent object types, playing roles in fact types, represented by boxes. This information structure contains three binary fact types and one ternary fact type.

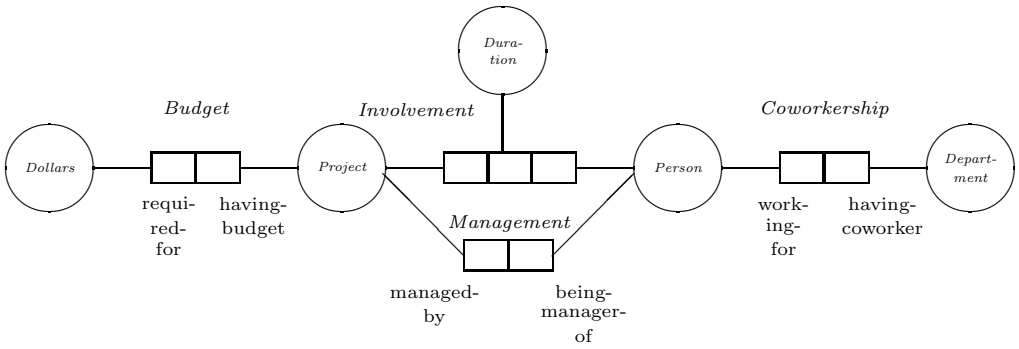


Figure 1.2: Information structure for the Project Case

It is important to realize that data models will in general be far more complex than the example described above. Firstly, the number of object types and fact types may be very large. Secondly, information structures are usually augmented with integrity constraints ([20], [124], [158]). Examples of constraints are: a project has exactly one budget, a project involves at least four persons, a person being involved in a project cannot be manager of that project. Thirdly, object types may be ordered in hierarchies (e.g. specialization), while fact types can be used as object types themselves (objectification).

Object-role models allow an elegant verbalization of queries by path expressions, built from names of object types and role names (see e.g. [81]). Role names verbalize the participation of object types in fact types. A path expression may for instance be: *Person being-manager-of Project having-budget* $> 10,000$. Basically, role names are the elementary constructors for all path expressions.

1.3.2 Basic transformations

Conceptual modelling is intended to provide a description of some information structure, without bothering about any implementation detail. In order to develop an efficient information system, a conceptual model has to be transformed into a model capturing the necessary implementation details, usually referred to as an internal model. An internal model can be considered to consist of two layers ([157]):

- The first layer specifies in a global way how different fact types from the conceptual model will be organized. As an example, it may require certain fact types to be combined in a single derived fact type (e.g. table) for reasons of efficiency. Sometimes this layer is called the *logical layer*.
- The second layer specifies how particular parts of the logical layer are realized on physical storage media using concrete data structures, such as B-trees and hash structures. Sometimes this layer is called the *physical layer*.

Typical approaches for the first layer are based on relational, hierarchical or network data models ([157]). Another possibility is to use nested relational models (sometimes called non-first-normal-form or NF^2). This technique can be seen as an intermediate between the relational, the hierarchical and the network modelling technique. In contrast to the relational approach, tables in NF^2 may contain entries that are tables themselves (see e.g. [138]).

Consider for example the nested table header in figure 1.3. This table is a correct internal representation for the Project Case information structure in the sense that all relevant information about projects is represented. This will be explained below.

Project	*Budget*	*Involvement*			*Management*
	Dollars	*Duration*	*Person*	*Coworkership*	*Person*
				Department	

Figure 1.3: A single nested table for the Project Case

The table in figure 1.3 consists of 4 columns, named *Project, Budget, Involvement* and *Management* respectively. For each project, column *Budget* contains all data about its budgets, column *Involvement* contains all data about the persons involved, while column *Management* registers the project managers. The latter three columns are sub-tables (nested relations). The nested relation *Budget* has a single column, named *Dollars*, which has been assigned an elementary domain (e.g. integer). The nested relation *Involvement* has three columns: *Duration*, *Person* and *Coworkership*. The columns *Duration* and *Person* have associated elementary domains, while column *Coworkership* again is a nested relation.

The nested table shown in figure 1.3 describes a possible internal representation for the Project Case information structure. However, internal representations consisting of more than one nested table are also possible. Figure 1.4 presents an example of an internal representation with two nested tables. More details about nested tables for object-role information structures are discussed in later chapters (see also [21], [14], [24]). It will also be shown that the number of alternative internal representations implementing the same information structure is exponential in the number of fact types (chapter 3).

Person	*Involvement*		*Coworkership*
	Project	*Duration*	*Department*

Person	*Management*	
	Project	*Budget*
		Dollars

Figure 1.4: Implementation for the Project Case consisting of two nested tables

As was mentioned in section 1.3.1, a conceptual model usually has constraints on the possible populations of the fact types. This also has consequences for the internal representations of such a conceptual model. As an example, if each project has at most one budget, the nested relation *Budget* in figure 1.3 and figure 1.4 will contain a single budget for each project, rather than for a group of budgets.

1.3.3 Access profile and response time

For a given conceptual model, there exists a huge number of transformations. In order to discriminate between good and bad transformations, it is necessary to estimate the time needed for actions on the

database, and the requested size of the storage capacity. For this purpose the notions of *access profile* and *data profile* are introduced.

Generally speaking, an access profile $\mathcal{A} \subseteq \mathcal{L}_{\mathsf{op}}$ is a set of operations defined in some language $\mathcal{L}_{\mathsf{op}}$. A data profile $\mathcal{D} \subseteq \mathsf{POP}_{\Sigma}$ is a collection of states (populations) for the schema Σ under consideration.

It was mentioned that data models are considered on the conceptual and the internal level. As a consequence, database operations will also be considered on these levels. Suppose for example that we want to know the projects managed by people from the personnel department. The following conceptual query will give the answer: *LIST Project managed-by Person working-for Department 'Personnel'.* Note that this query describes a path through the Project Case information structure (see figure 1.2). The corresponding internal query is expressed in terms of the chosen internal representation, for example by means of operators from a nested-relational algebra (see e.g. [138]).

The retrieval of the projects specified by the above query may be as follows (we assume the single table from figure 1.3). First, all occurrences of department 'Personnel' within sub-table *Coworkership* are located. Then all persons being coworker of that department are determined within sub-table *Involvement*. Finally the entries in sub-table *Management* containing these persons are determined and all projects managed by those persons are retrieved from column *Project*. The retrieval process described here can be speeded up by introducing an efficient access path (index) for the columns involved in the query (see e.g. [157]).

The relative frequency of different (classes of) conceptual database operations is specified in an access profile. Figure 1.5 presents an example access profile for the Project Case information structure. The operations in this profile are expressed as paths through the information structure ([49], ([81]).

Relative frequency	Operation
3	LIST Person being-manager-of Project x
4	LIST Project managed-by Person working-for Department x
2	LIST Person being-manager-of Project x AND working-for Department y
1	UPDATE Duration of (Project x, Person y)

Figure 1.5: Example access profile for Project Case

An access profile can be used to compare different internal representations with respect to their expected average response time. This is a highly complex task, since it will depend on the particular strategy for evaluating the queries contained in the access profile.

Consider for example the first query: *LIST Person being-manager-of Project x*. The strategy may be as follows. The query will first search for a certain project and then retrieve the persons being manager of that project. As a consequence, the implementation consisting of a single nested table (figure 1.3) will be more efficient for this query than the implementation consisting of two nested tables (figure 1.4), because the structure of the single table is dominated by *Project* and thus all managers of the same project are grouped together. Note that the introduction of query optimization strategies and indices (efficient access paths) can make figure 1.4 more efficient. Obviously, several other physical aspects of database design can be taken into account, for example underlying pointer mechanisms. This will also affect the storage requirements.

Note that the processing of update operations is affected by the question whether these operations are dependent or not. A sequence of update operations $U_1, U_2, U_3, ...$ is independent if the existence of U_i does not depend on the existence of U_j $(i > j)$. As a consequence, an independent sequence of update operations can be postponed by collecting the operations in a batch file to be processed at night (e.g. when update operations take too much time). A dependent sequence of update operations is more time-critical, as it has to be processed immediately.

1.3.4 Data profile and storage requirements

The expected size of the contents of the database is recorded in the data profile. In figure 1.6 we see an example of a simple data profile, in which the expected number of instances is specified for the object types in the Project Case information structure, relative to each other. So, for example, if a population contains $3 \times 50 = 150$ coworkerships, then it will also contain 3 times the number of departments, or, 15 departments.

object type	relative nr. of instances
project	10
person	50
department	5
involvement	75
management	10
coworkership	50

Figure 1.6: Example data profile for Project Case

Given the Project Case information structure and aforementioned data profile, different internal representations can be compared with respect to their storage usage. This is not a trivial task and therefore storage cost models are usually applied here. The following general considerations will be sufficient for the aim of this section.

Joining fact types may introduce redundancy. As a consequence, the implementation consisting of a single table (figure 1.3) will probably contain more redundancy than the implementation consisting of two tables (figure 1.4). This however will also depend on (1) the table structures, for example structured by *Project* in figure 1.3 and structured by *Person* in figure 1.4, (2) the number of instances of the corresponding object types, and (3) the integrity constraints on the fact types.

Redundancy usually increases storage cost, while the efficiency of retrieval operations may be improved. Update operations may become less efficient, as a result of maintenance (update overhead).

1.3.5 Choosing between different transformations

In general, the *fitness* of an internal representation takes into account the expected storage requirements Space and the expected average response time Time. The fitness function will specify how Time and Space affect the overall fitness of an implementation, depending on the aim of the optimization process. Example optimization options are:

1. Space should be as low as possible. In this case the optimization process will yield an implementation with minimal storage cost, irrespective of the average response time.

2. Time should be as low as possible. Now the result will be an implementation with minimal average response time.

3. Space may not exceed a certain threshold and Time should be as low as possible. This situation may occur when (a large amount of) information must be represented in storage having a predefined size (e.g. CD-ROM or main memory).

As usual, time and space are conflicting objectives for optimization. It is therefore very difficult to find an internal representation which has a satisfactory response time (for the associated access profile) in combination with satisfactory storage requirements (for the associated data profile). This is reflected in the well-known time/space trade-off.

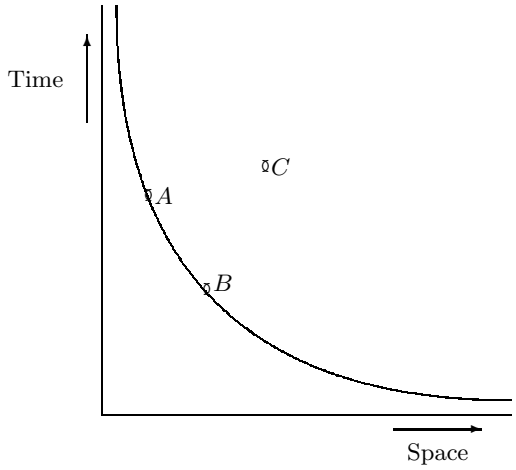


Figure 1.7: Time/space trade-off

Consider for example the internal representations A, B and C shown in figure 1.7. In this case representation A and B are both better than representation C. However, it is not clear whether A is better than B or vice versa. These representations are said to be part of the time/space trade-off, which corresponds to a convex Pareto-Optimal front ([160]). Several other trade-offs may be recognized, for example the update/retrieval trade-off.

1.4 Topics for discussion

1.4.1 Data modelling discussion topics

In the three level architecture for information systems modelling ([66]), a distinction is made between the conceptual level, the internal level and the external level. The conceptual level considers the question what kind of data will be manipulated by the system under development. The external level describes the data forms convenient in particular user views, while the internal level describes the interface to the storage facilities of a particular computer. This architecture is also explained in other work, for example [124].

Conceptual data modelling techniques often have an underlying object-role structure (for example [124], [164], [158], [144], [145], [73], [69]). Sometimes, the underlying object-role structure is not immediately clear (e.g. [36], [29]). More advanced object-role models can be specified on the basis of formalizations and extensions (see e.g. [82], [78], [80]). Other approaches are for example ([2]), [74], [120]).

Next we consider the transformation of a conceptual data model into an internal representation. Basic algorithms and tools are found in [124], [102], [166] and [144], [46], [72] respectively. Here the efforts are concentrated on the generation of normal forms, such as the Optimal Normal Form. According to [102], a relational schema is in Optimal Normal Form if it is in 5th Normal Form and the number of relations (tables) is minimal. These procedures may also take basic relational integrity rules into account, such as candidate keys, foreign keys and mandatory attributes (columns).

The number of relations (tables) can be further reduced by first 'optimizing' the conceptual model (e.g. [70], [72]). Also, more advanced integrity rules such as set constraints can be incorporated (e.g. [47], [135]).

The main goal of normalization is to construct a database schema where update anomalies cannot occur. This is a generally recognized database design theory (e.g. [157]). However, normalization does not consider the expected usage of the system, for example the frequency of different classes of (retrieval and update)

requests. Database *optimization* is therefore not supported by normalization theory. Moreover, in situations where updates do not occur (e.g. CD-ROM) there seems to be no justification for normalization at all.

Database optimization on the basis of expected patterns of database operations (access profiles) are for example the following. The approach in [132] considers a conceptual data model and describes an internal representation aiming at efficiency of retrieval and update requests. The intention is to restrict the number of disk accesses for *all possible* requests. The author of [141] goes a step further and describes general guidelines how expected query patterns can be used for tuning normalization as well as denormalization *in favour of important queries*. In [4], a conceptual model is used in combination with an access profile. An efficient internal representation in favour of the access profile is generated, while at the same time storage requirements are kept minimal. A general introduction to database tuning is found in [141].

1.4.2 Transformation discussion topics

Data schema transformations describe an evolutionary process. Evolution here indicates the gradual development of internal representations. The term *evolutionary algorithm* is often used for the class of optimization algorithms that simulate organic evolution.

The concept of evolution was introduced in the context of formal (technical) systems in e.g. [58], [130], [84]). This concept has become more powerful by a further formalization of evolutionary optimization (e.g. [140]). Several kinds of evolutionary optimization algorithms are commonly used. They are divided into three main streams: genetic algorithms ([62], [45]), evolution strategies ([140]), and evolutionary programming ([57]). An overview of these streams is given in [7]. Finally, although simulated annealing is in general not considered to be an evolutionary algorithm, this approach uses a more or less similar framework (e.g. [99], [131]).

Applications of evolutionary algorithms to optimization problems in the field of database research, deal with the problem of query optimization (e.g. [11], [86] and [87]). For the application of the evolutionary approach to information retrieval (document retrieval) we refer to [59] and [146], where focus is on a multiprocessor environment. The clustering of data in databases has been considered in e.g. [112].

1.4.3 Further discussion topics

The generation of data models considered here is a matter of *forward engineering*. In the case of existing database applications, techniques for *reverse engineering* should be used. Different strategies for database reverse engineering are possible, e.g. analysis of database tables or analysis of user interfaces (see e.g. [115]).

Generation of data models for Multi Media applications should be considered as well. Multi Media applications will have specific MM-oriented access profiles and data profiles. The generation of MM database structures therefore constitutes a separate class of generation (and transformation) strategies.

We need to compare traditional databases with other technologies such as JAVA. This also addresses databases in the context of networked applications (e.g. the WWW). Note that this requires a further refinement of *sensitivity* of database representations (see e.g. section 6.2.4).

Chapter 2

Conceptual data models

2.1 Introduction

The aim of this chapter is to give a summary of the basic components in conceptual models with an underlying object-role structure (see also [20], [162]). Our description has its roots in the relational data modelling technique and the relational algebra ([157]). Other descriptions of object-role models may be found in [48], [47] and [69]. These are logical formalizations.

An object-role model consists of object types playing roles in fact types. Depending on the power of the technique, fact types may be of any degree and can be treated as object types (objectified fact types). Furthermore, a distinction can be made between lexical and non-lexical object types and specialization (subtyping) of object types may be defined.

The organization of the chapter is as follows. In section 2.2 information structures are introduced. Section 2.3 considers populations (instantiations) of information structures. Integrity constraints will be treated in section 2.4, while section 2.5 describes possibilities for reforming conceptual schemata. A discussion on the specific constructs found in more advanced object-role modelling techniques is provided in chapter 4.

With respect to the graphical representation of information structures, we adopt the usual drawing conventions. Our approach however is applicable to any modelling technique with an underlying object-role structure, even if the notion of predicator is not explicitly used as drawing object (for example attributes [36] and functional relations [143]).

2.2 Information structures

2.2.1 Informal introduction

This section introduces the notion of *fact type*. A fact type (or relation type) is considered to be an association between object types. The graphical representation of a fact type is shown in figure 2.1. An alternative diagram is shown in figure 2.2. We prefer the style of figure 2.1, as this makes the concept of role more explicit.

We use the term *predicator* for the role played by an object type in a fact type. As an example, predicator p_1 is the role played by object type X_1 in fact type f.

2.2.2 Basic components of information structures

The discussion in the previous section leads to the following definition. An *information structure* $\mathcal{I}$ is a structure $\langle \mathcal{P}, \mathcal{O}, \mathcal{L}, \mathcal{E}, \mathcal{F}, \mathsf{Base}, \mathsf{Sub} \rangle$ consisting of the following basic components:

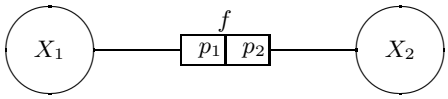


Figure 2.1: A fact type

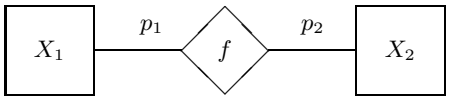


Figure 2.2: An alternative diagram

1. A set $\mathcal{P}$ of *predicators*.
2. A set $\mathcal{O}$ of *object types*. Object types are classified as follows.
   (a) Firstly we have atomic object types $\mathcal{A} \subseteq \mathcal{O}$. There are two different kinds of atomic object types: entity types and label types. The difference is that labels can, in contrast with entities, be represented (reproduced) on a communication medium. It is required that $\mathcal{A} = \mathcal{E} \cup \mathcal{L}$ and $\mathcal{E} \cap \mathcal{L} = \varnothing$.
   (b) Secondly, the object types in $\mathcal{F} = \mathcal{O} - \mathcal{A}$ are composed object types, also called *fact types*. The set $\mathcal{F}$ is a partition of the set of predicators $\mathcal{P}$. The fact type that corresponds with a predicator is obtained by the function $\mathsf{Fact} : \mathcal{P} \longrightarrow \mathcal{F}$ which is defined by: $\mathsf{Fact}(p) = f \iff p \in f$.

   All sets introduced above are assumed to be nonempty and finite.
3. A function $\mathsf{Base} : \mathcal{P} \longrightarrow \mathcal{O}$, specifying the object type associated to a predicator.
4. A binary relation $\mathsf{Sub} \subseteq \mathcal{E} \times \mathcal{E}$, with the convention that $a \,\mathsf{Sub}\, b$ is interpreted as: a is a subtype of b. Then b is called a supertype of a.

Example 2.2.1
Figure 2.3 shows a simple information structure, consisting of two binary fact types f and g. In this information structure we have e.g. $\mathsf{Fact}(r) = g$ and $\mathsf{Base}(r) = B$. Figure 2.3 also shows some integrity constraints. These will be discussed in section 2.4. □

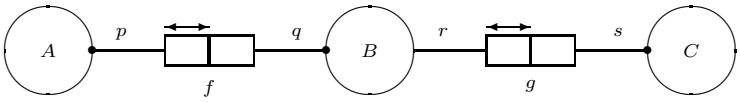


Figure 2.3: Information structure with two binary fact types

A fact type is called objectified, if it occurs as the base of a predicator. A predicator is called an objectification, if its base is a fact type. The set $\mathcal{H}$ contains all objectifications:

$$\mathcal{H} = \{p \in \mathcal{P} \mid \mathsf{Base}(p) \in \mathcal{F}\}$$

Example 2.2.2
Figure 2.4 shows a simple information structure with objectification, consisting of two binary fact types f and g. In this information structure we have $\mathcal{H} = \{r\}$, $\mathsf{Fact}(r) = g$ and $\mathsf{Base}(r) = f$. □

Instances (occurrences) of object types are *not* within the information structure. Instantiations (also called populations) will be introduced in section 2.3.

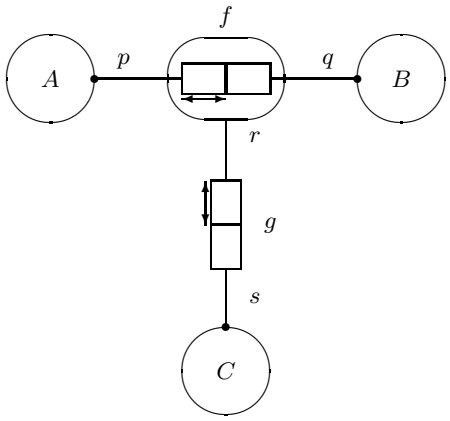


Figure 2.4: Simple information structure with objectification

2.2.3 Subtyping

The concept of *subtype* is defined as a binary relation $\mathsf{Sub} \subseteq \mathcal{E} \times \mathcal{E}$. This relation should have the property that with each element of $\mathcal{E}$ a unique top element can be associated. This so-called *pater familias* is found by the function $\sqcap : \mathcal{E} \to \mathcal{E}$. A formal definition of the function $\sqcap$ is given in [162], [78].

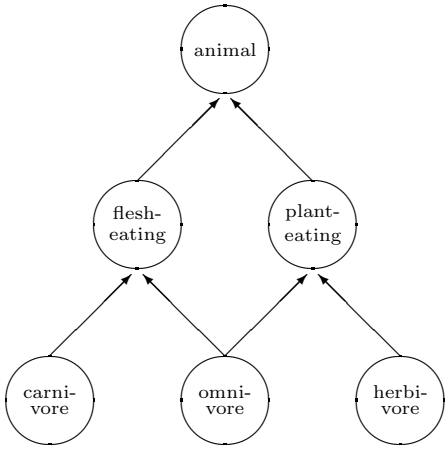


Figure 2.5: Example of a subtype hierarchy with top *animal*

Example 2.2.3

In figure 2.5 we have the following subtype hierarchy:

flesh-eating animal Sub *animal*
plant-eating animal Sub *animal*
carnivore Sub *flesh-eating animal*
omnivore Sub *flesh-eating animal*
omnivore Sub *plant-eating animal*
herbivore Sub *plant-eating animal*

Each element of the subtype relation is represented as an arrow in figure 2.5. As a consequence, the pater familias of object type carnivore *is* animal*, or:* $\sqcap(carnivore) = animal$. □

Two predicators p and q are called type related, denoted as $p \sim q$, if $\sqcap(\mathsf{Base}(p)) = \sqcap(\mathsf{Base}(q))$. A subtype s of object type t is used only if particular fact types are attached to s but not to t. For example, a fact type recording the kind of plant may be introduced for object type *plant-eating*, rather than *animal*.

2.2.4 An elaborated example

In figure 2.6 we see an example of an information structure with subtyping.

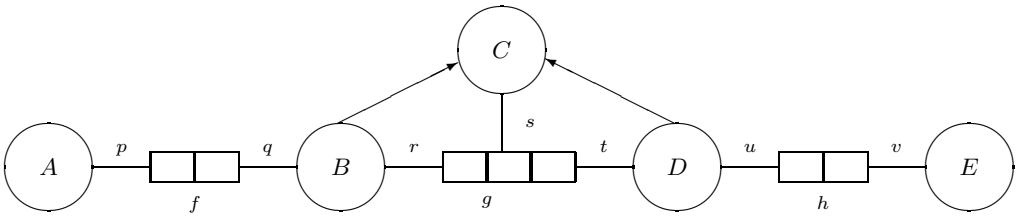


Figure 2.6: Example information structure with subtyping

This information structure is defined as follows:

$$\begin{aligned} \mathcal{P} &= \{p, q, r, s, t, u, v\} \\ \mathcal{O} &= \{A, B, C, D, E, f, g, h\} \\ \mathcal{F} &= \{f, g, h\} \end{aligned}$$

where $f = \{p, q\}$, $g = \{r, s, t\}$ and $h = \{u, v\}$. With respect to the predicators we have:

$$\begin{array}{lcllcl} \mathsf{Base}(p) & = & A & \mathsf{Fact}(p) & = & f \\ \mathsf{Base}(q) & = & B & \mathsf{Fact}(q) & = & f \\ \mathsf{Base}(r) & = & B & \mathsf{Fact}(r) & = & g \\ \text{etc.} & & & & & \end{array}$$

The subtype hierarchy is given by:

$$\begin{array}{lll} B & \mathsf{Sub} & C \\ D & \mathsf{Sub} & C \end{array}$$

2.3 Populations

An information structure is used as a frame for some part of the (real or fictive) world, the so-called Universe of Discourse (UoD). A *state* of the UoD then corresponds with a so-called instantiation or population of the information structure, and vice versa. The correspondence between populations and states is elaborated in more detail in [13] and [79], where state transitions are defined using homomorphisms.

In our approach, a population $\mathsf{Pop}_{\mathcal{I}}$ of an information structure $\mathcal{I} = \langle \mathcal{P}, \mathcal{O}, \mathcal{L}, \mathcal{E}, \mathcal{F}, \mathsf{Base}, \mathsf{Sub} \rangle$ is a value assignment to the object types in $\mathcal{O}$, conforming to the structure as prescribed in $\mathcal{P}$, $\mathcal{F}$ and Base, respecting the subtype hierarchy Sub. This is denoted as $\mathsf{IsPop}(\mathcal{I}, \mathsf{Pop}_{\mathcal{I}})$. The precise rules a population has to satisfy will be discussed below. When no confusion is likely to occur we write Pop rather then $\mathsf{Pop}_{\mathcal{I}}$.

The population of an atomic object type is just a set of values. The population of a label type comes from the corresponding concrete domain (e.g. string, natural number), while the population of an entity type

comes from an abstract domain containing unstructured values. This leads to the *universe of instances* Ω, consisting of all possible atomic values, and all possible composed values such as mappings from predicators to values. A formal treatment of aforementioned domains and universe of instances is found in [81] and [78].

The population of a fact type is a set of tuples. A tuple t of a fact type f is a mapping $t : f \longrightarrow \Omega$. This is a basic requirement for the insertion of tuples into the population of fact types (see also section 4.4). Furthermore, this mapping is required to assign values of the appropriate type. This is referred to as the *Conformity Rule*:

$$\forall_{f \in \mathcal{F}} \forall_{t \in \mathsf{Pop}(f)} \forall_{p \in f} \left[t(p) \in \mathsf{Pop}(\mathsf{Base}(p)) \right]$$

Respecting the subtype hierarchy is reflected by the *Subtype Rule*. This rule requires that the population of a subtype is contained in the population the supertype:

$$\forall_{x,y \in \mathcal{E}} \left[x \,\mathsf{Sub}\, y \Rightarrow \mathsf{Pop}(x) \subseteq \mathsf{Pop}(y) \right]$$

Example 2.3.1

Consider the information structure shown in figure 2.6. Suppose a, b, c, d_1, d_2, e are values that occur in the universal domain Ω. Then the following is a population of the information structure:

$$\begin{aligned}
\mathsf{Pop}(A) &= \{a\} \\
\mathsf{Pop}(B) &= \{b\} \\
\mathsf{Pop}(C) &= \{c, b, d_1, d_2\} \\
\mathsf{Pop}(D) &= \{d_1, d_2\} \\
\mathsf{Pop}(E) &= \{e\} \\
\mathsf{Pop}(f) &= \varnothing \\
\mathsf{Pop}(g) &= \{\{r : b, s : c, t : d_1\}, \{r : b, s : c, t : d_2\}\} \\
\mathsf{Pop}(h) &= \{\{u : d_1, v : e\}\}
\end{aligned}$$

□

When no confusion is likely to occur the predicator is omitted when writing tuples. In the above example this results in $\mathsf{Pop}(h) = \{\langle d_1, e \rangle\}$ and $\mathsf{Pop}(g) = \{\langle b, c, d_1 \rangle, \langle b, c, d_2 \rangle\}$.

2.4 Integrity constraints

Usually not all possible populations of an information structure are valid, i.e. correspond with some state in the UoD. Forbidden populations are excluded by so-called *static* integrity constraints. Dynamic constraints exclude forbidden transitions. These are not considered here.

2.4.1 Approach

In our approach, the semantics of constraints is expressed in terms of relational expressions constructed from fact types and relational operators, such as projection and selection ([20]). A relational expression has a structure (Schema) and a value (Val).

A schema $\Sigma = \langle \mathcal{I}, \mathcal{C} \rangle$ consists of an information structure $\mathcal{I}$ and a set of constraints $\mathcal{C}$. A population of a schema should be correct according to the information structure, and satisfy the requirements expressed by the constraints:

$$\mathsf{IsPop}(\Sigma, \mathsf{Pop}) \equiv \mathsf{IsPop}(\mathcal{I}, \mathsf{Pop}) \wedge \forall_{c \in \mathcal{C}} \left[\mathsf{Pop} \models c \right]$$

The following kinds of constraints are distinguished: total role, uniqueness, occurrence frequency, set exclusion, set equality, subset, enumeration, total subtype and exclusion subtype constraints (see e.g. [124] and [164]). In [20] these constraints are formalized. The set of all possible constraints is denoted by $\mathbf{\Gamma}(\mathcal{I})$, hence $\mathcal{C} \subseteq \mathbf{\Gamma}(\mathcal{I})$.

We discuss two kinds of constraints in more detail: uniqueness constraints and total role constraints. These constraints are of vital importance for the conceptual information structure, since they are used for identification purposes (see section 2.5.1 and [20],[124]). For related work on integrity constraints, see e.g. [47] and [123].

2.4.2 Total role constraint

We first consider the *total role constraint*. Object type may have the property that in any population all their instances must be involved in some set of predicators. An example might be that, in a personnel information system, *all* employees should have a department associated. This property is called a total role constraint.

Let τ be a nonempty subset of $\mathcal{P}$. A total role constraint $\mathsf{total}(\tau)$ only makes sense if all predicators in τ are type related:

$$\forall_{p,q \in \tau} [p \sim q]$$

A population Pop satisfies the total role constraint $\mathsf{total}(\tau)$, denoted as $\mathsf{Pop} \models \mathsf{total}(\tau)$, if it satisfies the following expression:

$$\bigcup_{q \in \tau} \mathsf{Pop}(\mathsf{Base}(q)) = \bigcup_{q \in \tau} \mathsf{Val}[\![\pi_q \, \mathsf{Fact}(q)]\!] \, (\mathsf{Pop})$$

where π is the usual projection operator (see e.g. [157], [108]). The projection operator is defined in terms of object-role models in [20]. Note the implicit coercion from a unary relation resulting from the projection into the range of values that are taken by this relation.

Example 2.4.1

Consider the total role constraint $\mathsf{total}(\{q\})$ *in the information structure of figure 2.3. The meaning of this constraint is:*

$$\mathsf{Pop}(B) = \mathsf{Val}[\![\pi_q f]\!] \, (\mathsf{Pop})$$

This expresses that each instance in the population of object type B *must occur in the population of fact type* f *via predicator* q*. When no confusion is likely to occur this is simply denoted as* $B = \pi_q f$*.* □

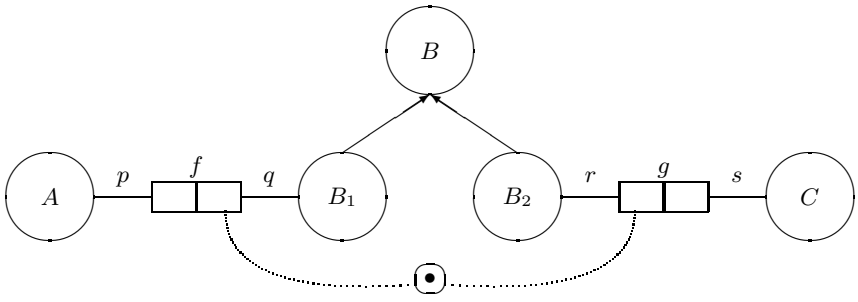


Figure 2.7: Example of total role constraint

Example 2.4.2

Consider the total role constraint $\mathsf{total}(\{q, r\})$ *in the information structure of figure 2.7. The meaning of this constraint is (using the shorthand notation):*

$$B_1 \cup B_2 = \pi_q f \cup \pi_r g$$

□

2.4.3 Uniqueness constraint

The uniqueness of values in some set of predicators has become a widely used concept in database technology, as a guarantee for integrity and as a base for efficient access mechanisms. In this section we will define some well-known concepts, such as functional dependency, identifiers (keys) and *uniqueness constraints*.

An important notion in relational algebra theory is functional dependency ([157]). Let r be a relational expression and let $\sigma, \tau \subseteq \mathsf{Schema}(r)$. Then $\sigma \xrightarrow{r} \tau$ is a boolean expression which holds in a population Pop iff in this population, $\mathsf{Val}[\![r]\!]\,(\mathsf{Pop})$ can be seen as a mapping from $\mathsf{Val}[\![\pi_\sigma r]\!]\,(\mathsf{Pop})$ into $\mathsf{Val}[\![\pi_\tau r]\!]\,(\mathsf{Pop})$. Then, in population Pop, we call τ functionally dependent on σ over r. This is denoted as:

$$\mathsf{Pop} \models \sigma \xrightarrow{r} \tau$$

The following property is obvious:

Lemma 2.4.1 $\mathsf{Schema}(r) \xrightarrow{r} \mathsf{Schema}(r)$

A set of predicators $\sigma \subseteq \mathsf{Schema}(r)$ is called an identifier (key) in population Pop of relational expression r iff:

$$\mathsf{Pop} \models \sigma \xrightarrow{r} \mathsf{Schema}(r)$$

We use $\mathsf{identifier}(r, \sigma)$ as an expression to denote that σ is an identifier of r. We then have:

Lemma 2.4.2 $\forall_{f \in \mathcal{F}} \exists_{\sigma \subseteq f} [\mathsf{Pop} \models \mathsf{identifier}(f, \sigma)]$

It should be noted that, using structural induction on the construction of relational expressions, this property is easily extended to all relational expressions. A uniqueness constraint $\mathsf{unique}(\sigma)$ is based on a non-empty set of predicators $\sigma \subseteq \mathcal{P}$. The semantics of this constraint will be expressed as an identifier of the form $\mathsf{identifier}(\xi(\sigma), \sigma)$, where ξ is an operator specifying a derived fact type. When σ does not exceed the boundaries of a single fact type f (i.e. $\sigma \subseteq f$), the uniqueness constraint is bound to this fact type: $\xi(\sigma) = f$. When more than one fact type is involved in a uniqueness constraint, these fact types may be joinable via common object types. If in these fact types objectification occurs, unnesting may be necessary. The operator ξ will produce the correct relational expression. This is defined in the *Uniquest Algorithm*. For more details, see [162] and [20].

2.5 Reforming conceptual schemata

In case a conceptual schema $\Sigma = \langle \mathcal{I}, \mathcal{C} \rangle$ has undesirable properties, it has to be *reformed*. As an example, if a particular algorithm for generating efficient database schemata assumes specific characteristics of the information structure, the conceptual schema may have to be preprocessed. The result of reforming a conceptual schema $\Sigma = \langle \mathcal{I}, \mathcal{C} \rangle$ is a new schema $\Sigma' = \langle \mathcal{I}', \mathcal{C}' \rangle$.

In this section two possibilities for reformation are considered: removing label types and removing unary fact types. The generation algorithm described in the next chapter assumes that all label types and unary fact types have been removed. For more advanced conceptual schema reformations the reader is referred to [47], [69], [70], [56] and [159].

In this book we use reformations as a preprocessing for implementation, i.e. conceptual models are first reformed before they are implemented.

2.5.1 Identification and defoliation

In section 2.2.2 two kinds of atomic object types were distinguished: entity types and label types. The difference is that labels can, in contrast with entities, be reproduced on a communication medium. Therefore it should be possible to uniquely identify an entity type via one or more label types. This unique identification is also called a *naming reference* or *standard name*. Usually, identification is guaranteed using uniqueness constraints and total role constraints (see e.g. [20]).

A naming reference very often consists of one single label type, specified via a so-called *bridge type*. A fact type f is called a (binary) bridge type, only if it has the form $f = \{p, q\}$ with $\mathsf{Base}(p) \in \mathcal{L}$ and $\mathsf{Base}(q) \in \mathcal{E}$. We assume that all bridge types are binary, and that no label types are involved in non-bridge types.

Example 2.5.1

Let Person *be an entity type and let* Number *be a label type. Suppose each person has exactly one number and each number belongs to exactly one person. Then instances of type* Person *can be identified by instances of type* Number*, using a bridge type between* Person *and* Number*.* □

In complex cases, identification is defined by more than one bridge type, or by a combination of bridge types and fact types. The following example describes such a complex case.

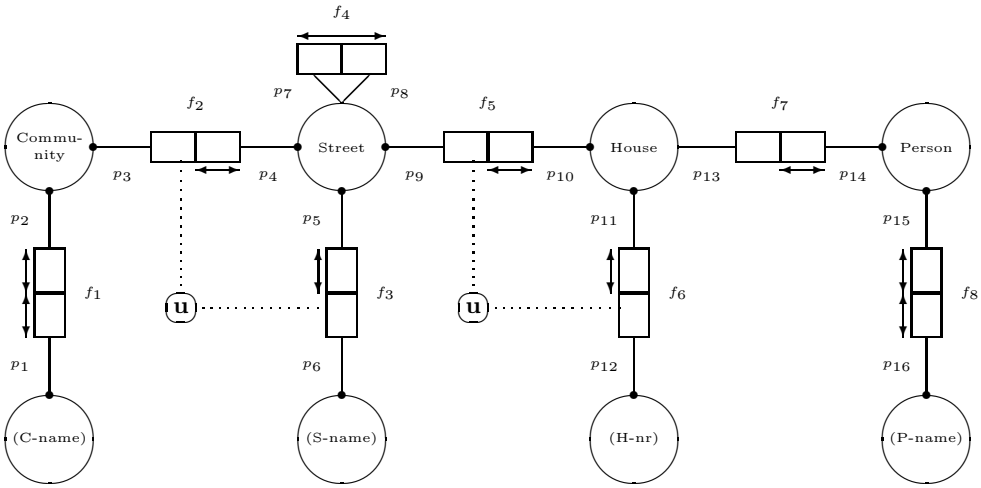


Figure 2.8: Schema with complex identification

Example 2.5.2

Consider the schema in figure 2.8. In this schema persons live in a house (f_7), houses are located in a street (f_5), and streets cross other streets (f_4) and are located in a community (f_2). By convention a label type is denoted in parenthesis.

There are two simple naming references: f_1 and f_8. The other naming references are complex. A street can be identified by the combination of its community (f_2) and its name (f_3). This identification is unique as a result of the constraints $\mathsf{unique}\{p_4\}$*,* $\mathsf{unique}\{p_5\}$ *and* $\mathsf{unique}\{p_3, p_6\}$*. Similarly, a house can be identified by the combination of its street (f_5) and its number (f_6). These naming references fulfil the requirements for structural identification (see [20]).* □

In internal representations of conceptual models the distinction between label types and entity types is not made. Therefore, an information structure must be *defoliated* before it is transformed into an internal structure. The effect of defoliation is the restriction of the information structure to the fact types that are not bridge types:

$$\mathcal{F}_d(\mathcal{I}) = \left\{ f \in \mathcal{F} \;\middle|\; \forall_{p \in f} \left[\mathsf{Base}(p) \notin \mathcal{L} \right] \right\}$$

Example 2.5.3

Consider the schema in figure 2.8 again. For this schema we have $\mathcal{F}_d = \{f_2, f_4, f_5, f_7\}$. *The defoliated information structure is presented in figure 2.9.* □

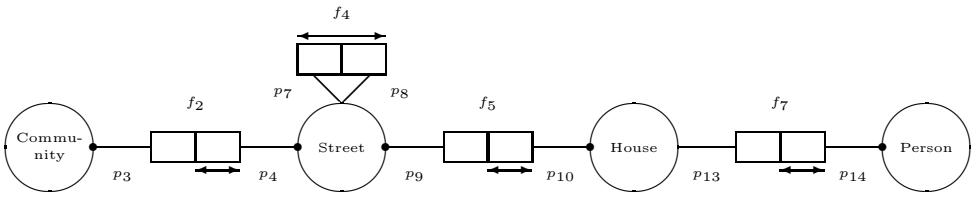


Figure 2.9: Corresponding defoliated information structure

The predicators in a defoliated information structure are specified by $\mathcal{P}_d = \cup\mathcal{F}_d$. The internal representations described in the next chapter focus on the defoliated information structure. When these internal representations are realized in a database system, each entity type is replaced by its naming reference. When no confusion is likely to occur, we will simply write $\mathcal{P}$ and $\mathcal{F}$ rather than $\mathcal{P}_d$ and $\mathcal{F}_d$.

Remark: The term defoliation is based on the Object Relation Network view of an information structure (see [20]). All leaves in the Object Relation Network are atomic object types. As a consequence, defoliation aims at removing the leaves that are label types.

2.5.2 Clean information structures

A set $F \subseteq \mathcal{F}$ is called *unary attached* if it consists of unary fact types attached to the same object type:

$$\forall_{f \in F}\,[|f| = 1] \;\wedge\; \forall_{\{p\},\{q\} \in F}\,[\mathsf{Base}(p) = \mathsf{Base}(q)]$$

A unary attached set $F \subseteq \mathcal{F}$ attached to object type X may be replaced by a single fact type, by introducing a new entity type Y and a new binary fact type between X and Y. This is a typical case of *role to object composition* ([56], [55]).

Example 2.5.4

Let Person *be an entity type and let* Is-Married *be a unary fact type attached to* Person. *This fact type can be removed by introducing an entity type* Marital Status *and a binary fact type between* Person *and* Marital Status. □

Example 2.5.5

Let Person *be an entity type and let* Is-Male *and* Is-Female *be unary fact types attached to* Person. *These fact types can be removed by introducing an entity type* Gender *and a binary fact type between* Person *and* Gender. □

The composition of unary attached fact types can be used to remove unary fact types from an information structure. An information structure without unary fact types is called a *clean information structure*.

It should be noted that, when removing unary attached fact types from an information structure, an additional enumeration constraint is needed for the new entity type. In terms of the previous example, this enumeration constraint should restrict the population of *Gender* to {Male, Female}.

2.6 Summary

In this chapter conceptual data modelling techniques with an underlying object-role structure were considered. After describing information structures and populations, the attention was focussed on integrity

constraints. Two kinds of constraints were discussed in detail: uniqueness constraints and total role constraints. In [20] other kinds of constraints (e.g. set constraints and subtype constraints) are described in detail, along with a full treatment of the relational operators used for specifying the semantics of these constraints.

The chapter was concluded with a section on schema reformation for getting rid of undesirable properties. This included the notions of identification and defoliation. We only considered structural identification, i.e. identification at the schema level. Identification at the instance level, called weak identification, is discussed in [20].

Chapter 3

Generation of tree representations

3.1 Introduction

For a given conceptual model the number of correct internal representations (internal views) may be very large, depending on the size and complexity of the conceptual model and the language used for specifying internal representations. Some internal representations will result in a system behaving in a desirable manner, whereas others will not. The desirability of a systems behaviour may for example be expressed in terms of response time and storage space. The problem then is how to find a 'good' candidate.

The aim of this chapter is to describe a mechanism for the generation of internal representations (see also e.g. [24]). Also we consider the possibility of restricting the generation process, in order to yield candidates having certain desirable properties. This sets the context for more sophisticated algorithms, such as algorithms aiming at particular optimizations. Furthermore, the generation of internal representations and the restriction of the generation process provides a basis for structural translations in automated prototyping. As design tools are becoming more important, the need for automatic translation of data structures becomes more pressing.

Various implementation-oriented data modelling techniques exist. Sometimes a distinction is made between relational, network and hierarchical models (see e.g. [157]). In approaches to the transformation of conceptual models into internal models focussing on the relational model, the result of the transformation is a relational schema in a certain normal form (see for instance [102], [106], [144], [151] and [50]). Other approaches can be found in [6], [92], and [156]. Also a lot of research has been done on nested relational models, also called non-first-normal-form or NF^2 models (see e.g. [138], [1] and [41]). Nested relational models can be seen as an intermediate level between relational, hierarchical and network models.

The organization of this chapter is as follows. Section 3.2 describes a mechanism for representing object-role information structures internally, such that the conventional internal models can be produced. Internal representations are defined *in terms of* the conceptual model, such that the underlying information structure of the original conceptual model is explicitly reflected in the internal representations. The advantage of this approach is, that conceptual-internal mappings can be described within the scope of a single specification language. Furthermore, this approach gives the opportunity to describe intermediate results easily. Information structures are assumed to be clean and defoliated (discussed in section 2.5). Section 3.3 shows how candidate internal representations can be generated for a given information structure. The attention will also be focussed on the question how this generation process can be guided, in order to generate structures having certain predefined characteristics, involving e.g. redundancy, optionals and size of the generated structure. In section 3.4 the generation algorithm is exemplified, starting from a simple object-role information structure and tracing the generation process. Examples also show how populations (or instantiations) of the information structure correspond to populations of the generated structures. This

topic is considered in more detail in chapter 4. In section 3.5 complex identification structures are discussed, resulting in a simplification during postprocessing. The chapter is concluded with a summary and outlook.

3.2 Internal representations for object-role models

In this section a mechanism is discussed for representing object-role information structures internally. The term 'internal' here means that users are not aware of the structure of the representation. Of course, designers may be aware of it.

Intuitively, such a representation is the result of lifting up certain atomic object types of the information structure and cutting the connections between these object types. Formally, a forest (set of trees) $T = \langle \mathcal{N}, E, \ell \rangle$ is called an *internal representation* for information structure $\mathcal{I} = \langle \mathcal{P}, \mathcal{O}, \mathcal{L}, \mathcal{E}, \mathcal{F}, \mathsf{Base}, \mathsf{Sub} \rangle$ if it is a labelled graph, satisfying the following wellformedness conditions $t_1, ..., t_6$ discussed below.

3.2.1 Base representation

t_1 : The set of nodes $\mathcal{N}$ is a partition of the set of predicators $\mathcal{P}$, such that predicators in the same node are type related:

$$\forall_{n \in \mathcal{N}, p,q \in n} [p \sim q]$$

A node is intended to model a field (or attribute) in the database. Since all predicators in the same node are type related, the common pater familias in node n can be denoted as $\mathsf{Base}(n)$.

t_2 : Loss of information should be excluded. Therefore, all predicators sharing the same node belong to different fact types:

$$\forall_{n \in \mathcal{N}, f \in \mathcal{F}} [|n \cap f| \leq 1]$$

Conditions t_1 and t_2 express the basic requirements for the nodes in an internal representation. An internal representation is called a *separately objectifying representation* if all objectifications (elements of $\mathcal{H}$) are treated separately: $\forall_{n \in \mathcal{N}} [|n \cap \mathcal{H}| \leq 1]$.

3.2.2 Fact representation

$E \subseteq \mathcal{N} \times \mathcal{N}$ is a set of edges. The pair $\langle m, n \rangle$ represents an edge from node m to node n. The edges in an internal representation should have the following properties.

t_3 : Edges are labelled by fact types in the obvious way, with some care to handle objectification. The function $\ell : E \to \mathcal{F}$ assigns $\ell(\langle m, n \rangle) = f$, iff both the source node and the destination node contain a predicator from f:

1. $n \cap f \neq \varnothing$
2. $m \cap f = \varnothing \Rightarrow \mathsf{Base}(m) = f$

t_4 : Fact types are not split and are therefore located around a single parent:

$$\ell(\langle m_1, n_1 \rangle) = \ell(\langle m_2, n_2 \rangle) \;\Rightarrow\; n_1 = n_2$$

t_5 : In order to keep predicators from the same fact type together, all predicators involved in an edge must occur in the label of of that edge, or in the label of some associated edge. More precisely, each edge $\langle m, n \rangle \in E$ has the following property:

$$\forall_{p \in m \cup n} \exists_{e = \langle x,y \rangle \in E} [p \in x \cup y \;\wedge\; \ell(e) = \mathsf{Fact}(p)]$$

We call T an *incomplete internal representation* for information structure $\mathcal{I}$, denoted as $\mathsf{Forest}(T, \mathcal{I})$, if it satisfies all wellformedness conditions discussed above.

3.2.3 Completeness

t_6 : In order to guarantee that the complete information structure is represented, each predicator must be involved in some edge:

$$\forall_{p \in \mathcal{P}} \exists_{\langle m,n \rangle \in E} [p \in m \cup n]$$

We call an (incomplete) internal representation T *complete*, denoted as $\mathsf{CompleteForest}(T, \mathcal{I})$, if it also satisfies wellformedness condition t_6. An internal representation is also called a *tree representation* in the sequel.

Example 3.2.1

Recall the information structure with two binary fact types f and g from figure 2.3. The tree in figure 3.1 is a possible tree representation for this information structure, according to:

$$\begin{aligned} \mathcal{N} &= \{n_1, n_2, n_3\} \text{ with:} \\ &\quad n_1 = \{q, r\}, n_2 = \{p\} \text{ and } n_3 = \{s\} \\ E &= \{e_1, e_2\} \text{ with:} \\ &\quad e_1 = \langle n_2, n_1 \rangle \text{ and } e_2 = \langle n_3, n_1 \rangle \\ \ell(e_1) &= f \\ \ell(e_2) &= g \end{aligned}$$

□

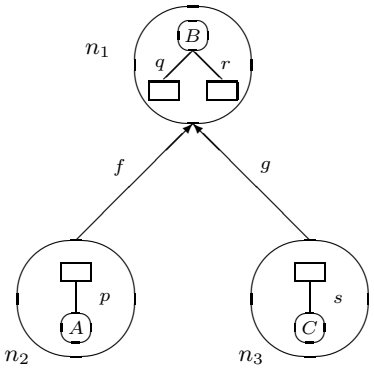


Figure 3.1: Tree containing two binary fact types

The tree representation in figure 3.1 consists of a single tree with root $\{q, r\}$. Other complete representations consisting of a single tree have root $\{p\}$ or $\{s\}$. Representations consisting of two trees are also possible for this information structure. The number of nodes will then be equal to the number of predicators, i.e. four.

An alternative notation for the tree representation in figure 3.1 is $[\{q, r\}, [\{p\}], [\{s\}]]$. This is called the nested-bracket notation for hierarchical record structures (will be discussed in section 3.2.5). Since predicator r is unique but not total and predicator q is total but not unique, this record structure can be written as follows:

$$\left[\overline{\{q, r\}}\, , [\{p\}]\ \mathsf{rep}, [\{s\}]\ \mathsf{op} \right]$$

Here $\{q, r\}$ is the primary key, $\{p\}$ is a repeating attribute type and $\{s\}$ is an optional attribute type. The record structure can be simplified to: $\left[\overline{B}\, , A\ \mathsf{rep}, C\ \mathsf{op} \right]$. Further details about optional and repeating attribute types are given in section 3.2.5.

Example 3.2.2

Recall the information structure from figure 2.4: binary fact type f is objectified and plays a role in fact type g. The tree in figure 3.2 is a possible tree representation for this information structure, according to:

$$\begin{aligned}
\mathcal{N} &= \{n_1, n_2, n_3, n_4\} \text{ with:} \\
&\quad n_1 = \{q\}, n_2 = \{p\}, n_3 = \{r\} \text{ and } n_4 = \{s\} \\
E &= \{e_1, e_2, e_3\} \text{ with:} \\
&\quad e_1 = \langle n_2, n_1 \rangle, e_2 = \langle n_3, n_1 \rangle \text{ and } e_3 = \langle n_4, n_3 \rangle \\
\ell(e_1) &= f \\
\ell(e_2) &= f \\
\ell(e_3) &= g
\end{aligned}$$

□

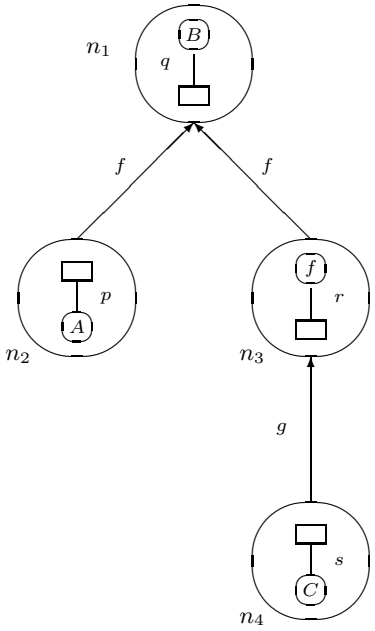


Figure 3.2: Tree containing objectified fact type f

The conditions $t_1, ..., t_6$ will guarantee the wellformedness of an internal representation with respect to a specific information structure. They do not specify the generation process of internal representations. In section 3.3 a generation algorithm will be described. This algorithm yields all possible tree representations for a given information structure. Several guidance parameters will be introduced in order to exclude candidates with undesirable properties from the generation process.

3.2.4 Basic properties of internal representations

Note the duality of both partitions $\mathcal{F}$ and $\mathcal{N}$ of the set of predicators $\mathcal{P}$. The first is based on equality of fact type, the second is related to attachedness. The unique node in which a predicator is contained is found by the function:

$$\mathsf{Node} : \mathcal{P} \longrightarrow \mathcal{N}$$

It is defined by: $\mathsf{Node}(p) = n \iff p \in n$, analogous with the function Fact (see section 2.2.2). A node m, being the source of an edge, is anchored to this edge by a unique predicator $p \in m$:

Lemma 3.2.1 Let $e = \langle m, n \rangle$ be an edge and let $\ell(e) = f$. If node m has an atomic base, it has the following property:

$$|m \cap f| = 1$$

Proof:

Let $\langle m, n \rangle \in E$ with $\mathsf{Base}(m) \in \mathcal{A}$ and $\ell(\langle m, n \rangle) = f$. From condition t_3 we conclude that $m \cap f \neq \varnothing$. Now the result can be obtained from condition t_2. □

For node m this unique predicator is denoted as $\mathsf{Anchor}(m)$. The application of this lemma to the example tree representations is trivial:

Example 3.2.3

In the tree shown in figure 3.1 we have $\mathsf{Anchor}(n_2) = p$ *and* $\mathsf{Anchor}(n_3) = s$. *For* n_1 *it is not defined, since* n_1 *is not the source of any edge. In the tree shown in figure 3.2 we have* $\mathsf{Anchor}(n_2) = p$ *and* $\mathsf{Anchor}(n_4) = s$. *For* n_1 *it is not defined, since this node is not the source of any edge. For* n_3 *it is not defined, since this node does not have an atomic base.* □

For the nodes, being the destination of some edge, an analogous property holds:

Lemma 3.2.2 Edge $e = \langle m, n \rangle$ with $\ell(e) = f$ has the following property:

$$|n \cap f| = 1$$

Proof:

Let $\langle m, n \rangle \in E$ with $\ell(\langle m, n \rangle) = f$. From condition t_3 we conclude that $n \cap f \neq \varnothing$. Now the result can be obtained from condition t_2. □

The corresponding predicator in destination node n is unique for fact type f and is called its *hook*. It is denoted as $\mathsf{Hook}(f)$. Note that each fact type has a hook, while not all nodes have an anchor. Obviously there may be several hooks in the same node.

Example 3.2.4

In the trees shown in figure 3.1 and figure 3.2 we have $\mathsf{Hook}(f) = q$ *and* $\mathsf{Hook}(g) = r$. □

The properties of anchor and hook define the layout within a node. A 'top' in a node is an anchor and a 'bottom' in a node is a hook. This allows us to draw internal representations in a specific way (see for example figure 3.1).

The notions of anchor and hook will also play an important role during the construction of internal representations for guiding the generation process. Furthermore, they will be used for the transformation of populations and operations from the conceptual to the internal level in chapter 4, and for the mutation of internal representations into other internal representations in chapter 5.

Let T be a complete tree representation for information structure $\mathcal{I}$ and let $\mathcal{R} \subseteq \mathcal{N}$ be the set of nodes being the root of some tree:

$$\mathcal{R} = \left\{ x \in \mathcal{N} \;\middle|\; \forall_{\langle m,n \rangle \in E} \left[x \neq m \right] \right\}$$

A basic property of forest T involves the number of nodes in T. It can be expressed in terms of the number of predicators, fact types and objectifications in $\mathcal{I}$, and the number of trees in T:

Lemma 3.2.3 $|\mathcal{N}| = |\mathcal{P}| - |\mathcal{F}| + |\mathcal{H}| + |\mathcal{R}|$

Proof:

Suppose $e_1, \ldots, e_k$ are all edges labelled with the same fact type f. Then all these edges have the same destination node (condition t_4). This destination node contains precisely one predicator of f (see lemma 3.2.2). From lemma 3.2.1 we conclude that the sources of the edges either contain precisely one predicator of f, or correspond to an objectification of f. We denote the number of objectifications of f as $\|f\|$. We therefore have:

$$\begin{aligned} & \text{number of edges labelled with } f \\ = \;& |f| - 1 + \|f\| \end{aligned}$$

Summation of all fact types yields:

$$\begin{aligned} |E| &= \sum_{f \in \mathcal{F}} \text{number of edges labelled with } f \\ &= \sum_{f \in \mathcal{F}} (|f| - 1 + \|f\|) \\ &= |\mathcal{P}| - |\mathcal{F}| + |\mathcal{H}| \end{aligned}$$

The number of nodes equals the number of edges plus the number of trees. As a result:

$$|\mathcal{N}| = |E| + |\mathcal{R}| = |\mathcal{P}| - |\mathcal{F}| + |\mathcal{H}| + |\mathcal{R}|$$

□

In the proof given above it is assumed that T is a separately objectifying representation, i.e. all objectifications (elelements of $\mathcal{H}$) constitute their own node. If this is not the case, this lemma specifies an upperbound for the number of nodes.

A *singleton internal representation* for information structure $\mathcal{I}$ is an internal representation where each fact type from $\mathcal{I}$ is represented in a separate tree ($|\mathcal{R}| = |\mathcal{F}|$). The set of all possible singleton representations is denoted as $\mathcal{S}_s$. This set has the following property:

Lemma 3.2.4 $|\mathcal{S}_s| \geq 2^{|\mathcal{F}|}$

Proof:

The fact types in $\mathcal{I}$ are binary fact types or fact types of a higher degree (section 2.5.2). As a consequence, at least 2 different trees can be constructed for each fact type f, because each predicator $p \in f$ can be used as a hook ($p = \mathsf{Hook}(f)$). The number of fact types in $\mathcal{I}$ is $|\mathcal{F}|$ and all combinations of trees are valid. From this the lemma follows. More precisely, we have:

$$|\mathcal{S}_s| = \prod_{1 \leq i \leq |\mathcal{F}|} |f_i|$$

□

The set of all possible internal representations is denoted as $\mathcal{S}$. This is also called the *internal solution space*. Note that $\mathcal{S}_s \subseteq \mathcal{S}$. The following property is obvious:

Theorem 3.2.1 $|\mathcal{S}| \geq 2^{|\mathcal{F}|}$

Proof:

This property follows directly from lemma 3.2.4 in combination with the fact that $\mathcal{S}_s \subseteq \mathcal{S}$. □

3.2.5 Target environments

Tree representations can be interpreted in several ways. Let $T = \langle \mathcal{N}, E, \ell \rangle$ be a tree representation. In general, each tree in T can be interpreted as a (relational, hierarchical or network) record type where each node corresponds to an atomic attribute type.

We give a more detailed description of the interpretation in terms of the *nested relational model* (see e.g. [138], [1] and [41]). Let $m \in \mathcal{N}$ be a node with children as depicted in figure 3.3. Here each fact type f_i has $\mathsf{Hook}(f_i) \in m$, and the corresponding anchors from f_i are in nodes $n_{i1}, ..., n_{il_i}$.

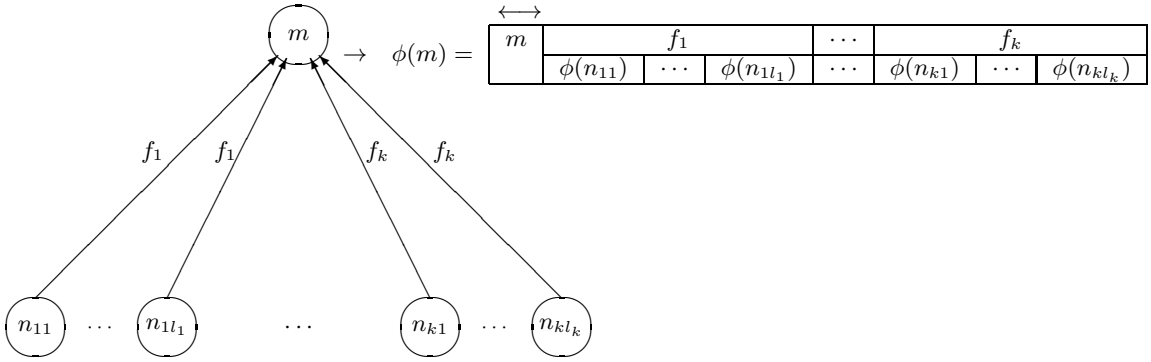


Figure 3.3: Interpretation in terms of nested tables

We then construct a table with a column for m-values, along with a column for each fact type hooking to node m. As a consequence, we get a sub-table for each such fact type. This is described by the operator ϕ. The general behaviour of ϕ is shown in figure 3.3. The column for m-values is chosen as the primary key ([157]).

Using the nested-bracket notation ([54]) the behaviour of ϕ is as follows:

$$\phi(m) = [\overline{m}, [\phi(n_{11}), \cdots, \phi(n_{1l_1})]\ c(p_1), \cdots, [\phi(n_{k1}), \cdots, \phi(n_{kl_k})]\ c(p_k)]$$

Here $c(p_i)$ is a collection of integrity constraints which can be derived from the integrity constraints for $p_i = \mathsf{Hook}(f_i)$. For the basic constraints (uniqueness and total role constraints), the resulting $c(p_i)$ is expressed using the keywords op and rep for optional and repeating attribute types respectively (see also figure 3.4). This is done as follows:

$$\begin{aligned} \mathsf{rep} \in c(p_i) &\iff \neg\, \mathsf{unique}(p_i) \\ \mathsf{op} \in c(p_i) &\iff \neg\, \mathsf{total}(p_i) \end{aligned}$$

This definition was illustrated by a simple example in section 3.2.3. Further examples will be given in section 3.4. Note that for a given nested table a flat relational table can be obtained by unnesting (flattening) the hierarchical structure. Further details are given in section 5.2. The use of the network modelling technique (e.g. CODASYL [157]) is illustrated in section 4.5.2.

3.3 The generation process

3.3.1 Basic generation strategy

The intention of the *generation process* that we describe is to generate all internal representations for a given information structure $\mathcal{I}$. However, in order to avoid the overhead of the backtrack mechanism, we

notation	explanation
$\overline{m}$	m is the primary key
$[n_1, ..., n_l]$ op	optional attribute group
$[n_1, ..., n_l]$ rep	repeating attribute group

Figure 3.4: Legend of symbols used in record structures

let the algorithm search for a random internal representation.

The construction first sets up an initial incomplete tree representation. This incomplete representation is extended step by step until it is complete. In each step a fact type will be incorporated in the internal representation. As a consequence, the number of steps equals the number of fact types in the information structure.

Initially each predicator constitutes its own isolated node, not linked to any other node. This initialization establishes the following precondition:

Lemma 3.3.1 Precondition

$\mathcal{I}$ is an information structure

$\mathcal{N} := \{ \{p\} \mid p \in \mathcal{P} \}$;
$E := \varnothing$;
$\ell := \varnothing$;
$T := \langle \mathcal{N}, E, \ell \rangle$;

Forest$(T, \mathcal{I})$

Proof:

Let $\mathcal{I}$ be an information structure. After the initialisation, $\langle \mathcal{N}, E, \ell \rangle$ satisfies the conditions t_1 and t_2, because each node contains exactly one predicator. The conditions t_3, t_4 and t_5 are void, because there are no edges. From this we conclude that T is an incomplete tree representation of $\mathcal{I}$, denoted as Forest$(T, \mathcal{I})$. □

In each step of the algorithm a fact type is selected to be incorporated in the tree representation. This is done by selecting a predicator of a yet unprocessed fact type. This predicator then will act as a handle for the extension of the tree representation.

We use the set $\mathcal{U}$ to record the unprocessed isolated nodes. The procedure CanExtend selects an unprocessed node $m \in \mathcal{U}$, in combination with another node $n \in \mathcal{N}$. The actual extension is performed by the procedure ProcessFactType. The algorithm:

proc GenerateForest ($\mathcal{I}$: Information Structure):Forest;
  $\mathcal{N} := \{ \{p\} \mid p \in \mathcal{P} \}$;
  $E := \varnothing$;
  $\ell := \varnothing$;
  $\mathcal{U} := \mathcal{N}$;
  while CanExtend (m, n) **do**
    ProcessFactType (m, n)
  od ;
  $T := \langle \mathcal{N}, E, \ell \rangle$
endproc GenerateForest

The extension of a tree representation either enlarges an already existing tree, or adds a new tree to the forest. If the unprocessed node $m \in \mathcal{U}$ is chosen in combination with a processed node $n \in \mathcal{N} - \mathcal{U}$, an existing tree is extended. A choice for $n = m$ leads to the creation of a new tree. Other combinations of m and n are not allowed, because they do not result in a correct tree representation. The procedure CanExtend will check this. Furthermore it will guarantee that all object types within the same node belong to the same subtype hierarchy:

```
proc CanExtend (var m, n : Node) : Boolean;
    take random m = {p} ∈ U, n ∈ (N − U) ∪ {m} with Base(m) = Base(n)
endproc CanExtend
```

It may be desirable to further restrict the possibilities for extension, e.g. when the result of the generation must fulfil specific design criteria. This further restriction is discussed in the next section.

For the extension of an existing tree and for the creation of a new tree one single strategy can be used. This strategy consists of the processing of the fact type, which is implicitly specified via the unprocessed unary node m.

The extension starts with the union of the selected nodes m and n. Then the entire fact type f, specified via m, is processed by the addition of edges with label f:

```
proc ProcessFactType (m, n : Node);
    Let m = {p};
    if m ≠ n then N := N − {m};
    n := m ∪ n;
    for each x ∈ Fact(p) − m do
        E := E ∪ {e} where e = ⟨{x}, n⟩;
        ℓ(e) := Fact(p)
    od ;
    U := U − {{x}|x ∈ Fact(p)};
    if ∃{x}∈U [Base(x) = Fact(p)] then ProcessObjectifications(Fact(p), n);
endproc ProcessFactType
```

Finally, we have to handle the objectifications of the fact type f under consideration. For each fact type g, which contains an objectification of f, we have to choose a predicator for unnesting. This unnesting is performed by the addition of edges with label f. The fact type g then is processed by recursively calling procedure ProcessFactType:

```
proc ProcessObjectifications (f : FactType; n : Node);
    for each m ∈ U with Base(m) = f do
        E := E ∪ {⟨m, n⟩};
        ℓ(⟨m, n⟩) := f;
        ProcessFactType(m, m)
    od
endproc ProcessObjectifications
```

The procedure ProcessObjectifications results in a separately objectifying representation (all objectifications are taken separately). Next we show that the loop in procedure GenerateForest is governed by the following invariant:

Lemma 3.3.2 Invariant

Forest($T, \mathcal{I}$) **and** CanExtend(m, n)

ProcessFactType(m, n)

Forest($T, \mathcal{I}$)

Proof:

Suppose Forest($T, \mathcal{I}$) and CanExtend(m, n). Let f be the fact type implicitly specified by m. After the execution of ProcessFactType, condition t_1 is satisfied as a result of the checks in CanExtend(m, n).

Each predicator in $f - m$ is the source of a new edge with destination n and label f. From this we conclude that the conditions t_2, t_4 and t_5 are satisfied.

Furthermore, the recursive call in the processing of objectifications guarantees condition t_3. As a consequence, the resulting forest T is an incomplete tree representation of $\mathcal{I}$. □

The previous lemmas guarantee the correctness of the entire generation:

Theorem 3.3.1 Postcondition

$\mathcal{I}$ is an information structure

$T :=$ GenerateForest($\mathcal{I}$)

$\mathcal{U} = \varnothing \ \wedge$ CompleteForest($T, \mathcal{I}$)

Proof:

Let $\mathcal{I}$ be an information structure. The result of the initialization is an incomplete tree representation (see lemma 3.3.1). In each iteration the procedure ProcessFactType yields an incomplete tree representation (see lemma 3.3.2). Procedure CanExtend guarantees that GenerateForest terminates with $\mathcal{U} = \varnothing$, i.e. each predicator has been classified. Consequently T is a complete tree representation according to condition t_6. □

3.3.2 Further restricting the generation process

The generation process described in the previous section results in a correct internal representation, according to the definition in section 3.2. However, in many cases the property of correctness is not strong enough, for example when the search space in schema transformation is to be reduced. Then, database structures to be generated must be characterised in advance, in order to exclude candidates with undesirable properties from the generation process.

This characterization can be embedded within the generation framework as follows. In each iteration the procedure CanExtend selects two nodes m and n as a base for further processing. This selection leaves us many opportunities to influence the nature of the resulting tree representation, simply by restricting the number of possible combinations of m and n. We will discuss some restrictions in this section.

Let $m = \{p\} \in \mathcal{U}$ be an unprocessed isolated node and let n be either a processed node ($n \in \mathcal{N} - \mathcal{U}$) or a new root ($n = m$), such that their bases belong to the same subtype hierarchy. This forms the starting-point for further restriction (see figure 3.5).

First we deal with lossless tree representations. To prevent loss of information in the result, the anchor of node n should have a total role constraint:

$$n \in \mathcal{R} \vee \text{total}(\text{Anchor}(n))$$

proc CanExtendReduced (**var** m, n : Node) : Boolean;
take random $m = \{p\} \in \mathcal{U}$, $n \in (\mathcal{N} - \mathcal{U}) \cup \{m\}$ with Base(m) = Base(n)
such that: $n \in \mathcal{R} \vee$ total(Anchor(n))
and if NoNesting **then** unique(p)
and if NoRedundancy **then** $n \in \mathcal{R} \vee$ unique(Anchor(n))
and if NoOptionals **then**
if $n \in \mathcal{R}$
then $\forall_{x \in m \cup n}$ [total(x)]
else total(p)
and if SizePreference **then** |Facts(n)| $< \gamma$
and if DepthPreference **then** Depth(n) $< \delta$
endproc CanExtendReduced

Figure 3.5: Guidance conditions for the extension of incomplete representations

Absence of this constraint allows Fact(p) to have instances that cannot be related to instances of Fact(Anchor(n)) higher up in the tree. These instances of Fact(p) can not be reached via node n, which results in loss of information.

Next we deal with several other structural properties, such as absence of nesting, redundancy or optionals, and preferences with respect to the size or depth of the tree. A certain combination of these properties can be specified via the guidance parameters NoNesting, NoRedundancy, etcetera (see figure 3.5).

The meaning of these parameters is expressed in so-called *guidance conditions*. For example, for the construction of flat relational structures (parameter NoNesting), a uniqueness constraint unique(p) is required. Freedom from redundancy (parameter NoRedundancy) is obtained when Anchor(n) has a uniqueness constraint, provided n is not a root:

$$n \in \mathcal{R} \vee \text{unique}(\text{Anchor}(n))$$

A structure will be free from optionals (parameter NoOptionals) when the equality of population Pop(m) and population Pop(n) can be guaranteed. This is expressed in terms of total role constraints. Furthermore, a maximal number γ of fact types per tree can be specified (parameter SizePreference) and a maximal depth δ of the tree can be specified (parameter DepthPreference).

3.4 Elaborated examples

In this section the behaviour of the generation algorithm is illustrated. This is done by showing how an example information structure is transformed into tree structures, and how different settings of the guidance parameters influence the result.

Consider the information structure shown in figure 3.6. We assume that this structure has already been defoliated. The population in figure 3.7 will be used to illustrate the characteristics of some generated trees at the level of instances.

The selection of a parameter setting will on the one hand be determined by quantitative aspects of the conceptual schema under consideration. On the other hand it will be determined by aspects of the target environment. For example, if the application is to be stored on CD-ROM parameter NoRedundancy can be switched off, because there are no updates. In a relational target environment, parameter NoNesting may be set to true.

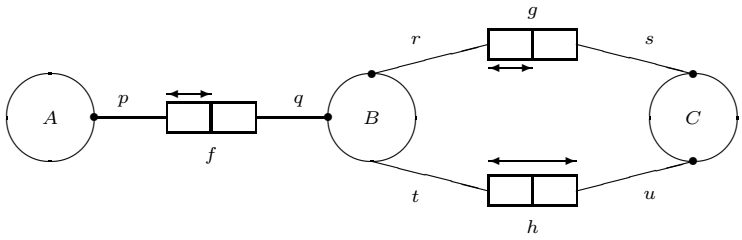


Figure 3.6: Example schema containing three binary fact types

f	
A	B
a_1	b_1
a_2	b_1
a_3	b_2
a_4	b_3

g	
B	C
b_1	c_1
b_2	c_1
b_3	c_2

h	
B	C
b_1	c_1
b_2	c_1
b_1	c_2

Figure 3.7: Possible population for example schema

3.4.1 Size preference without redundancy

Suppose we aim at a tree representation where the maximal number of fact types per tree is 2 and redundancy cannot occur. This can be obtained by the following setting of parameters:

NoRedundancy = True
SizePreference = True
$\gamma = 2$

In the information structure in figure 3.6 we have $\mathcal{P} = \{p, q, r, s, t, u\}$ and therefore both $\mathcal{N}$ and $\mathcal{U}$ initially consist of 6 nodes, each containing one predicator. There are no edges yet: $E = \varnothing$ and $\ell = \varnothing$.

We will show how the generation proceeds step by step, in each step choosing two nodes m and n that fulfil the guidance criteria, followed by the processing of the fact type implicitly specified by m. During this processing the set of unprocessed nodes $\mathcal{U}$ will be reduced. Furthermore, nodes in $\mathcal{N}$ will be joined and edges between these nodes will be constructed.

1. Let the first selection of nodes consist of $m = \{r\}$ and $n = \{r\}$. Note that the choice $m = n$ will lead to the creation of a new root (see section 3.3). This choice passes all checks in CanExtend. Procedure ProcessFactType reacts as follows: an edge labelled with g from $\{s\}$ to $\{r\}$ is created, and $\{s\}$ and $\{r\}$ are removed from $\mathcal{U}$. $\mathcal{N}$ does not change, because no nodes have to be joined. The resulting situation is shown in figure 3.8.

2. Now $\mathcal{U}$ contains the unprocessed isolated nodes $\{p\}, \{q\}, \{t\}$ and $\{u\}$. Since the maximal grouping rate $\gamma = 2$ has not yet been reached, the next step may either extend the tree in figure 3.8 or create a new root. Suppose the existing tree is extended. There are three possibilities to do this:

   (a) Choose $m = \{u\}$ and $n = \{s\}$, and process fact type h.

   (b) Choose $m = \{t\}$ and $n = \{r\}$, and process fact type h.

   (c) Choose $m = \{q\}$ and $n = \{r\}$, and process fact type f.

   Consider the first alternative. It attempts to join the nodes $\{u\}$ and $\{s\}$, and to add an edge from node $\{t\}$ to node $\{u, s\}$. However, Anchor$(\{u, s\}) = s$ and predicator s is not unique. Therefore, the

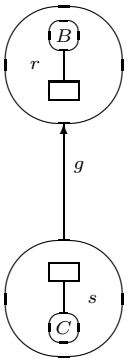


Figure 3.8: The first step processes fact type g

guidance condition of parameter NoRedundancy is not fulfilled and as a consequence, this possibility is rejected. In section 3.4.2 the meaning of this guidance condition will be illustrated at the instance level.

Next we consider the second alternative. ProcessFactType reacts as follows: after joining $\{t\}$ and $\{r\}$, an edge labelled with h from $\{u\}$ to $\{r,t\}$ is added. Furthermore, the nodes $\{u\}$ and $\{t\}$ are deleted from $\mathcal{U}$. The resulting tree is shown in figure 3.9.

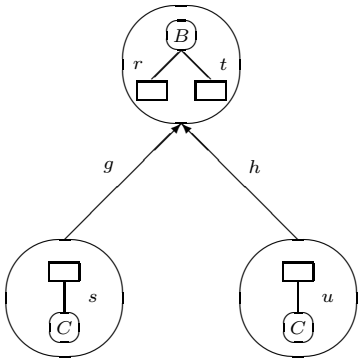


Figure 3.9: The second step adds fact type h

In case the third alternative is selected, the result is an edge labelled f from $\{p\}$ to $\{q,r\}$. We do not consider this alternative and proceed with the tree shown in figure 3.9.

3. At this moment $\mathcal{U}$ contains two unprocessed nodes $\{p\}$ and $\{q\}$. This leaves three possibilities for the processing of fact type f in the final step:

   (a) Join $\{q\}$ and $\{r,t\}$.

   (b) Create a new root $\{q\}$.

   (c) Create a new root $\{p\}$.

   Consider the first alternative. Observing the guidance parameters (especially $\gamma = 2$), we see that this alternative is rejected, as it would result in a tree containing three fact types.

   Next we consider the second alternative. In this alternative $\{q\}$ is chosen to become the root of a new tree. In the reaction of ProcessFactType, $\mathcal{N}$ does not change, $\mathcal{U}$ is made empty and an edge from

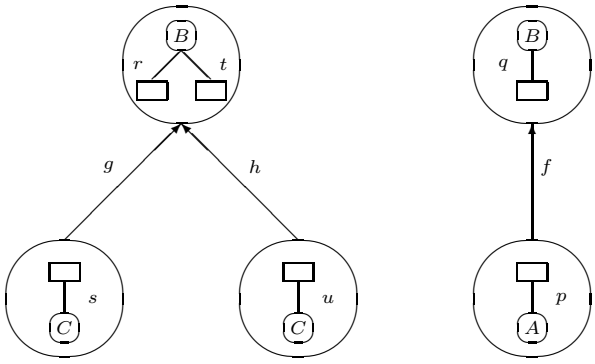


Figure 3.10: The final step adds fact type f

node $\{p\}$ to node $\{q\}$ is added. The label of this new edge is f. This alternative leads to the final situation shown in figure 3.10.

If the third alternative is chosen the new root is $\{p\}$. Then an edge from $\{q\}$ to $\{p\}$ is added.

The forest in figure 3.10 corresponds to the following relation types:

- $\left[\overline{\{r,t\}}, \{s\}, \{u\}\ \ \mathsf{op}\ \ \mathsf{rep}\right]$
- $\left[\overline{\{q\}}, \{p\}\ \ \mathsf{rep}\right]$

The keys of these relation types are $\{r,t\}$ and $\{q\}$ respectively. This is expressed by a line (or arrow) on top of the corresponding attribute type. There are two relation-valued attribute types $\{u\}$ and $\{p\}$. This is expressed by rep, which is a shorthand for *repeating*. $\{u\}$ is an optional relation-valued attribute type for instances of C. It is optional according to the absence of constraint total(t), while it is relation-valued according to the absence of constraint unique(t) (see figure 3.6).

The example population from figure 3.7 is represented in the generated structure in figure 3.11.

$\overleftrightarrow{\{r,t\}}$	$\{s\}$	$\{u\}$ op rep
b_1	c_1	c_1
		c_2
b_2	c_1	c_1
b_3	c_2	

$\overleftrightarrow{\{q\}}$	$\{p\}$ rep
b_1	a_1
	a_2
b_2	a_3
b_3	a_4

Figure 3.11: Example population in generated structure without redundancy

We have used parameter NoRedundancy for generating the structure in figure 3.11. Note that this structure is indeed free from redundancy, as every fact instance is mentioned only once. Note furthermore that attribute type $\{u\}$ is optional.

The scenario discussed in this section shows the strong influence of the first step of the generation on the final result. The choice for node $\{r\}$ in the first step (in combination with parameter NoRedundancy) fully determined the height of the tree(s) in the resulting structure.

3.4.2 Maximal grouping without optionals

In case a maximal grouping of fact types is preferred, such that optional attribute types cannot occur, the parameters should be set as follows:

NoOptionals = True
SizePreference = True
$\gamma = |\mathcal{F}| = 3$

The ultimate result of maximal grouping is a forest consisting of a single tree. However, since γ expresses the *maximal* number of fact types per tree, the setting $\gamma = |\mathcal{F}|$ will also allow representations consisting of more than one tree.

Initially both $\mathcal{U}$ and $\mathcal{N}$ consist of 6 nodes, each containing one predicator. Furthermore $\mathcal{E} = \varnothing$ and $\ell = \varnothing$.

1. Suppose the first step chooses $m = \{q\}$ and $n = \{q\}$. This choice passes all checks in CanExtend. Then ProcessFactType reacts as follows: an edge labelled with f is constructed from $\{p\}$ to $\{q\}$, and both processed nodes are removed from $\mathcal{U}$. $\mathcal{N}$ does not change. The resulting situation is shown in figure 3.12.

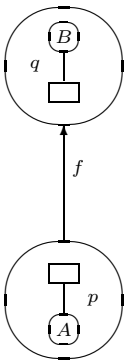


Figure 3.12: The first step processes fact type f

2. Now $\mathcal{U}$ consists of the nodes $\{r\}, \{s\}, \{t\}$ and $\{u\}$. There are six possibilities for the next step:
   (a) Choose $m = \{t\}$ and $n = \{q\}$, and process fact type h.
   (b) Choose $m = \{r\}$ and $n = \{q\}$, and process fact type g.
   (c) Create a new root for either $\{r\}, \{s\}, \{t\}$ or $\{u\}$.

   Consider the first alternative. It attempts to join node $\{t\}$ and node $\{q\}$, adding an edge from node $\{u\}$ to node $\{q, t\}$. However, the root $\{q, t\}$ causes an optional attribute type $\{u\}$, since predicator q is total and t is not. Therefore the guidance condition of parameter NoOptionals is not fulfilled and CanExtend rejects this possibility.

   Next we consider the second alternative. It selects unprocessed node $\{r\}$ in combination with processed node $\{q\}$. Procedure ProcessFactType will react as follows: after joining $\{r\}$ and $\{q\}$, an edge labelled with g from $\{s\}$ to $\{q, r\}$ is added. Furthermore, the nodes $\{r\}$ and $\{s\}$ are deleted from $\mathcal{U}$. The situation is shown in figure 3.13.

   In the third alternative a new root is created. This possibility will not be worked out further.

3. At this moment $\mathcal{U}$ contains the unprocessed nodes $\{t\}$ and $\{u\}$. This leaves four possibilities for the processing of fact type h in the last step:

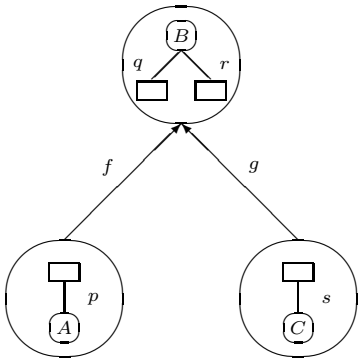


Figure 3.13: The second step adds fact type g

(a) Choose $m = \{t\}$ and $n = \{q, r\}$.

(b) Choose $m = \{u\}$ and $n = \{s\}$.

(c) Create a new root for either $\{t\}$ or $\{u\}$.

Consider the first alternative. It attempts to join $\{t\}$ and $\{q, r\}$. As a result of parameter NoOptionals this alternative will be rejected (see also the previous step).

Consider the second alternative. It selects $\{u\}$ and $\{s\}$ to be joined. In the reaction of ProcessFactType the set $\mathcal{N}$ is adapted by joining $\{u\}$ and $\{s\}$, and $\mathcal{U}$ is made empty. Furthermore an edge labelled with h from node $\{t\}$ to node $\{u, s\}$ is added. This leads to the final situation shown in figure 3.14.

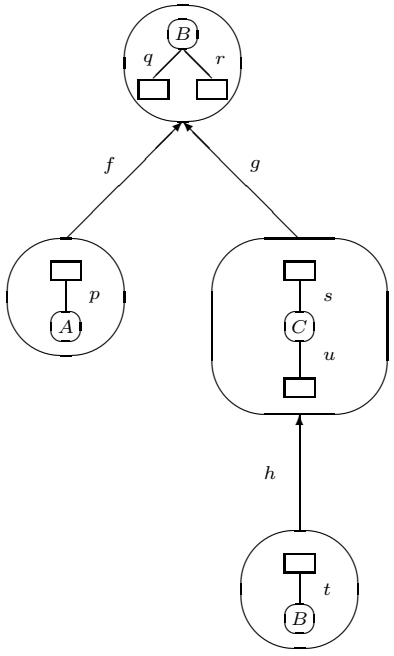


Figure 3.14: The third step adds fact type h

The tree in figure 3.14 corresponds to the following relation type:

$$\left[\overleftrightarrow{\{q,r\}}, \{p\}\ \mathsf{rep}, \{s,u\}, \{t\}\ \mathsf{rep}\right]$$

The key of this relation type is $\{q,r\}$. There are two relation-valued attribute types $\{p\}$ and $\{t\}$, for instances of A and B respectively. The nesting of these attribute types is the result of $\neg\,\mathsf{unique}(q)$ and $\neg\,\mathsf{unique}(u)$ (see figure 3.6).

The population from figure 3.7 is represented in the generated relation type in figure 3.15. Note that this structure does not have optional attribute types, as was specified by the guidance parameters.

$\overleftrightarrow{\{q,r\}}$	$\{p\}$ rep	$\{s,u\}$	$\{t\}$ rep
b_1	a_1	c_1	b_1
	a_2		b_2
b_2	a_3	c_1	b_1
			b_2
b_3	a_4	c_2	b_1

Figure 3.15: Example population in generated structure without optionals

The tree representation shown in figure 3.14 is essential for the meaning of figure 3.15. There are three tuples, the first one of which is read as follows: *b_1 is related to a_1 and a_2 (via f) and to c_1 (via g). c_1 is related to b_1 and b_2 (via h).* An incorrect interpretation could be: *a_1 is related to b_1, and a_2 is related to b_2.*

We see that the generated structure allows for redundancy: the instances $\{c_1, b_1\}$ and $\{c_1, b_2\}$ of fact type h are mentioned twice. A closer inspection of the original schema in figure 3.6 and the tree representation in figure 3.14 leads to the cause of this redundancy. In the tree structure we see that an instance of fact type h containing object x of type C is mentioned as often as x is involved in g. Since predicator s is not unique, some instances of h may be mentioned more than once.

3.4.3 Loss of information

At the end of the previous section we considered the cause of redundancy. In this section we will show that the point that was made there is closely related to the question of loss of information.

Consider the schema in figure 3.6 and the tree representation in figure 3.14 again. Assume the absence of constraint $\mathsf{total}(s)$. This assumption does not affect the relation type expressed by the tree in figure 3.14:

$$\left[\overleftrightarrow{\{q,r\}}, \{p\}\ \mathsf{rep}, \{s,u\}, \{t\}\ \mathsf{rep}\right]$$

Let c_3 be an object of type C which is not involved in fact type g. If c_3 is involved in an instance of fact type h, a problem arises: this instance of h cannot be represented in a tree structure according to figure 3.14. This problem does not occur if the anchor of non-root and non-leave nodes is required to be total (see section 3.3.2).

3.5 Complex identification

Very often the identification of an entity type can be done by a single label type. However, it may be necessary to use a combination of label types as identifier. In this section we discuss the effect of such complex naming references on the resulting relation types. We take the Address Identification Schema from example 2.5.2 as input for the generation process. The defoliated information structure for that schema is recalled in figure 3.16.

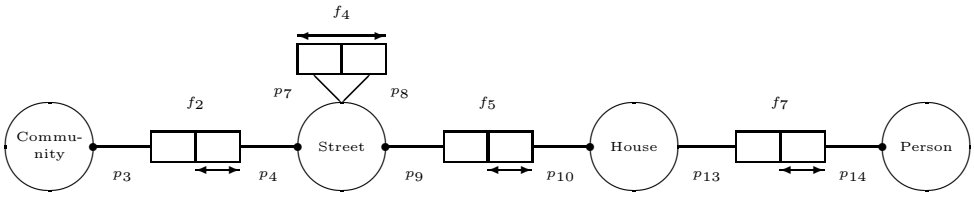


Figure 3.16: Defoliated structure for Address Identification Schema

3.5.1 Maximal grouping

Maximal grouping is obtained by the following setting of parameters:

SizePreference = True
$\gamma = |\mathcal{F}|$

Consider the tree representation in figure 3.17. This representation implements the Address Identification Schema under maximal grouping around *House*, or H for short. It is easily checked that this tree represents the defoliated information structure from figure 3.16 correctly and that no guiding conditions are violated.

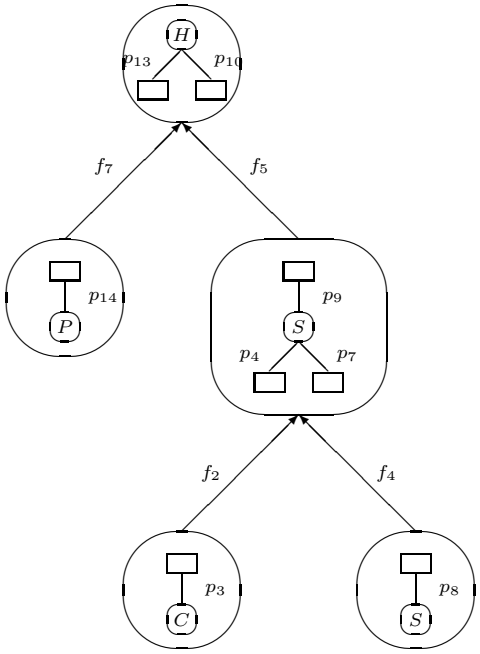


Figure 3.17: Address Identification Schema under maximal grouping around *House*

The corresponding relation type will contain two optional relation-valued attribute types $\{p_{14}\}$ and $\{p_8\}$, since predicators p_{13} and p_7 are neither total nor unique (see example 2.5.2). Furthermore, redundancy is allowed for fact types f_2 and f_4. This redundancy is caused as follows: the anchor of node $\{p_4, p_7, p_9\}$ is predicator p_9, which is not unique.

Usually different scenarios can result in the same tree representation. The tree in figure 3.17 may be the result of a generation starting with predicator p_{13}:

> **Breadth-first scenario:** Start with node $\{p_{13}\}$ and process fact type f_7. Then join $\{p_{10}\}$ and $\{p_{13}\}$ and add an edge labelled by fact type f_5. Proceed with $\{p_4\}$ and $\{p_7\}$.

Another scenario that generates this tree starts with predicator p_{10}:

> **Depth-first scenario:** Start with node $\{p_{10}\}$ and process fact type f_5. Then join $\{p_7\}$ and $\{p_9\}$ and add an edge labelled by fact type f_4. Proceed with $\{p_4\}$ and $\{p_{13}\}$.

Both scenarios mentioned above contain a permutation of the predicators p_{13}, p_{10}, p_4 and p_7. Not every possible sequence of these predicators will result in the same tree. Firstly, it depends on the choice for nodes to be extended. Furthermore, only if both p_4 and p_7 are chosen after p_{10}, the result will be the same.

The tree structure shown in figure 3.17 corresponds to the following hierarchical relation type:

$$\left[\, \overline{\{p_{13}, p_{10}\}}\,, \{p_{14}\} \;\; \mathsf{op} \;\; \mathsf{rep}, \{p_9, p_4, p_7\}, \{p_3\}, \{p_8\} \;\; \mathsf{op} \;\; \mathsf{rep} \right]$$

Each set of predicators can be replaced by the naming reference (identification) of its base. These naming references were discussed in example 2.5.2. For example, $\{p_4, p_7, p_9\}$ has base *Street*, or S for short. Entity type S has naming reference $\langle$S-name, C-name$\rangle$, or $\langle S, C\rangle$ for short. Furthermore, $\{p_{13}, p_{10}\}$ has base *House*, or H for short. Entity type H has naming reference $\langle$H-nr, S-name, C-name$\rangle$, or $\langle H, S, C\rangle$ for short. The resulting relation type then is:

$$\left[\, \overline{\langle H, S, C\rangle}\,, \langle P\rangle \;\; \mathsf{op} \;\; \mathsf{rep}, \langle S, C\rangle\,, \langle C\rangle\,, \langle S, C\rangle \;\; \mathsf{op} \;\; \mathsf{rep} \right]$$

This relation type can be simplified to a large extend. It is dominated by the combination $\langle H, S, C\rangle$. Naming reference $\langle S, C\rangle$ corresponding to node $\{p_9, p_4, p_7\}$ contains the same S and C values as naming reference $\langle H, S, C\rangle$. This $\langle S, C\rangle$ therefore can be omitted. For the same reason $\langle C\rangle$ is obsolete. Note that $\langle S, C\rangle$ corresponding to node $\{p_8\}$ cannot be treated in the same way, as it is the result of fact type f_4 which is *not* part of the identification of another base.

Finally, leaving out the sharp brackets, we get:

$$\left[\, \overline{H, S, C}\,, P \;\; \mathsf{op} \;\; \mathsf{rep}, [S, C] \;\; \mathsf{op} \;\; \mathsf{rep} \right]$$

3.5.2 Size/depth preference without optionals

Consider the following setting of parameters:

NoOptionals = True
SizePreference = True
$\gamma = 2$
DepthPreference = True
$\delta = 2$

The result of the generation will be a forest where each tree contains at most 2 fact types and has maximal depth 2, such that optional attribute types do not occur. The tree representation in figure 3.18 satisfies these conditions.

It can be reached via the following scenario. Start with node $\{p_4\}$ and process fact type f_2. Then join $\{p_4\}$ and $\{p_9\}$ and add an edge labelled by fact type f_5. Proceed with $\{p_{14}\}$. It becomes a new root because the tree does not contain a node with base P yet. Add an edge from $\{p_{13}\}$ to $\{p_{14}\}$ labelled by f_7. Finally, create a new root $\{p_8\}$. For two reasons it is not possible to add p_8 to $\{p_4, p_9\}$. Firstly, parameter NoOptionals only allows the extension of node $\{p_4, p_9\}$ with predicators having a total role constraint. Secondly, the tree having $\{p_4, p_9\}$ as root already contains the maximal number of fact types.

Obviously there is no difference between depth-first scenarios and breadth-first scenarios for the tree representation in figure 3.18. This representation corresponds to the following relation types:

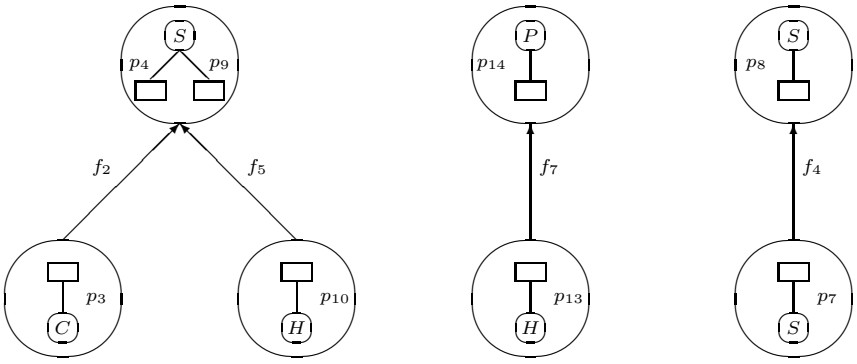


Figure 3.18: Address Identification Schema under specific design parameters

$\left[\overline{\{p_4, p_9\}}, \{p_3\}, \{p_{10}\} \;\; \mathsf{rep}\right]$

$\left[\overline{\{p_{14}\}}, \{p_{13}\}\right]$

$\left[\overline{\{p_8\}}, \{p_7\} \;\; \mathsf{rep}\right]$

Replacing the sets of predicators by the naming references of their bases, we get:

$\left[\overline{\langle S, C\rangle}, \langle C\rangle, \langle H, S, C\rangle \;\; \mathsf{rep}\right]$

$\left[\overline{\langle P\rangle}, \langle H, S, C\rangle\right]$

$\left[\overline{\langle S, C\rangle}, \langle S, C\rangle \;\; \mathsf{rep}\right]$

In the first relation type the values of $\langle C\rangle$ also occur in $\langle S, C\rangle$, since f_2 is part of the identification of entity type S. Therefore, this $\langle C\rangle$ can be omitted.

3.6 Summary

In this chapter, basic transformations for conceptual data models have been considered. First a mechanism for specifying internal representations was introduced and basic properties were examined. It was shown that the number of candidate internal representations is exponential in the number of fact types from the conceptual model. Then the generation process was described. A distinction was made between a basic generation strategy and possibilities for restricting the generation process in order to yield candidates having specific predefined characteristics. The generation process was illustrated using several examples.

Chapter 4

Transformation of populations and operations

4.1 Introduction

In this chapter a general framework for database generation is described, including the transformation of populations and operations (see also [21]). Transformation of populations may be used to guarantee so-called losslessness. Also it can be applied during prototyping, which can be further enhanced by the transformation of operations. This can be explained as follows.

When a database designer examines a candidate internal representation for a given conceptual model, a prototype can be generated with several sample populations. This will provide a better understanding of the internal representation under consideration. Also it may be desirable to examine the (dynamic) behaviour of that design, by simulating update operations. For related work see e.g. [155].

The organization of the chapter is as follows. Section 4.2 presents an overview of different transformations. In section 4.3 populations are considered, while section 4.4 focusses on operations. These sections first introduce populations and operations for both the conceptual and the internal level, without considering the relationship between these levels. Then, conceptual populations and operations will be mapped to internal populations and operations. We discuss several basic population rules, such as the Partitioning Rule and the Fitting Rule, and consider their violation for each operation we introduce. Usually, several other rules can be specified by an information analyst, for example constraints involving uniqueness and mandatory roles. These will be considered in section 4.5. For other constraints (e.g. set constraints) we refer to [47] and [135]. Section 4.5 will also consider additional transformations dealing with traditional network models, design alternatives for objectifications, and advanced modelling constructs (e.g. power types and generalization hierarchies).

4.2 Framework for database generation

Let Σ_1 be a structure description of a database. Suppose we perform a transformation resulting in a new structure description, say Σ_2. Although different terms may be used for these descriptions, e.g. information structure, database schema or database structure, they have the common property that they focus on types, rather than instances of those types (i.e. populations conforming to the structure description). As a result, we must also consider the consequences of the transformation mentioned above for these instances. More concretely, if $\mathsf{Pop}(\Sigma_1)$ is a population of structure Σ_1 and this structure Σ_1 is transformed into Σ_2, how is a population $\mathsf{Pop}(\Sigma_2)$ of structure Σ_2 obtained, such that all data in $\mathsf{Pop}(\Sigma_1)$ is represented? For database operations a similar problem occurs.

Usually, databases are considered on different levels of abstraction. We use the well-known distinction between the conceptual level and the internal level ([66]). The conceptual level focusses on correct specification of semantics, while the internal level focusses on efficient realization. On both levels we consider structures, populations and operations. We will describe a transformation from the conceptual level to the internal level. The overall transformation process is shown in the activity graph in figure 4.1. In this graph a box represents an activity, while a circle represents the input or output of an activity.

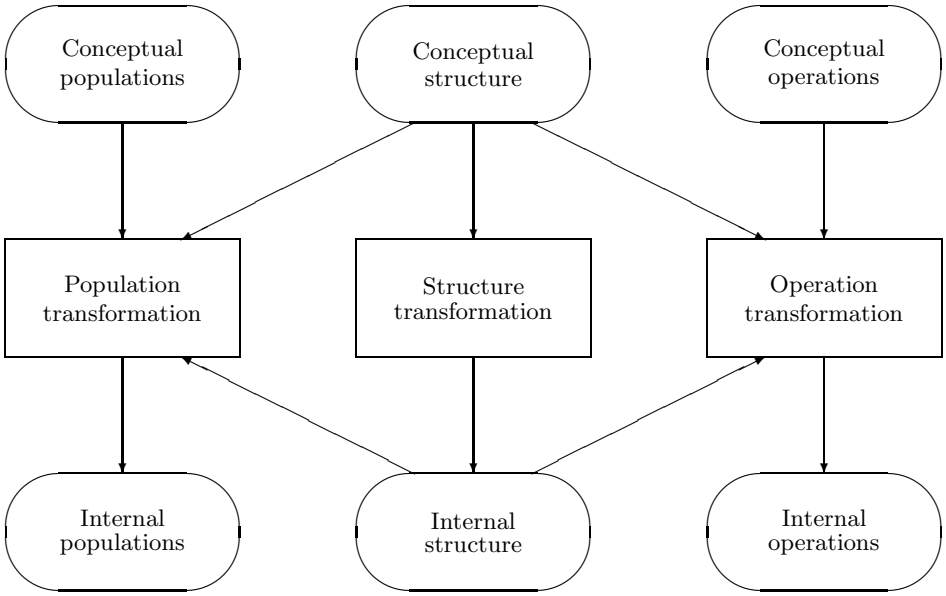


Figure 4.1: Transformation of structure, populations and operations

In figure 4.1, we see three transformation activities. The middle transformation of the three involves the database structure. A conceptual structure is transformed into an internal structure. This transformation has been described in chapter 3.

On the left hand side we see the population transformation. Populations of the conceptual structure are transformed into populations of the internal structure at hand. As a consequence, conceptual populations as well as the conceptual structure and the internal structure are used for this transformation.

On the right hand side we see the operation transformation. Conceptual operations are transformed into internal operations. Again, conceptual and internal structures are needed for this transformation.

Next we discuss the use of population rules. A population is not an isolated notion, since it is associated with a structure. Therefore a population must satisfy certain rules. The aim of these rules is to protect the evolution of populations during system usage against unwanted situations. This evolution is caused by (update) operations. As a consequence, for each operation it should be checked under what circumstances these rules are violated. If violation occurs, the operation must be rejected or some auxiliary actions have to be undertaken. We discuss several basic population rules and consider their violation for each operation we introduce.

4.3 Transformation of database populations

In this section we discuss populations (instantiations) of information structures, tree representations, and their relationship. We first discuss the population of information structures (4.3.1). Then we introduce populations for tree representations (4.3.2). Next, we show how a conceptual population (of an information structure) is translated into an internal population (of a tree representation) (4.3.3).

4.3.1 Information structure population

A population of an information structure $\mathcal{I}$ assigns to each object type in $\mathcal{O}$ a set of values of some universal domain, conforming to the structure as prescribed by $\mathcal{P}$ and $\mathcal{F}$ (see section 2.3). The population of an atomic object type is a set of values. The population of a composed object type (fact type) is a set of tuples. A tuple t of a fact type f is a mapping of all its predicators to values of the appropriate type. This is referred to as the *Conformity Rule*.

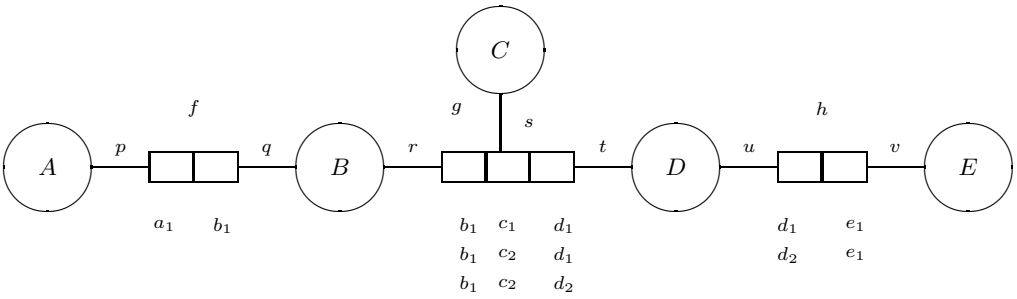


Figure 4.2: Information structure with example population

Example 4.3.1

Consider the information structure from figure 4.2. The population of fact type f contains one tuple t, defined as follows:

$$\begin{aligned} t(p) &= a_1 \\ t(q) &= b_1 \end{aligned}$$

As a consequence, an alternative denotation for tuple t is $\{(p, a_1), (q, b_1)\}$. *When no confusion is likely to occur, t may simply be written as* $\langle a_1, b_1 \rangle$ *or* $\langle b_1, a_1 \rangle$. □

4.3.2 Tree population

In this section we introduce how tree representations are populated. Our way of populating tree representations is analogous with [41], [136] and [138], where the population of a nested relation consists of *nested tuples*. We describe such tuples in terms of nodes in the forest.

A population Pop_T of a tree representation T assigns to each node $m \in \mathcal{N}$ a set of values of the population of $\mathsf{Base}(m)$. When no confusion is likely to occur we write Pop rather than Pop_T. Let $m \in \mathcal{N}$ be a node. Then, a population $\mathsf{Pop}(m)$ is a set of tuples, where each tuple assigns a unique value to m:

$$\forall_{t_1, t_2 \in \mathsf{Pop}(m)} \left[t_1(m) = t_2(m) \Rightarrow t_1 = t_2 \right]$$

This is referred to as the *Partitioning Rule* for populations (cf. [136]). The value $t(m)$ is called the *root value* of tuple t.

Note that from now onwards Pop may be used with two different kinds of arguments. For a conceptual population (of an information structure) the argument is an object type, while for an internal population (of a tree representation) the argument is a node.

Furthermore, a tuple $t \in \mathsf{Pop}(m)$ assigns to each child n of node m a subset of $\mathsf{Pop}(n)$. This assignment takes the structure of fact types into account. Let f be a fact type with $\mathsf{Hook}(f) \in m$. The extension of

f, denoted as $\mathsf{Ext}(f)$, contains all children of node m dealing with fact type f. Let $\mathsf{Ext}(f) = \{n_1, ..., n_l\}$. Then, each $\langle t_1, ..., t_l \rangle \in t(\mathsf{Ext}(f))$ should fit in the population of the corresponding children:

$$\forall_{1 \leq i \leq l} \left[t_i \in \mathsf{Pop}(n_i) \right]$$

This is referred to as the *Fitting Rule*. In the sequel we will use $t(f)$ as a shorthand for $t(\mathsf{Ext}(f))$.

The *Partitioning Rule* and the *Fitting Rule* establish the basis for the mapping of populations from the conceptual level to the internal level.

4.3.3 Transformation of populations

In this section we consider the translation of a conceptual population (of an information structure) into an internal population (of a tree representation). Let $\mathcal{I}$ be an information structure and let T be a tree representation of $\mathcal{I}$. From a population of information structure $\mathcal{I}$ we will generate a population of tree representation T.

Let m be a node in T. Then, for each instance of the corresponding base a (unique) tuple will result in $\mathsf{Pop}(m)$:

$$\mathsf{Pop}(\mathsf{Base}(m)) = \left\{ t(m) \;\middle|\; t \in \mathsf{Pop}(m) \right\}$$

This is referred to as the *Base Representation Rule*. Next we consider the children of node m. Let f be a fact type with $\mathsf{Hook}(f) \in m$. Furthermore let $\mathsf{Ext}(f) = \{n_1, ..., n_l\}$ be the children of node m dealing with fact type f. We describe how the internal representation of $\mathsf{Pop}(f)$ is obtained. For each tuple $t \in \mathsf{Pop}(m)$ the assignment $t(n_1, ..., n_l)$ is defined by:

$\langle t_1, ..., t_l \rangle \in t(n_1, ..., n_l)$ iff:

$\langle t(m), t_1(n_1), ..., t_l(n_l) \rangle \in \mathsf{Pop}(f)$

This is referred to as the *Fact Representation Rule*. This concludes the transformation of populations from the conceptual to the internal level. Obviously, the resulting population of the tree representation satisfies the *Partitioning Rule* and the *Fitting Rule*.

Example 4.3.2

Let $\mathcal{I}$ be the information structure from figure 4.2 and let T be the tree representation from figure 4.3. We transform the population of $\mathcal{I}$ into a population of T. Following our definitions, this population will consist of one nested tuple with root value b_1. The result of the transformation process is shown in figure 4.4.

Besides root value b_1 the nested tuple in figure 4.4 contains a sub-table for the values related to b_1 via fact type f, and a sub-table for the values related to b_1 via fact type g. The sub-table corresponding to fact type f contains a single tuple having only a root value. The sub-table corresponding to fact type g consists of three tuples containing a sub-table for fact type h. □

Another (more tree-like) view of the nested tuple in figure 4.4 is presented in figure 4.5. It is obtained by lifting up root value b_1, incorporating the instances of the associated fact types (fields in the nested table). The complete formal definition of this population is given in example 4.3.3.

Example 4.3.3

Consider the population shown in figure 4.5. We formally define this population in terms of nodes of the tree representation in figure 4.3. The population of the leaves is straightforward:

$$\begin{array}{lcll} \mathsf{Pop}(n_5) & = & \{t_1\} & \text{with: } t_1(n_5) = e_1 \\ \mathsf{Pop}(n_2) & = & \{t_2\} & \text{with: } t_2(n_2) = a_1 \\ \mathsf{Pop}(n_3) & = & \{t_3, t_4\} & \text{with: } t_3(n_3) = c_1 \\ & & & \phantom{\text{with: }} t_4(n_3) = c_2 \end{array}$$

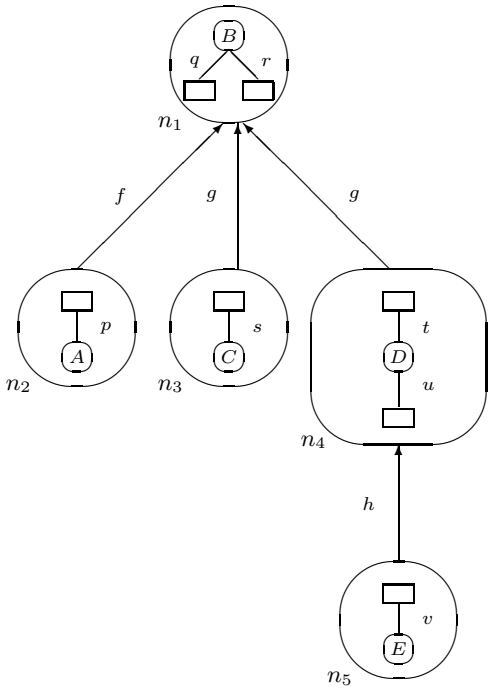


Figure 4.3: Maximal grouping around object type B

n_1	f	g		
	n_2	n_3	n_4	h
				n_5

b_1	a_1			
		c_1	d_1	e_1
		c_2	d_1	e_1
		c_2	d_2	e_1

Figure 4.4: Example population in nested table

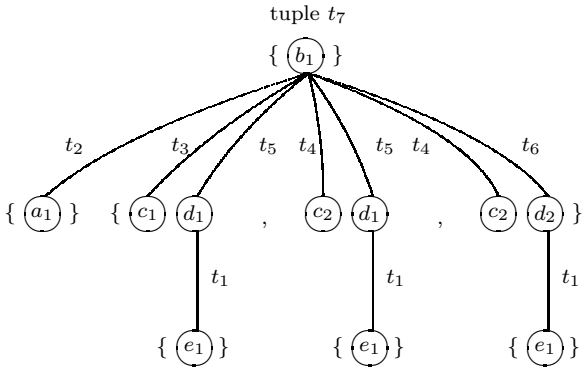


Figure 4.5: Tree-like view of example population

For node n_4 we have:

$$\mathsf{Pop}(n_4) = \{t_5, t_6\} \quad \text{with:} \quad \begin{array}{lcl} t_5(n_4) & = & d_1 \\ t_5(n_5) & = & \{t_1\} \\ t_6(n_4) & = & d_2 \\ t_6(n_5) & = & \{t_1\} \end{array}$$

The population of the root n_1 consists of tuple t_7, defined by:

$$\begin{array}{lcl} t_7(n_1) & = & b_1 \\ t_7(n_2) & = & \{t_2\} \\ t_7(n_3, n_4) & = & \{\langle t_3, t_5\rangle, \langle t_4, t_5\rangle, \langle t_4, t_6\rangle\} \end{array}$$

□

4.4 Transformation of database operations

In this section we will introduce operations for conceptual information structures and internal tree representations. Without losing the generality of our framework for database generation, we will focus on update operations. We introduce operations for insertion, deletion and modification of populations. For each operation a general definition is given, which is applicable to both the conceptual and the internal level. In order to shortly address retrieval operations, a global strategy for transforming queries from the conceptual to the internal level is discussed in section 4.4.7.

Our emphasis on update operations is motivated as follows. As was mentioned in section 4.1, the evolution of populations during system usage is caused by update operations. This has several consequences. For example, checking whether population rules are violated should be done when update operations are performed. Furthermore, the set of all possible populations (also called the state space) for a given information structure is induced from the empty population by applying update operations. This is also important for the use of structural induction over the state space.

4.4.1 Insertion

Let Γ be a set of tuples (a population) and let $t \notin \Gamma$ be a tuple. Then, the general definition of the insert operation is as follows:

$$\mathsf{Insert}(\Gamma, t) = \Gamma \cup \{t\}$$

First, we consider the insert operation on the conceptual level. Let f be a fact type of our information structure and let $\Gamma = \mathsf{Pop}(f)$. Furthermore, let t be a tuple with domain $\mathsf{Dom}(t)$. For the insertion of t into Γ we require t to be well defined: $\mathsf{Dom}(t) = f$.

Example 4.4.1

Recall the information structure and population from figure 4.2. The population of fact type $g = \{r, s, t\}$ *contains three tuples:* $\mathsf{Pop}(g) = \{g_1, g_2, g_3\}$. *Suppose we want to insert* $\langle b_2, c_1, d_2\rangle$ *into* $\mathsf{Pop}(g)$. *Then we have to perform* $\mathsf{Insert}(\mathsf{Pop}(g), g_4)$, *where tuple* g_4 *is defined as follows:*

$$\begin{aligned} g_4(r) &= b_2 \\ g_4(s) &= c_1 \\ g_4(t) &= d_2 \end{aligned}$$

The result of this operation is $\mathsf{Pop}(g) = \{g_1, g_2, g_3, g_4\}$. □

Example 4.4.2

We give a more practical example in terms of the Project Case shown in figure 4.6. The insertion from the previous example corresponds to the recording of a new involvement, such as: Insert the fact that person 'Smith' is involved in project 'database design' for the duration of '2 days per week'. □

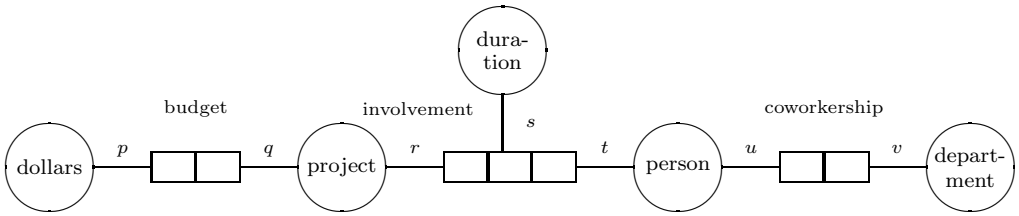


Figure 4.6: Part of the Project Case information structure

Note that the insertion of a fact instance may violate the *Conformity Rule*. This violation will not occur if tuple t to be inserted uses values of the appropriate type:

$$\forall_{p \in \mathsf{Dom}(t)} \left[t(p) \in \mathsf{Pop}(\mathsf{Base}(p)) \right]$$

Next, we discuss the insert operation on the internal level. It is important to note that we do not yet consider any relationship with the insert operation on the conceptual level. This relationship will be discussed in a later section.

Let n be a node in a given tree representation and let $\Gamma = \mathsf{Pop}(n)$. For the insertion of tuple t into Γ we require this tuple to be well defined:

$$\mathsf{Dom}(t) = \{n\} \cup \left\{ \mathsf{Ext}(f) \;\middle|\; \mathsf{Hook}(f) \in n \right\}$$

where $\mathsf{Ext}(f)$ is the set of nodes, containing all children of node n, dealing with fact type f.

Example 4.4.3

Recall the tree representation in figure 4.3 and its population in figure 4.4. Suppose we want to insert a tuple, say t, *defined as follows:*

$$\begin{aligned} t(n_1) &= b_2 \\ t(n_2) &= \{t'\} \text{ where } t'(n_2) = a_1 \\ t(n_3, n_4) &= \varnothing \end{aligned}$$

The insertion is performed by $\mathsf{Insert}(\mathsf{Pop}(n_1), t)$, *resulting in the population shown in figure 4.7.* □

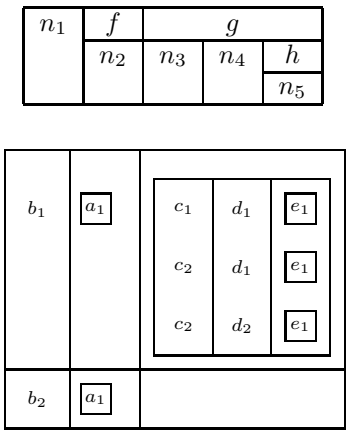


Figure 4.7: One row has been inserted

Example 4.4.4

A more practical example is given in terms of the nested table for the Project Case in figure 4.8. The insertion from the previous example corresponds to the recording of a new budget, such as: Insert the fact that project 'database design' has a budget of 7000 dollars. □

project	budget	involvement		
	dollars	duration	person	coworkership
				department

Figure 4.8: Nested table for the Project Case

Note that the insertion of a tuple t into the population of node n may violate the *Partitioning Rule*. This will not occur if the following condition is satisfied:

$$\forall_{t' \in \mathsf{Pop}(n)} [t'(n) \neq t(n)]$$

The *Fitting Rule* may also be violated by the insertion of tuple t into population $\mathsf{Pop}(n)$. For $\mathsf{Ext}(f) = \{n_1, ..., n_k\} \in \mathsf{Dom}(t)$ this violation will not occur if tuple t uses correct instances of the corresponding children nodes:

$$t(n_1, ..., n_k) \subseteq \prod_{i=1}^{k} \mathsf{Pop}(n_i)$$

4.4.2 Deletion

Let Γ be a set of tuples (a population) and let t be a tuple in Γ. Then, the general definition of the delete operation is as follows:

$$\mathsf{Delete}(\Gamma, t) = \Gamma - \{t\}$$

On the conceptual level this operation is simple. Violation of the *Conformity Rule* will not happen.

Example 4.4.5

Suppose we want to delete the inserted tuple from example 4.4.1. Then we have to perform $\mathsf{Delete}(\mathsf{Pop}(g), g_4)$. □

Next, we consider the Delete operation on the internal level, without considering the transformation process from the conceptual to the internal level. This mapping will be discussed in a later section.

Example 4.4.6
Suppose we want to delete the inserted tuple from example 4.4.3. This deletion is performed by $\mathsf{Delete}(\mathsf{Pop}(n_1), t)$. □

Let n be a node with $\Gamma = \mathsf{Pop}(n)$ and let t be a tuple to be deleted. Obviously, the operation $\mathsf{Delete}(\Gamma, t)$ will not violate the *Partitioning Rule*. However, the *Fitting Rule* may be violated if node n has a father, say $\mathsf{Father}(n)$. This violation does not occur if the father node does not occupy the tuple to be deleted:

$$\forall_{t' \in \mathsf{Pop}(\mathsf{Father}(n))} [t \notin t'(n)]$$

4.4.3 Modification

In this section, we consider the modification of populations. We first introduce the notion of a *modifier*. A modifier (also called modification function) for a tuple $t \in \Gamma$ is a function m with domain $\mathsf{Dom}(\mathsf{m}) \subseteq \mathsf{Dom}(t)$. For modifier m for tuple t the *complementary domain* is defined by:

$$\mathsf{ComDom}(\mathsf{m}) = \mathsf{Dom}(t) - \mathsf{Dom}(\mathsf{m})$$

Modifiers are used for the definition of *modified* tuples. Such a tuple is defined by: $M(t, \mathsf{m}) = \mathsf{m} \cup t[\mathsf{ComDom}(\mathsf{m})]$. If m is a modifier for t, then the modify operation is defined as follows:

$$\mathsf{Modify}(\Gamma, t, \mathsf{m}) = \Gamma - \{t\} \cup \{M(t, \mathsf{m})\}$$

First, we consider the modify operation on the conceptual level.

Example 4.4.7
Recall the information structure and population from figure 4.2. Suppose we want to modify tuple $g_1 = \langle b_1, c_1, d_1 \rangle$ *into* $\langle b_1, c_3, d_1 \rangle$. *Then we need a modification function* m *defined by* $\mathsf{Dom}(\mathsf{m}) = \{s\}$ *and* $\mathsf{m}(r) = c_3$. *The actual modification is performed by* $\mathsf{Modify}(\mathsf{Pop}(g), g_1, \mathsf{m})$. □

Example 4.4.8
A more practical example is given in terms of the Project Case. The modification from the previous example may correspond to the following change: Person 'Blake' being involved in project 'system maintenance' for '2 days per week' will from now on work '4 days per week'. □

The modify operation may violate the *Conformity Rule*. Let m be a modification function of tuple t. Then, violation is avoided if m uses values of the appropriate type:

$$\forall_{p \in \mathsf{Dom}(\mathsf{m})} [\mathsf{m}(p) \in \mathsf{Pop}(\mathsf{Base}(p))]$$

Next, we consider the modify operation on the internal level. Let $t \in \mathsf{Pop}(n)$ be a tuple in the population of node n and let m be a modifier (modification function) of t. The modification of t according to m may violate the *Partitioning Rule* if $n \in \mathsf{Dom}(\mathsf{m})$. This will not happen if m has the following property:

$$\forall_{t' \in \mathsf{Pop}(n)} [t'(n) \neq \mathsf{m}(n)]$$

The *Fitting Rule* may also be violated. For $\mathsf{Ext}(f) = \{n_1, ..., n_k\} \in \mathsf{Dom}(\mathsf{m})$ this rule will not be violated if m uses correct instances of the corresponding children nodes:

$$\mathsf{m}(n_1, ..., n_k) \subseteq \prod_{i=1}^{k} \mathsf{Pop}(n_i)$$

In the above definition of the modify operation, we did not make any choice with respect to the implementation of this operation. A straightforward implementation would be as follows:

$$\mathsf{Modify}(\Gamma, t, \mathsf{m}) = \mathsf{Insert}(\mathsf{Delete}(\Gamma, t), M(t, \mathsf{m}))$$

Note that this option uses no intermediate results. As a consequence, this implementation strategy will in general be inefficient. Another option is:

$$\mathsf{Modify}(\Gamma, t, \mathsf{m}) = \mathsf{Delete}(\mathsf{Insert}(\Gamma, M(t, \mathsf{m})), t)$$

This option is quite dangerous. For example, the situation $t = M(t, \mathsf{m})$ will lead to the unwanted deletion of tuple t.

4.4.4 Transformation of insertion

In this section we describe the transformation process from conceptual operations into internal operations. For each operation on the conceptual level, we give the necessary activities in terms of internal operations. The operations we use were introduced in the previous sections.

We first consider the transformation of insertions from the conceptual to the internal level. Let f be a fact type. Suppose we want to insert a tuple t into the population of fact type f. This insertion is performed by $\mathsf{Insert}(\mathsf{Pop}(f), t)$. Let T be a tree representation in which f is represented internally. We describe how the population of T must be changed. This will happen in two steps:

proc TransformInsertion (f : Fact Type; t : Tuple);
    InsertAtomicTuples (f, t);
    Connect (f, t)
end

First the values in t that are new must be inserted. Then they have to be interconnected, such that the current population is correctly modified. Both steps will be discussed in more detail.

1. In the first step, atomic tuples are inserted into the tree population using the internal Insert operation. An atomic tuple is a tuple having only a root value. Such a tuple should only be inserted if there is no other tuple with this root value:

   for each $p \in f$
   do
       let $n = \mathsf{Node}(p)$
       if $\forall_{x \in \mathsf{Pop}(n)} [t(p) \neq x(n)]$
       then $\mathsf{Insert}(\mathsf{Pop}(n), t')$ where:
                $t'(n) = t(p)$
                $t'(g) = \varnothing$ (for all fact types g with $\mathsf{Hook}(g) \in n$)
       end
   end

   Example 4.4.9
   Recall the information structure from figure 4.2 and the tree representation from figure 4.3. The insertion of $\langle b_2, c_1, d_2 \rangle$ into $\mathsf{Pop}(g)$ was discussed in example 4.4.1. Now we show the necessary actions to be performed on the corresponding tree population.
   In this first step, we see that there is no tuple in the population of node n_1 with root value b_2 (section 4.3.3). As a consequence, we have to perform $\mathsf{Insert}(\mathsf{Pop}(n_1), t')$, where atomic tuple t'

is defined as follows:

$$\begin{aligned} t'(n_1) &= b_2 \\ t'(n_2) &= \varnothing \\ t'(n_3, n_4) &= \varnothing \end{aligned}$$

□

2. Next, we consider the second step of the mapping process of $\mathsf{Insert}(\mathsf{Pop}(f), t)$ from the conceptual to the internal level. After the first step, all values in t will occur in the corresponding tree population. However, they have to be interconnected. In the second step this interconnection is established using the internal Modify operation.

   Let $n = \mathsf{Node}(\mathsf{Hook}(f))$ be the upper node involving fact type f and let t' be the tuple in $\mathsf{Pop}(n)$ with the correct root value: $t'(n) = t(\mathsf{Hook}(f))$. Then, we have to perform the operation $\mathsf{Modify}(\mathsf{Pop}(n), t', \mathsf{m})$, where m is a modification function with domain consisting of all children of node n dealing with fact type f (i.e. $\mathsf{Dom}(\mathsf{m}) = \{\mathsf{Ext}(f)\}$). If $\mathsf{Ext}(f) = \{n_1, ..., n_l\}$, modification function m is defined as follows:

$$\mathsf{m}(f) = t'(f) \cup \varepsilon(f, t)$$

   where $\varepsilon(f, t)$ denotes the existing tuples in $\mathsf{Ext}(f)$ with a root value occurring in tuple t:

$$\varepsilon(f, t) = \left\{ \langle t_1, ..., t_l \rangle \;\middle|\; \forall_{1 \le i \le l} \left[t_i \in \mathsf{Pop}(n_i) \wedge t_i(n_i) = t(\mathsf{Anchor}(n_i)) \right] \right\}$$

Example 4.4.10

In the second step, the new (atomic) tuple t' from example 4.4.9 is modified by $\mathsf{Modify}(\mathsf{Pop}(n_1), t', \mathsf{m})$*, with the following modification function* m*:*

$$\begin{aligned} \mathsf{Dom}(\mathsf{m}) &= \{(n_3, n_4)\} \\ \mathsf{m}(n_3, n_4) &= \{\langle t_3, t_6 \rangle\} \end{aligned}$$

where tuples t_3 and t_6 are defined as in example 4.3.3. The result is shown in figure 4.9. This example shows that an update of a fact type on the conceptual level may cause updates of several fact types on the internal level. For instance, the population of column h in the nested table has been changed, while on the conceptual level it was not. □

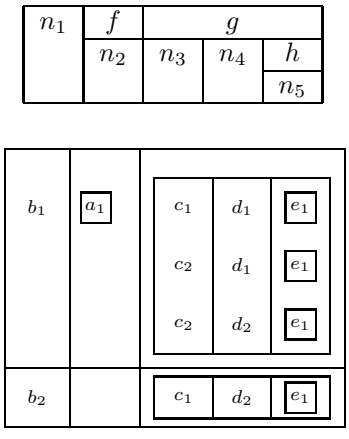


Figure 4.9: Result of transformation

As a result of their formal nature, the examples in this section may seem rather artificial. Therefore we give a more practical example in terms of the Project Case.

Example 4.4.11

This example is similar to examples 4.4.9 and 4.4.10. Suppose we record the fact that person 'Smith' is involved in project 'database design' for the duration of '2 days per week'. Then what happens on the internal level?

In the first step, the algorithm detects the absence of project 'database design'. As a consequence an atomic tuple with root value 'database design' will be inserted in the project column. In the second step, the values 'database design', 'Smith' and '2 days per week' are connected with each other.

Note that the second step properly deals with the nested nature of the internal representation, since the resulting tuple with root value 'database design' will also contain the department of person 'Smith'. □

Next we will shortly discuss the computational complexity of the insert operation. Suppose we insert a tuple t into the population of fact type f. We express the complexity of this operation in terms of the number of internal updates needed.

Let C_i and C_m be the average cost of an internal insertion and modification respectively. In the first step, the number of insertions will not exceed the number of predicators in f, while in the second step there is exactly one modification. As a consequence, the complexity of the insert operation is given by:

$$C(\mathsf{Insert}(\mathsf{Pop}(f), t)) \leq |f| * C_i + C_m$$

Note that this is only a global estimation. For instance, in both steps of the transformation process some search must be done, in order to determine the precise actions to be performed. We will not elaborate on this further.

4.4.5 Transformation of deletion

In this section we consider the transformation of deletions from the conceptual to the internal level. Let f be a fact type. Suppose we want to delete a tuple t from the population of fact type f. This deletion is performed by $\mathsf{Delete}(\mathsf{Pop}(f), t)$. Let T be a tree representation in which f is represented internally. We describe how the corresponding tree population must be changed. This will (again) happen in two steps:

proc TransformDeletion (f : Fact Type; t : Tuple);
  Disconnect (f, t);
  DeleteAtomicTuples (f, t)
end

First the values in tuple t must be disconnected such that the current population is correctly modified. Then the unnecessary atomic tuples can be deleted. Both steps will be discussed in more detail.

1. In the first step, values in the tree population are disconnected, according to the tuple to be deleted from $\mathsf{Pop}(f)$. This disconnection will be established using the internal Modify operation. Obviously the disconnection is described by the inverse of the second step in the previous section.

   Let $n = \mathsf{Node}(\mathsf{Hook}(f))$ be the upper node involving fact type f and let t' be the tuple in $\mathsf{Pop}(n)$ with the correct root value: $t'(n) = t(\mathsf{Hook}(f))$. Then, we have to perform the operation $\mathsf{Modify}(\mathsf{Pop}(n), t', \mathsf{m})$, where m is a modification function with domain $\mathsf{Dom}(\mathsf{m}) = \{\mathsf{Ext}(f)\}$. If $\mathsf{Ext}(f) = \{n_1, ..., n_l\}$, the modification function m is defined as follows:

   $$\mathsf{m}(f) = t'(f) - \varepsilon(f, t)$$

2. In the second step, the unnecessary tuples will be deleted from the tree population, using the internal Delete operation. As a consequence, all atomic tuples for which there is no reference from the father node will be deleted:

for each $p \in f$
do
  let $n = \mathsf{Node}(p)$
  let $t' \in \mathsf{Pop}(n)$ such that $t'(n) = t(p)$
  if $\neg\exists_{t^* \in \mathsf{Pop}(\mathsf{Father}(n))}\,[t' \in t^*(n)]$ **and** $\forall_{g \in \mathcal{F}}\,[n = \mathsf{Node}(\mathsf{Hook}(g)) \Rightarrow t'(g) = \varnothing]$
  then $\mathsf{Delete}(\mathsf{Pop}(n), t')$
  end
end

Note that this second step is the inverse of the first step in the previous section.

Next we will shortly discuss the computational complexity of the delete operation. This will be similar to the insert operation. Let C_d and C_m be the avarage cost of an internal deletion and modification respectively. Then the complexity of the delete operation has the following upperbound:

$$C(\mathsf{Delete}(\mathsf{Pop}(f), t)) \leq |f| * C_d + C_m$$

4.4.6 Transformation of modification

In this section we consider the transformation of modifications from the conceptual to the internal level. Let f be a fact type. Suppose we want to modify a tuple t in the population of fact type f, according to modification function m. This modification is performed by $\mathsf{Modify}(\mathsf{Pop}(f), t, \mathsf{m})$. Let T be a tree representation in which f is represented internally. The population of T must be changed as follows:

proc TransformModification (f : Fact Type; t, m : Tuple);
  TransformDeletion (f, t);
  TransformInsertion $(f, M(t, \mathsf{m}))$
end

First the old tuple t is deleted and then the modified tuple $M(t, \mathsf{m})$ is inserted. In section 4.4.3 it was mentioned that this approach will in general be inefficient, since it may cause a deletion of atomic tuples, followed by an insertion of the same atomic tuples. A more efficient approach is obtained by restricting tuple t to the domain of m before it is deleted, and by interleaving the execution of TransformDeletion and TransformInsertion.

This restriction and interleaving of deletion and insertion is done as follows. First tuple t is partly disconnected (corresponding to $\mathsf{Dom}(\mathsf{m})$). Then the necessary atomic tuples are inserted and connected. Finally, the unnecessary atomic tuples can be deleted.

4.4.7 Query transformation

In this section a general strategy for retrieval operations and their transformation is discussed. We consider the following elementary search action FactSearch for information structures:

> Let p and q be predicators in the same fact type f. Then, for a given value $x \in \mathsf{Pop}(\mathsf{Base}(p))$ occurring in $\mathsf{Pop}(f)$ via predicator p, find the values from $\mathsf{Pop}(\mathsf{Base}(q))$ being associated with x via p and q.

This elementary search action is an example of a low-level action, to be used when queries on information structures are performed. It can be applied during the evaluation of path expressions through information structures, e.g. occurring in conceptual query languages ([49], [81]).

Next we consider the transformation of the above search action for information structures into the search action NodeSearch for tree representations of information structures. Let m be the node containing predicator p, and let n be the node containing predicator q. Then the search action is from m to n. Following the wellformedness conditions for tree representations discussed in chapter 3, the following cases are distinguished:

1. If there is an edge $\langle n, m \rangle$, the search action is directed downwards. Then for the unique tuple $t \in \mathsf{Pop}(m)$ with root value x, those tuples in $\mathsf{Pop}(n)$ that are contained in t are yielded.
2. If there is an edge $\langle m, n \rangle$, the search action is directed upwards. Then for the unique tuple $t \in \mathsf{Pop}(m)$ with root value x, those tuples in $\mathsf{Pop}(n)$ containing t are yielded.
3. If there is a node z with (equally labelled) incoming edges $\langle m, z \rangle$ and $\langle n, z \rangle$, the search action occurs between siblings. Let t be the unique tuple in $\mathsf{Pop}(m)$ with root value x and let $\Gamma \subseteq \mathsf{Pop}(z)$ be the set of tuples containing t. Then those tuples in $\mathsf{Pop}(n)$ that are contained in Γ are yielded.

Obviously a path through an information structure will result in a path through the corresponding tree representation. The case distinction given above shows how the latter path can be derived.

4.5 Additional transformations

In this section we consider several additional transformations involved in database design. We first consider transformation of total role and uniqueness constraints (section 4.5.1). Then, in section 4.5.2 functional representations are considered, while section 4.5.3 focusses on design alternatives for objectifications. Finally, advanced modelling constructs are considered in section 4.5.4, for example powertypes and generalization hierarchies.

4.5.1 Constraint transformation

Constraints on the possible populations of an information structure are used for specifying integrity rules. Obviously, when considering internal representations of an information structure, these constraints must be translated (see also section 3.2.5).

We discuss two kinds of (static) constraints in more detail: uniqueness constraints and total role constraints. These constraints are of vital importance for the conceptual information structure, since they are used for identification purposes (section 2.5.1). Furthermore, these constraints are important for internal representations, because several basic properties of these representations can be derived from them (chapter 3).

First we consider total role constraints. A total role constraint for predicator p expresses that each instance of $\mathsf{Base}(p)$ must occur in at least one instance of $\mathsf{Fact}(p)$. For an internal representation containing predicator p this has the following consequences:

1. If in this representation predicator p is the anchor of a non-leaf node, then predicator p must have a total role constraint. In case this predicator is not total, the internal representation is *illegal*, because it may result in loss of information.
2. If in this representation predicator p is the hook of a fact type and p does not have a total role constraint, then the sub-table corresponding to $\mathsf{Fact}(p)$ is *optional* (see also section 3.2.5).

Example 4.5.1

Recall the tree representation from figure 4.3. Suppose predicator t is not total. Then there can be instances of fact type h that cannot be represented in this structure. Suppose predicator u is not total. Then the sub-table corresponding to fact type h is optional. □

Next we consider uniqueness constraints. The meaning of a uniqueness constraint is expressed in terms of functional dependencies (see section 2.4.3). As a consequence, presence or absence of a uniqueness constraint will not cause an internal representation to be illegal. However, a special situation occurs if the hook of a fact type is unique. Then the sub-table corresponding to that fact type is flat rather than nested, i.e. it is not a real sub-table but a normal non-repeating attribute type. This situation occurs in several examples in section 3.4 and 3.5.

4.5.2 Functional representations

In this section the relation between object-role models (including the corresponding tree structures) and functional representations (such as traditional network models) is considered. For a full treatment of network models, the reader is referred to [157] or [43].

Consider the information structure in the left part of figure 4.10. A candidate internal representation is given in the right part.

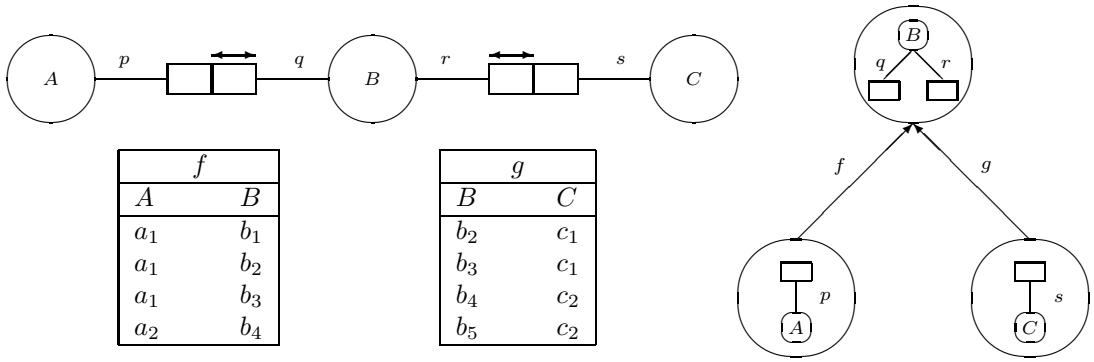


Figure 4.10: Simple information structure, population and candidate tree representation

The tree representation in figure 4.10 can be interpreted in terms of traditional networks as follows. An edge in the tree corresponds to a link (or set) from a parent (or owner) to a child (or member). A link occurrence (or instance) contains one parent occurrence and a list of child occurrences.

Example 4.5.2

Consider for example the link e defined by $\{p\} \xrightarrow{f} \{q, r\}$. *An occurrence of this link will consist of one A-value and a list of B-values. As is usual in traditional network models, A-values (parent occurrences) are used in exactly one occurrence of e and B-values (child occurrences) are used in at most one occurence of e. This leads to the situation given in figure 4.11.* □

Obviously, an object-role information structure may contain fact types with uniqueness constraints over more than one role. If the associated tree representations have to be interpreted as CODASYL structures according to the above principle, the information structure has to be prepared. This preparation results in a new information structure, where all fact types are functional. For a binary fact type this is illustrated in figure 4.12.

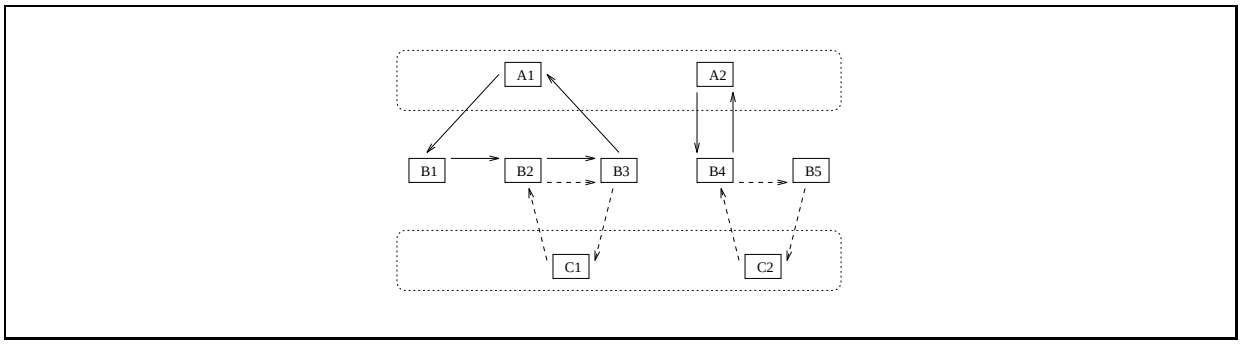


Figure 4.11: Data instances in traditional network structure

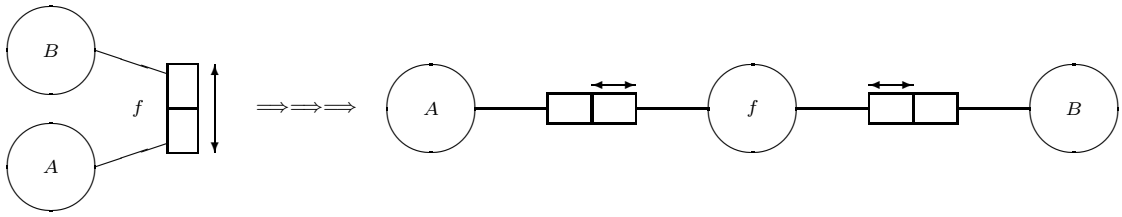


Figure 4.12: Obtaining functional fact types

4.5.3 Design alternatives for objectifications

This section introduces different kinds of design alternatives for objectified fact types. Let f be an objectified fact type:

$$\exists_{g \in \mathcal{F}, p \in g} [\mathsf{Base}(p) = f]$$

The situation is given in the left part of figure 4.14. We consider several possibilities for specifying internal representations of such objectified fact types. A distinction is made between *direct representations* (atomic-first and composed-first representations), and *indirect representations* (flattened, partitioning, and redundancy representations).

Atomic-first representations

In *atomic-first representations*, the objectified fact type f is treated by first representing fact type f, followed by the objectifications of f (lower in the tree). There are $|f|$ alternatives to obtain an atomic-first representation. In each alternative a predicator $x \in f$ is chosen as the root of the representation. In the left part of figure 4.13 an example is shown where the root node contains object type B. The associated tree structure was shown in figure 3.2.

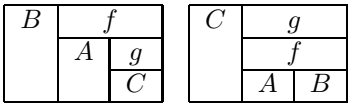


Figure 4.13: Examples of atomic-first representation and composed-first representation

Composed-first representations

In *composed-first representations*, the objectified fact type f is treated by first representing the fact type in which f plays a role, i.e. fact type g, followed by the components of f (lower in the tree). A maximum of

$|g|-1$ composed-first representations may be obtained. In each alternative a predicator $x \in g - \mathcal{H}$ is chosen as the root of the representation, where $\mathcal{H}$ is the set of objectifications. In the right part of figure 4.13 an example is given where the root node contains object type C. Note that composed-first representations do not satisfy wellformedness condition t_3 discussed in section 3.2.

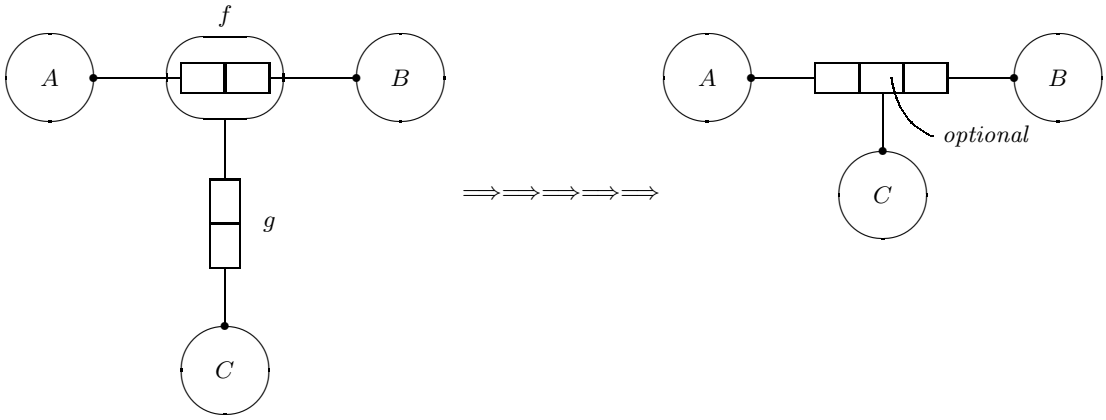


Figure 4.14: Unnesting results in additional constraint

Indirect representations

A representation is called indirect if the information structure is reorganized before applying the representation mechanism. In *flattened representations*, the objectified fact type f is treated by first flattening the objectification, resulting in a derived fact type h containing a maximum of $|f| \times |g|$ predicators. As a consequence, a maximum of $|h|$ flattened representations can be obtained. In each alternative a predicator $x \in h$ is chosen as the root of the representation.

Flattened representations may result in *optional* predicators when objectifications are not total (see figure 4.14). Note that optional predicators and non-total predicators are not the same. An optional predicator allows populations of fact types to contain partial tuples, whereas a non-total predicator does not.

In *partitioning representations*, the objectified fact type f is treated by first flattening and partitioning the objectification, resulting in a derived fact type h containing a maximum of $|f| \times |g|$ predicators, and a derived fact type h' containing f predicators. Partitioning can be used to avoid optional predicators. An example of this would be an additional binary fact type in the right part of figure 4.14 containing A and B values.

4.5.4 Advanced modelling constructs

Object-role modelling techniques have been extended with constructs for modelling complex object structures, such as power types and generalization hierarchies (see e.g. [82], [78], [80]). When these advanced constructs have to be implemented using our mechanism for specifying internal representations, several strategies can be applied. We consider two extremes.

On the one hand, the *direct representation approach* aims at a direct coupling between advanced modelling constructs and the internal representation mechanism. On the other hand, the *indirect representation approach* aims at removing all advanced modelling constructs such that 'standard' object-role models remain. The previous section considered both approaches for objectified fact types.

For power types, an example of the direct approach would be to relate a power type to a repeating attribute type (sub-table). The indirect approach would be to express a power type in terms of more basic modelling constructs. This can for example be done using *existensional uniqueness constraints* (see [83]). Consider

powertype B in the left part of figure 4.15, defined as a set of $A-$values. This can also be modelled as a binary fact type between B and A, where the role played by object type A is existensional unique (see the right part of figure 4.15). This constraint requires that no two instances of object type B can be associated via f to the same set of instances of object type A.

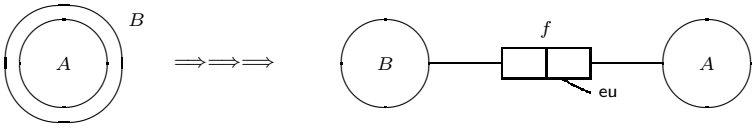


Figure 4.15: Removing power type B using eu-constraint

For generalization hierarchies, the indirect approach can be applied using *optional predicator constraints* (see section 4.5.3). Consider for example object type A in the left part of figure 4.16. This object type is a generalization of object types B and C. An alternative way to model this situation is given in the right part. Both predicators in the new fact type are optional. For more details we refer to [8].

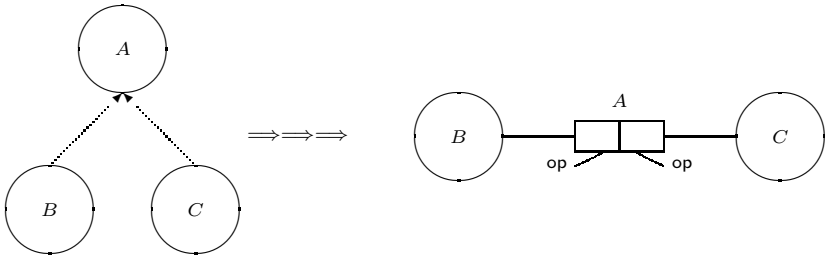


Figure 4.16: Modelling generalization using optional predicators

4.6 Summary

This chapter considered the transformation of database populations and operations from the conceptual to the internal level. Populations and operations were first described on both the conceptual and the internal level, without considering the relationship between these levels. Then, conceptual populations and operations were mapped to internal populations and operations. Basic population rules such as the Fitting Rule and the Partitioning Rule were considered, along with their violation for each operation we introduced.

The focus of chapter 5 is the mutation of internal representations into other internal representations. Different kinds of mutations will be considered. Then in chapter 6 the attention is focussed on the question how these mutations can be used for optimization purposes.

Chapter 5

Tree transformations

5.1 Introduction

For a given internal representation an alternative representation can be constructed by means of *mutation*. A sequence of mutations then describes a path through the internal solution space and can be seen as a database design process. As an example, operators for joining and splitting relations can be used for searching the set of candidate relational models (e.g. for normalization or optimization purposes). Similarly, operators for adding and dropping indices can be used for searching through the set of candidate index configurations.

To draw a parallel between our case and the area of query optimization, consider the case where so-called *join processing trees* are used ([86]). A join processing tree describes a specific join order, together with a join method for each join. In order to find the best join processing tree, a mutation system $\langle \mathcal{J}, \mu_1, \mu_2, \mu_3 \rangle$ is introduced for searching the solution space $\mathcal{J}$ of candidate join processing trees. The search process is based on three mutation operators $\mu_1 =$ join method choice, $\mu_2 =$ join commutativity, $\mu_3 =$ join associativity. In addition to these basic mutation operators, two extra operators *left join exchange* and *right join exchange* are described. These operators are expressed in terms of join commutativity and associativity.

In this chapter a mutation system $\langle \mathcal{S}, \mathsf{Prune}, \mathsf{Graft}, \mathsf{Promote} \rangle$ is introduced, where $\mathcal{S}$ is the solution space of candidate internal representations for a given conceptual data model (see also [25], [14] and [15]. Prune, Graft and Promote are basic mutation operators for elements of $\mathcal{S}$. The additional operators AddIndex and DropIndex are basic index manipulation operators. It will be clear that a mutation system has to be chosen carefully. Mutation operators have to be *correct* in the sense that the result of a mutation is a correct internal representation, and *complete* in the sense that all candidate representations can be produced.

The chapter is organized as follows. In order to get accustomed to the notion of mutation, we restrict ourselves to *relational* representations and their mutation (joining and splitting) in section 5.2. The idea behind interpreting a tree representation $T = \langle \mathcal{N}, E, \ell \rangle$ as a (flat) relational model is to ignore the edges (E) and the labels (ℓ). In section 5.3 the attention will be focussed on the complete internal representations (including edges and labels). An encoding mechanism for these representations is introduced, so that representations and their mutation can be described on a high level of abstraction. In section 5.4 the operators Prune, Graft and Promote are introduced, whereas section 5.5 considers additional aspects of mutation systems, such as closedness and completeness, distance and crossover. The chapter is concluded with a summary and outlook.

5.2 Relational representations and mutations

The representation mechanism introduced in chapter 3 allows us to describe (flat) relational models in several ways. As an example, in case all hook predicators are unique there will be no repeating groups (see also guidance condition NoNesting in figure 3.5). In this section a more general approach to interpreting a tree representation $T = \langle \mathcal{N}, E, \ell \rangle$ as a relational model is discussed. The idea is to ignore the edges (E) and their labels (ℓ). We give an inductive definition of the solution space $\mathcal{S}_r$ of relational representations. The nodes in a relational representation are called attribute types here.

5.2.1 Direct relational representations

Let $\mathcal{I}$ be an information structure. A relational representation for $\mathcal{I}$ is a set of relation types. A relation type is a set of attribute types, where each attribute type is defined as a set of predicators. Following this definition, the information structure of a conceptual model corresponds to a relational representation, called the *direct relational representation*.

The direct relational representation δ of $\mathcal{I}$ is the set of relation types, where each fact type $f \in \mathcal{F}$ has a corresponding relation type $r \in \delta$, defined by: $r = \{\{p\} | p \in f\}$.

Example 5.2.1

Figure 5.1 shows an information structure which will be used as a running example for this chapter. The direct relational representation for the running example is given in figure 5.2. □

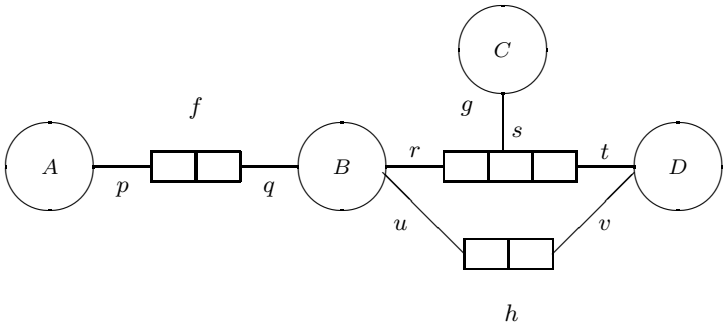


Figure 5.1: Running example for this chapter

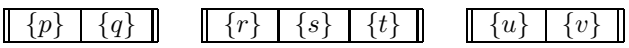


Figure 5.2: Direct relational representation for running example

Populations (instantiations) of information structures are treated analogously. Following the mapping-oriented approach for relations (see chapter 2 and 4), the population of the direct relational representation is straightforward.

Let $\mathcal{I}$ be an information structure with population Pop and let δ be the corresponding direct relational representation. The direct relational population Pop(δ) is a population of the relation types in δ conforming to the population Pop. More precisely, if $f \in \mathcal{F}$ is a fact type and $r \in \delta$ is the corresponding relation type, each fact instance $t \in \mathsf{Pop}(f)$ results in a similar relation instance $t' \in \mathsf{Pop}(r)$, defined by:

$$\forall_{p \in f} \left[t'(\{p\}) = t(p) \right]$$

Example 5.2.2
The example information structure population from figure 5.3 results in the direct relational population shown in figure 5.4. □

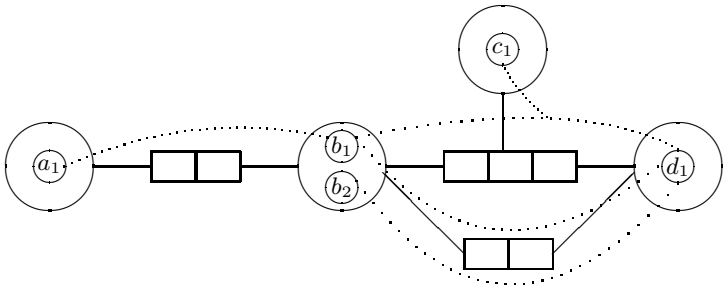


Figure 5.3: Example population for given information structure

$\{p\}$	$\{q\}$
a_1	b_1

$\{r\}$	$\{s\}$	$\{t\}$
b_1	c_1	d_1

$\{u\}$	$\{v\}$
b_1	d_1
b_2	d_1

Figure 5.4: Direct relational population for example information structure population

5.2.2 Derived relational representations

The direct relational population shown in figure 5.4 is similar to the original population shown in figure 5.3. In order to go beyond this original population, the relational join operator (e.g. [157]) is defined in terms of predicators. The resulting operator is called the *P-join*.

Let $\mathcal{I}$ be an information structure and let R be a relational representation for $\mathcal{I}$. Furthermore, let $r_1, r_2 \in R$ be relation types ($r_1 \neq r_2$) with type related attribute types $a_1 \in r_1, a_2 \in r_2$. Then the result of joining a_1 and a_2, denoted as $\mathsf{Join}(R, a_1, a_2)$, is a relational representation R', defined by:

$$R' = R - \{r_1, r_2\} \cup \{r\}$$

where $r = r_1 \cup r_2 - \{a_1, a_2\} \cup \{a_1 \cup a_2\}$.

As is usual, joining results in a reduction of the number of relation types. This is illustrated by the following example.

Example 5.2.3
The result of joining $\{q\}$ and $\{r\}$ is shown in figure 5.5 (top). The result of joining $\{r\}$ and $\{u\}$ is shown in the bottom of figure 5.5. □

$\{p\}$	$\{q, r\}$	$\{s\}$	$\{t\}$

$\{u\}$	$\{v\}$

$\{p\}$	$\{q\}$

$\{r, u\}$	$\{s\}$	$\{t\}$	$\{v\}$

Figure 5.5: Relational representations $\mathsf{Join}(\delta, \{q\}, \{r\})$ and $\mathsf{Join}(\delta, \{r\}, \{u\})$

The sets $\{q, r\}$ and $\{r, u\}$ in figure 5.5 specify the join structure of the relational representation under consideration (see also [100]). Next the population of the P-join is considered.

Let $\mathcal{I}$ be an information structure and let R be a relational design for $\mathcal{I}$. Furthermore, let $r_1, r_2 \in R$ be relation types ($r_1 \neq r_2$) with type related attribute types $a_1 \in r_1, a_2 \in r_2$. Then the population of $\mathsf{Join}(R, a_1, a_2)$ is defined by the (outer) join (e.g. [40]) applied to the new relation type.

Example 5.2.4

The population of relational design $\mathsf{Join}(\delta, \{q\}, \{r\})$ *is shown in figure 5.6.* □

$\{p\}$	$\{q, r\}$	$\{s\}$	$\{t\}$
a_1	b_1	c_1	d_1

$\{u\}$	$\{v\}$
b_1	d_1
b_2	d_1

Figure 5.6: Population of $\mathsf{Join}(\delta, \{q\}, \{r\})$

Usually, redundancy and null values are not considered on the conceptual level. The following example illustrates how redundancy and null values may be introduced on the internal level.

Example 5.2.5

The population of representation $\mathsf{Join}(\delta, \{r\}, \{u\})$ *is shown in figure 5.7. This population contains null values. The population of representation* $\mathsf{Join}(\delta, \{t\}, \{v\})$ *containing redundant information is shown in figure 5.8.* □

$\{p\}$	$\{q\}$
a_1	b_1

$\{r, u\}$	$\{s\}$	$\{t\}$	$\{v\}$
b_1	c_1	d_1	d_1
b_2	–	–	d_1

Figure 5.7: Population of $\mathsf{Join}(\delta, \{r\}, \{u\})$ containing null values

$\{p\}$	$\{q\}$
a_1	b_1

$\{r\}$	$\{s\}$	$\{u\}$	$\{t, v\}$
b_1	c_1	b_1	d_1
b_1	c_1	b_2	d_1

Figure 5.8: Population of $\mathsf{Join}(\delta, \{t\}, \{v\})$ containing redundancy

5.2.3 A relational mutation system

Let $\mathcal{I}$ be an information structure and let δ be the corresponding direct relational representation. Then the solution space $\mathcal{S}_r$ of candidate relational representations is inductively defined as follows:

Direct representation: $\delta \in \mathcal{S}_r$

Derived representations: $R \in \mathcal{S}_r \wedge \exists_{a_1, a_2 \in R} [R' = \mathsf{Join}(R, a_1, a_2)] \Rightarrow R' \in \mathcal{S}_r$

Note that formally the solution space $\mathcal{S}_r$ is *not* contained in the solution space $\mathcal{S}$ of candidate tree representations ($\mathcal{S}_r \not\subseteq \mathcal{S}$), because elements of $\mathcal{S}_r$ do not contain edges whereas elements of $\mathcal{S}$ do. Next we consider the number of joins in relational representations. Let $\mathcal{I}$ be an information structure with fact types $\mathcal{F}$. The corresponding direct relational representation δ consists of $|\mathcal{F}|$ relation types. Consequently the number of fact type joins in δ is equal to zero, i.e. no fact types have been joined. In the other extreme *all* fact types

are joined in a single relation type. In that case the number of fact type joins is equal to $|\mathcal{F}|-1$. Relational representations consisting of a single relation type are also called *universal* relation types (see e.g. [157]).

This leads us to the following. The solution space $\mathcal{S}_r$ of candidate relational representations for a given information structure $\mathcal{I}$ with fact types $\mathcal{F}$ consists of $|\mathcal{F}|$ hyperplanes, where a hyperplane $\nu_i \subseteq \mathcal{S}_r$ is the subset of $\mathcal{S}_r$ containing the relational representations with i joins ($0 \leq i \leq |\mathcal{F}|-1$).

Example 5.2.6

As an example, the information structure from figure 5.1 has a solution space consisting of three hyperplanes. In figure 5.9 these planes are shown. In this example, the lowest plane only contains the direct relational representation, while the highest plane contains the candidate universal relation types. □

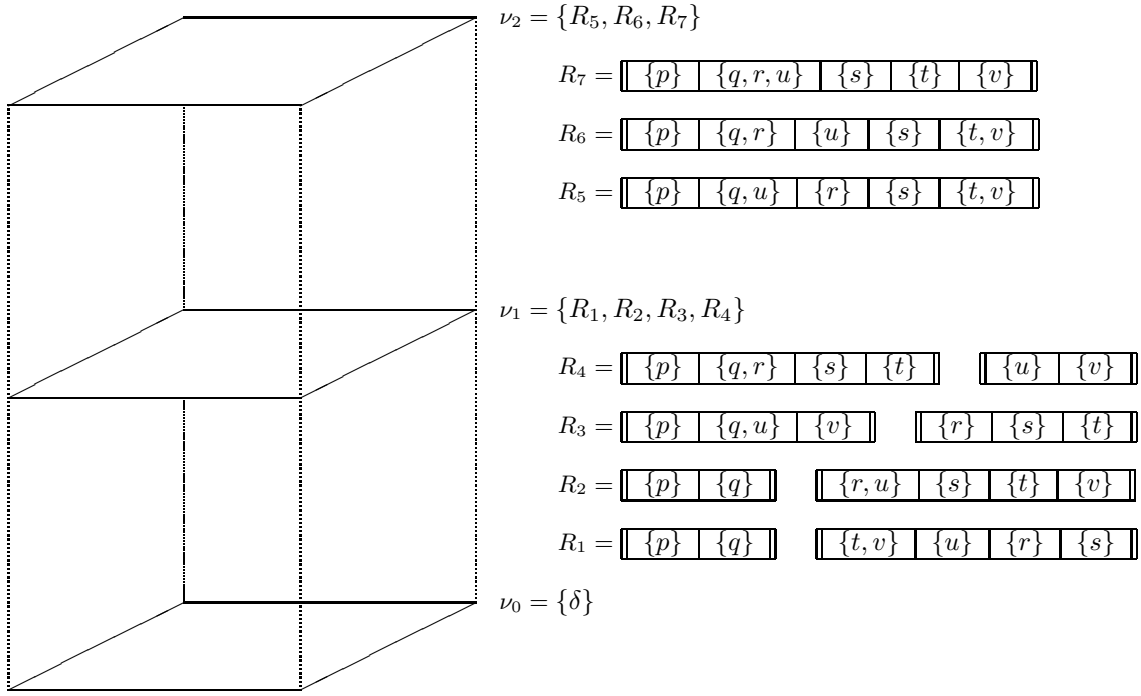


Figure 5.9: Hyperplanes for running example

Using the above definition of solution space $\mathcal{S}_r$, the navigation process through $\mathcal{S}_r$ is based on mutation system $\mathcal{M}_r = \langle \mathcal{S}_r, \mathsf{Join}, \mathsf{Split} \rangle$. Operator Split is defined as the inverse of operator Join. Let $R \in \nu_i \subseteq \mathcal{S}_r$ be a relational representation for information structure $\mathcal{I}$, where $0 \leq i \leq |\mathcal{F}|-1$. Now for the mutation of R into an alternative representation R' two basic possibilities can be recognized:

- $R' \in \nu_{i+1}$: R' is the result of joining two relation types in R.
- $R' \in \nu_{i-1}$: R' is the result of splitting a relation type in R.

Example 5.2.7

In figure 5.10 the possible mutations for the solution space from figure 5.9 are shown. This figure also shows how the solution space $\mathcal{S}_r$ is induced from the direct relational representation δ written as R_0 here. □

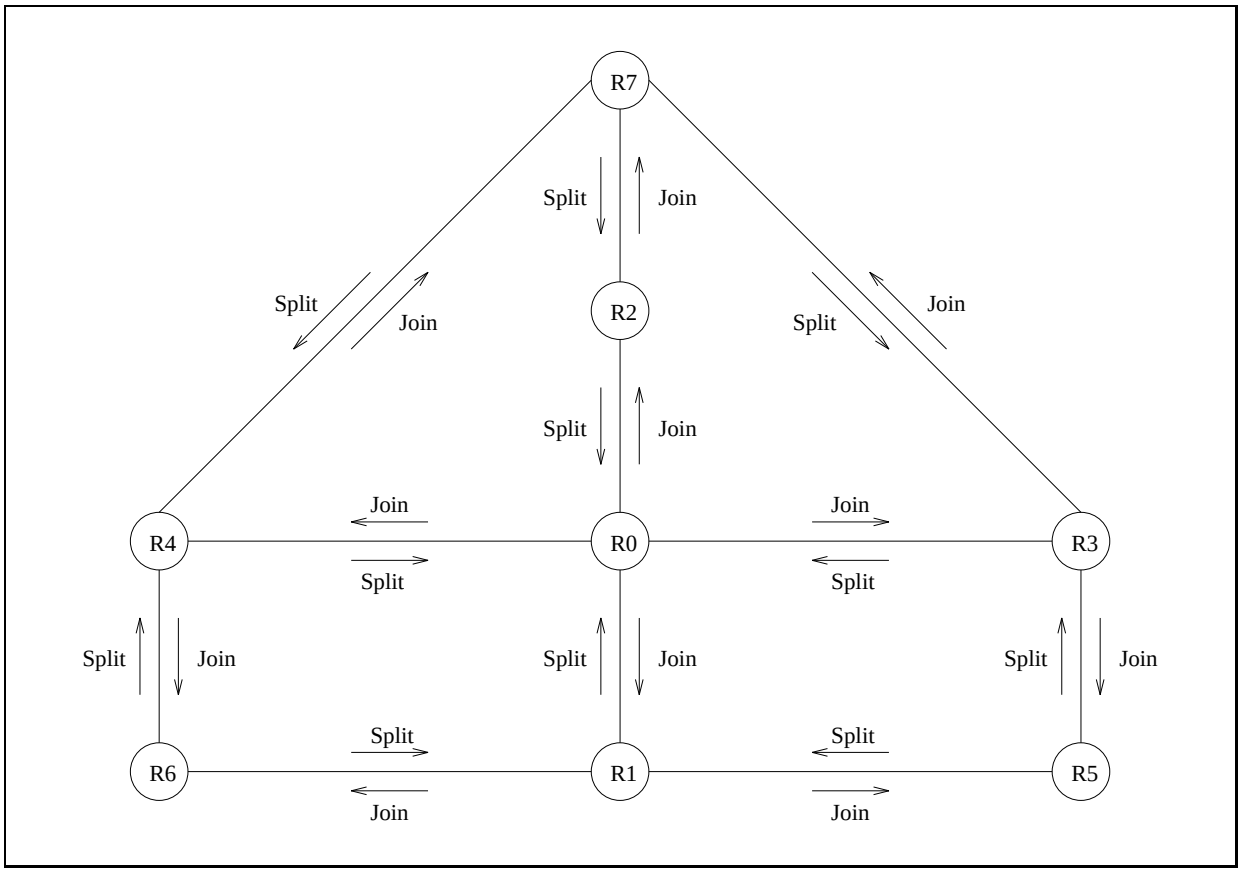


Figure 5.10: Mutations for example solution space

5.3 Encoding of solution space

Encoding mechanisms are often used to obtain a suitable problem representation. For example, in information retrieval (e.g. [59]) an allocation of documents may be encoded as a document vector. If d is the number of documents, a document vector is a sequence of integers between 0 to $d-1$. As another example, in query optimization (e.g. [11]) a join processing tree is encoded by linearizing it into an ordered list of cells containing join processing details.

In this section we present an encoding mechanism for the description of tree representations by a total function $\alpha : \mathcal{P} \longrightarrow \mathcal{P} \cup \mathcal{R}$. This encoding mechanism enables us to describe operators for initial generation and further mutation (section 5.4) on a high level of abstraction.

5.3.1 Strategy for encoding tree representations

The encoding we introduce is based on the concept of *anchors* (see lemma 3.2.1). All nodes being the source of some edge and having an atomic base have a unique anchor. As a result all nodes have an anchor, except for root nodes and nodes containing objectifications. This leads us to the encoding of a tree representation by *anchor function* $\alpha : \mathcal{P} \longrightarrow \mathcal{P} \cup \mathcal{R}$, defined as follows.

$$\alpha(p) = \begin{cases} \mathsf{Node}(p) & (\mathsf{Node}(p) \in \mathcal{R}) \\ p & (\mathsf{Node}(p) \notin \mathcal{R} \wedge \mathsf{Base}(p) \in \mathcal{F}) \\ \mathsf{Anchor}(\mathsf{Node}(p)) & (\mathsf{Node}(p) \notin \mathcal{R} \wedge \mathsf{Base}(p) \notin \mathcal{F}) \end{cases}$$

So for predicator $p \in \mathcal{P}$ the function α yields either a node (in case p is in a root), or it yields predicator p (in case p is an objectification), or the anchor of the node containing predicator p (otherwise). In order to simplify the encoding, predicators will be numbered from 1 to $|\mathcal{P}|$ and trees from -1 to $-|\mathcal{R}|$.

Example 5.3.1

In figure 5.11 we see a tree representation for the running example (figure 5.1), using predicators 1, ..., 7 and tree -1. The function α assigns e.g. $\alpha(1) = 1$, $\alpha(2) = -1$ and $\alpha(6) = 5$. The entire tree representation is encoded as:

p:	*1*	*2*	*3*	*4*	*5*	*6*	*7*
α(p):	*1*	*-1*	*-1*	*4*	*5*	*5*	*7*

In examples we use the following shorthand notation: | *1* | *-1* | *-1* | *4* | *5* | *5* | *7* | □

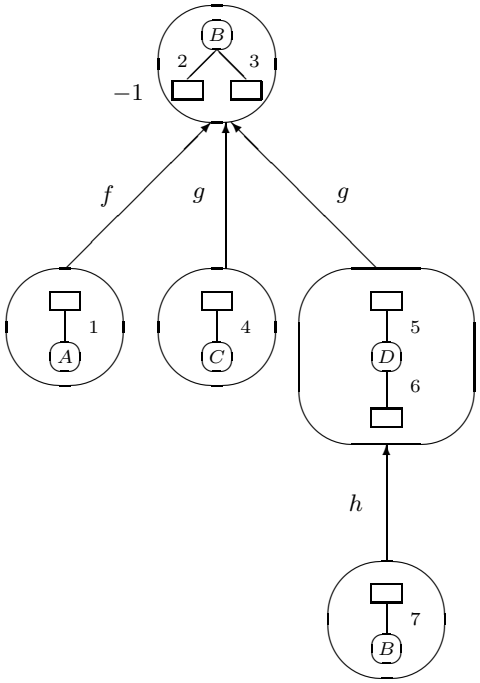


Figure 5.11: Tree representation with numbers

Decoding a function α into a tree representation is done by operator DecodeForest, which uses elementary operator DecodeFactType. Starting with the roots of the trees (negative positions in the encoding), operator DecodeForest traverses trees downwards using the properties of Anchor and Hook. Let $p = \mathsf{Hook}(f)$ be the hook of some fact type f. We first consider the question how to find the positions in encoding α for the children of node $\mathsf{Node}(p)$ dealing with fact type f. This is done by the operator DecodeFactType. For hook p this operator yields the following classes of children.

extension: The first class contains the nodes in $\mathcal{N}$ containing a predicator $q \in \mathsf{Fact}(p) - \{p\}$. For hook p this class is called the *extension* of $\mathsf{Fact}(p)$, denoted as $\mathsf{Ext}(\mathsf{Fact}(p))$. Each predicator $q \in \mathsf{Fact}(p) - \{p\}$ is the anchor of the associated node: $q = \mathsf{Anchor}(\mathsf{Node}(q))$. As a consequence, in encoding α these nodes are indicated by $\alpha(q) = q \ \wedge \ \mathsf{Base}(q) \in \mathcal{A})$.

objectifications: The second class contains those nodes in $\mathcal{N}$ containing an objectification r of $\mathsf{Fact}(p)$, indicated in α by $\alpha(r) = r \ \wedge \ \mathsf{Base}(r) = \mathsf{Fact}(p)$.

The traversal downwards in DecodeForest can now be described as follows. Let $n \in \mathcal{N}$ be a node which is visited during traversal. We consider the question how to find the positions in encoding α corresponding to the children of node n. If n is a root ($\forall_{p \in n}\,[\alpha(p) < 0]$), each predicator $p \in n$ is a hook and can be handled by DecodeFactType. If n is not a root ($\forall_{p \in n}\,[\alpha(p) > 0]$) and $\mathsf{Base}(n) \in \mathcal{F}$, then again each predicator $p \in n$ is a hook. If n is not a root and $\mathsf{Base}(n) \notin \mathcal{F}$, node n contains a unique predicator $p = \mathsf{Anchor}(n)$. Each predicator $q \in \mathsf{Fact}(p) - \{p\}$ is the hook of the associated fact type $q = \mathsf{Hook}(\mathsf{Fact}(q))$ and can again be handled by DecodeFactType.

5.3.2 Generation of encoded tree representations

In this section the generation of *encoded* tree representations is considered. We describe the effect of procedure ProcessFactType from section 3.3.1 in terms of the encoding mechanism introduced in the previous section. Procedure ProcessFactType is the action to be performed in the main loop of procedure GenerateForest (see section 3.3.1).

The generation process is initialized by $\alpha(x) := 0$ for all predicators x. In each iteration of procedure GenerateForest, procedure ProcessFactType is called with nodes m and n as parameters. These parameters were selected by procedure CanExtend. Let $m = \{p\}$ be a singleton set containing an unused predicator, i.e. $\mathsf{Fact}(p)$ has not yet been represented ($\alpha(p) = 0$). Furthermore, let $q \in n$ be a used predicator ($\alpha(q) \neq 0$) such that p and q are type related ($p \sim q$). The processing of $\mathsf{Fact}(p)$ by $\mathsf{ProcessFactType}(m, n)$ consists of two phases which will now be described. In the first phase the new hook p is installed, where either p is added to $\mathsf{Node}(q)$ or a new root $\{p\}$ is created:

$$\alpha(p) = \begin{cases} \alpha(q) & \text{if } m \neq n \\ -|\mathcal{R}| - 1 & \text{otherwise} \end{cases}$$

In the second phase all new anchors are installed and the objectifications of $\mathsf{Fact}(p)$ are treated. This is performed by the assignment $\alpha(x) := x$, for new anchors $x \in \mathsf{Fact}(p) - \{p\}$ and for objectifications $x \in \mathcal{P}$ with $\mathsf{Base}(x) = \mathsf{Fact}(p)$.

Recall from section 3.3.1 that during the generation process a tree representation will grow from an *incomplete* representation to a *complete* representation. As a consequence $|\mathcal{R}| = 0$ in the initial stage and $|\mathcal{R}| \leq |\mathcal{F}|$ after termination.

Example 5.3.2

The tree representation in figure 5.11 (and figure 4.3) may be constructed by the following sequence of actions: $\mathsf{ProcessFactType}(\{2\}, \{2\})$, $\mathsf{ProcessFactType}(\{3\}, \{2\})$, $\mathsf{ProcessFactType}(\{6\}, \{5\})$. *Note that other sequences may lead to the same result (see also the breadth-first and depth-first scenarios mentioned in section 3.5.1).* □

The call $\mathsf{ProcessFactType}(m, n)$ where $m = \{p\}$ involves each predicator in $\mathsf{Fact}(p)$ and each predicator x with $\mathsf{Base}(x) = \mathsf{Fact}(q)$. Let $||\mathsf{Fact}(p)||$ be the number of objectifications of $\mathsf{Fact}(p)$. Then the number of integer assignments needed for $\mathsf{ProcessFactType}(\{p\}, n)$ is as follows:

Lemma 5.3.1 $\mathsf{NrAss}(\mathsf{ProcessFactType}(\{p\}, n)) = |\mathsf{Fact}(p)| + ||\mathsf{Fact}(p)|| \leq |\mathcal{P}|$

5.3.3 Embedding index selection

Often the population of an internal representation T can only be accessed in a primitive way, for example sequential access. If however a particular node in T needs faster access, e.g. *direct* access, an efficient access path (also called index) can be provided. If the number of nodes in T equals k, there are 2^k possible index

configurations. The problem of *index selection* deals with the question how a suitable index configuration can be found.

In this section the basic operators AddIndex and DropIndex are embedded within our problem encoding. These operators can be used for searching through the set of candidate index configurations, either interactively or on the basis of index selection algorithms (see e.g. [118]).

Let α be the encoding of representation T. Then the candidate index configurations can be encoded as follows:

$$\alpha_{\mathsf{i}}(p) = \begin{cases} \alpha(p) & (\alpha(p) = 0 \text{ or } \mathsf{Node}(p) \text{ is not indexed}) \\ \alpha(p) - |\mathcal{P}| & (\alpha(p) < 0 \text{ and } \mathsf{Node}(p) \text{ is indexed}) \\ \alpha(p) + |\mathcal{P}| & (\alpha(p) > 0 \text{ and } \mathsf{Node}(p) \text{ is indexed}) \end{cases}$$

Recall that predicator p is in a root node if $\alpha(p) < 0$. Consequently, indexed root nodes are still encoded by negative numbers. The corresponding operator AddIndex(n) where n is a node changes the value of $\alpha_{\mathsf{i}}(x)$ accordingly (for $x \in n$):

$$\alpha_{\mathsf{i}}(x) := \begin{cases} \alpha_{\mathsf{i}}(x) - |\mathcal{P}| & (-|\mathcal{P}| \leq \alpha_{\mathsf{i}}(x) < 0) \\ \alpha_{\mathsf{i}}(x) + |\mathcal{P}| & (0 < \alpha_{\mathsf{i}}(x) \leq |\mathcal{P}|) \end{cases}$$

Example 5.3.3

Consider the tree representation and its encoding from section 5.3.1. Suppose two indexes are added by performing AddIndex($\{1\}$) *and* AddIndex($\{2,3\}$). *The encoding of the resulting tree representation is as follows:*

8	-8	-8	4	5	5	7

□

The operator DropIndex(n) restores the effect of AddIndex(n). It is defined as follows ($x \in n$):

$$\alpha_{\mathsf{i}}(x) := \begin{cases} \alpha_{\mathsf{i}}(x) + |\mathcal{P}| & (-2\,|\mathcal{P}| \leq \alpha_{\mathsf{i}}(x) < -\,|\mathcal{P}|) \\ \alpha_{\mathsf{i}}(x) - |\mathcal{P}| & (|\mathcal{P}| < \alpha_{\mathsf{i}}(x) \leq 2\,|\mathcal{P}|) \end{cases}$$

5.3.4 Similarity templates

Similarities among different solutions for a given problem can be expressed using *similarity templates* (see also [84], [62]). We will define this concept in terms of the encoding of tree representations.

A similarity template specifies a set of encoded representations, with similarities at certain positions. The symbol $*$ is used as *don't care* symbol. A similarity template τ then is an extension of the anchor function introduced in section 5.3:

$$\tau : \mathcal{P} \longrightarrow \mathcal{P} \cup \mathcal{R} \cup \{*\}$$

Let α be an anchor function and let τ be a similarity template. Anchor function α matches template τ if the following condition is satisfied:

$$\forall_{p \in \mathcal{P}} \left[\tau(p) = * \ \vee \ \tau(p) = \alpha(p)\right]$$

Example 5.3.4

Recall the tree representation from figure 5.11. The encoding α of this representation is given below, along with a similarity template τ:

α :	1	-1	-1	4	5	5	7
τ :	1	*	-1	4	5	*	7

Then clearly α matches τ. Here τ specifies all tree representations where predicator 2 is Hook(f), *predicator 3 is* Hook(g), *and predicator 6 is* Hook(h), *with the additional requirement that predicator 2 must be in a root.* □

This section is concluded with two elementary properties. Let τ be a similarity template with d don't care positions. We then have the following property:

Lemma 5.3.2 The number of matchings of τ is restricted by $(2\,|\mathcal{P}|)^d$.

Proof:
Each don't care position can be filled either with a predicator or with a root (see section 5.3). In both cases the number of possibilities will not exceed $|\mathcal{P}|$ (index selection is not considered here, see section 5.3.3). Now the result is easily derived, since there are d don't care positions. □

The number of don't care positions will not exceed the number of predicators ($d \leq |\mathcal{P}|$). Using this information and lemma 5.3.2, a rather pessimistic upperbound for the size of the internal solution space is as follows:

Lemma 5.3.3 $|\mathcal{S}| \leq (2 * |\mathcal{P}|)^{|\mathcal{P}|}$

5.4 Mutations for tree representations

In this section we introduce a mutation system $\mathcal{M} = \langle \mathcal{S}, \mathsf{Prune}, \mathsf{Graft}, \mathsf{Promote} \rangle$, where $\mathcal{S}$ is the internal solution space from chapter 3 and Prune, Graft and Promote are mutation operators for elements of $\mathcal{S}$. These operators can be used for searching through $\mathcal{S}$ on the basis of so-called evolutionary algorithms (automated database design algorithms, see chapter 6), or for the definition of more advanced transformation operators, such as crossover (see section 5.5).

5.4.1 The Prune mutation

Let $T = \langle \mathcal{N}, E, \ell \rangle$ be a tree representation. The Prune mutation cuts off a part of a tree in T in order to create a new tree (within T). It is not exactly a subtree which is cut off, because the new tree (also called cutting) is obtained by *node splitting*. The node splitting process is defined in terms of three elementary operators ψ, Ψ and $\oplus$.

The cutting is specified by the operator ψ, the remainder of the representation is specified by the operator Ψ, and the union of these two is described by the merge operator $\oplus$. From the viewpoint of the cutting operator ψ, a set of properties is *isolated* from a representation. From the viewpoint of the remainder operator Ψ, a set of properties is *removed* from a representation. The starting-point for the Prune mutation is a predicator. Predicator p is a valid application point for pruning, denoted as $\mathsf{ValidPrune}(T, p)$, if it is a hook. This fact type will be completely disconnected from the accomodating node, including descendant nodes and edges. With respect to objectified fact types we restrict ourselves to internal representations with connected objectifications. Therefore hook p is required to have an atomic base. Now first an example is given.

Example 5.4.1
Recall the tree representation T from figure 5.11 consisting of a single tree. Figure 5.12 shows this representation after pruning fact type h. The resulting representation $T' = \mathsf{Prune}(T, u)$ consists of two trees with root $\{q, r\}$ and $\{u\}$.

Other applications of the Prune *mutation to this representation are $\mathsf{Prune}(T, q)$ and $\mathsf{Prune}(T, r)$. In case cutting-point q is chosen, fact type f will be represented in a separate tree. In case cutting-point r is chosen, the subtree containing fact type g and h will be represented in a separate tree. As a consequence, cutting-points q and r have the same effect: $\mathsf{Prune}(T, q) = \mathsf{Prune}(T, r)$.* □

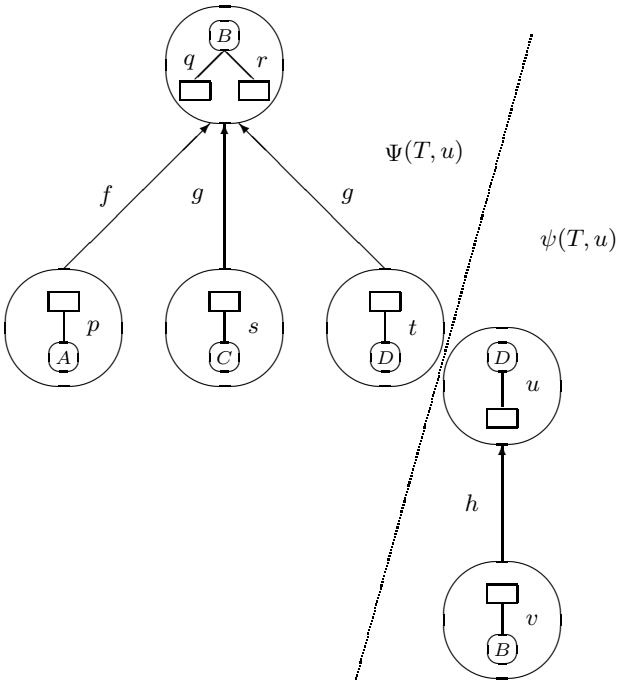


Figure 5.12: Representation $T' = \mathsf{Prune}(T, u)$ consists of two trees

The effect of the Prune mutation is easily expressed in terms of the anchor function α for encoding tree representations (see section 5.3). Let α be an anchor function for representation T and let p be a hook in T. Then the effect of $\mathsf{Prune}(T, p)$ is expressed by:

$$\alpha(p) := -|\mathcal{R}| - 1$$

Consequently, the number of integer assignments needed for $\mathsf{Prune}(T, p)$ is as follows:

Lemma 5.4.1 $\mathsf{NrAss}(\mathsf{Prune}(T, p)) = 1$

The effect of the Prune mutation can also be expressed in terms of the tree representation mechanism (nodes, edges, labels), although this is more complicated. Let T be a tree representation and let $p = \mathsf{Hook}(f)$ be a hook in T. Furthermore, let $n = \mathsf{Node}(p)$. Then, the local effect of pruning T in p is as follows: $n := n - \{p\}$. In order to give an exact definition of the global effect of pruning T in hook p, we base the Prune mutation on the operators Ψ, ψ and $\oplus$:

$$\mathsf{Prune}(T, p) = \Psi(T, p) \oplus \psi(T, p)$$

First the merge operator is considered. The merge operator $\oplus$ is used for the union of two disjoint representations, i.e. having no predicators in common: $(\cup\mathcal{N}_1) \cap (\cup\mathcal{N}_2) = \varnothing$. This merging is performed by simply uniting the separate components of the representations: nodes, edges and labelling.

Next we describe the remainder operator Ψ detailed, the cutting operator ψ can be handled analogously. Let $T = \langle \mathcal{N}, E, \ell \rangle$ be a representation and let $p = \mathsf{Hook}(f)$ be a valid cutting point with $n = \mathsf{Node}(p)$. Let $\mathsf{DescNodes}(p)$ and $\mathsf{DescEdges}(p)$ be the set of descendant nodes and edges respectively:

$$\begin{aligned} \mathsf{DescNodes}(p) &= \{x \in \mathcal{N} | x \rightarrow^* z \rightarrow n \wedge \ell(\langle z, n \rangle) = f\} \\ \mathsf{DescEdges}(p) &= \{\langle x, y \rangle \in E | x \in \mathsf{DescNodes}(p)\} \end{aligned}$$

The remainder of T after cutting in predicator p is the (incomplete) representation $T' = \Psi(T, p)$. Let $T' = \langle \mathcal{N}', E', \ell' \rangle$. This representation is defined as follows:

$$\begin{aligned} \mathcal{N}' &= \mathcal{N} - \mathsf{DescNodes}(p) - \{n\} \cup \begin{cases} \varnothing & (n = \{p\}) \\ \{n - \{p\}\} & (\text{otherwise}) \end{cases} \\ E' &= E - \mathsf{DescEdges}(p) \\ \ell' &= \ell[E'] \end{aligned}$$

If point of application p is not valid for pruning, or does not even occur in the representation at hand, the remainder Ψ yields the original representation: $T = \Psi(T, p)$. The cutting ψ then yields the empty representation $\varepsilon = \langle \varnothing, \varnothing, \varnothing \rangle$.

This section is concluded with a basic property of the Prune mutation. Let T be a tree representation.

Lemma 5.4.2 Pruning a singleton root has no effect: $\forall_{\{p\} \in \mathcal{R}} [T = \mathsf{Prune}(T, p)]$.

Proof:

No node splitting is necessary, since $\{p\}$ is a root. Consequently, the cutting $\psi(T, p)$ is a representation consisting of a single tree. This tree is the tree from T with root $\{p\}$. The remainder $\Psi(T, p)$ is a representation consisting of the remaining $|\mathcal{R}| - 1$ trees. Therefore, merging the cutting and the remainder yields the original tree representation. □

5.4.2 Correctness of the Prune mutation

In this section the correctness of the Prune mutation is considered. Let $T = \langle \mathcal{N}, E, \ell \rangle$ be an internal representation for information structure $\mathcal{I}$ containing predicators $\mathcal{P}$. Furthermore, let predicator p be the hook of some fact type. We consider the question whether $T' = \mathsf{Prune}(T, p)$ is still a correct internal representation for $\mathcal{I}$. The wellformedness conditions t_1 to t_6 for internal representations discussed in section 3.2 are informally summarized in figure 5.13.

condition	explanation
	base representation (nodes):
t_1	predicators in the same node are type related
t_2	predicators in the same node belong to different fact types
	fact representation (edges):
t_3	edges are labelled by fact types to reflect the information structure
t_4	fact types are located around a single parent
t_5	predicators involved in an edge occur in the label of some associated edge
	completeness:
t_6	each predicator is involved in some edge

Figure 5.13: Wellformedness conditions for tree representations

First the correctness of the remainder operator Ψ is considered:

Lemma 5.4.3 Pruning does not kill: $\mathsf{Forest}(T, \mathcal{I}) \Rightarrow \forall_{p \in \mathcal{P}} [\mathsf{Forest}(\Psi(T, p), \mathcal{I})]$

Proof:

If predicator p is *not* a valid point for pruning, then $T = \Psi(T, p)$ and the lemma is obvious. If p is a hook in T, then $T' = \Psi(T, p)$ is the remainder of T after disconnecting $\mathsf{Fact}(p)$ including descendants. If $\mathsf{Node}(p) = \{p\}$ then $\{p\}$ is a root. In that case the lemma is also obvious, since no node splitting is necessary, and T' contains all trees from T except for the tree with root $\{p\}$ (see also lemma 5.4.2).

However, if $\mathsf{Node}(p) \neq \{p\}$ then fact type $\mathsf{Fact}(p)$ is disconnected by splitting $n = \mathsf{Node}(p)$ in $n - \{p\}$ and $\{p\}$, where T' contains node $n - \{p\}$. In order to guarantee $\mathsf{Forest}(T', \mathcal{I})$ the wellformedness conditions t_1 to t_5 from figure 5.13 are checked as follows.

First the nodes in T' are considered. Since no predicators are added to existing nodes and representation T satisfies conditions t_1 and t_2, it can be concluded that representation T' also satisfies t_1 and t_2.

Next the edges in T' are considered. We have to consider conditions t_3, t_4 and t_5 for (a) the single edge e_s in T' with source m, and (b) the edges e_d in T' having destination m. For condition t_3 combined with case (a) observe that $p \notin \ell(e_s)$ and therefore $m \cap \ell(e_s) \neq \varnothing$. For condition t_3 combined with case (b) observe that $p \notin \ell(e_d)$ and therefore $m \cap \ell(e_d) \neq \varnothing$.

For condition t_4 it is sufficient to note that T' contains all edges from T, except the edges in $\mathsf{DescEdges}(p)$. Condition t_5 is satisfied for the same reasons as condition t_3. □

The cutting operator ψ has a similar property, which can be treated analogously:

Lemma 5.4.4 After pruning the cutting is alive: $\mathsf{Forest}(T, \mathcal{I}) \Rightarrow \forall_{p \in \mathcal{P}} [\mathsf{Forest}(\psi(T, p), \mathcal{I})]$

Next the correctness of merging is considered:

Lemma 5.4.5 $\mathsf{Forest}(T_1, \mathcal{I}) \wedge \mathsf{Forest}(T_2, \mathcal{I}) \Rightarrow \mathsf{Forest}(T_1 \oplus T_2, \mathcal{I})$

Proof:

Merging of two representations $T_1 = \langle \mathcal{N}_1, E_1, \ell_1 \rangle$ and $T_2 = \langle \mathcal{N}_2, E_2, \ell_2 \rangle$ results in a new representation $T_1 \oplus T_2$ by uniting the nodes, edges and labels. Representations T_1 and T_2 are assumed disjoint: $(\cup \mathcal{N}_1) \cap (\cup \mathcal{N}_2) = \varnothing$. Therefore representation $T_1 \oplus T_2$ contains exactly the nodes, edges and labels from $\mathcal{N}_1$, $\mathcal{N}_2$, E_1, E_2, ℓ_1 and ℓ_2. From this the lemma can be derived. □

The correctness of the entire Prune mutation is a property of main interest:

Theorem 5.4.1 $\mathsf{CompleteForest}(T, \mathcal{I}) \Rightarrow \forall_{p \in \mathcal{P}} [\mathsf{CompleteForest}(\mathsf{Prune}(T, p), \mathcal{I})]$

Proof:

From the previous lemmas it is now clear that pruning is at least *partially* correct: if $\mathsf{CompleteForest}(T, \mathcal{I})$ then $\forall_{p \in \mathcal{P}} [\mathsf{Forest}(\mathsf{Prune}(T, p), \mathcal{I})]$. So it is sufficient to show that $\mathsf{Prune}(T, p)$ also satisfies condition t_6 expressing completeness (see figure 5.13).

Is each predicator from $\mathcal{I}$ involved in some edge in $\mathsf{Prune}(T, p)$? The answer is *yes*. Let remainder $\Psi(T, p)$ and cutting $\psi(T, p)$ contain predicators $\mathcal{P}_\Psi$ and $\mathcal{P}_\psi$ respectively. Then $\mathcal{P}_\Psi \cap \mathcal{P}_\psi = \varnothing$ and $\mathcal{P} = \mathcal{P}_\Psi \cup \mathcal{P}_\psi$, where each predicator in $\mathcal{P}_\Psi$ occurs in some edge in $\Psi(T, p)$ and each predicator in $\mathcal{P}_\psi$ occurs in some edge in $\psi(T, p)$. □

5.4.3 The Graft mutation

The intention of the Graft mutation is to describe an inversion of the Prune mutation. Let T be a tree representation including predicators p and q, where predicator p is the hook of $\mathsf{Fact}(p)$ and has an atomic base. Predicator q is a valid application point for grafting predicator p, denoted as $\mathsf{ValidGraft}(T, p, q)$, if p and q are type related and q does not belong to a descendant node of p:

$$p = \mathsf{Hook}(\mathsf{Fact}(p)) \ \wedge \ \mathsf{Base}(p) \in \mathcal{A} \ \wedge \ p \sim q \ \wedge \ \mathsf{Node}(q) \notin \mathsf{DescNodes}(p)$$

Predicator q is not allowed to belong to a descendant node of p, as this would result in a *cyclic* representation rather than a *tree* representation. If $\mathsf{ValidGraft}(T,p,q)$ then $T' = \mathsf{Graft}(T,p,q)$ is the representation that has $\psi(T,p)$ grafted into $\mathsf{Node}(q)$, by adding predicator p to $\mathsf{Node}(q)$. Additionally it is assumed from now onwards that $\mathsf{Node}(p) = \{p\}$ is a root. If this is not the case, mutation $\mathsf{Prune}(T,p)$ can be used to achieve this.

The effect of the Graft mutation is expressed in terms of the anchor function α as follows. Let α be an anchor function for representation T and let q be a valid application point for grafting predicator p. Then the effect of $\mathsf{Graft}(T,p,q)$ is expressed by:

$$\alpha(p) := \alpha(q)$$

So the number of integer assignments needed for $\mathsf{Graft}(T,p,q)$ equals the number of assignments needed for $\mathsf{Prune}(T,p)$:

Lemma 5.4.6 $\mathsf{NrAss}(\mathsf{Graft}(T,p,q)) = 1$

Example 5.4.2
Recall the tree representation T from figure 5.11. Suppose node $\{2,3\}$ is split by mutation $\mathsf{Prune}(T,2)$. Now mutation $\mathsf{Graft}(T,2,7)$ adds predicator 2 to node $\{7\}$. As a consequence, in the new representation fact type f is represented below fact type h. In the encoding this mutation is simply expressed by $\alpha(2) := 7$. □

The effect of the Graft mutation applied to representation $T = \langle \mathcal{N}, E, \ell \rangle$ is expressed in terms of nodes, edges and labels as follows. Let representation $T' = \mathsf{Graft}(T,p,q)$ be the result of grafting hook p onto $m = \mathsf{Node}(q)$, and let $n = \mathsf{Node}(p)$. Suppose $T' = \langle \mathcal{N}', E', \ell' \rangle$. Then the new set of nodes $\mathcal{N}'$ is defined as follows:

$$\mathcal{N}' \quad = \quad \mathcal{N} - \{m,n\} \cup \{m \cup n\}$$

These nodes are connected by all edges from T, with the modification of the (incoming and outgoing) edges for m into (incoming and outgoing) edges for $m \cup n$. Furthermore, each edge $\langle x,n \rangle \in E$ is replaced by $\langle x, m \cup n \rangle$. The inherited edges keep their labels from T, while the modified edges are labelled according to their origin in T.

5.4.4 Correctness of the Graft mutation

The correctness of the Graft mutation is a property of main interest. Let $T = \langle \mathcal{N}, E, \ell \rangle$ be an internal representation for information structure $\mathcal{I}$ containing predicators $\mathcal{P}$. Then the following property holds:

Theorem 5.4.2 The Graft mutation yields correct representations:

$$\mathsf{CompleteForest}(T,\mathcal{I}) \Rightarrow \forall_{p,q \in \mathcal{P}, \mathsf{ValidGraft}(T,p,q)} [\mathsf{CompleteForest}(\mathsf{Graft}(T,p,q),\mathcal{I})]$$

The proof of this property is analogous with the proof of lemma 5.4.3 (pruning does not kill) and is omitted here.

A basic property of the Prune – Graft mutations is that pruning restores grafting (see also figure 5.14):

Lemma 5.4.7 $\forall_{p,q \in \mathcal{P}, \mathsf{ValidGraft}(T,p,q)} [T = \mathsf{Prune}(\mathsf{Graft}(T,p,q),p)]$

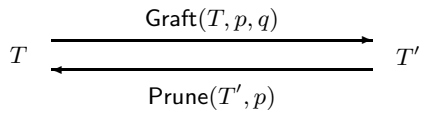


Figure 5.14: Pruning restores grafting

Proof:

Let $p, q \in \mathcal{P}$ be predicators s.t. q is a valid application point for grafting predicator p. Then in representation $T' = \mathsf{Graft}(T, p, q)$ predicator p has been added to $\mathsf{Node}(q)$. Now we have: $\mathsf{ValidPrune}(T', p)$. According to the definition of the Prune mutation, in representation $T'' = \mathsf{Prune}(T', p)$ predicator p is removed from $\mathsf{Node}(q)$. Consequently $T = T''$. □

Similarly, grafting restores pruning:

Lemma 5.4.8 $\forall_{p \in \mathcal{P}, \mathsf{ValidPrune}(T,p), q \in \mathsf{Node}(p) - \{p\}} [T = \mathsf{Graft}(\mathsf{Prune}(T, p), p, q)]$

This section is concluded with the property that mutations Prune and Graft are each others inverse.

Theorem 5.4.3 $\exists_{p,q \in \mathcal{P}} [T' = \mathsf{Graft}(T, p, q)] \iff \exists_{r \in \mathcal{P}} [T = \mathsf{Prune}(T', r)]$

Proof:

This theorem expresses that ($\Rightarrow$) if the Graft mutation has modified a representation T into T', it is always possible to restore the original situation using the Prune mutation, and ($\Leftarrow$) vice versa. For part ($\Rightarrow$) choose $r = p$ and apply lemma 5.4.7.

For part ($\Leftarrow$) consider the following. Let r be a valid application point for pruning. If $\mathsf{Node}(r) = \{r\}$ then pruning has no effect (lemma 5.4.2). Consequently $T = T'$. Now choose $p = q = r$ to obtain the theorem. If $\mathsf{Node}(r) \neq \{r\}$ then choose $p = r$ and $q \in \mathsf{Node}(r) - \{r\}$ and apply lemma 5.4.8 to obtain the theorem. □

5.4.5 The Promote mutation

The next mutation operator generates a variation of a tree representation by promoting an anchor predicator to a hook predicator. Let p be the anchor of $\mathsf{Node}(p)$ and let q be the hook of $\mathsf{Fact}(p)$. Then anchor p can become the hook of $\mathsf{Fact}(p)$ by swapping edge $\langle \mathsf{Node}(p), \mathsf{Node}(q) \rangle$ to $\langle \mathsf{Node}(q), \mathsf{Node}(p) \rangle$. This is the local effect of the Promote mutation (see also figure 5.15).

However, the shape of the tree must be changed in a consistent manner. Therefore all edges with $f = \mathsf{Fact}(p)$ have to be redirected: their target node was originally $\mathsf{Node}(q)$, but will be $\mathsf{Node}(p)$ after promotion. So $T' = \mathsf{Promote}(T, p)$ is a tree representation with the same set of nodes, only some edges redirected, and all labels unaffected. In order to restrict the affected edges to those with label f, we require $\mathsf{Node}(q)$ to be a root node. Now a predicator p in representation T is a valid application point for promotion, denoted as $\mathsf{ValidPromote}(T, p)$, if:

$$\exists_{n \in \mathcal{N}} [p = \mathsf{Anchor}(n)] \wedge \mathsf{Node}(\mathsf{Hook}(\mathsf{Fact}(p))) \in \mathcal{R}$$

The effect of the Promote mutation is expressed in terms of the anchor function α as follows. Let p be an anchor in representation T and let α be an anchor function for T. Furthermore, let $q = \mathsf{Hook}(\mathsf{Fact}(p))$ be a predicator in $\mathsf{Fact}(p)$. The effect of promotion $\mathsf{Promote}(T, p)$ then is expressed by:

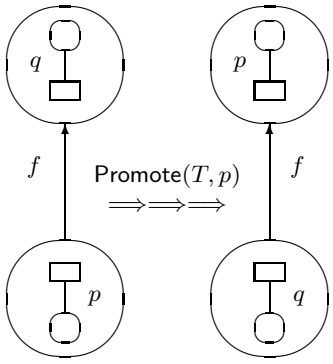


Figure 5.15: Local effect of promotion: anchor p becomes hook

$$\alpha(x) := \begin{cases} q & \text{if } x \in \mathsf{Node}(q) \\ \alpha(q) & \text{if } x \in \mathsf{Node}(p) \end{cases}$$

As a consequence, the number of integer assignments needed for $\mathsf{Promote}(T, p)$ equals:

Lemma 5.4.9 $\mathsf{NrAss}(\mathsf{Promote}(T, p)) = |\mathsf{Node}(p)| + |\mathsf{Node}(\mathsf{Hook}(\mathsf{Fact}(p)))| \leq |\mathcal{P}|$

Example 5.4.3

Consider predicator s in the left part of figure 5.16. The result of $\mathsf{Promote}(T, s)$ *is given in the right part.* □

Let $T = \langle \mathcal{N}, E, \ell \rangle$ be a tree representation and let p be an anchor to be promoted. What would happen if $\mathsf{Node}(\mathsf{Hook}(\mathsf{Fact}(p)))$ is *not* a root node? Then the edges to be redirected are more difficult to determine. Suppose $\mathsf{Promote}'$ is a promotion operator allowing anchor p to be in any node. Now in $T' = \mathsf{Promote}'(T, p)$ predicator p is in general not promoted a single level upwards, it has to be promoted upto the root of its tree. Otherwise T' would be an incorrect representation.

In order to determine the edges to be redirected, consider the edges $E'(p)$ on the path from node $n = \mathsf{Node}(p)$ to the root of the tree:

$$E'(p) = \{\langle x, y \rangle \in E | \, \mathsf{Node}(p) \rightarrow^* x\}$$

In $\mathsf{Promote}'(T, p)$ the edges in $E'(p)$ are replaced by their reversal: $\{\langle y, x \rangle \mid \langle x, y \rangle \in E'(p)\}$. Furthermore, the edges labelled with the same fact type as the edges in $E'(p)$ should be redirected. Let $F(p)$ be the set containing these fact types, defined as:

$$F(p) = \{f \in \mathcal{F} | \exists_{e \in E'(p)} [\ell(e) = f]\}$$

Then the edges in $\{e \in E - E'(p) | \ell(e) \in F(p)\}$ should be replaced by the following edges:

$$\{\langle y, x \rangle \, | \exists_{\langle y,z \rangle \in E, \langle x,z \rangle \in E'(p), x \neq y} [\ell(\langle y, z \rangle) \in F(p)]\}$$

The definition of $\mathsf{Promote}(T, p)$ in terms of the representation mechanism from chapter 3 (nodes, edges, labels) has in fact been given above by $\mathsf{Promote}'(T, p)$, because the $\mathsf{Promote}$ mutation is a special (elementary) case of the $\mathsf{Promote}'$ mutation. For the $\mathsf{Promote}$ mutation there is just one edge in $E'(p)$.

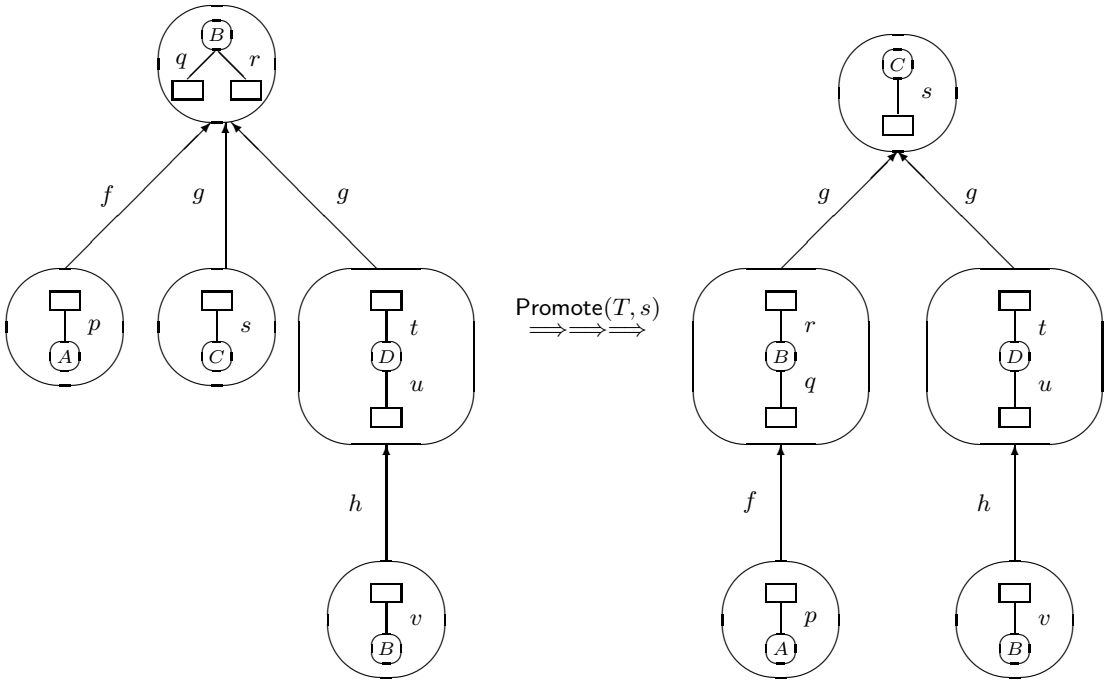


Figure 5.16: Promoting predicator s to become hook

5.4.6 Correctness of the Promote mutation

In this section the correctness of the Promote mutation is considered. Let $T = \langle \mathcal{N}, E, \ell \rangle$ be an internal representation for information structure $\mathcal{I}$ containing predicators $\mathcal{P}$. Furthermore, let p be a predicator s.t. $\mathsf{ValidPromote}(T, p)$. We consider the question whether $T' = \mathsf{Promote}(T, p)$ is still a correct internal representation for $\mathcal{I}$. The wellformedness conditions t_1 to t_6 for internal representations are summarized in figure 5.13. The full definition of these conditions is found in chapter 3.

Theorem 5.4.4 Promotion is correct:

$$\mathsf{CompleteForest}(T, \mathcal{I}) \Rightarrow \forall_{p \in \mathcal{P}, \mathsf{ValidPromote}(T,p)} \left[\mathsf{CompleteForest}(\mathsf{Promote}(T, p), \mathcal{I})\right]$$

Proof:

Let $T = \langle \mathcal{N}, E, \ell \rangle$ be a correct tree representation for information structure $\mathcal{I}$ and let predicator p be a valid application point for promotion. Let $T' = \mathsf{Promote}(T, p)$ be the result after promotion. Then T' satisfies wellformedness conditions t_1 and t_2, because promotion does not affect the nodes.

Next the edges are considered (conditions t_3, t_4 and t_5). The set $E'(p)$ contains a single edge: $\langle \mathsf{Node}(p), \mathsf{Node}(\mathsf{Hook}(\mathsf{Fact}(p)))\rangle$. This edge is reversed and consequently the resulting edge satisfies condition t_3 (note that $\mathsf{Node}(p)$ as well as $\mathsf{Node}(\mathsf{Hook}(\mathsf{Fact}(p)))$ have an atomic base). The set $F(p)$ contains a single fact type: $\mathsf{Fact}(p)$. So all edges e with $\ell(e) = \mathsf{Fact}(p)$ are redirected. Their source node remains the same, but their target node is changed. Their target node was $\mathsf{Node}(\mathsf{Hook}(\mathsf{Fact}(p)))$ in T, and will be $\mathsf{Node}(p)$ in T'. Consequently the redirected edges also satisfy t_3, since $\ell(e) \cup \mathsf{Node}(p) \neq \varnothing$.

For condition t_4 it is sufficient to observe that in T' all edges e with label $\ell(e) = \mathsf{Fact}(p)$ have the same target node: $\mathsf{Node}(p)$. Condition t_5 expresses that all predicators involved in an edge must occur in the label of that edge, or in the label of an associated edge. Consequently the predicators involved in the edge in T' that has been reversed satisfies t_5, but what about the redirected edges

(i.e. the other edges with label $\mathsf{Fact}(p)$)? Let $\langle m, \mathsf{Node}(\mathsf{Hook}(\mathsf{Fact}(p)))\rangle$ be such an edge in T and let $\langle m, \mathsf{Node}(p)\rangle$ be the resulting edge in T' after redirection. Then the predicators in target node $\mathsf{Node}(p)$ satisfy t_5, as this is also the target node of the reversed edge. Furthermore the predicators in source node m also satisfy t_5, because the labels of the incoming and outgoing edges of node x did not change.

Finally condition t_6 is considered (completeness). This condition requires that each predicator must occur in some edge. Consequently this condition is also satisfied, since all nodes from T will again be source or target of some edge in T' (particularly the nodes m, $\mathsf{Node}(p)$ and $\mathsf{Node}(\mathsf{Hook}(\mathsf{Fact}(p)))$). □

This section is concluded with the property that the Promote mutation describes its own inverse.

Theorem 5.4.5 $T' = \mathsf{Promote}(T, p) \Rightarrow \exists_{q \in \mathcal{P}} [T = \mathsf{Promote}(T', q)]$

Proof:

This theorem expresses that in case the Promote mutation has modified a representation T resulting in T', it is always possible to restore the original situation using the same mutation operator. Let $T' = \mathsf{Promote}(T, p)$. Now choose $q = \mathsf{Hook}(\mathsf{Fact}(p))$ in the original representation T. In T' predicator q is a valid application point for promotion, and $\mathsf{Promote}(T', q)$ results in T. □

5.5 Additional considerations on mutations

In this section several additional aspects of mutations are discussed. We first address properties of mutation systems, such as *closedness* and *completeness* (5.5.1). Then the attention is focussed on the notion of *distance* between internal representations (5.5.2), and *recursive* nested tables are introduced (5.5.3). Finally the notion of *crossover* is addressed (5.5.4). The aim of crossover is to combine two internal representations into a single representation containing a part of *both* original representations.

5.5.1 Properties of mutation systems

An n-ary mutation system $\mathsf{M} = \langle \mathsf{S}, \mu_1, ..., \mu_n \rangle$ over a given solution space S consists of S along with a collection of mutation operators $\mu_1, ..., \mu_n$ over S. Mutation system M over solution space S is called *closed* if application of mutation operators always produces a representation within S. As an example, in the previous section it was shown that mutation system $\mathcal{M} = \langle \mathcal{S}, \mathsf{Prune}, \mathsf{Graft}, \mathsf{Promote} \rangle$ is closed.

Let $\mathsf{M} = \langle \mathsf{S}, \mu_1, ..., \mu_n \rangle$ be a mutation system. A *mutation path* $T_0, ..., T_k$ of length k in M is a sequence of elements of S, such that T_i can be mutated into T_{i+1} (for $1 \leq i \leq k-1$). Mutation system M is called *complete* if it has the following property:

$$\forall_{x,y \in \mathsf{S}} [\text{There exists a mutation path from } x \text{ to } y \text{ in } \mathsf{M}]$$

Let $\mathcal{M}_r = \langle \mathcal{S}_r, \mathsf{Join}, \mathsf{Split} \rangle$ be the mutation system for relational representations introduced in section 5.2. This mutation system has the following property:

Theorem 5.5.1 Mutation system $\mathcal{M}_r$ is complete.

The completeness of $\mathcal{M}_r$ is a direct consequence of $\mathcal{S}_r$ being inductively defined using mutation Join, combined with the fact that mutation Split is defined as the inverse of mutation Join. For tree representations a similar property holds:

Theorem 5.5.2 Mutation system $\mathcal{M} = \langle \mathcal{S}, \mathsf{Prune}, \mathsf{Graft}, \mathsf{Promote} \rangle$ is complete.

Proof:

Let $x, y \in \mathcal{S}$ be internal representations. We construct a mutation path from x to y in three phases.

The pruning phase: First the trees in x are broken down as much as possible using the Prune mutation. Let T' be the resulting representation. *The promoting phase:* Now the trees in T' are inspected and all predicators being a hook in y but not in T' are promoted. *The grafting phase:* Finally, new trees are constructed according to the structure of y. □

Note that the completeness considered in theorem 5.5.2 assumes the representation mechanism introduced in chapter 3. For other representation mechanisms, other mutation operators may be necessary.

5.5.2 Distance between internal representations

Equivalence between data models is an important topic in data modelling (see e.g. [93], [6], [88] and [28]). However, even if data models are equivalent in some sense, they often are quite different in another sense. So it is desirable to have a mechanism for expressing the degree of inequality between (equivalent) data models. In this section, such a degree of inequality is introduced in the context of the internal solution space $\mathcal{S}$. It is called the *distance* between internal representations.

First the *anchor distance* is introduced. This distance is based on the encoding α for tree representations and is similar to the well-known *Hamming distance* (see e.g. [75]). Let $x, y \in \mathcal{S}$ be tree representations and let α_x and α_y be their encoding. Then the anchor distance $\mathsf{Dist_a}(\alpha_x, \alpha_y)$ is defined as the number of predicators having a different encoding. More concretely, if $P(\alpha_x, \alpha_y) = \{p \in \mathcal{P} | \, \alpha_x(p) \neq \alpha_y(p)\}$ then $\mathsf{Dist_a}(\alpha_x, \alpha_y) = |P(\alpha_x, \alpha_y)|$.

Example 5.5.1

Let α be an anchor function for tree representation T containing predicators p, q, r and s, such that $\mathsf{ValidPrune}(T, p), \mathsf{ValidGraft}(T, q, r)$ and $\mathsf{ValidPromote}(T, s)$. Furthermore, let α_{Prune}, $\alpha_{\mathsf{Promote}}$, α_{Graft} be the anchor functions resulting from $\mathsf{Prune}(T, p), \mathsf{Graft}(T, q, r)$, $\mathsf{Promote}(T, s)$ respectively. Following lemma 5.4.1, 5.4.6 and 5.4.9 there are the following anchor distances:

$$\begin{aligned} \mathsf{Dist_a}(\alpha, \alpha_{\mathsf{Prune}}) &= 1 \\ \mathsf{Dist_a}(\alpha, \alpha_{\mathsf{Graft}}) &= 1 \\ \mathsf{Dist_a}(\alpha, \alpha_{\mathsf{Promote}}) &= |\mathsf{Node}(p)| + |\mathsf{Node}(\mathsf{Hook}(\mathsf{Fact}(p)))| \end{aligned}$$

□

Next the *mutation distance* is introduced. Let $\mathsf{M} = \langle \mathsf{S}, \mu_1, ..., \mu_n \rangle$ be a mutation system over solution space S, and let x and y be elements of S. Then the mutation distance $\mathsf{Dist_m}(x, y)$ is defined as the length of the shortest mutation path in M from x to y. This is equal to the minimal number of steps needed to mutate x into y, using the mutation operators $\mu_1, ..., \mu_n$.

Example 5.5.2

Let $\mathcal{M}_r = \langle \mathcal{S}_r, \mathsf{Join}, \mathsf{Split} \rangle$ be the mutation system for relational representations introduced in section 5.2. For the running example the mutation distance between elements of $\mathcal{S}_r$ is easily determined from figure 5.10. For instance:

$$\begin{aligned} &\mathsf{Dist_m}(R_1, R_1) = 0 \\ &\mathsf{Dist_m}(R_1, R_3) = 1 \\ &\mathsf{Dist_m}(R_5, R_6) = 2 \\ &\mathsf{Dist_m}(R_1, R_7) = 3 \end{aligned}$$

□

Depending on the mutation operators the mutation distance may be symmetric, but this is not necessarily the case. Mutation system $\mathcal{M}_r$ clearly has the following property:

Lemma 5.5.1 The mutation distance in $\mathcal{M}_r$ is symmetric.

The above lemma is a direct consequence of the fact that mutation Split is defined as the inverse of mutation Join. For tree representations a similar property holds:

Lemma 5.5.2 The mutation distance in $\mathcal{M} = \langle \mathcal{S}, \mathsf{Prune}, \mathsf{Graft}, \mathsf{Promote} \rangle$ is symmetric.

Proof:
Mutations Prune and Graft are each others inverse (theorem 5.4.3) and mutation Promote describes its own inverse (theorem 5.4.5). □

5.5.3 Constructing recursive nested tables via mutation

Although the nested tables discussed so far provide a powerful mechanism for implementing database structures, they do not allow *recursion*. In this section we describe how the representation mechanism from chapter 3 is related to the notion of recursive nested tables.

This notion is based on the following observations. In a nested table (or nested relation) an attribute may be a table itself ([41], [136], [138]). The interpretation of a tree representation as a nested table is based on tree traversal from the root downwards (see section 3.2.5). As a consequence, a special situation occurs if a node belongs to its own descendants. Then there is a cycle. Note that cyclic representations are *not* within the internal solution space $\mathcal{S}$, since elements of $\mathcal{S}$ are required to consist of trees (see chapter 3).

We first consider the construction of such cycles in a given representation $T = \langle \mathcal{N}, E, \ell \rangle$, and then give an example. Let $p \in m \in \mathcal{N}$ be a hook and let $q \in n \in \mathcal{N}$ be a type related predicator, such that $n \in \mathsf{DescNodes}(p)$. Then q is not a valid application point for grafting predicator p. Suppose Graft' is a graft mutation accepting q as a valid application point for grafting predicator T. Now mutation $\mathsf{Graft}'(T, p, q)$ forces p to be moved downwards within its own descendants. So the resulting representation contains a cycle.

Example 5.5.3
Recall the tree representation T from figure 5.11 for the running information structure (figure 5.1). Let $T' = \mathsf{Prune}(T, 3)$ be the representation with root nodes $\{2\}$ and $\{3\}$. Mutation $\mathsf{Graft}'(T', 3, 7)$ results in a cycle from node $\{3, 7\}$ to node $\{5, 6\}$, and then again to node $\{3, 7\}$. Figure 5.17 shows the corresponding nested tables with recursion level 0 and recursion level 1 (the recursion level is the number of extra traversals through the cycle). □

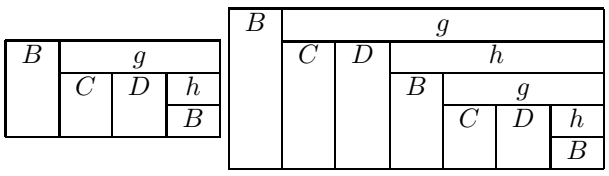


Figure 5.17: Nested tables with recursion level 0 and 1

5.5.4 The Crossover operator

In this section the Crossover operator is addressed. The aim of crossover is to combine two internal representations into a single representation containing a part of *both* original representations. This can be done by exchanging subtrees between internal representations.

Let T_1, T_2 be tree representations and let p_1, p_2 be predicators in T_1, T_2 respectively. Furthermore let $D_1 = \cup\,\mathsf{DescNodes}(p_1)$ be the set of predicators occurring in descendants of p_1 in T_1 and let D_2 be defined analogously for p_2. Now predicators p_1 and p_2 are valid application points for crossover, denoted as $\mathsf{ValidCrossover}(T_1, p_1, T_2, p_2)$ if the following property holds:

$$p_1 = \mathsf{Hook}(\mathsf{Fact}(p_1)) \ \wedge\ p_2 = \mathsf{Hook}(\mathsf{Fact}(p_2)) \ \wedge\ p_1 \sim p_2 \ \wedge\ D_1 = D_2$$

We first express the Crossover operator in terms of encoded tree representations. Let α_1, α_2 be anchor functions for representations T_1, T_2 respectively, and let p_1, p_2 be predicators s.t. $\mathsf{ValidCrossover}(T_1, p_1, T_2, p_2)$. Then crossover based on p_1, p_2 results in two new anchor functions α_1' and α_2', defined as follows:

$$\alpha_1'(x) = \begin{cases} \alpha_2(x) & \text{if } x \in D_2 \cup \{p_2\} \\ \alpha_1(x) & \text{otherwise} \end{cases}$$

$$\alpha_2'(x) = \begin{cases} \alpha_1(x) & \text{if } x \in D_1 \cup \{p_1\} \\ \alpha_2(x) & \text{otherwise} \end{cases}$$

In order to define the Crossover operator in terms of the tree representation mechanism, a generalization of the Graft mutation is used. The simple case where $\mathsf{Node}(p_1) = \{p_1\}$ can be expressed in using the merge operator from section 5.4.1:

$$\mathsf{Crossover}(T_1, p_1, T_2, p_2) \quad = \quad \Psi(T_1, p_1) \oplus \psi(T_2, p_2)$$

Note that if $T_1 = T_2$ this is similar to the Prune mutation (see section 5.4.1). More complex cases where $\mathsf{Node}(p_1) \neq \{p_1\}$ are treated as follows. The original mutation $\mathsf{Graft}(T, p_1, p_2)$ grafts $\psi(T, p_1)$ into $\mathsf{Node}(p_2)$. Now let $\mathsf{GraftCross}(T_1, p_1, T_2, p_2)$ be the graft operator that grafts $\psi(T_2, p_2)$ into $\mathsf{Node}(p_1)$ in representation T_1. Then the Crossover operator is defined by:

$$\mathsf{Crossover}(T_1, p_1, T_2, p_2) \quad = \quad \mathsf{GraftCross}\ (\Psi(T_1, p_1), q, T_2, p_2) \text{ where } q \in \mathsf{Node}(p_1) - \{p_1\}$$

It may however be hard to find valid application points for crossover, as the requirement defined above is quite restrictive (particularly $D_1 = D_2$). To solve this problem the requirement ValidCrossover can be relaxed. Then, in case violation of wellformedness occurs, either repairment (patching) or lethalization (penalization) should be performed (see e.g. [62], [45], [116]). We will not elaborate this further in this book.

5.6 Summary

In this chapter a mutation system $\langle \mathcal{S}, \mathsf{Prune}, \mathsf{Graft}, \mathsf{Promote} \rangle$ was introduced. It was shown that the mutation operators Prune, Graft and Promote are *correct* in the sense that the result of a mutation is a correct internal representation, and *complete* in the sense that all internal representations in $\mathcal{S}$ can be produced. An encoding mechanism α for these representations was introduced, so that the representations and their mutation can be described on a high level of abstraction. The embedding of index selection in the encoding mechanism was considered.

Chapter 6

Optimizer transformations

6.1 Introduction

Conceptual data models are intended to specify what kind of data must be handled by an information system ([66]). These models do not consider implementation oriented aspects, such as time/space utilization. As a result, a typical problem to be addressed during the database implementation process is *database optimization*, dealing with the question what internal representation is the 'best' for a particular application environment.

Each application environment has a corresponding optimal internal representation (or more than one) which is attuned to the requirements of that environment. The requirements can for example be as follows: many updates (not to be postponed) for 5000 hospital patients, relatively few updates (some of them may be postponed, others not) for 600 hospital co-workers, and no updates for 50 surgeons. Summarizing: one and the same conceptual data model can have different optimal internal representations, depending on the requirements of the application environment, and database optimization is concerned with finding that optimal representation.

The aim of this chapter is to identify basic objectives for database optimization, and to set up a framework in which these objectives can be treated (see also [17], [22] and [23]). This framework is based on evolution of internal representations, using the mutation systems from the previous chapter. Optimization of response time and storage space will gain extra attention.

This chapter is organized as follows. First database optimization drivers and constraints are considered (section 6.2). Then database profiles are considered in section 6.3. These profiles can be used for computing the expected response time and the expected storage space requirements for a given internal representation. Section 6.4 describes such a cost model. Finally, in section 6.5 a general framework for (evolutionary) optimization is discussed, which can be used for different drivers and constraints. The chapter is concluded with a summary and outlook.

6.2 Basic database optimization drivers

In this section basic database optimization drivers are considered. Special emphasis is given to optimization of response time and storage space.

6.2.1 Framework for database optimization

Database optimization may be driven by different objectives, such as structural properties, integrity constraints or time/space cost functions. For a given objective, there may be a constraint expressed in terms

of another objective. Such a constraint then is used to explicitly exclude certain representations from the optimization process. As an example, when optimizing the response time of a CD-ROM database application, there exists a strict storage capacity threshold. The table in figure 6.1 presents further examples of database optimization drivers under different constraints. These examples will be shortly addressed below. More details will be discussed in the next sections.

approach	optimization driver					constraint				
	structure	integrity	sensitivity	time	space	structure	integrity	sensitivity	time	space
ONF [102]	×					×	×			
INDP [33]		×				×	×			
URSI [4]			×	×	×	×				
INSEL [110]				×	×	×				×
CD-ROM [38]				×		×				×
REALT [61]				×		×			×	
MMDB [51]					×	×				×

Figure 6.1: Examples of database optimization drivers and constraints

All examples considered here are constrained by *structure*, since it must be guaranteed that the result of the optimization process is a *correct* internal representation for the conceptual model under consideration.

The first line of the table in figure 6.1 contains the Optimal Normal Form (ONF [102], [101]), where the number of tables is minimized under the condition that the representation is at least in 5th normal form. So, for ONF the optimization process is driven by a structural property (number of trees or tables), while it is constrained by integrity constraints from the conceptual model. The ONF requirement and the question whether particular algorithms indeed produce representations in ONF is examined in detail in [124], [133] and [134].

Next, inclusion dependencies (INDPs) are considered. Minimization of inclusion dependencies is a typical case of integrity-driven optimization (e.g. [33], [32]). The number of inclusion dependencies cannot be reduced to zero, as this would introduce anomalies in the form of e.g. redundancy. So optimization here is driven by as well as constrained by integrity.

Other important objectives are response time and storage space. Different approaches can be identified, such as time optimization without space threshold (e.g. [4]) and time optimization with space threshold (e.g. CD-ROM [38]). In index selection (INSEL) response time is optimized, under the condition that no storage space is unnecessarily wasted ([110]). Algorithms for index selection may also consider a storage capacity threshold (see e.g. [118]).

Time optimization with a strict time threshold occurs in the design of databases used in real-time systems. Problems with the choice of data(base) structures for real time databases are addressed in e.g. [61] and [127]. The problem of space optimization with a space threshold occurs for example during the design of data structures for (parts of) databases residing in main memory (see e.g. [51]).

The aim of an optimization driver is usually described in terms of a fitness function $\mathsf{Fitness}_{\mho_f} : \mathsf{S} \longrightarrow \Re$, where S is some solution space of candidate representations and $\Re$ is the set of real numbers. The function $\mathsf{Fitness}$ is controlled by the parameters in $\mho_f$. The database optimization problem is now formulated as follows. *Find* $x \in \mathsf{S}$ *with:*

$$\mathsf{Fitness}_{\mho_f}(x) = \max \left\{ \mathsf{Fitness}_{\mho_f}(y) \;\middle|\; y \in \mathsf{S} \right\}$$

$\mho_f$ contains parameters used for e.g. constraint specification. In this context, the term 'constraint' refers to constraints that exclude certain representations from the optimization process. These constraints may be related to the integrity constraints occurring in the conceptual data model, but this is not necessarily true. The storage capacity threshold in CD-ROM database applications is an example of a constraint which is not related to integrity constraints from the conceptual model.

6.2.2 Structure-driven optimization

In *structure-driven* optimization the fitness function only depends on the internal representation at hand. It does not consider integrity constraints occurring in the conceptual data model, nor does it consider

information about database usage (e.g. expected query patterns). First an example is given where the fitness function is *not* parameterized ($\mho_f = \varnothing$).

Example 6.2.1

The optimization process may be driven by the height of the trees in internal representations: $\mathsf{Fitness}(T) = \mathsf{Height}(T)$. *An optimizer driven by this fitness function will search for representations consisting of trees with maximum height.* □

Other possibilities are $\mathsf{Breadth}(T)$ and $\mathsf{NrTrees}(T)$. An objective threshold K can be used here by specifying fitness parameters $\mho_f = \{K, f\}$ where $1 \leq K \leq |\mathcal{F}|$ and function $f : \Re \longrightarrow \Re$ specifies how representations exceeding K are treated.

Example 6.2.2

Consider the fitness function from the previous example. Suppose this function is used with parameters $\mho_f = \{K, f\}$, *where function* f *is defined by* $f(x) = 0$ *for all* $x \in \Re$*:*

$$\mathsf{Fitness}_{\{K,\,f\}}(T) = \begin{cases} 0 & (\text{if } \mathsf{Height}(T) > K) \\ \mathsf{Height}(T) & (\text{otherwise}) \end{cases}$$

An optimizer driven by this fitness function will search for representations with maximum attainable height not exceeding K. □

However, structure-driven optimization is usually constrained by other properties. These properties may be related to the integrity constraints $\mathcal{C}$ in the conceptual data model, or to the time/space utilization of the representation under consideration. The following example is based on $\mho_f = \{\mathcal{C}\}$.

Example 6.2.3

Let $\mathsf{Anomaly}$ *be a boolean operator, checking whether particular anomalies occur in representation* T *(e.g. update anomalies). Then, minimization of the number of trees under the condition that there are no anomalies can be based on the following fitness function:*

$$\mathsf{Fitness}_{\{\mathcal{C}\}}(T) = \begin{cases} 0 & (\text{if } \mathsf{Anomaly}(T, \mathcal{C})) \\ \frac{1}{\mathsf{NrTrees}(T)} & (\text{otherwise}) \end{cases}$$

□

The fitness function described in example 6.2.3 typically describes the fitness of candidate representations in cases where normalization is combined with structure-driven optimization. A well-known example is the so-called Optimal Normal Form (ONF). According to [102] a relational representation is in ONF if it is at least in 5th Normal Form and the number of relations is minimal. Further details about ONF are found in [124], [133] and [134].

6.2.3 Integrity-driven optimization

Integrity constraints may become objective of optimization by counting the number of anomalies or dependencies (of a particular kind) in a representation. Here the fitness parameters $\mho_f = \{\mathcal{C}\}$ are directly used for *evaluating* the fitness function, rather than for *constraining* it (see example 6.2.3).

Example 6.2.4

Assume an objective NrDep *that expresses the number of dependencies of a particular kind. Then the fitness function may be as follows:*

$$\mathsf{Fitness}_{\{\mathcal{C}\}}(T) = \frac{1}{\mathsf{NrDep}(T, \mathcal{C})}$$

□

Note that the fitness function in the above example is used for *undesirable* dependencies, since their number is *minimized*. Also it is possible to use a fitness function which is driven as well as constrained by integrity constraints.

Example 6.2.5

Assume an objective NrDep *and a boolean operator* Anomaly *checking whether particular anomalies occur (e.g. update anomalies). Then, minimization of the number of dependencies under the condition that there are no anomalies can be based on the following fitness function:*

$$\mathsf{Fitness}_{\{\mathcal{C}\}}(T) = \begin{cases} 0 & (\text{if } \mathsf{Anomaly}(T,\mathcal{C})) \\ \frac{1}{\mathsf{NrDep}(T,\mathcal{C})} & (\text{otherwise}) \end{cases}$$

□

This approach is applied in e.g. [33] and [32], where the number of inclusion dependencies is minimized. The motivation for minimizing the number of inclusion dependencies between trees (or tables) is that these dependencies complicate update transactions.

The fitness functions considered above provide a global framework for optimization driven by integrity constraints. Usually, different classes of dependencies are considered, and within each class there may be different kinds of update overhead.

6.2.4 Sensitivity-driven optimization

An internal representation is *sensitive* to changes in the conceptual schema, if minor changes of the conceptual schema result in major changes of the internal representation. So sensitivity is an undesirable property and may therefore be objective in database optimization. This is applied in [4].

6.2.5 Time/space-driven scalar optimization

In time/space optimization there are two objectives: response time and storage space. Several problems are related to these objectives. As an example, it is well-known that the properties of Time and Space express *conflicting*, rather than cooperative objectives. This results in the so-called time/space trade-off. Another problem is the fact that Time and Space are noncommmensurable. So it is not possible to compare these objectives directly: bytes cannot be compared to seconds. This complicates the question how to find an appropriate internal representation.

One way of treating optimization problems with more than one objective is *scalarization*, also called weighting of several objectives. Other approaches will be discussed in the next sections.

In scalarization ([160]) the objectives for optimization are combined in a single fitness function. So for time/space optimization this leads to the following:

$$\mathsf{Fitness}(T) = f(\mathsf{Time}(T), \mathsf{Space}(T))$$

Example 6.2.6

A typical approach to specify a scalar function is to compute a weighted sum. Let $k_1, k_2 \geq 0$ be weight coefficients ($k_1, k_2 \in \mho_f$). The fitness function then is defined as follows:

$$\mathsf{Fitness}(T) = \frac{1}{k_1 \times \mathsf{Time}(T) + k_2 \times \mathsf{Space}(T)}$$

□

An alternative for the weighted sum from example 6.2.6 is to multiply Time and Space. Then the coefficients k_1 and k_2 lose their importance. For the weighted sum we consider the choice of k_1 and k_2 in more detail.

On the one hand the coefficients k_1 and k_2 can be related to each other (e.g. $k_1 = \beta$, $k_2 = 1 - \beta$ with $\beta \in [0, 1]$). On the other hand (if unrelated) these coefficients can be used to compute the *cost* of an internal representation. If $k_1 = \gamma_t$ is the unit cost of (response) time and $k_2 = \gamma_s$ is the unit cost of the storage media to be used, then the total cost is defined by $\gamma_t \times \mathsf{Time}(T) + \gamma_s \times \mathsf{Space}(T)$. The parameters considered so far are presented in the table in figure 6.2.

parameter	explanation
β	weight coefficient for time optimization
γ_t	unit cost of response time
γ_s	unit cost of storage media

Figure 6.2: Parameters for scalarization

Combination of all parameters leads to the following general fitness function:

$$\mathsf{Fitness}(T) = \frac{1}{\beta \times \gamma_t \times \mathsf{Time}(T) + (1 - \beta) \times \gamma_s \times \mathsf{Space}(T)}$$

Finally the definition of scalar optimality is given. Let T be a representation in solution space $\mathcal{S}$. Representation T is called a *scalar optimum* in $\mathcal{S}$ under weight coefficient β, denoted as $\mathsf{ScalarOpt}(T, \beta, \mathcal{S})$, if there are no representations with a higher fitness:

$$\forall_{T' \in \mathcal{S}} \left[\mathsf{Fitness}(T) \geq \mathsf{Fitness}(T')\right]$$

6.2.6 Time/space optimality

In this section we consider optimality with respect to response time and storage space. Let T be a representation in solution space $\mathcal{S}$. Representation T is called a *selfish time optimum* in $\mathcal{S}$, denoted as $\mathsf{SelfishTimeOpt}(T, \mathcal{S})$, if there are no representations with a better response time:

$$\forall_{T' \in \mathcal{S}} \left[\mathsf{Time}(T) \leq \mathsf{Time}(T')\right]$$

Selfish space optimality is defined in a similar way. Representation T is called a *selfless time optimum* in $\mathcal{S}$, if there are no representations with equal space but better time:

$$\forall_{T' \in \mathcal{S}} \left[\mathsf{Space}(T) = \mathsf{Space}(T') \Rightarrow \mathsf{Time}(T) \leq \mathsf{Time}(T')\right]$$

This is denoted as $\mathsf{SelflessTimeOpt}(T, \mathcal{S})$. Again, selfless space optimality is defined in a similar way. The general fitness function for scalar optimization from section 6.2.5 guarantees selfless time/space optimality:

Lemma 6.2.1 $\forall_{\beta \in (0,1)} \left[\mathsf{ScalarOpt}(T, \beta, \mathcal{S}) \Rightarrow \mathsf{SelflessTimeOpt}(T, \mathcal{S}) \wedge \mathsf{SelflessSpaceOpt}(T, \mathcal{S})\right]$

Proof:

By contradiction. Suppose representation T is a scalar optimum which is *not* selfless time optimal. Then there is a T' with equal space but better time. So $\mathsf{Fitness}(T') > \mathsf{Fitness}(T)$ since weight coefficient $\beta \neq 0$. Consequently T is not a scalar optimum (contradiction). In the same way it is shown that representation T is selfless space optimal. □

It is easily seen that scalar optimality under $\beta \in (0, 1)$ does not guarantee selfish time/space optimality. Let representation T be a scalar optimum in solution space $\mathcal{S}$. Then there may be other representations with better time, since $\beta \neq 1$. Then T is not selfish time optimal. For the same reason T is not guaranteed to be selfish space optimal ($\beta \neq 0$).

6.2.7 Time/space-driven vector optimization

In *vector optimization* the optimization process is driven by more than one objective, without combining these criteria in a single fitness function. Then two representations T_1 and T_2 with properties $\mathsf{Time}(T_1)$, $\mathsf{Space}(T_1)$, $\mathsf{Time}(T_2)$ and $\mathsf{Space}(T_2)$ can only be compared by comparing their properties directly. This is done using a variety of binary relations.

Let T_1, T_2 be internal representations. Then representation T_2 is called *better than* representation T_1, denoted as $T_2 < T_1$, if there is improvement on both criteria:

multiple improvement: $\mathsf{Time}(T_2) < \mathsf{Time}(T_1) \wedge \mathsf{Space}(T_2) < \mathsf{Space}(T_1)$

Example 6.2.7

Let x be an internal representation with average response time T_x and storage space requirements S_x. Suppose another representation y is obtained (e.g. by means of mutation) with response time $T_y < T_x$ and storage space $S_y < S_x$. The situation is shown in figure 6.3. In this case representation y is better than representation x: $y < x$. □

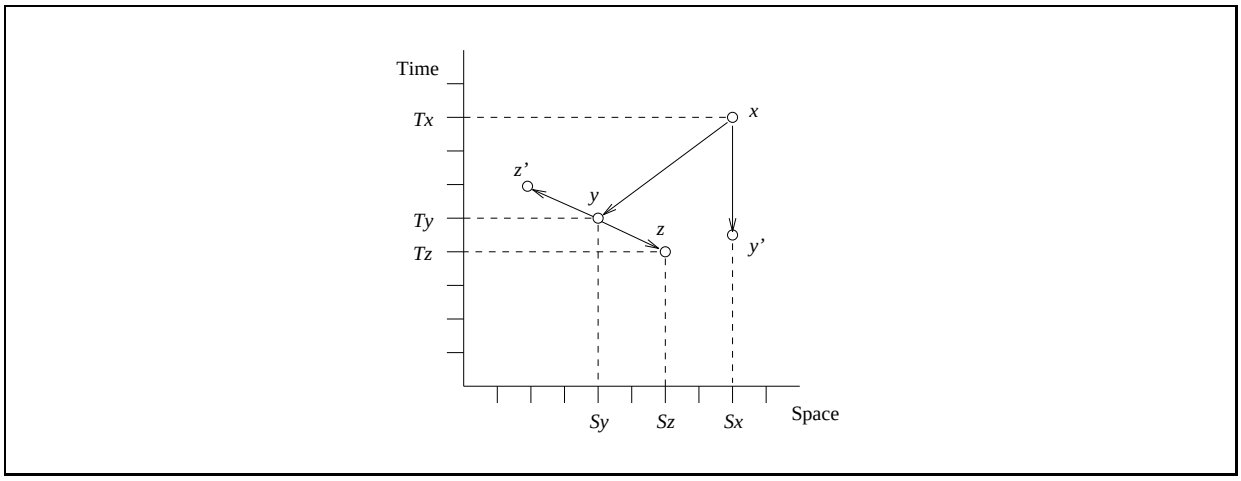


Figure 6.3: Representations with different time/space properties

Binary relation $<$ is irreflexive and transitive. It is neither symmetric nor antisymmetric. The definition of the reflexive counterpart ($T_1 \leq T_2$) is analogous. So in figure 6.3 $y' \leq x$ (and of course $y \leq x$). Representation T_2 is called *an alternative for* representation T_1, denoted as $T_2 <> T_1$, if there is improvement on at least one criterion:

single improvement: $\mathsf{Time}(T_2) < \mathsf{Time}(T_1) \vee \mathsf{Space}(T_2) < \mathsf{Space}(T_1)$

Example 6.2.8

Consider representation y in figure 6.3. Suppose another representation z is obtained with response time $T_z < T_y$ and storage space $S_z > S_y$. In this case representation z is an alternative for representation y: $z <> y$. Furthermore $y <> z$ and $z < x$. □

Relation $<>$ is irreflexive. Following the above definitions we get $T_2 < T_1 \Rightarrow T_2 <> T_1$. Note that in figure 6.3 representation z' is also an alternative for representation y. A preference for mutation $y \rightarrow z$ or $y \rightarrow z'$ will depend on the requirements of the particular application environment. This will be considered in a more general context in section 6.2.8.

The fact that representations T_1 and T_2 have equal response time as well as storage space is denoted as $T_1 \equiv T_2$, rather than $T_1 = T_2$. Relation $\equiv$ is reflexive, symmetric and transitive. Here we have $T_2 \equiv T_1 \Rightarrow T_2 \not\prec T_1$, expressing that equality excludes multiple improvement (which is obvious). The same holds for single improvement.

Next we consider domination, a frequently used notion in multi-objective optimization ([125], [160]). Representation T_2 is said to *dominate* representation T_1, denoted as $T_2 \prec T_1$, if $T_2 \leq T_1$ and $T_2 <> T_1$.

Example 6.2.9
In figure 6.3 we have $y \prec x$ and $z \prec x$, but also $y' \prec x$. □

The condition for domination can be rewritten as $T_2 \prec T_1 \Leftrightarrow T_2 \leq T_1 \wedge T_2 \not\equiv T_1$. Following [125] and [160] we now define the set of nondominated representations. This set is called the *Pareto Optimal* set of representations $\mathbf{PO}(\mathcal{S}) \subseteq \mathcal{S}$, after V. Pareto:

$$\mathbf{PO}(\mathcal{S}) = \{T \in \mathcal{S} | \neg \exists_{T' \in \mathcal{S}} [T' \prec T]\}$$

Why is the set **PO** of interest for time/space optimization? The reason is that in time/space optimization, the optimum is *always* in **PO**. This holds for different time/space optimization approaches. Two basic possibilities will be discussed in the next section: using preferences or thresholds.

This section is concluded with the property that scalar optimality guarantees Pareto optimality:

Lemma 6.2.2 $\forall_{\beta \in (0,1)} [\mathsf{ScalarOpt}(T, \beta, \mathcal{S}) \Rightarrow T \in \mathbf{PO}(\mathcal{S})]$

Proof:
By contradiction. Suppose representation T is a scalar optimum which is *not* in **PO**. Then there is a representation $T' \in \mathcal{S}$ such that $T' \prec T$. This means that (a) $T' \leq T$ and (b) $T' <> T$.

According to (b) we have $\mathsf{Time}(T') < \mathsf{Time}(T)$ or $\mathsf{Space}(T') < \mathsf{Space}(T)$. If however $\mathsf{Time}(T') < \mathsf{Time}(T)$ then it follows from (a) that $\mathsf{Space}(T') = \mathsf{Space}(T)$. This leads to $\mathsf{Fitness}(T') > \mathsf{Fitness}(T)$ since $\beta \neq 0$. But this is not possible because T is a scalar optimum. For the same reason $\mathsf{Space}(T') < \mathsf{Space}(T)$ is not possible ($\beta \neq 1$). As a result, $T \in \mathbf{PO}$. □

6.2.8 Preferences and thresholds in time/space optimization

In this section preferences and thresholds in optimizing response time and storage space are considered. In *multi-objective optimization with time preference* a representation T is optimal, if selfish time optimality is guaranteed:

$$\mathsf{SelfishTimeOpt}(T, \mathcal{S}) \wedge \mathsf{SelflessSpaceOpt}(T, \mathcal{S})$$

Here first aim is optimization of response time. In the upper left corner of figure 6.4 optimization with time preference is illustrated. There is a selfish time optimum, whereas selfish space optimality can not be guaranteed. Similarly the lower left corner illustrates optimization under space preference:

$$\mathsf{SelflessTimeOpt}(T, \mathcal{S}) \wedge \mathsf{SelfishSpaceOpt}(T, \mathcal{S})$$

Multi-objective optimization with time preference is applied in e.g. [4]. An internal representation is called optimal ([4]), if (1st preference) *update* time is minimal, (2nd preference) *retrieval* time is minimal, (3rd

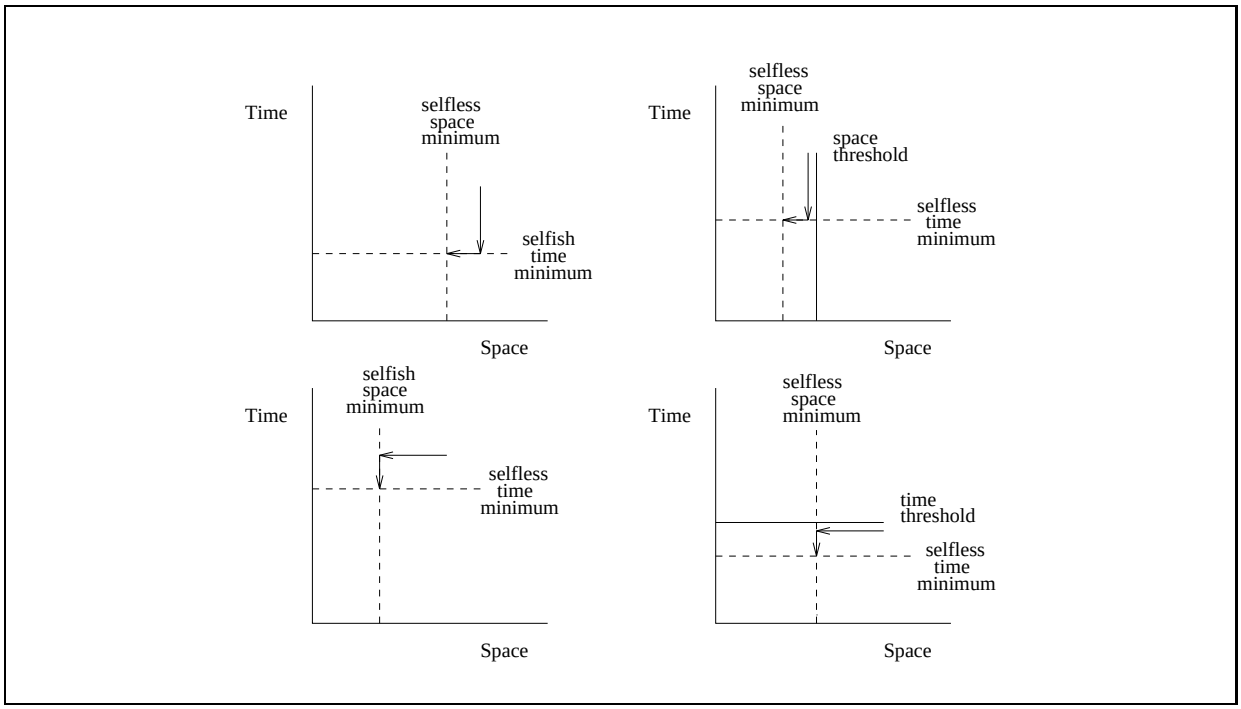


Figure 6.4: Selfish optimization (preference, left) and selfless optimization (threshold, right)

preference) *storage space* is minimal, and (4th preference) the representation is *insensitive* as much as possible to changes in the underlying conceptual data model[1].

Another example comes from the area of *index selection* (e.g. [118], [110]). In most cases algorithms for index selection perform multi-objective optimization with time preference. Here retrieval time is improved by adding efficient access paths, which will cost extra storage space and may lead to worse update behaviour. An index configuration with optimal time is to be determined (selfish time optimality), such that no storage space is unnecessarily wasted (selfless space optimality). There may be a threshold with respect to storage space. Then access paths are added as long as storage space is below the threshold. Optimization with thresholds is discussed in more detail below.

The following lemma expresses that time preference optimality guarentees Pareto optimality.

Lemma 6.2.3 $\mathsf{TimePrefOpt}(T, \mathcal{S}) \Rightarrow T \in \mathbf{PO}(\mathcal{S})$

Proof:

By contradiction. Suppose $T \notin \mathbf{PO}$. Then there is a $T' \in \mathcal{S}$ such that (a) $T' \leq T$ and (b) $T' <> T$. First we consider (a). This leads to $\mathsf{Time}(T') = \mathsf{Time}(T)$, since time preference optimality guarantees selfish time optimality (by definition).

Next we consider (b): $T' <> T$ means $\mathsf{Time}(T') < \mathsf{Time}(T)$ or $\mathsf{Space}(T') < \mathsf{Space}(T)$. According to (a), time improvement is not possible. So $\mathsf{Space}(T') < \mathsf{Space}(T)$. But this is not possible, since $\mathsf{Time}(T') = \mathsf{Time}(T)$ by (a) and T is selfless space optimal by definition of $\mathsf{TimePrefOpt}$. This leads to a contradiction. As a consequence $T \in \mathbf{PO}$. □

A similar property holds for space preference optimality. In (single-objective) *time optimization with constrained space* a representation T is optimal for space threshold K, if the following property holds:

[1]This explains why the approach is designated URSI in the table in figure 6.1.

$$\mathsf{Space}(T) \leq K \ \wedge \ \forall_{T' \in \mathcal{S}} [\mathsf{Space}(T') \leq K \Rightarrow \mathsf{Time}(T) \leq \mathsf{Time}(T')]$$

This is denoted as $\mathsf{SpaceConstrTimeOpt}(T, K, \mathcal{S})$. Here an important requirement is the space maximum K. In multi-objective optimization with space threshold K a representation T is optimal, if it has the following property:

$$\mathsf{SpaceConstrTimeOpt}(T, K, \mathcal{S}) \wedge \mathsf{SelflessSpaceOpt}(T, \mathcal{S})$$

This definition is illustrated in the upper right corner of figure 6.4. Optimization with a time threshold is illustrated in the lower right corner. In both cases there is selfless time optimality and selfless space optimality. The optima in multi-objective optimization with thresholds are also Pareto optimal:

Lemma 6.2.4 $\mathsf{SpaceConstrTimeOpt}(T, K, \mathcal{S}) \wedge \mathsf{SelflessSpaceOpt}(T, \mathcal{S}) \Rightarrow T \in \mathbf{PO}(\mathcal{S})$

Proof:

By contradiction. Suppose $T \notin \mathbf{PO}$. Then there is a $T' \in \mathcal{S}$ such that (a) $T' \leq T$ and (b) $T' <> T$. First we consider (a). This leads to $\mathsf{Space}(T') \leq K$ and thus $\mathsf{Time}(T) \leq \mathsf{Time}(T')$ by definition of space constrained time optimality. Another application of (a) now results in $\mathsf{Time}(T) = \mathsf{Time}(T')$.

Therefore, since T is selfless time optimal $\mathsf{Space}(T) \leq \mathsf{Space}(T')$. This is in contradiction with (b) since $\mathsf{Time}(T) = \mathsf{Time}(T')$. As a consequence $T \in \mathbf{PO}$. □

Space constrained time optimization occurs for example in the design and optimization of CD-ROM database applications, where the storage space capacity is restricted. Basic file organizations and access methods for CD-ROM are discussed in [38]. The fact that the storage capacity of a CD is huge but still limited is further addressed in [91], where emphasis is given to compression and encryption techniques.

The definitions given in this section lead to the matrix in figure 6.5. This matrix should be read as follows. When there is a threshold on one of the objectives, optimal internal representations are selfless time and selfless space optimal (upper left corner). The same holds for time/space scalar optimization. When there is a preference for one objective over the other, optimal representations are selfish optimal with respect to the objective with preference, and selfless optimal for the other objective.

	selfless space optimal	selfish space optimal
selfless time optimal	space threshold time threshold scalar	space preference
selfish time optimal	time preference	Utopia

Figure 6.5: Selfless/selfish time/space optimality

The situation where selfish time optimality is combined with selfish space optimality rarely occurs. This is called a Utopian situation ([165], [160]). This situation is characterized by $\forall_{T_1, T_2 \in \mathbf{PO}} [T_1 \equiv T_2]$ which means that even single improvement is impossible.

6.2.9 Clustering algorithms

Basically, finding a suitable internal representation for a given conceptual model is a matter of *clustering*. Different kinds of clustering algorithms can be used for this purpose. The following classification of clustering algorithms is based on [163]:

graph theoretic algorithms: Clustering is performed on the basis of graph theoretic properties and algorithms.

construction algorithms: Clustering is performed in a single phase.

evolutionary algorithms: Clustering is performed in iterations.

hierarchical algorithms: Clustering is performed in a hierarchic way. Consequently, the resulting clusters also have a hierarchic nature.

In section 6.5 a general framework for the evolutionary approach is given.

6.3 Characterization of application environment

In order to characterize the environment in which the database under development will operate, several kinds of database profiles can be used. We consider *access profiles*, *data profiles* and *device profiles*. These profiles will be discussed below and will be illustrated for the information structure shown in figure 6.6.

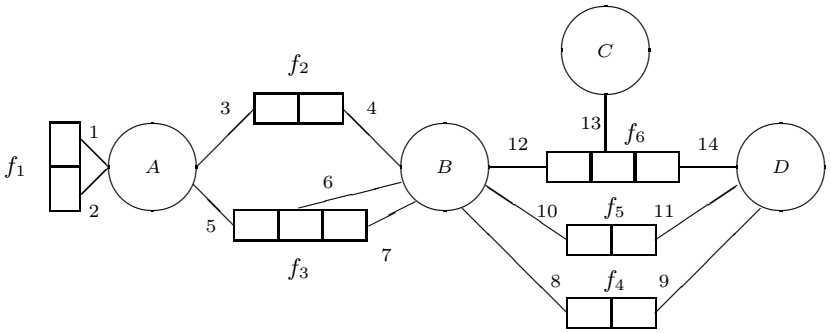


Figure 6.6: Example information structure for access and data profiles

6.3.1 Access profiles

In this section access profiles are considered. An access profile contains information about the user requests to be performed, including retrieval as well as update requests. The intention of an access profile is to specify the frequency of different (classes of) requests. This frequency may be absolute or relative. In some cases it is more convenient to interpret the values in an access profile as the probability or priority of the requests.

We describe two kinds of access profiles: *path profiles* and *affinity profiles*. A path profile specifies paths through an information structure, along with their frequency. Each path is defined in terms of predicators from the information structure. A path profile $\langle \mathsf{Paths}, \mathsf{Freq} \rangle$ consists of a set of paths and a frequency function $\mathsf{Freq} : \mathsf{Paths} \longrightarrow \mathbb{N}$. Two predicators p and q occurring as a sequence in a path should be type related or belong to the same fact type:

$$p \sim q \ \vee \ \mathsf{Fact}(p) = \mathsf{Fact}(q)$$

Example 6.3.1

Figure 6.7 shows a path profile, specifying paths through the information structure from figure 6.6 along with their relative frequency. Note that the latter two paths in this profile are overlapping and that the highest priority is given to joining fact types f_3 and f_6. □

Relative frequency	Path through information structure
1	p_3, p_4, p_{10}, p_{11}
5	p_6, p_7, p_{12}, p_{13}
4	$p_1, p_2, p_5, p_6, p_8, p_9$

Figure 6.7: Example path profile for given information structure

Path profiles can be further generalized by specifying additional properties of the paths under consideration. For example, the kind of access (retrieval or update) of the paths under consideration can be added. Other examples are maximum response time and criticality of paths. Criticality here can be interpreted as the severity of the effect when the system fails to respond in time. More details about criticality are found in [119].

A similar approach is used in [122] and [103], where an access profile consists of transactions (instead of paths) and attributes (instead of predicators). More details about paths through information structures are found in [81] and [49].

Next we consider *affinity profiles*. An affinity profile is a $p \times p$ matrix R with $p = |\mathcal{P}|$, where $R(i, j)$ gives the affinity between predicator p_i and predicator p_j. The affinity between predicators p_i and p_j can be interpreted as the frequency of path traversal from p_i to p_j. Then the matrix R can be computed as follows:

$$R_{\langle \mathsf{Paths},\mathsf{Freq}\rangle}(i,j) = \sum_{\text{sequence } p_i,p_j \text{ occurs in } P \in \mathsf{Paths}} \mathsf{Freq}(P)$$

An alternative way of computing an affinity profile can be based on so-called atomic operations. Atomic operations are used in e.g. [132], though this approach does not compute affinity profiles.

6.3.2 Data profiles

In this section *data profiles* are considered. A data profile contains a quantification of the database contents. The first property to quantify is the *size* of the database contents. A size profile $\langle \mathsf{NrInst}, \mathsf{LabelSize}\rangle$ consists of the following functions:

$$\mathsf{NrInst} : \mathcal{O} \to \mathbb{N}$$
$$\mathsf{LabelSize} : \mathcal{L} \to \mathbb{N}$$

The function NrInst specifies the expected number of instances for each object type, while LabelSize specifies the expected size of the labels of a given type.

Example 6.3.2

Figure 6.8 shows an example function NrInst*, specifying the relative number of instances for each object type in the running example (figure 6.6).* □

Besides the size of the database contents, a data profile may specify several other properties. The following quantitative descriptors can be distinguished ([109]):

1. descriptors of central tendency such as mode, mean and median.

Object type:	A	B	C	D	f_1	f_2	f_3	f_4	f_5	f_6
Relative nr. of instances:	3	2	2	1	1	2	3	1	2	1

Figure 6.8: Example data profile for running example

2. descriptors of dispersion such as range (maximum and minimum), variance and standard deviation.
3. descriptors of size such as the number of instances (cardinality) and the number of distinct values.
4. descriptors of frequency distribution such as normality, uniformity, and value intervals and counts.

6.3.3 Device profiles

In this section *device profiles* are considered. A device profile is a characterization of the platform the database will run on. This includes properties of software as well as hardware.

Typical software parameters are related to the target DBMS. As an example, a device profile may specify the kinds of indices (access paths) the DBMS supports and the query optimization strategies that are applied. Typical hardware parameters are related to storage media, e.g. seek time, rotational delay and block transfer time. Another hardware parameter is the storage capacity, though this capacity may also be restricted by the software used.

The approach in [118] uses access profiles in combination with device profiles. The device profile here specifies the total number of disk blocks available for the database, the size of a block, the cost of a block, average access time and cost of access.

6.3.4 Techniques for database profile acquisition

In this section the question how to obtain profiles is addressed. Profiles may be obtained during information system analysis and during information system maintenance. These possibilities are described below.

During information system analysis the system and its environment are described, for example using (conceptual) data models and (conceptual) process or transaction models. Profiles may be obtained here by characterizing user requests in an access profile and database contents in a data profile. If there is a coupling between data models and process or transaction models, the access profile can be defined based on this coupling. Then the retrieval and update behaviour for particular parts of the data model should be analysed and quantified according to the associated process model. Further details about the coupling between data models and process models for information systems may be found in e.g. [77], [76], [78] and [128].

During system maintenance, profiles can be computed from a log-book describing the user requests that have been performed in a specific time interval. Besides an access profile, these requests can also be used to estimate an average data profile (particularly the update requests). A more direct way to obtain a data profile is to examine the database contents at particular points in a given time interval.

In [117] a data profile is extracted from an operational database for the purpose of estimating the size of query results in the context of query optimization. In [109] data profiles are considered and corresponding manipulation operators are described, such as BuildProfile, UpdateProfile and EstimateProfile.

In [119] profiles are considered for the purpose of software-reliability engineering. A step by step method for developing profiles is discussed, and different kinds of profiles are defined, such as customer profiles, user profiles, system-mode profiles and functional profiles. The use of these profiles for other purposes is considered, including performance engineering.

Finally there is the possibility of a library of predefined profiles for different types of applications in different environments, e.g. personnel information systems in an environment with many updates for particular parts

of the personnel conceptual data model. Another example of applications in a specific environment is a database for image processing (see e.g. [161]). Note that the use of predefined profiles are more or less related to benchmarking. In [64] several benchmarks are considered, including a portable benchmark for relational database systems.

6.3.5 Identity database profiles

When judging the quality of candidate internal representations, database profiles are used to characterize the target environment of the database under development. In this section we introduce so-called *identity profiles*. These profiles may be used when only a global comparison between candidate representations is required, or when it is not possible to define non-identity profiles.

First identity access profiles are considered. Let $p = |\mathcal{P}|$ and let $\langle R, U \rangle$ be an access profile consisting of a $p \times p$ retrieval matrix R and a p vector U. Entry $R(i,j)$ specifies the affinity between predicator p_i and predicator p_j, and entry $U(i)$ specifies the frequency of update operations on predicator p_i. Profile $\langle R, U \rangle$ is an identity access profile, if it has the following properties:

1. Unconnected predicators have no affinity:

$$\forall_{p,q \in \mathcal{P}} \left[p \not\sim q \ \vee \ \mathsf{Fact}(p) \neq \mathsf{Fact}(q) \Longleftrightarrow R(p,q) = 0 \right]$$

2. All connected predicators have the same affinity:

$$\forall_{p_1,p_2,q_1,q_2 \in \mathcal{P}} \left[R(p_1,p_2) \neq 0 \wedge R(q_1,q_2) \neq 0 \Longrightarrow R(p_1,p_2) = R(q_1,q_2) \right]$$

3. All predicators have the same update pattern:

$$\forall_{p,q \in \mathcal{P}} \left[U(p) = U(q) \right]$$

Next identity data profiles are considered. Let D be a data profile, defined as a k vector with $k = |\mathcal{O}|$ where $D(i)$ specifies the number of instances in the population of object type $o_i \in \mathcal{O}$. Profile D is called an identity data profile if it has the following properties:

1. All atomic object types have the same number of instances:

$$\forall_{a_1,a_2 \in \mathcal{A}} \left[D(a_1) = D(a_2) \right]$$

2. All fact types have the same number of instances:

$$\forall_{f_1,f_2 \in \mathcal{F}} \left[D(f_1) = D(f_2) \right]$$

So identity profiles assume uniformity. Consequently these profiles don't characterize the actual access patterns and database contents. For more details about the implications of assuming uniformity in database profiles, the reader is referred to [37].

6.4 Computation of objectives for optimization

In this section the computation of expected storage requirements and expected average response time is discussed. First some general considerations are given.

6.4.1 Approach to objective computation

As was mentioned in the previous section, values in profiles may be absolute or relative. In our case, relative values will suffice since we are mainly interested in *comparing* internal representations. We therefore express storage requirements in terms of abstract space units (s.u.), and average response time in terms of abstract time units (t.u.). It is stressed here that the evolution mechanism for searching through the solution space of internal representations (section 6.5) is independent of the underlying cost model for those representations. As a consequence, different cost models may be embedded in this framework. The cost model we describe is based on the following definitions:

1. $\mathcal{I}$ is a (defoliated) flattened information structure, consisting of n-ary fact types.
2. $D = \langle \mathsf{NrInst}, \mathsf{LabelSize} \rangle$ is a data profile, specifying the number of instances of object types in $\mathcal{I}$, along with the expected size of label instances.
3. A is an access profile, specifying the expected pattern of retrieval and update requests to be performed on $\mathcal{I}$.
4. T is an internal representation of $\mathcal{I}$ consisting of nodes $\mathcal{N}$, where $\mathbf{I} \subseteq \mathcal{N}$ is the set of nodes accessible by index.

6.4.2 Storage space requirements

A data profile D can be used to estimate the expected storage requirements $\mathsf{Space}_D(T)$ for a given internal representation:

$$\mathsf{Space}_D(T) = \sum_{n \in \mathcal{R}} \mathsf{DataSpace}(n, D) + \sum_{n \in \mathbf{I}} \mathsf{IndexSpace}(n, D)$$

where $\mathcal{R}$ and $\mathbf{I}$ contain all root nodes and indexed nodes, respectively. For a given entity type x the expected size ($\mathsf{LabelSize}$) of the identifying labels is denoted as $\mathsf{Size}(x)$. Then the space needed for an index on node n is restricted by the number of instances in the population of that node:

$$\mathsf{IndexSpace}(n, D) = |\mathsf{Pop}(\mathsf{Base}(n))| \times \mathsf{Size}(\mathsf{Base}(n))$$

where $\mathsf{Base}(n)$ is the common pater familias in node n. Next we consider the computation of $\mathsf{DataSpace}(n, D)$. Let n be a node with incoming edges labelled by $f_1, ..., f_k$ (see figure 6.9). If $k = 0$, then node n is a leaf node. For $k > 0$ fact type f_i is represented using the nodes $m_{i1}, ..., m_{il_i}$ being children of node n. So $l_i = |f_i| - 1$.

Now the expected storage requirements for node n is recursively defined by:

$$\mathsf{DataSpace}(n, D) = |\mathsf{Pop}(\mathsf{Base}(n))| \times \left[\mathsf{Size}(\mathsf{Base}(n)) + \sum_{1 \le j \le l_i}^{1 \le i \le k} |\mathsf{Pop}(f_i)| \times \mathsf{DataSpace}(m_{ij}, D) \right]$$

In this formula $|\mathsf{Pop}(\mathsf{Base}(n))|$ is the maximum number of tuples in node n. Each tuple t has exactly one root value in n and at most $|\mathsf{Pop}(f_i)|$ sub-tuples in child m_{ij}, nested within t ($1 \le i \le k, 1 \le j \le l_i$). The size of the root value (say of type x) is given by $\mathsf{Size}(x)$.

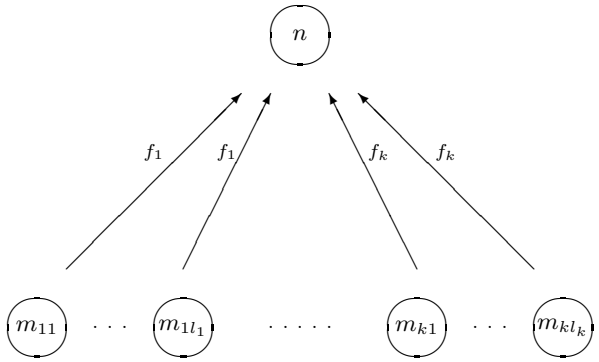


Figure 6.9: General structure of tree

6.4.3 Average response time

Next we discuss the processing of access profiles. We use access profile $A = \langle R, U \rangle$ consisting of a $p \times p$ matrix R, for the expected pattern of retrieval requests, and a p-vector U, for the expected pattern of update requests ($p = |\mathcal{P}|$). Entry U_i gives the expected number of updates for predicator p_i per time unit (e.g. per day). Entry R_{ij} gives the expected number of accesses to be performed from predicator p_i into predicator p_j. Then, the average response time of representation $\mathcal{T}$ based on profiles D and A is defined as:

$$\mathsf{Time}_{D,A}(T) = \sum_{i,j \leq p} c_{ij} \times R_{ij} + \sum_{i \leq p} d_i \times U_i$$

where c_{ij} is the cost for an access from $\mathsf{Node}(p_i)$ into $\mathsf{Node}(p_j)$, and d_i is the cost for updating $\mathsf{Node}(p_i)$.

For the computation of c_{ij} and d_i several approaches can be used. Usually a detailed estimation is given in terms of memory blocks, pages, number of I/O operations, buffer size, etc. Here a global cost estimation in terms of nodes of the internal representation will suffice. Let ξ be the average time needed for processing a single storage unit (e.g. a byte). Furthermore, let $S(y)$ be a shorthand for $\mathsf{Size}(\mathsf{Base}(p_y)) \times |\mathsf{Pop}(\mathsf{Base}(p_y))|$. Then, the cost c_{ij} for access from $\mathsf{Node}(p_i)$ into $\mathsf{Node}(p_j)$ is defined by:

$$c_{ij} = \begin{cases} \xi \times S(i) & (\mathsf{Node}(p_j) \in \mathbf{I}) \\ \xi \times S(i) \times S(j) & (\text{otherwise}) \end{cases}$$

Next we consider the cost d_i for updating predicator p_i. In the absence of an index on $\mathsf{Node}(p_i)$, the maximum number of changes will be equal to the number of instances in $\mathsf{Base}(p_i)$. An index on $\mathsf{Node}(p_i)$ will double this, as a result of index maintenance:

$$d_i = \begin{cases} \xi \times S(i) & (\mathsf{Node}(p_j) \notin \mathbf{I}) \\ 2 \times \xi \times S(i) & (\text{otherwise}) \end{cases}$$

6.5 Evolutionary algorithms for database optimization

In this section a framework for evolutionary database optimization is introduced. The framework is based on the mutation operators from the previous chapter.

6.5.1 Searching the internal solution space by evolution

A conceptual data model has a large number of candidate internal representations. This number is exponential in the number of fact types contained in the conceptual model (see chapter 3). As a consequence, the enumeration of all candidate internal representations is far from attractive. Therefore a flexible mechanism for navigating through the internal solution space is introduced. This mechanism is embedded in a general framework for *evolutionary algorithms*.

The intention of evolutionary algorithms for internal representations is to enable a database designer to generate different internal representations for a given conceptual data model, and to manipulate these (preliminary) representations, i.e. to modify them and to make them evolve into more desirable designs. The situation is illustrated in figure 6.10. Each small circle corresponds to an internal representation.

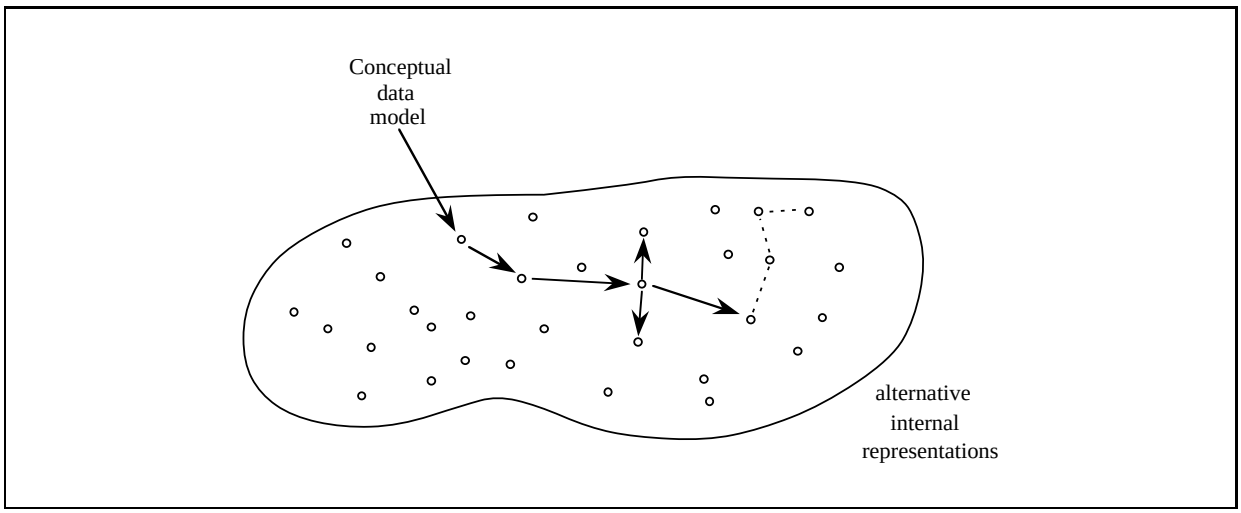


Figure 6.10: Searching the solution space

An evolutionary approach to database optimization uses the following framework. First an initial internal representation is generated, using an algorithm in the style of chapter 3. Then a database designer has the opportunity to activate evolutionary algorithms, modifying representations into new representations. In this way it is possible to interactively 'walk' through the internal solution space.

The evolutionary algorithms we describe are based on mutation systems. As an example, in mutation system $\mathcal{M} = \langle \mathcal{S}, \mathsf{Prune}, \mathsf{Graft}, \mathsf{Promote} \rangle$ the operators Prune, Graft and Promote are used for searching the internal solution space $\mathcal{S}$. The operators AddIndex and DropIndex can also be used here. Furthermore, the operator Crossover can be used.

Often evolutionary algorithms apply mutation operators in a (multi-)set of internal representations, which will be called a *pool* here[2]. This leads to the general description in figure 6.11.

In figure 6.11 operator Mutate is a *macro-operator* ([7]), transforming a pool of internal representations into a new pool. This macro-operator then is defined in terms of *local-operators* Prune, Graft and Promote, transforming an internal representation into another internal representations.

Evolution within solution space $\mathcal{S}$ is driven by the fact that each internal representation has an associated *fitness*, specified by the function $\mathsf{Fitness} : \mathcal{S} \longrightarrow \Re$. Depending on the objective of the optimization process, a particular parameterization of this fitness function is chosen. In section 6.2 several basic database optimization drivers were introduced, using different kinds of parameters.

[2]Usually the term *population* is used (see e.g. [7]). We prefer the term *pool*, because in this book the term population refers to data instances (see chapter 2 and 4).

```
Evolutionary algorithm:
    Generate initial pool of internal representations;
    while Not ready do
        Pool := Mutate (Pool);
        Select candidates
    end .
```

Figure 6.11: Generic description of evolutionary algorithm based on mutation

6.5.2 Outline of an evolutionary algorithm

In this section an outline of an evolutionary algorithm is described, which is a rewrite of the outline given in [7]. The initial pool of internal representations is denoted as Pool_0. This pool evolves as Pool_t for $t \geq 0$ until $\mathsf{Terminate}(\mathsf{Pool}_t)$ yields true. Furthermore, at point in time t there are temporary pools Pool'_t and Pool''_t. The algorithm is given in figure 6.12 and will be explained below.

```
Evolutionary algorithm:
    t := 0;
    Pool_0 := [T_1, ..., T_μ] ∈ S^μ;
    Evaluate [Fitness_{Ʊ_f}(Pool_0(1)), ..., Fitness_{Ʊ_f}(Pool_0(μ))];
    while Not Terminate_{Ʊ_t}(Pool_t) do
        Pool'_t := Recombine_{Ʊ_r}(Pool_t);
        Pool''_t := Mutate_{Ʊ_m}(Pool'_t);
        Evaluate [Fitness_{Ʊ_f}(Pool''_t(1)), ..., Fitness_{Ʊ_f}(Pool''_t(λ))];
        Pool_{t+1} := Select_{Ʊ_s}(Pool''_t ∪ Q_t);
        t := t + 1;
    end .
```

Figure 6.12: Outline of an evolutionary algorithm

The function Fitness is controlled by parameters $\mho_f$. The macro-operator Mutate is defined in terms of the mutation operators Prune, Graft and Promote. Operator Mutate decides which internal representations are used for mutation, and which points (predicators) are used for applying specific mutation operators. This process is controlled by mutation parameters $\mho_m$. More precisely, parameters $\mho_m$ are used by operator Mutate in order to determine $T \in \mathsf{Pool}'_t$ and $p, q \in \mathcal{P}$ such that either $\mathsf{ValidPrune}(T, p)$, $\mathsf{ValidGraft}(T, p, q)$ or $\mathsf{ValidPromote}(T, p)$. Furthermore there is a macro-operator Recombine which is defined in terms of the operator Crossover. The operator Recombine is controlled by parameters $\mho_r$.

In each iteration a parent pool of size μ is modified using operators Recombine and Mutate. This results in offspring pool Pool'_t and Pool''_t of size λ. Then a selection routine Select controlled by parameters $\mho_s$ determines the new pool Pool_{t+1} of size μ from $\mathsf{Pool}''_t \cup Q_t$, where $Q_t \in \{\varnothing, \mathsf{Pool}_t\}$.

6.5.3 Setting up an evolutionary algorithm

In this section further details about setting up an evolutionary algorithm are discussed. The topics that will be addressed are mentioned in the informal description of an evolutionary algorithm in figure 6.13 (this description is based on [22], [105]).

The description shown in figure 6.13 will now be further refined and related to the problem of database optimization. We first discuss the preliminary actions.

```
Evolutionary algorithm: {informal description}
    preliminary actions; {parameterization and initialization}
    while further evolution is needed do
        generate offspring; {reproduction, pairing, recombination and mutation}
        determine fitness; {fitness evaluation and scaling}
        survival of the fittest {selection and replacement}
    end .
```

Figure 6.13: Informal description of evolutionary algorithm

Preliminary actions

Parameterize fitness function: The fitness function is the yardstick for optimization. Typically, this function uses domain-dependent parameters. By choosing a particular parameterization $\mho_f$ this function is concretized for a specific application. In our case the underlying conceptual data model is a domain parameter, along with the associated database profiles (including time/space thresholds).

Parameterize evolution routines: Evolution routines describe the process of evolution. One may choose for very general evolution routines (domain-independent), or more specific (domain-dependent) routines. In our case, the evolution routines Mutate and Recombine parameterized with $\mho_m$ and $\mho_r$ are based on the operators introduced in chapter 5. So these routines are domain-dependent. Examples of domain-independent evolution routines are the standard mutation and crossover operators for bitstrings (see e.g. [62]). Examples of parameters in $\mho_m$ and $\mho_r$ are the mutation rate and the crossover rate, specifying the probability that mutation or crossover must be performed. Furthermore, the size μ of parent pool Pool_t and the size λ of offspring pool Pool'_t and Pool''_t are evolution parameters. Finally, the control parameters $\mho_t$ and $\mho_s$ for routines Terminate and Select are important evolution parameters. These will be discussed in more detail below.

Generate initial pool: In this step an initial pool is created. Each element in this pool should describe some candidate solution to the optimization problem under consideration. In our case, each solution is an (encoded) internal representation for the original conceptual data model.

Generation of offspring

The next refinement of the description from figure 6.13 involves the generation of offspring. Operator Recombine constructs a pool Pool'_t for a given pool Pool_t, followed by operator Mutate constructing a pool Pool''_t (see the outline of an evolutionary algorithm in figure 6.12). This involves the following activities.

Reproduce representations: The operators Recombine and Mutate first set up a context for applying the operators Crossover, Prune, Graft and Promote, by reproducing (copying) some of the representations from the current pool. A selection criterion that shows bias for representations with comparatively high (scaled) fitness values can be used here.

Pair representations: As a preparation for operator Recombine pairs of representations are formed. The pairing may be random or more sophisticated.

Recombination: Apply operator Crossover to each pair of representations. This procedure swaps fractions of representations ("partial candidate solutions") so that new representations are formed in the temporary pool. For a pair T_1, T_2 the fractions are selected by operator Recombine by determining predicators p_1, p_2 such that $\mathsf{ValidCrossover}(T_1, p_1, T_2, p_2)$.

Mutation: Apply mutation operators Prune, Graft and Promote. For representation T the application points are selected by operator Mutate by determining predicators p, q such that either $\mathsf{ValidPrune}(T, p)$, $\mathsf{ValidGraft}(T, p, q)$ or $\mathsf{ValidPromote}(T, p)$.

Determination of fitness

Next the fitness of each representation in the temporary pool Pool''_t is determined in two steps.

Evaluate pool: In evolutionary algorithms usually each solution in the current pool is decoded to obtain its actual meaning (see e.g. [116], [129]). This is then passed on to the fitness function in order to calculate and store its quality. In our case fitness can be determined without explicitly decoding the individuals, i.e. decoding and fitness computation occurs simultaneously.

Scale fitnesses: Sometimes fitness values are referred to as "raw" when they are not suitable as such for the replacement procedure that will be applied after fitness computation. This is for instance the case when this procedure processes fitness values (rather than fitness ranks) and large fitness fluctuations occur. In order to become suitable, the fitness values are scaled on behalf of the selection procedures used. The scaling may be linearly or more sophisticated.

Selection and replacement

In the final step of the evolution cycle, the principle of survival of the fittest will be applied. The representations in the temporary pool Pool''_t replace a number of representations in the current pool Pool_t. The latter are determined by procedure Select according to some criterion, for example one with bias for the worst performing representations.

Before the next evolution step is started, a termination criterion for the main program loop is applied, based on either the observed fitness or on a maximum number of transformation steps. If it succeeds the algorithm is made to terminate, otherwise it continues.

Note that the recombination and mutation steps, as they modify representations, are concerned with how the solution space is *explored*, whereas all other steps are concerned with the rate at which currently useful information (as indicated by current fitnesses) is *exploited*. In general, the performance of an evolutionary algorithm depends critically on the proportion between the rates of exploration and exploitation.

6.5.4 Fitness function versus objective function

The function Fitness may be defined in terms of a more elementary objective function o. In simple cases the functions o and Fitness are identical. The more complex case, where objective function and fitness function are different, occurs for example when there exists an objective threshold. This is illustrated in the following example.

Example 6.5.1

Optimization using an objective threshold can be based on fitness parameters $\mho_f = \{K, f\}$, where K is an objective threshold and $f : \Re \longrightarrow \Re$ is a function specifying how objective values exceeding K are mapped to fitness values. The fitness function then may be defined as:

$$\mathsf{Fitness}\{K, f\}(T) = \begin{cases} f(o(T)) & (o(T) > K) \\ o(T) & (\text{otherwise}) \end{cases}$$

□

In section 6.2 the use of such fitness functions in database optimization was illustrated.

6.5.5 Basic selection criteria

In this section basic selection criteria are introduced. Let $P = \mathsf{Pool}_t$ be a parent pool of size μ and let $R = \mathsf{Pool}''_t$ be an offspring pool of size λ.

First a random walk is considered. This runs under $Q = \varnothing$, i.e. the current pool P is not needed for the selection of the new pool. It is simply defined as follows:

Random walk:
$\mathsf{Select}(R) = R$

From now on we assume $Q = P$. Individual hill climbing may be defined using a configuration with a single parent representation and a single offspring representation: ($\mu = \lambda = 1$). Selection then is performed as follows:

Individual climbing:
$\mathsf{Select}(R) = \mathsf{max}(P \cup R)$

Steepest ascent aims at the maximal improvement that can be obtained (in a single iteration). It may be defined using a larger offspring pool size ($\mu = 1$ and $\lambda > 1$). Offspring R here is obtained by performing all possible mutations to P. Then, again, selection is done as follows:

Steepest ascent:
$\mathsf{Select}(R) = \mathsf{max}(P \cup R)$

Pool climbing may be defined using a larger parent pool ($\mu > 1$ and $\lambda = 1$). Selection then is performed as follows:

Pool climbing:
$$\mathsf{Select}(R) = \begin{cases} P & (\text{if } \mathsf{min}(P \cup R) \neq \mathsf{min}(P)) \\ P - \mathsf{min}(P) \cup R & (\text{otherwise}) \end{cases}$$

Here offspring R can be obtained in several ways, under control of parameters $\mho_m$ and $\mho_r$. It may for instance be obtained by performing a mutation on a randomly chosen representation in P. In section 6.5.6 a pool climber is described where R is obtained by performing a mutation on the *best* representation in P.

An overview of more advanced selection criteria is given in [7]. This includes basic techniques applied in *evolution strategies* ([140], [130]), *evolutionary programming* ([57], [58]), and *genetic algorithms* ([62], [84]). Some aspects involved in these advanced approaches were discussed in section 6.5.3.

6.5.6 An example pool climber

In the previous section basic selection criteria were introduced: individual climbing, pool climbing and steepest ascent. In this section an example pool climber is described. In this pool climber a representation with minimal fitness may be replaced by a mutation of a representation with maximum fitness. This is done if the result of the mutation is not worse than the representation with minimal fitness. Crossover will not be considered here.

A single evolution step is in general made by first creating a context for mutation, followed by the actual mutation. Both actions are taken care of by macro operator Mutate. Creating a suitable context for mutation is done using five simple selection routines. These are listed in figure 6.14. For each routine the effect is described.

We describe how a single pool climbing step is made using the Promote mutation. For other mutation operators the same scenario can be applied.

1. A context for promotion is created as follows:

   $T :=$ SelectMax(Pool)
   $p :=$ SelectValidPromote (T)

2. The promotion is now performed as follows:

Selection routine	Explanation
SelectMax(Pool)	Select a representation from Pool with maximal fitness
SelectMin(Pool)	Select a representation from Pool with minimal fitness
SelectValidPrune(T)	Select a predicator from T for pruning
SelectValidGraft(T)	Select two predicators from T for grafting
SelectValidPromote(T)	Select a predicator from T for promotion

Figure 6.14: Selection routines

$T' :=$ Promote(T, p)

3. Next the current worst element may be replaced by a mutation of the current best element:

   $T'' :=$ SelectMin(Pool)
   if Fitness$(T'') \leq$ Fitness(T') **then** Replace (T'', T')

The replacement in the third step is performed by operator Select, rather than by operator Mutate (see the outline of an evolutionary algorithm in figure 6.12). Note that this step is fully deterministic, whereas the first two steps are not.

More advanced replacement mechanisms have a nondeterministic nature, for example replacement based on fitness proportional selection (see e.g. section 8.4.5 and 8.4.6).

6.6 Summary

In this chapter the attention was focussed on database optimization. Basic possibilities for optimization were distinguished, including structure-driven, integrity-driven, sensitivity-driven and time/space-driven optimization. Also, database profiles and the computation of expected storage requirements and expected average response time was considered. Finally a framework for evolutionary database optimization was described.

In chapter 7 a prototype optimizer will be discussed. This prototype (at the time of writing this book) contains some of the basic strategies described in this chapter, such as random walk, hill climbing, steepest ascent, single improvement and multiple improvement. A further formalization of the evolution process using Markov chain theory is presented in chapter 8.

Chapter 7

Implementing data transformations

7.1 Introduction

One of the requirements in the context of data transformations is *executability of approach*. More concretely, we want to be able to activate transformation algorithms, for example by typing:

```
        edo -h750 -sql     {generate SQL code after 750 steps hill climbing}
or :
        edo -a5 -sql2      {generate Nested-SQL code after 5 steps steepest ascent}
or :
        edo -h100 -xml     {generate an XML based web site after 100 steps hill climbing}
or :
        edo -s500 -cod     {generate CODASYL code after 500 steps single improvement}
```

This illustrates *batch optimization*. However, since we don't have much experience with this kind of data transformation, we also need an *interactive environment* where transformation algorithms can be interactively called, interrupted, and adjusted by changing control parameters.

The aim of this chapter is twofold. Firstly, we describe how data transformation algorithms can be embedded in an interactive tool. Secondly, this chapter aims at presenting illustrative experiences with basic transformation algorithms such as random walk, hill climbing, steepest ascent, single improvement and multiple improvement. These algorithms were introduced in chapter 6. The transformation tool we implemented and used for the experiments is called EDO for Evolutionary Database Optimizer. Our experiences with data transformation experiments were published in [16], [18], and [137].

The organization of this chapter is as follows. In section 7.2 the intention and architecture of data transformation tools is described. Furthermore, a piece of program is discussed. Section 7.3 gives a further acquaintance. We present the Main Menu, the Status Report and the Evolution Screen. In section 7.4 some experimental results for basic transformation algorithms are discussed. Finally, in section 7.5 a framework for combining basic evolutionary algorithms is described, and illustrated with experimental results. The chapter is concluded with a summary and outlook.

7.2 Overview of data transformers

7.2.1 Intention

The intention is to support profile-based transformations, in combination with constraint-based transformations. In profile-based transformations, the evolution process in the internal solution space $\mathcal{S}$ is guided on

the basis of profiles estimating the expected usage of the database, while constraint-based transformations guide the evolution process in $\mathcal{S}$ on the basis of integrity constraints from the conceptual data model. As was mentioned before, emphasis is given to profile-based transformations.

7.2.2 Architecture

Figure 7.1 presents a typical architecture for data transformers. This is explained as follows. First, the *Converter* generates internal representations for the conceptual model under consideration. Next, the *Evolver* modifies the current pool of internal representations. Routines from these modules may be activated interactively via a main menu, which is described in the next section. The Evolver consists of three main parts: the *Evaluator* for fitness evaluation, the *Selector* for selecting possibilities for mutation and for selecting mutated representations to be accepted, and the *Transformer* for applying mutation operators.

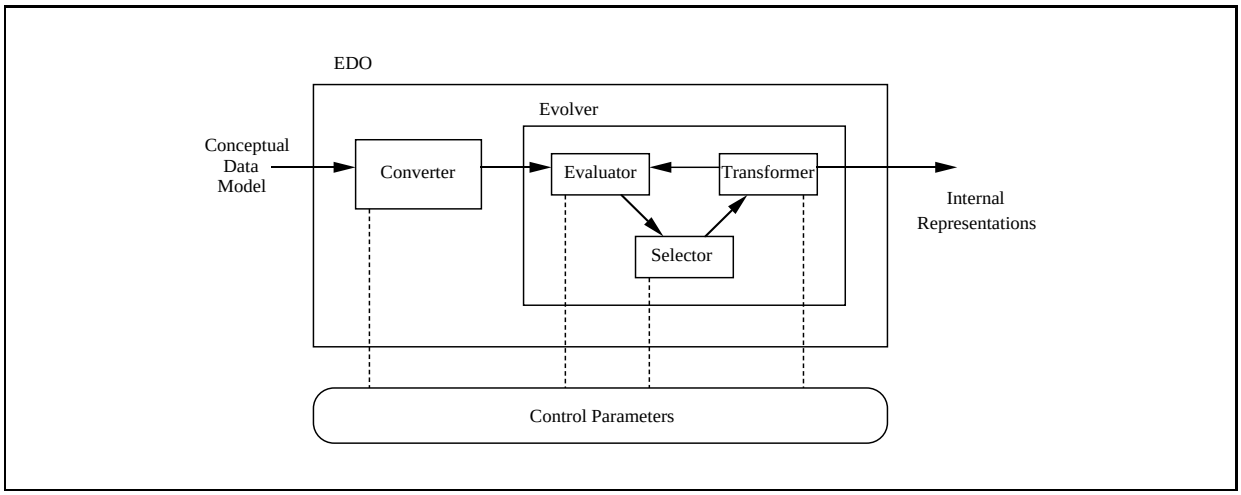


Figure 7.1: Architecture for data transformers

7.2.3 A piece of program

In this section a piece of program is given in order to illustrate the realization of evolution mechanisms. This program concerns the Promote mutation. Recall from section 5.4.5 that the effect of $\mathsf{Promote}(T, p)$ is expressed by:

$$\alpha(x) := \begin{cases} q & \text{if } x \in \mathsf{Node}(q) \\ \alpha(q) & \text{if } x \in \mathsf{Node}(p) \end{cases}$$

where predicator p is an anchor and predicator $q := \mathsf{Hook}(\mathsf{Fact}(p))$ is a hook in a root node. The realization of this mutation is given in figure 7.2.

The realization of the Promote mutation in figure 7.2 is based on a two dimensional array of integers. Here Pool is a $\mathsf{PoolSize} \times |\mathcal{P}|$ array, where $\mathsf{Pool}[T, p]$ contains the encoding for predicator p in representation T. This encoding was introduced in section 5.3.1.

First routines IsAnchor and InRoot check whether promotion is allowed. Then each predicator in $\mathsf{Node}(q)$ gets coding value q and each predicator in $\mathsf{Node}(p)$ gets the old coding value of q (i.e. AnchorQ). All other predicators remain unchanged.

```
proc Promote(T, p : Integer):
{ This procedure performs a promote mutation on representation T }
{ Predicator p will be promoted }
var Counter, AnchorP, q, AnchorQ : Integer;
begin
    q := Hook(Fact(p));
    if IsAnchor (T, p) and InRoot (T, q) then begin
        AnchorP := Pool [T, p];
        AnchorQ := Pool [T, q];
        for Counter := 1 to NrPreds do begin
            if Pool [T, Counter] = AnchorQ then begin { Going down }
                Pool [T, Counter] := q
            endif ;
            if Pool [T, Counter] = AnchorP then begin { Going up }
                Pool [T, Counter] := AnchorQ
            endif
        end
    endif
end .
```

Figure 7.2: Realization of promote mutation using array of integers

7.3 User interface

The aim of this section is to describe a typical user interface for data transformers. The interface will be illustrated with an example random walk.

7.3.1 Overview of user interface

In this section the user interface is summarized. We first consider the Main Menu shown in figure 7.3.

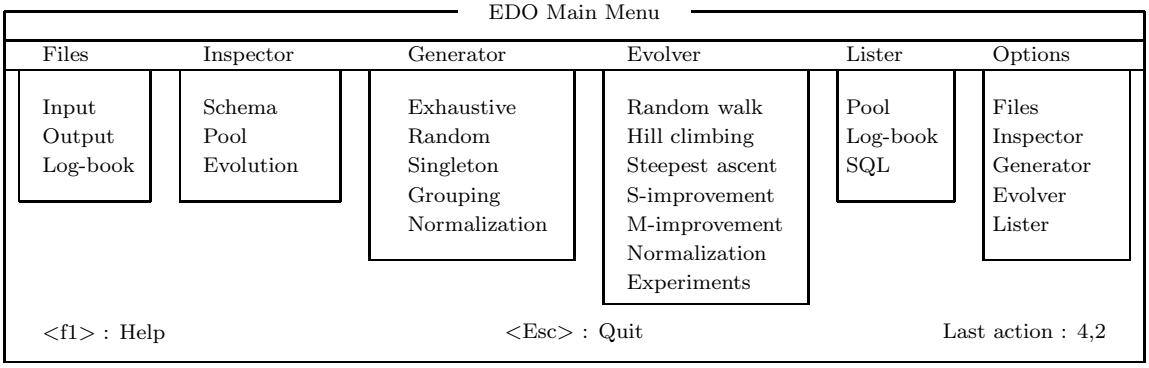


Figure 7.3: Main Menu of prototype optimizer

The main menu shown in figure 7.3 is explained as follows. There are six sub-menus: *Files*, for editing files; *Inspector*, for getting detailed information about the conceptual data model, the current pool of internal representations or the evolution process until the current moment; *Generator*, for generating an initial pool

of internal representations; *Evolver*, for activating evolutionary algorithms modifying the current pool; *Lister*, for producing output; *Options*, for setting control parameters of the tool.

Apart from the Main Menu, the user interface includes two other parts: the Status Report and the Evolution Screen. The Status Report can be positioned below the Main Menu. It gives general information about current parameter settings and fitness properties. More detailed information about parameter settings can be obtained via sub-menu *Options*, while more detailed fitness properties are found via sub-menu *Inspector*.

The Evolution Screen is displayed after activating the *Evolver*. It presents information about the evolution process. The Evolution Screen can be a simple ascii screen or a more advanced graphical screen. Although simple, an ascii screen is sufficient to illustrate evolution processes. The Evolution Screen will be discussed in more detail in the next sections.

7.3.2 Realization of random walk

In this section we consider the realization of random walk (see also section 6.5.5). The intention of the algorithm is as follows. First an initial pool of internal representations is generated. Then this pool is iteratively modified. In a pool modification each element x of the current pool is replaced by a randomly generated mutation y. The algorithm is given in figure 7.4.

```
Random walk:
    Generate initial pool and evaluate fitness;
    while Not ready do
        for each x in pool do
            Generate y := Mutate (x) and evaluate fitness;
            Replace (x, y)
        end
    end .
```

Figure 7.4: Realization of random walk

Here Mutate is a generic term for mutation operators (e.g. Prune, Graft and Promote). Let PoolSize be the number of internal representations in a single pool and let Steps be the number of evolution steps made. Then the complexity of the algorithm shown in figure 7.4 is given by:

$$\mathsf{PoolSize} \times (\mathsf{g} + \mathsf{f} + \mathsf{Steps} \times (\mathsf{m} + \mathsf{f}))$$

where g is the cost of generating an initial internal representation, f is the cost of a single fitness evaluation, and m is the cost of a mutation. For generation and mutation, the number of (re)construction steps is restricted by the number of predicators (see chapter 5). In each (re)construction step a simple and rather cheap assignment is made. On the other hand, fitness evaluation requires a number of multiplications which is at least quadratic in the number of predicators (see the cost model in section 6.4). As a result, the formula given above shows that the bottle-neck in the performance of evolutionary algorithms is fitness evaluation, rather than the actual mutation mechanism.

7.3.3 The Status Report

The first activity in the random walk algorithm in figure 7.4 is the generation of an initial pool of internal representations, using the *Generator* from the Main Menu. After completing the generation process, the system gives the Status Report shown in figure 7.5.

From the first column of the Status Report in figure 7.5 we conclude that the conceptual model under consideration is called 'Schema 2'. The profiles used for optimization are 'Data profile 1' and 'Access

Schema:	2	Current fitness:	0.00317	Space weight:	5
Data profile:	1	Max fitness:	0.00317	Pool size:	4
Access profile:	1			Randseed:	1
		Current space:	311		
Object types:	10	Min space:	311	Steps:	30
Fact types:	5	Current time:	320	Display mode:	0
Predicators:	13	Min time:	320	Step mode:	0

Figure 7.5: Status Report after initial generation

profile 1'. As the Status Report shows, the conceptual model consists of 10 object types, where 5 object types are composed object types (fact types), with 13 predicators in total.

Next we consider the second column of the Status Report. In the current pool the minimal space is 311 s.u. (space units), the minimal time is 320 t.u. (time units), and the highest fitness is 0.00317 f.u. (fitness units). Since this is the initial pool, the best properties in the current pool (e.g. Current fitness) are equal to the best properties found in the evolution process until now (e.g. Max fitness).

In the third column of the Status Report we find some system parameters. The importance of storage optimization (Space weight) is 0.5. The number of representations in a single pool (Pool size) is 4, while the number of evolution steps to be made by evolutionary algorithms (Steps) is 30. Also, we find some other parameters (Randseed, Display mode, Step mode) which are outside the scope of this book.

7.3.4 The Evolution Screen

Next we will activate a random walk, using the *Evolver* from the Main Menu. During the evolution process an Evolution Screen is constructed. In the left part of the Evolution Screen a graph is given with horizontal axis *age* and vertical axis *plane*. In the right part a graph is given with horizontal axis *age* and vertical axis *fitness*. We first explain the age on the horizontal axes.

Let Pool_t be the result after evolution step t (see also 6.5.2). The initial pool of internal representations, denoted as Pool_0, is produced by the *Generator*. Each next pool Pool_{t+1} is produced by the *Evolver*, and is the result of mutating pool Pool_t where $t \geq 0$. The age on the horizontal axes corresponds to t.

Next we explain the plane-axis. The solution space $\mathcal{S}$ of candidate internal representations for a given information structure $\mathcal{I}$ with fact types $\mathcal{F}$ consists of $|\mathcal{F}|$ hyperplanes, where a hyperplane $\nu_j \subseteq \mathcal{S}$ is the subset of $\mathcal{S}$ containing the representations with j joins ($0 \leq j \leq |\mathcal{F}| - 1$). The numbers on the plane-axis correspond to values of j. More details about these planes are found in section 5.2.

As a consequence, the left figure in the Evolution Screen shows in which parts of the solution space $\mathcal{S}$ the evolution process is performed. For instance, the pool resulting from the first iteration of the evolver, denoted as Pool_1, is shown in the first column of graph plane/age. Suppose the elements of this pool are in hyperplanes ν_0 and ν_1.

Next we explain the *fitness* on the second vertical axis. As was mentioned in section 6.2, each internal representation in $\mathcal{S}$ has a certain fitness. The graph fitness/age is expressed as a percentage of an upperbound for the conceptual model under consideration. In this example, we assume that Pool_1 contains internal representations with a fitness of 30 % and 40 % of the upperbound, while the average fitness in that pool is 40 %.

7.3.5 Inspecting the results

When an evolution process has been completed, there are several ways to examine the results. Of course the Evolution Screen itself has given information about the evolution process. After returning to the Main Menu further investigation is possible as follows.

Firstly, the Status Report contains new values with respect to the fitness. In the excerpt of the Status Report shown in figure 7.6, we see that the maximal fitness in the current pool is 0.00156 (Current fitness), while the maximal fitness found during the evolution process is 0.00323 (Max fitness). Compared to the original values in figure 7.5 the former property has worsened, while the latter property has improved. This is a typical feature of random walk. Similar properties with respect to Space and Time are given. Note that an improvement is indicated by an increasing fitness and by a decreasing Space and Time.

Current fitness:	0.00156
Max fitness:	0.00323
Current space:	969
Min space:	309
Current time:	280
Min time:	260

Figure 7.6: Excerpt of Status Report after evolution

As was mentioned before, the Status Report only gives global information about fitness properties. Therefore, a second possibility for further investigation is provided by the *Inspector* from the Main Menu. The Pool Inspector gives more detailed information about the current pool, while the Evolution Inspector gives more information about the evolution process which resulted in the current pool.

Finally there are several possibilities for writing information to output files. Such a file may for example contain the current pool of internal representations specified in terms of SQL. This then can be used for prototyping. Also, the file may contain a so-called logbook of the entire evolution process. This can be used for further processing by plotting programs. As an example, the graphs in section 7.5.4 and appendix B were produced using Gnuplot. Moreover, the file may contain Latex commands for further processing. The graph in section 7.4.6 was produced in this way.

7.4 Using basic evolutionary algorithms

In this section we describe experimental results of basic evolutionary algorithms introduced in chapter 6: random walk, hill climbing, steepest ascent, single improvement, multiple improvement. Appendix B contains the typical fitness development graphs for these algorithms.

7.4.1 Realization of hill climbing

In this section we consider the realization of hill climbing (see also 6.5.5). We describe a simple climbing algorithm and formulate some basic properties.

The intention of the algorithm is as follows. First an initial pool of internal representations is generated. Then this pool is iteratively modified. In a pool modification each element x of the current pool may be replaced by a randomly generated mutation y of x. Whether x will actually be replaced by y depends on their fitness. In this way the hill climbing nature is guaranteed. The algorithm is given in figure 7.7.

```
Hill climbing:
    Generate initial pool and evaluate fitness;
    while Not ready do
        for each x in pool do
            Generate y := Mutate (x) and evaluate fitness;
            if Fitness (x) ≤ Fitness (y)
            then Replace (x, y)
            end
        end
    end .
```

Figure 7.7: Realization of hill climbing

Next we formulate some basic properties of the algorithm in figure 7.7, involving the fitness of the internal representations found during a search process:

1. In each stage of the evolution process the current pool contains the best internal representation found as yet:
$$\forall_{i \geq 0} \left[\mathsf{MaxFitness}(i) = \mathsf{CurFitness}(i) \right]$$
where $\mathsf{CurFitness}(i)$ and $\mathsf{MaxFitness}(i)$ express the properties introduced in section 7.3.3 with respect to Pool_i.

2. Therefore, the best representation in the current pool cannot be worse that the best representation in previous pools:
$$\forall_{j \geq 0} \forall_{0 \leq i \leq j} \left[\mathsf{CurFitness}(i) \leq \mathsf{CurFitness}(j) \right]$$

3. As a direct consequence, this property also holds for the best representation found as yet:
$$\forall_{j \geq 0} \forall_{0 \leq i \leq j} \left[\mathsf{MaxFitness}(i) \leq \mathsf{MaxFitness}(j) \right]$$

The computational complexity of the hill climbing algorithm shown in figure 7.7 is basically the same as the complexity of the random walk algorithm from section 7.3. If however a 'steepest ascent' hill climber is used, each step evaluates all possible mutations, rather than a single one. Then the complexity $\mathsf{Steps} \times (\mathsf{m} + \mathsf{f})$ becomes $\mathsf{Steps} \times N \times (\mathsf{m} + \mathsf{f})$, where N is the average number of neighbours.

7.4.2 An example hill climbing run

In this section we make an example hill climbing walk. We use the information structure from figure 7.8 and the access profile and data profile from figure 7.9. In this example run, the weight coefficient for time optimization is set to 0.7.

After generating an initial pool of internal representations using the *Generator*, hill climbing is activated using the *Evolver*. During the evolution process an Evolution Screen is constructed. In this case, the graph fitness/age clearly shows the climbing nature. From the fitness distribution it can be concluded that the maximal fitness that was found in the evolution process is 0.00544 f.u.. We examine the corresponding internal representation. In the optimal internal representation that was found, fact type f_3 is represented in a separate nested table with key p_8:

X_2 of p_8	f_3	
	X_1 of p_6	X_2 of p_7

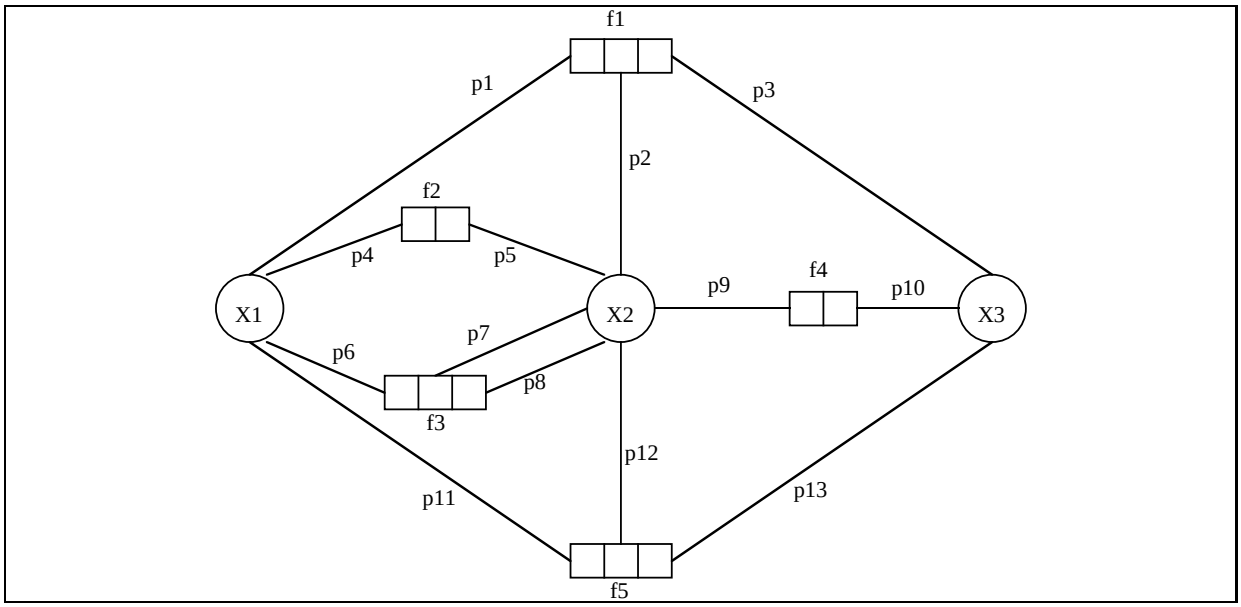


Figure 7.8: Information structure

Relative frequency	Path expression
0.10	$p_4, p_5, p_8, p_6, p_{11}, p_{12}$
0.50	$p_9, p_{10}, p_{13}, p_{12}$
0.40	p_5, p_4, p_1, p_2

Type:	x_1	x_2	x_3	f_1	f_2	f_3	f_4	f_5
Relative nr. of instances:	3	2	1	1	2	3	1	2

Figure 7.9: Example access profile (top) and data profile (bottom)

Fact types f_1 and f_2 are represented in a common table with key p_3 (obtained from fact type f_1). Fact type f_2 is nested within the subtable for fact type f_1:

X_3 of p_3	f_1		
	X_2 of p_2	X_1 of p_1, p_4	f_2
			X_2 of p_5

Fact types f_5 and f_4 are represented in a common table with key p_{12} (obtained from fact type f_5). Fact type f_4 is nested within the subtable for fact type f_5:

X_2 of p_{12}	f_5		
	X_1 of p_{11}	X_3 of p_{10}, p_{13}	f_4
			X_2 of p_9

As was mentioned at the beginning of this section, in this example hill climbing run the weight coefficient for time optimization was 0.7. Therefore, the highest priority is given to the access profile. It is easily checked

that the internal representation discussed above supports the access profile from figure 6.8 appropriately, e.g. in terms of the number of inter-table accesses needed for that profile, without going to the extreme of representing the complete information structure in a single table.

The values of the properties of Time and Space for this internal representation are given in section 7.4.4. Also, a comparison with other values of the weight coefficient for storage space will be given. We now first make a comparison with random walk.

7.4.3 Comparison with random walk

In this section we make a random walk for the same information structure under the same conditions. In figure 7.10 a graph is given for these example runs. We see that random walk at first gives better results than hill climbing (maximum fitness after 6 and 10 mutations). Later hill climbing finds representations that are not found by random walk.

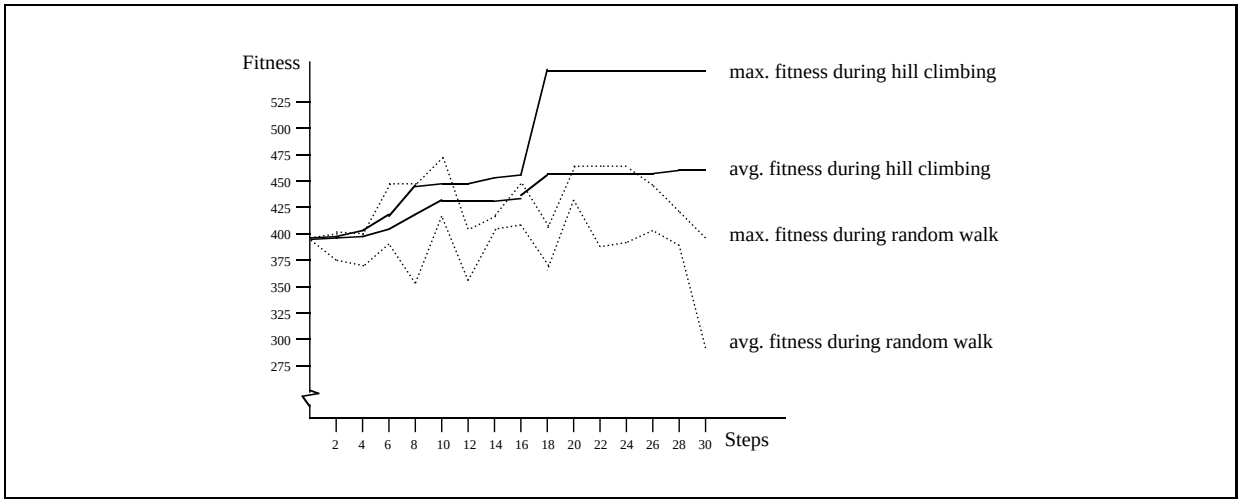


Figure 7.10: Average and maximum fitness

7.4.4 Time/space trade-off

The focus of this section is the time/space trade-off. This trade-off can be examined in several ways, for example by means of scalar optimization varying the weight coefficient for time optimization β from 0 to 1 (see also section 6.2.5). As a result of this variation, representations with different time/space properties will be found.

We describe the results of an experiment using the information structure given in figure 7.8. We make hill climbing runs for $\beta = 0, \frac{1}{10}, \frac{2}{10}, ..., 1$. In each case the optimal internal representation is recorded after 30 and 60 evolution steps. Each run is initiated from the same internal representation. This representation is indicated by Steps = 0 in figure 7.11.

After 30 evolution steps with different values of β, five different optima are found. These optima are indicated by the line Steps = 30 in figure 7.11. The optimal representation discussed in section 7.4.2 is depicted as a small square. The optima that are found after 30 more evolution steps are indicated by the line Steps = 60. In accordance with intuition, this line is closer to the origin of the time/space axes than the line indicated by Steps = 30.

The optimum after 30 steps which ignores space optimization ($\beta = 1$) is given in position (190,260). This optimum is dominated by other optima with better properties for Time as well as Space. However, after

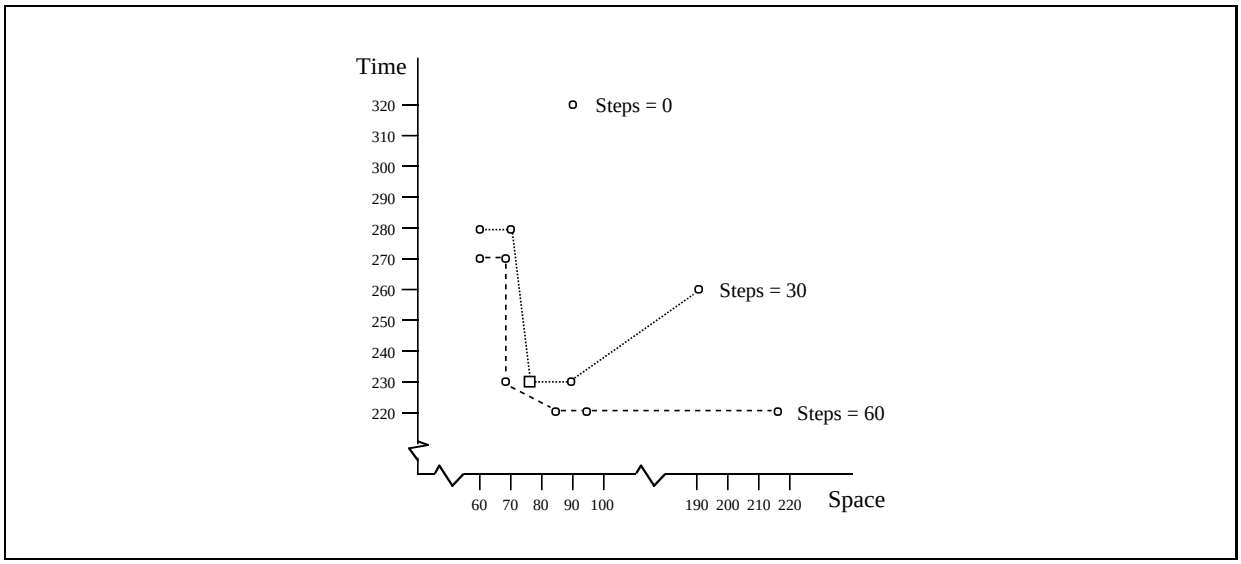


Figure 7.11: Representations with different time/space properties

30 more evolution steps this optimum has a value for Time which is not dominated by any of the other solutions.

Next we consider the results of a similar experiment with a larger conceptual model (14 fact types). Again scalar optimization is applied with different values for weight coefficient β. The result is shown in figure 7.12.

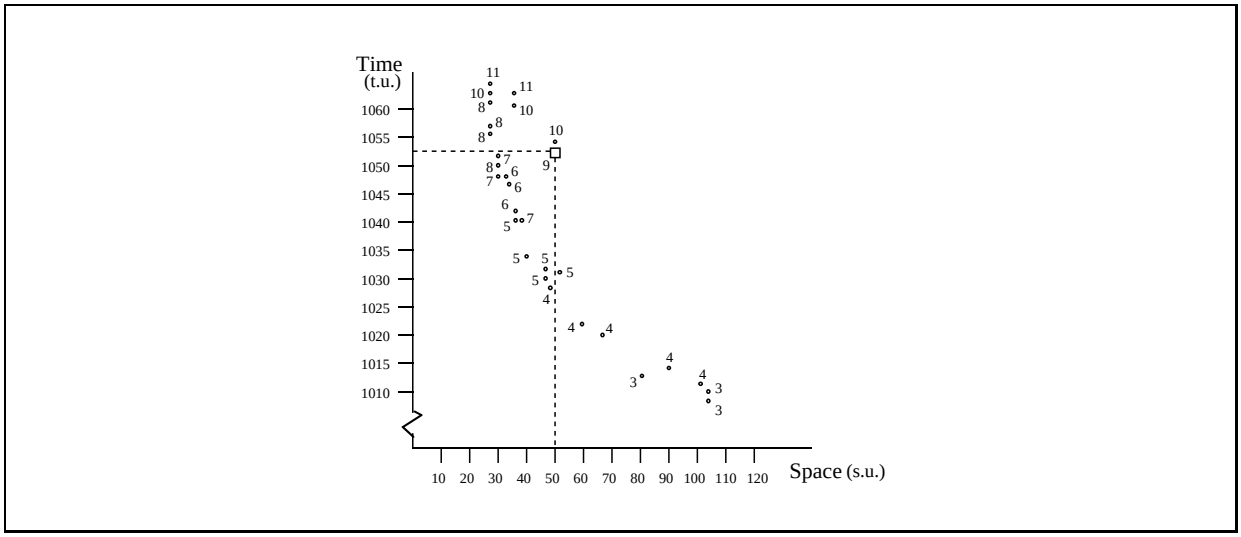


Figure 7.12: Different time/space properties with number of tables

Several candidate internal representations are given in figure 7.12, with an indication for their storage requirements and average response times. Also, the number of tables in a candidate is given. As expected, representations with a low time property (obtained by high β) consist of relatively few tables (e.g. 3, 4). These representations however need relatively much space. On the other hand, representations with a high time property (obtained by low β) consist of relatively many tables, and need relatively little space.

7.4.5 Number of tables versus time/space/fitness

The previous section was concluded with an example concerning the number of tables (trees) in an internal representation. In this section the relation between number of tables and time/space values is examined in more detail. Also the relation with fitness values is considered.

We use a simple conceptual model containing 5 fact types. For this model 100 evolution steps are made using random walk. Each representation found during the evolution process is evaluated with respect to response time, storage space and fitness. The results are shown in figure 7.13.

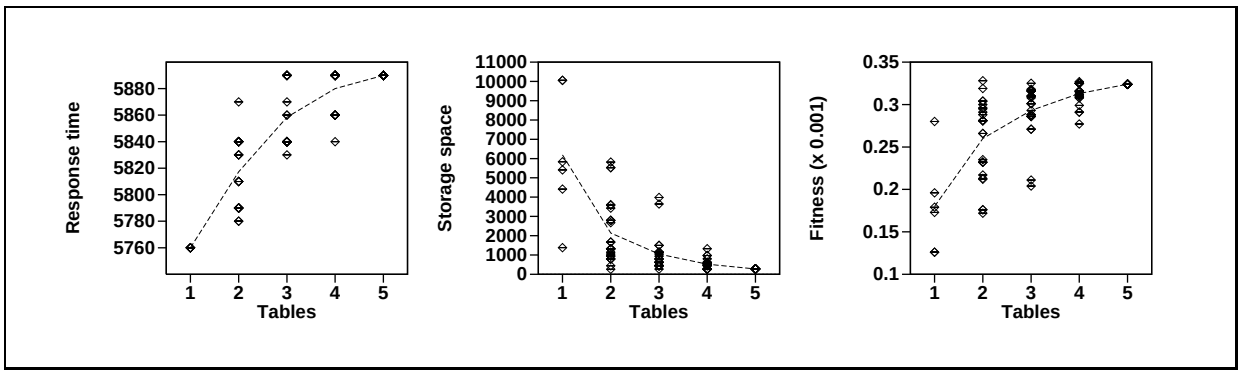


Figure 7.13: Tables-time, tables-space and tables-fitness relation

In the left part of figure 7.13 a graph tables-time is given. The middle part presents a graph tables-space. Again, representations with many tables have a high time value and a low space value, and vice versa (see also 7.12). The graphs also give the average time and space per value of the number of tables (dashed line). In the right part of figure 7.13 a graph with the corresponding fitness values are given.

Although the form of the dashed lines (average properties) in figure 7.13 seems likely, this cannot be generalized. Other experiments give different results. This not only depends on the underlying cost models, but also on the specific realization of the evolutionary algorithm used (in this case random walk).

7.4.6 Generating Latex output

During an evolution process the user receives information about time, space, and fitness development via the Evolution Screen. An Evolution Screen can be printed easily using the Print Screen key on the keyboard. A data transformer may also write the corresponding Latex commands to a file. In figure 7.14 we see an example of the resulting graph. Such a Latex graph can be printed separately or it can be integrated in database design documentation.

In figure 7.14 we see a Pareto-Optimal front consisting of five internal representations. During the evolution process, the Pareto-Optimal front moves towards the origin of the time/space axes.

7.5 Combining evolutionary algorithms

In this section we consider the possibility of combining different evolutionary algorithms. The experimental results were obtained using a more realistic solution space: the conceptual model contains 50 fact types; so the solution space contains at least $2^{50} \approx 10^{15}$ internal representations.

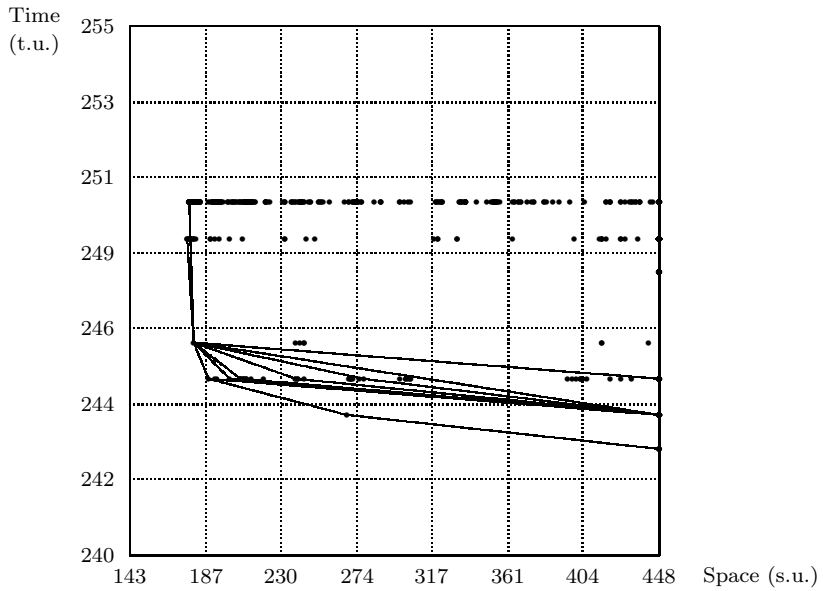


Figure 7.14: Graph constructed by Latex generator

7.5.1 Approach to combining evolutionary algorithms

An evolutionary algorithm runs under a specific setting of control parameters, such as the number of evolution steps to be made and the weight coefficient for time/space optimization. We now define an *experiment* as an algorithm, which (1) contains one or more evolutionary algorithms and (2) automatically manipulates control parameters. Figure 7.15 presents an overview of the situation.

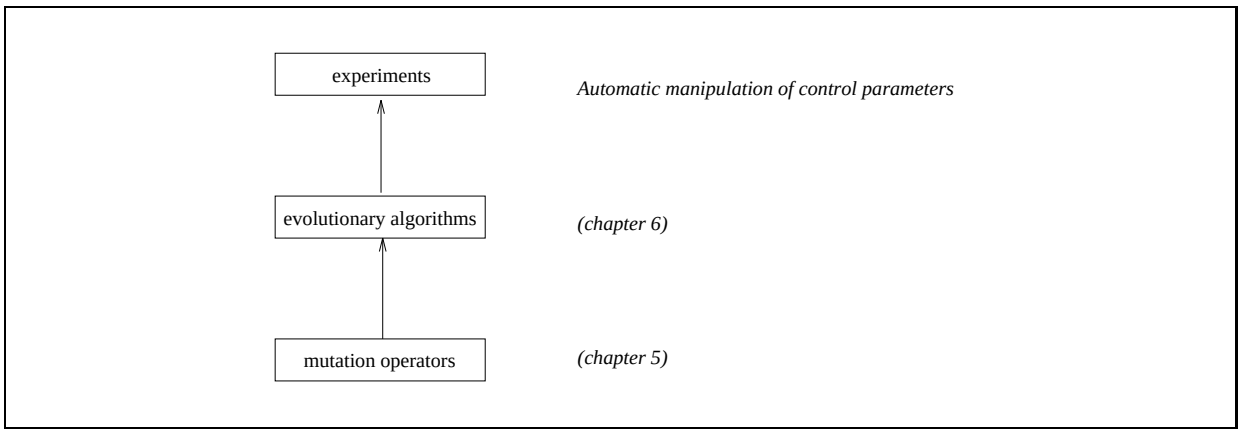


Figure 7.15: Mutations, evolutionary algorithms and experiments

In the previous sections some experiments were described. As an example, in order to examine the time/space trade-off, hill climbing was used with different weight coefficients for time/space optimization (section 7.4.4). In this section we describe a more complex experiment based on layered evolution.

7.5.2 Framework for layered evolution

Evolutionary algorithms can be combined by dividing the evolution process into a number of levels. As an example, the process can be started at level zero with k_0 steps random walk. From the k_0 representations that are produced this way, ℓ_0 representations are selected as crossroads for further development. Different selection strategies are possible, and therefore the selected representations are not necessarily the ones with the highest fitness. Taking these selected representations as starting point, the evolution process is continued in level one with k_1 steps. The evolutionary algorithm here could again be random walk, but could also be a more directed one, such as hill climbing. Now, a selection of ℓ_1 representations is made. The process of further developing selected representations can be repeated several times. In this way the evolution process consists of a number of levels, each of which applies a specific evolutionary algorithm under a specific setting of control parameters.

This framework, called *survey search*, leaves enough room to vary different control parameters. In figure 7.16 the framework is illustrated for four levels.

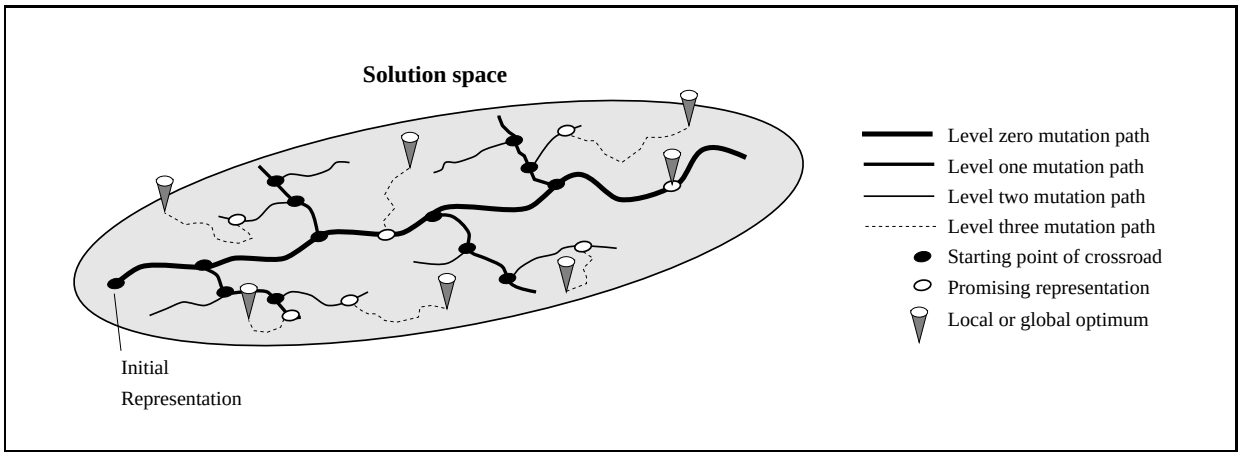


Figure 7.16: Survey search consisting of four levels

In figure 7.16, four search levels are defined: the first three levels perform a random walk, whereas the last level uses a directed evolutionary algorithm, for instance a hill climber or steepest ascent. For levels zero and one, the crossroads are selected such that they are uniformly distributed over the mutation path performed. For level two, a number of steps random walk is performed, but no crossroads are selected.

At this point, the solution space is covered with a network of mutation paths. From this network the most promising representations are further developed in the last level. So these promising representations serve as initial representations for a directed algorithm.

7.5.3 Set-up of experiment with layered search

In this section the set-up of an experiment aiming at comparison between survey search and conventional hill climbing is described. We use a four level survey search. First, three levels random walk provide us with ten promising representations. The remaining fourth level consists of hill climbing mutation paths, starting from these promising representations. The configuration of the different levels in survey search experiment is given in figure 7.17.

The configuration in figure 7.17 is explained as follows:

Level 0: This is the start level. 900 evolution steps are performed here, starting from a randomly chosen starting-point, and using random walk. Each 100th step is selected as a crossroad.

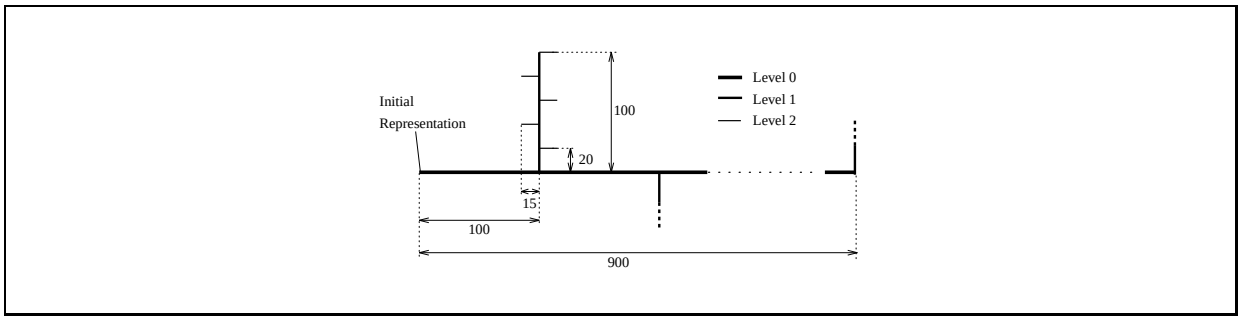


Figure 7.17: Survey search configuration (first three levels only)

Level 1: Starting from each of the 9 obtained crossroads, 100 evolution steps are performed, again using random walk. At this level each 20th representation is selected as a crossroad.

Level 2: The previous level provided us with 45 new crossroads. From each of these, 15 steps random walk are performed.

Level 3: Until now, 2475 internal representation were examined. The ten best representations will be starting-point for 50 steps hill climbing. This makes the total number of steps:

$$2475 + (10 \times 50) = 2975$$

For conventional hill climbing, four starting-points are used: two random representations, one *singleton* representation (i.e. each fact type is represented in a separate tree), and one *grouping* representation (i.e. the entire information structure is represented in a single tree). The number of evolution steps to be performed from each starting point then becomes:

$$\frac{2975}{4} \approx 744$$

7.5.4 Evaluation of results

In this section the results of the experiment discussed in the previous section are evaluated. The fitness development for survey search is given in figure 7.18. Figure 7.19 presents the fitness development for conventional hill climbing.

The following conclusions can be drawn from the fitness development graphs in figure 7.18 and 7.19.

optima: Conventional hill climbing finds the best representations. Even the worst optimum for hill climbing (found by the grouping starting-point) is better than the best optimum for survey search.

convergence: Both survey search and conventional hill climbing are still improving the fitness at the end of the evolution process. This indicates that there is no convergence yet. This is not surprising since a conceptual model with 50 fact types has an internal solution space of $2^{50} \approx 10^{15}$ representations.

In order to characterize the fitness development during the evolution process, let f_x be the maximum fitness after x evolution steps. Then the following properties can be used to express further differences between figure 7.18 and 7.19:

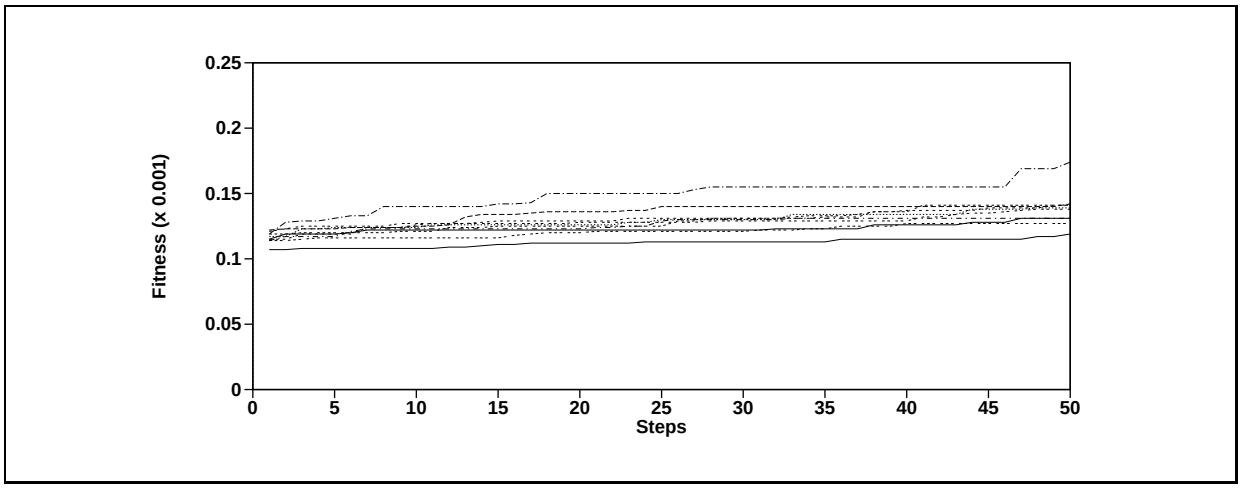


Figure 7.18: Fitness development for survey search

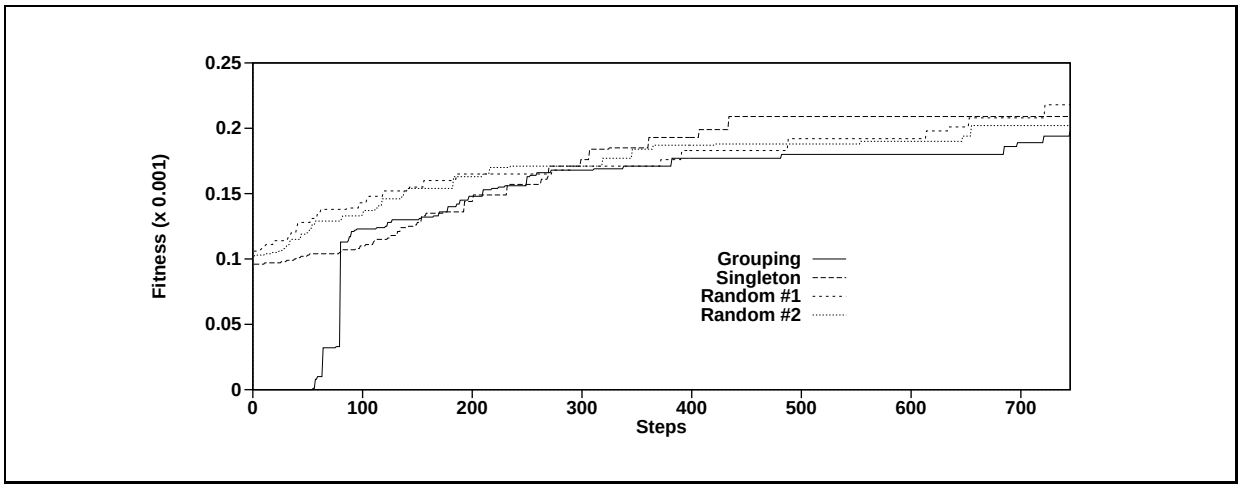


Figure 7.19: Fitness development for conventional hill climbing

- The fitness improvement $\mathsf{Fitness}_{imp}$ expresses the improvement of fitness in a specific step interval (i, j):

$$\mathsf{Fitness}_{imp} = f_j - f_i$$

- The relative fitness improvement $\mathsf{Fitness}_{rimp}$ expresses the fitness improvement compared to the fitness of the first representation of a specific step interval (i, j):

$$\mathsf{Fitness}_{rimp} = \frac{f_j - f_i}{f_i}$$

- The stepwise fitness improvement $\mathsf{Fitness}_{simp}$ expresses the average fitness improvement per evolution step in a specific step interval (i, j):

$$\mathsf{Fitness}_{simp} = \frac{f_j - f_i}{j - i}$$

- The fitness efficiency $\mathsf{Fitness}_{eff}$ expresses the ratio of the number of improving evolution steps to the total number of evolution steps in a specific step interval (i, j):

$$\mathsf{Fitness}_{eff} = \frac{|\mathsf{Im}|}{j-i} \text{ where: } \mathsf{Im} = \{i < x \leq j \mid f_x > f_{x-1}\}$$

These quantities are computed for survey search in interval (1,50) and for conventional hill climbing in interval (1, 50) and (1, 745). The results are given in figure 7.20. The figure shows the maximum values found.

Property		Survey search after 50 steps	Hill climbing after 50 steps	Hill climbing after 745 steps
Fitness	($\times 10^{-3}$)	0.174	0.128	0.218
Fitness improvement	($\times 10^{-3}$)	0.055	0.022	0.198
Rel. fitness imp.		0.464	0.671	1387
Stepwise fitn. imp.	($\times 10^{-3}$)	1.127	0.452	0.266
Fitness efficiency		0.327	0.367	0.081

Figure 7.20: Maximum values for step interval characterization

We first consider the results after 50 evolution steps. At this point, survey search has found a maximum fitness of 0.174, whereas hill climbing has found a maximum of 0.128. The difference is even more clear for the fitness improvement and for the stepwise fitness improvement. So this might lead us to the conclusion that survey search is not that bad.

However, survey search had a preparation and is finished after these 50 steps, while hill climbing has just started. This aspect becomes clear in the relative fitness improvement and the fitness efficiency. Then hill climbing has the highest values. As a result of the first three levels, the starting-points for survey search are better than those for hill climbing. Therefore it may be expected that hill climbing is able to perform more improving steps, leading to a higher $\mathsf{Fitness}_{rel}$ and $\mathsf{Fitness}_{eff}$.

Next we consider the results after 745 steps. Now hill climbing has found the best fitness and has the highest fitness improvement. This fitness improvement is the result of the grouping starting-point (see figure 7.19), which has an extremely low fitness, leading to an extremely high relative fitness improvement. Note that hill climbing has a lower stepwise fitness improvement and fitness efficiency than survey search. This means (1) that survey search relatively makes more improvements than hill climbing (fitness efficiency) and (2) that these improvements are relatively larger. But since hill climbing makes more evolution steps it results in the best fitness (0.218).

7.5.5 Further examination

The results from the previous section show differences in the performance of two approaches to optimization. Conventional hill climbing found better representations than survey search. However, this may be otherwise when the approaches are adapted. There are several possibilities for adaptation:

extension: Increase the number of evolution steps for conventional hill climbing and for the last level of survey search.

reconfiguration: Change the configurations e.g. by changing the number of starting-points. Less starting-points will allow longer climbing paths for hill climbing and for the last level of survey search.

Another possibility is extension combined with reconfiguration. We have tried extension with 7000 evolution steps. For conventional hill climbing this results in $\frac{7000}{4} = 1750$ additional steps. For survey search this results in $\frac{7000}{10} = 700$ additional steps. Now both approaches found the same fitness (0.240).

The definitions for step interval characterization given in the previous section (e.g. $\mathsf{Fitness}_{impr}$) can be used to compare each combination of step intervals. As an example, figure 7.21 presents the fitness improvement for the extended experiment. The intervals contain 75 steps.

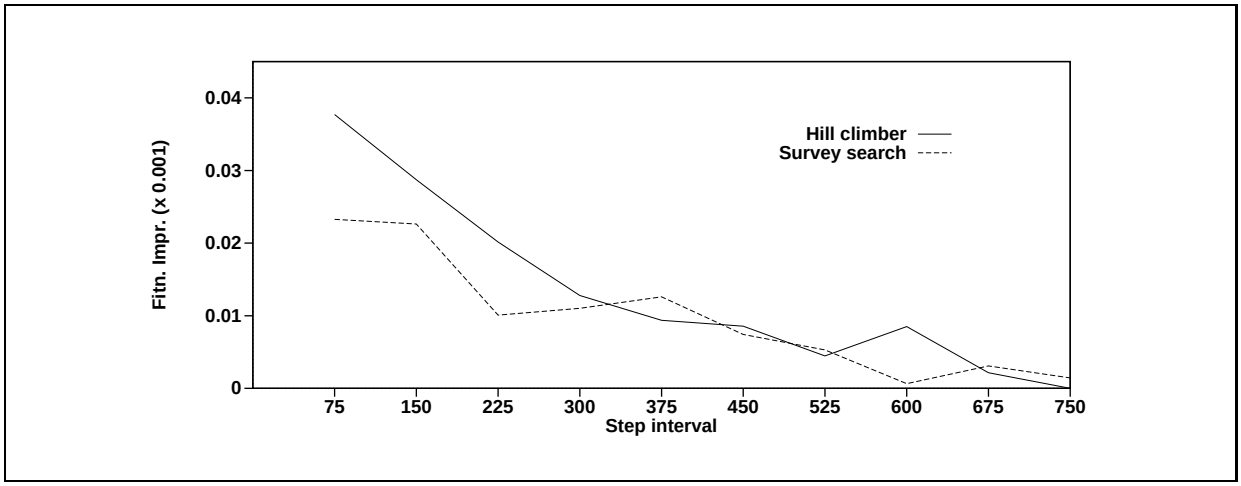


Figure 7.21: Fitness improvement for different intervals

The graphs in figure 7.21 should be read as follows. We consider the dashed line (survey search). In the first 75 steps there is a fitness improvement of 0.022. The second interval gives an improvement being somewhat smaller. In the third interval the improvement descends drastically, but it is restored a little in the fourth interval (225 - 300). Note that both graphs descend in the long run, as could be expected (as a result of convergence).

7.6 Summary

In this chapter we considered the implementation of a data transformation tool. For a given conceptual data model, this interactive tool allows a database designer to generate internal representations and to make these representations evolve into more desirable ones. The fitness of internal representations is estimated on the basis of profiles, characterizing the environment in which the database under development will operate.

In chapter 8 a formal basis for describing the dynamics of the underlying algorithms is introduced. This will be based on Markov theory.

Chapter 8

Markov chain fundamentals

8.1 Introduction

Data schema transformations gain a lot of attention in the world of database theory (see e.g. [47], [68], [71], [92]). However, there is no technique for describing and predicting the dynamics of schema transformation processes. The aim of this chapter is to define such a technique. See also [153] and [154].

More concretely, we describe mutation, evolution and optimization of internal representations using Markov chain theory (see e.g. [67]). We show how this description can be used for considering the dynamics of database design processes, in terms of success probability, first success probability, local optimum probability, and success entropy. In [153] several other properties are considered, including the expected time to find a global optimum and the expected fitness after a given number of evolution steps.

Several Markov chain models have been introduced for the purpose of describing and predicting the behaviour of evolutionary algorithms. In [63] a basic approach was introduced. Further extensions were presented in [150], [85] and [107]. These models are representation dependent, i.e. the underlying problem representation encoded as strings of zeros and ones appears directly in the given formulae. Consequently these models cannot be used for data schema transformations. We therefore take a slightly different approach. The definitions given here are representation independent. So, the formulae for transition matrices and success probabilities do not assume a particular problem representation. This has the advantage that our definitions may be used for entirely different problem domains. However, we focus on basic evolutionary algorithms, instead of the more complex algorithms considered in the above references.

The organization of the chapter is as follows. In section 8.2 the use of Markov chains for describing evolution dynamics is explained. Section 8.3 presents several new properties of the solution space of internal representations. These properties include local and global optima, which then are used for the definition of *critical* and *dead* representations. In section 8.4 transition matrices are given for basic evolutionary algorithms: random walk, hill climbing, steepest ascent and single/multiple improvement (see also chapter 6). Furthermore, fitness proportional selection mechanisms are considered. In section 8.5 probabilities are considered. We define success probability, first success probability, local optimum probability, and success entropy. The formulae given will be illustrated using the transition matrices from the previous section. The chapter is concluded with a discussion (section 8.6) and a summary.

8.2 Using Markov theory for evolution dynamics

In this section we show how Markov theory can be used for describing the dynamics of evolutionary algorithms. Let X_0 be the stochastic variable giving an initial internal representation, and let X_t be the

stochastic variable giving the internal representation resulting from evolution step t where $t \geq 1$. Then the series (chain) of variables $\{X_0, X_1, X_2, ...\} = \{X_n\}$ describes a database design process.

The *state space* of a chain $\{X_n\}$ coincides with the internal solution space $\mathcal{S}$ introduced in chapter 3. The *transitions* for a chain $\{X_n\}$ correspond with the mutations from chapter 5. The evolutionary algorithms from chapter 6 determine the *transition probabilities*. Since only the current representation, and not its predecessors, determines the possible transitions to a next representation, the chain satisfies the Markov property ([67]), and hence the chain is a Markov chain.

The evolutionary algorithms from chapter 6 have *stationary* transition probabilities, i.e. the probabilities are independent of the number of evolution steps that have already been made. This property allows the analysis to be based on the one step transition matrix P containing the transition probabilities.

The running example for this chapter is based on the information structure in figure 8.1. The solution space for this information structure consists of the following representations (for each representation the root set $\mathcal{R}$ is given). There are three representations with a single root: $\{p\}, \{q\}, \{q, r\}$, and four representations with two roots: $\{\{p\}, \{r\}\}, \{\{p\}, \{s\}\}$, $\{\{q\}, \{r\}\}$ and $\{\{q\}, \{s\}\}$. These representations are given in figure 8.2.

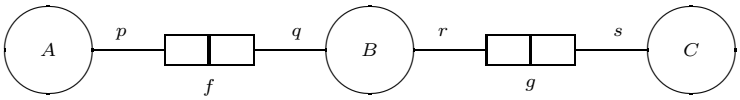


Figure 8.1: Running example

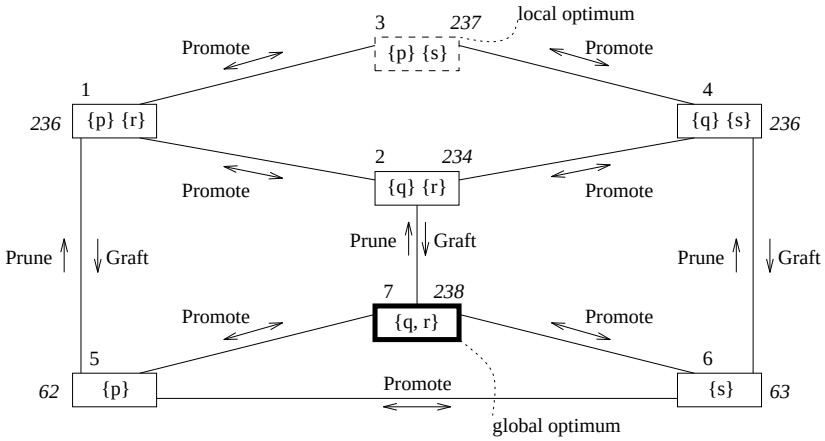


Figure 8.2: Solution space for running example

The representations in figure 8.2 are numbered from 1 to 7. Each representation has an associated fitness which is printed in italic. For example, representation 7 has fitness *238*. This is the highest fitness (global optimum). Representation 3 is a optimum?local. This terminology will be further explained in section 8.3.3. Figure 8.2 also indicates the possible mutations.

Note that all evolutionary algorithms have the same state space. They only differ in the way their transition matrix is defined (section 8.4).

The state space of a chain $\{Y_n\}$, describing the evolution of a *pool* consisting of N representations, is defined by the vectors $Y_n = (X_n^{(1)}, \ldots, X_n^{(N)})$. Here $X_n^{(i)}$ is the i-th representation in the pool at time n. Again the

Markov property is satisfied, and hence a chain describing the evolution of a pool is also a Markov chain. Pool evolution will not be considered in this chapter.

8.3 Properties of solution space

In order to prepare the Markov chain analysis we first examine the internal solution space $\mathcal{S}$ in more detail. The properties considered will be illustrated for the running example (see figure 8.2). Let P be a transition matrix, where $P(i,j)$ corresponds to the probability of a transition from representation i to representation j.

8.3.1 Solution space diameter

In section 5.5.2 the mutation distance $\mathsf{Dist_m}$ between two internal representations has been considered. We now define the *diameter* of solution space $\mathcal{S}$ as the maximum mutation distance between two representations:

$$\mathsf{Diam}(\mathcal{S}) = \max\{\mathsf{Dist_m}(i,j) \mid i,j \in \mathcal{S}\}$$

The diameter of the running example is 3. The following lemma gives an upperbound for the diameter:

Lemma 8.3.1 $\mathsf{Diam}(\mathcal{S}) \leq 3 \times |\mathcal{F}| - 2$

Proof:

In the proof of theorem 5.5.2 about completeness of the Prune, Graft and Promote operators, a maximum of $|\mathcal{F}| - 1$ prune applications, $|\mathcal{F}|$ promote applications and $|\mathcal{F}| - 1$ graft applications has to be performed. Hence the total number of mutations to be applied to representation i to mutate it into j never exceeds $3 \times |\mathcal{F}| - 2$. □

From this lemma it follows that given an arbitrary initial representation, there is a way to derive any other representation (e.g. a global optimum) in at most $3 \times |\mathcal{F}| - 2$ mutation steps. Unfortunately the mutation path itself is very hard, if not impossible, to obtain.

The n-steps transition matrix P^n can be used to consider the reachability of representations in $\mathcal{S}$. In order to guarantee that *all* representations can be reached, choose $n \geq \mathsf{Diam}(\mathcal{S})$.

8.3.2 Neighbour representations

Representation j is called a *neighbour* of representation i, if $\mathsf{Dist_m}(i,j) = 1$. We then write $\mathsf{Neighbour}(i,j)$. The set of neighbours of a given representation i now is defined by: $\mathsf{Neighbours}(i) = \{j \in \mathcal{S} | \mathsf{Neighbour}(i,j)\}$.

The number of neighbours provides an indication for the possible transitions and their probability. We denote this number by $\mathsf{NNb}(i)$. If $N_p \subseteq \mathcal{N}$ is the set of nodes containing predicators being type related to predicator p, the maximum number of neighbours of representation $i \in \mathcal{S}$ is given by:

Lemma 8.3.2 $\mathsf{NNb}(i) \leq |\mathcal{P}| - |\mathcal{F}| + \sum_{p:\exists f \in \mathcal{F}[\mathsf{Hook}(f)=p]} |N_p|$

Proof:

By counting the possibilities of the mutation operators. Since each neighbour is obtained via the application of a mutation operator, the number of neighbours never exceeds the number of possible mutations. The converse does not hold in general, since certain mutations produce the same neighbour

(e.g. cutting a hook from a root node consisting of two hooks: cutting the one or the other produces the same representation).

First consider the Promote mutation. The number of possibilities is $|\mathcal{N}| - |\mathcal{R}|$, assuming that the anchor to be promoted can be in any node except a root node (see section 5.4.5). According to lemma 3.2.3 this equals $|\mathcal{P}| - |\mathcal{F}|$.

Next consider the operators Prune and Graft. For each hook p the number of possibilities for pruning is 1. The number of possibilities for grafting does not exceed $|N_p| - 1$ (the node of which p is a part should not be considered, hence the subtraction of 1). Summarize over all hooks to obtain the lemma. □

A rather pessimistic, but *simple*, upper bound for the number of neighbours is as follows:

Lemma 8.3.3 $\forall_{i \in \mathcal{S}} [\mathsf{NNb}(i) \leq |\mathcal{P}| + |\mathcal{F}| \times (|\mathcal{P}| - |\mathcal{A}|)]$

Proof:

Consider the possible values for $|N_p|$ for a given predicator p. Since there are $|\mathcal{A}|$ atomic object types, there are also $|\mathcal{A}|$ classes of type related predicators. In each class at least one predicator must be contained, and hence a class holds a maximum of $|\mathcal{P}|-|\mathcal{A}|+1$ predicators. Now use $|N_p| \leq |\mathcal{P}|-|\mathcal{A}|+1$ in the previous lemma:

$$\mathsf{NNb}(i) \leq |\mathcal{P}| - |\mathcal{F}| + \sum_{p:\exists f \in \mathcal{F}[\mathsf{Hook}(f)=p]} |N_p|$$

We then get $\mathsf{NNb}(i) \leq |\mathcal{P}| - |\mathcal{F}| + |\mathcal{F}| \times (|\mathcal{P}| - |\mathcal{A}| + 1)$. From this the lemma follows. □

In the running example lemma 8.3.3 yields 4+2×(4-3)=6. Lemma 8.3.2 yields 4-2+1+1=4 for e.g. representation 7.

8.3.3 Local and global optimal representations

A representation $i \in \mathcal{S}$ is called a *local optimum*, denoted as $\mathsf{LocOpt}(i)$, if it has the following property:

$$\mathsf{Fitness}(i) \geq \max \{\mathsf{Fitness}(j) | \, \mathsf{Neighbour}(i,j)\}$$

A local optimum is a 'mountain top' in the so-called fitness landscape. A representation i is said to be a strong local optimum, denoted as $\mathsf{StrongLocOpt}(i)$, if:

$$\mathsf{Fitness}(i) > \max \{\mathsf{Fitness}(j) | \, \mathsf{Neighbour}(i,j)\}$$

Strong local optima may become absorbing states (i.e. a state that cannot be left) for hill climbing algorithms, since the only way to abandon such an optimum would be to descend (temporarily). A local optimum i is called a weak local optimum, denoted as $\mathsf{WeakLocOpt}(i)$, if:

$$\mathsf{Fitness}(i) = \max \{\mathsf{Fitness}(j) | \, \mathsf{Neighbour}(i,j)\}$$

Weak local optima do not necessarily become absorbing states for climbing algorithms. However, algorithms that accept only improvements (and no '0-improvements') have absorbing states for weak local optima as

well. Note that all algorithms have the same strong and weak local optima, but only for certain algorithms these form a problem as they become absorbing.

A *mathematical global optimum* is a representation with maximal fitness:

$$\mathsf{Fitness}(i) = \max\{\mathsf{Fitness}(j) \mid j \in \mathcal{S}\}$$

The global optima are the (high) 'mountain tops' one generally is looking for. A global optimum is also a local optimum.

We prefer a more general definition, since mathematical optima are not always aimed for in practice. A representation $i \in \mathcal{S}$ is called a *bounded global optimum*, denoted as $\mathsf{GlobOpt}(i)$, if $\mathsf{Fitness}(i) \geq F$, where F is some minimal required fitness value. Note that by setting

$$F = \max\{\mathsf{Fitness}(j) \mid j \in \mathcal{S}\}$$

the mathematical definition is obtained. A global optimum in the bounded definition is not necessarily a local optimum as well. Note that the number of bounded global optima may be high, depending on the definition of F.

In the left-hand part of figure 8.3 the number of global optima for an arbitrary information structure is plotted for various values of F. In this figure G is the number of 'mathematical' global optima. Of course the number of global optima as a function of F is discontinuous: each fitness value corresponds to the end of a 'step' in figure 8.3 (a black dot). The number of representations having exactly fitness F determines the difference with the next 'step'. The right-hand part of figure 8.3 gives the number of global optima for the running example.

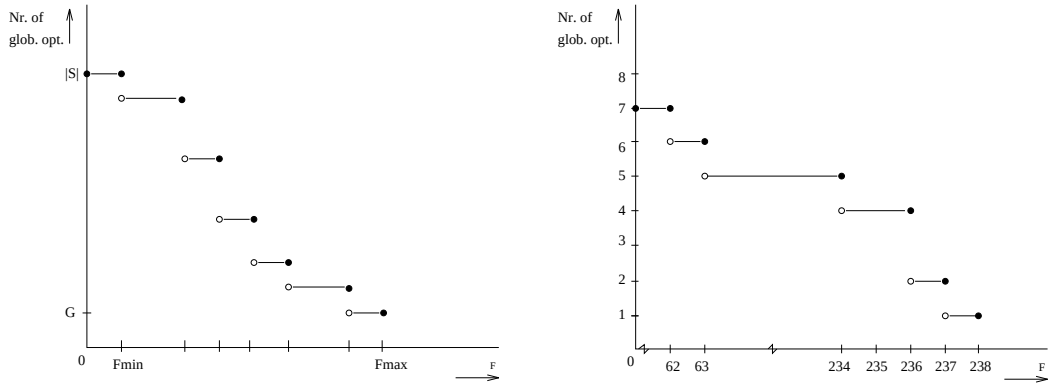


Figure 8.3: Number of global optima for various values of minimal required fitness F

8.3.4 Fitness plateaus

In section 5.5.2 the mutation distance and anchor distance between two representations was introduced. We now define the *fitness distance* between two representations i and j by:

$$\mathsf{Dist}_\mathsf{f}(i,j) = |\mathsf{Fitness}(i) - \mathsf{Fitness}(j)|$$

This definition for fitness distance is used for introducing *fitness plateaus*. A set of representations $\mathsf{Plat} \subseteq \mathcal{S}$ is called a δ-plateau if all $i, j \in \mathsf{Plat}$ have the following properties:

1. $\mathsf{Dist}_\mathsf{f}(i,j) \leq \delta$, where δ is some constant determining the maximum fitness difference between two representations of the plateau.

2. There exists a mutation path from representation i to j consisting only of representations being element of the plateau:

$$\exists_{k\geq 1,(a_1,\ldots,a_k)\in \mathsf{Plat}^k}[a_1 = i \wedge a_k = j \wedge \forall_{1\leq n<k}[\mathsf{Neighbour}(a_n, a_{n+1})]]$$

Consequently, the elements of a plateau are points of a connected graph. Note that any strong local optimum i implies a 0-plateau Plat which is a singleton set containing only i. For weak global optimum i a corresponding 0-plateau contains at least 2 elements.

8.3.5 Critical and dead representations during climbing

When climbing algorithms are used, there are representations from which no path (respecting the possible transitions) to a global optimum exists: these representations do not communicate with any global optimum. Once such a 'dead' representation is visited, no global optimum can be reached. More precisely, a representation i is called *dead*, denoted as $\mathsf{Dead}(i)$, if:

$$\neg\exists_{\mathsf{GlobOpt}(g)}[i \to g]$$

Non-dead representations will be called *living*. Of course this definition depends upon the specific evolutionary algorithm in use. This dependence is reflected through the transition matrix P (which is the basis for the communication relation).

For algorithms with a local optimum problem, the local optima are dead. However, dead representations are not necessarily local optima, since a dead representation can still allow improvement. In situations where a 0-plateau Plat forms a closed set ($\forall_{i\in\mathsf{Plat},j\notin\mathsf{Plat}}[p_{ij} = 0]$) not containing a global optimum, no global optimum can be reached from any representation of the plateau. Such a plateau is also called 'dead', because all its representations are dead.

Furthermore there are representations from which a global optimum can be reached and which communicate with at least one dead neighbour. Although these representations do communicate with a global optimum, there is a possibility that a neighbour which does not communicate with a global optimum is accepted. If the algorithm makes the wrong decision at such a point (i.e. move to a dead neighbour), again no global optimum will be found. If on the other hand a living neighbour is chosen, there is still a possibility of finding a global optimum. Representations with these properties are called *critical*. More precisely, a representation i is called critical, denoted as $\mathsf{Critical}(i)$, if:

$$\neg\mathsf{Dead}(i) \wedge \exists_{j\in\mathcal{S}}[P(i,j) > 0 \wedge \mathsf{Dead}(j)]$$

The following lemma provides a way to indentify the dead representations when the transition matrix P is given:

Lemma 8.3.4 $\mathsf{Dead}(i) \Longleftrightarrow \forall_{0\leq k\leq|\mathcal{S}|,\mathsf{GlobOpt}(g)}[P^k(i,g) = 0]$

Proof:

Dead representations do not communicate with global optima, or:

$$\forall_{k\geq 0,\mathsf{GlobOpt}(g)}[P^k(i,g) = 0]$$

Note that P^k covers exactly those paths that have length k. A result known from graph theory is that if there exists a path with length $k > |\mathcal{S}|$, there also exists a path of length $\ell \leq |\mathcal{S}|$ (by repeated removal of cycles). □

With this lemma and the transition matrix P, the dead points can be identified by computing $P^0, P^1, P^2, P^3, \ldots, P^{|\mathcal{S}|}$. If for a given i all entries $P^k(i, g)$, with g a global optimum, are zero, then i is dead. If on the other hand at least one entry $P^k(i, g)$ is non zero, i is living.

In figure 8.4 the solution space is divided into two components: a dead and a living part.

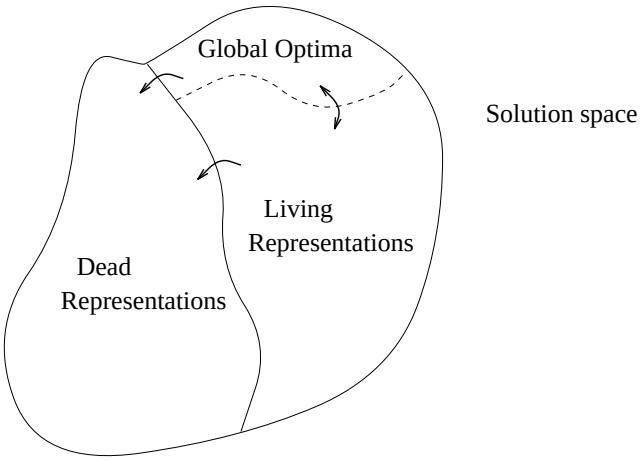


Figure 8.4: Living and dead points in solution space

The global optima in figure 8.4 form a special component of the living part. The critical representations will be located close to the border, as they are alive themselves, but communicate with at least one dead neighbour. Local optima may be dead or living, depending on the evolutionary algorithm and its parameter settings.

By definition, there are no transitions from dead representations to living representations, and hence the dead representations form the state space for a *sub Markov chain*, which is a Markov chain itself. It is in this part all absorbing local optima can be found.

For the position of strong local optima (SLO) three cases can be distinguished. These are illustrated in figure 8.5.

1. All strong local optima are alive ($\mathsf{AllStrongAlive}(\mathcal{S}, P)$):

$$\forall_{i \in \mathcal{S}, \neg \mathsf{GlobOpt}(i)} [\mathsf{StrongLocOpt}(i) \Rightarrow \neg \mathsf{Dead}(i)]$$

   This case corresponds to the lefthand situation in figure 8.5.

2. The opposite case where all strong local optima are dead ($\mathsf{AllStrongDead}(\mathcal{S}, P)$):

$$\forall_{i \in \mathcal{S}, \neg \mathsf{GlobOpt}(i)} [\mathsf{StrongLocOpt}(i) \Rightarrow \mathsf{Dead}(i)]$$

   The center part of figure 8.5 illustrates this case.

3. And the general case, where some strong local optima are dead and others are alive ($\mathsf{StrongPartialDead}(\mathcal{S}, P)$):

$$\neg(\mathsf{AllStrongAlive}(\mathcal{S}, P) \vee \mathsf{AllStrongDead}(\mathcal{S}, P))$$

   The righthand side of figure 8.5 illustrates the general case.

For weak local optima similar cases exist. The precidates AllWeakAlive, AllWeakDead and WeakPartialDead are defined analogously. From the above it can be concluded that the 'dead representation' problem is even bigger than the more apparent local optimum problem.

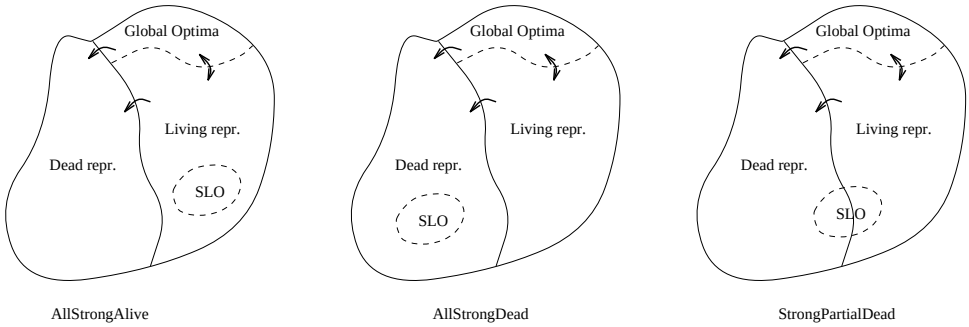


Figure 8.5: Three possibilities w.r.t. the location of strong local optima

8.4 Transition matrices

In this section transition matrices are given for random walk, hill climbing, steepest ascent and single/multiple improvement. Furthermore, random walk and hill climbing are generalized to fitness proportional walk and fitness proportional hill climbing.

8.4.1 Random walk

First random walk is considered (see also section 6.5.5 and 7.3.2). In a random walk, for a given representation i a neighbour j is chosen without method or conscious choice. We assume that the transitions are fair: transition to one neighbour is equally likely as transition to another. Then, the corresponding transition matrix P_r is defined by:

$$p_{ij} = \begin{cases} \frac{1}{\mathsf{NNb}(i)} & \text{if } \mathsf{Neighbour}(i,j) \\ 0 & \text{otherwise} \end{cases}$$

It is easily verified that P_r is a stochastic matrix, i.e. $0 \leq p_{ij} \leq 1$ and $\sum_{j=1}^{|\mathcal{S}|} p_{ij} = 1$. The following lemma expresses that random walk is fully alive:

Lemma 8.4.1 $\mathsf{AllStrongAlive}(\mathcal{S}, P_r) \wedge \mathsf{AllWeakAlive}(\mathcal{S}, P_r)$

Proof:

In a random walk all local optima can be starting-point of a sequence of transitions leading to a global optimum. Consequently all local optima are alive. □

The above lemma indicates that random walk has no problems with dead or local optimal representations. However, random walk is a 'blind' search, which suggests that it suffers from convergence problems.

The transition matrix for the running example is given in the left part of figure 8.6.

$$\begin{bmatrix} 0 & 1/3 & 1/3 & 0 & 1/3 & 0 & 0 \\ 1/3 & 0 & 0 & 1/3 & 0 & 0 & 1/3 \\ 1/2 & 0 & 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1/3 & 1/3 & 0 & 0 & 1/3 & 0 \\ 1/3 & 0 & 0 & 0 & 0 & 1/3 & 1/3 \\ 0 & 0 & 0 & 1/3 & 1/3 & 0 & 1/3 \\ 0 & 1/3 & 0 & 0 & 1/3 & 1/3 & 0 \end{bmatrix} \qquad \begin{bmatrix} 2/3 & 0 & 1/3 & 0 & 0 & 0 & 0 \\ 1/3 & 0 & 0 & 1/3 & 0 & 0 & 1/3 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1/3 & 2/3 & 0 & 0 & 0 \\ 1/3 & 0 & 0 & 0 & 0 & 1/3 & 1/3 \\ 0 & 0 & 0 & 1/3 & 0 & 1/3 & 1/3 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 8.6: Example matrices for random walk and hill climbing

The random walk transition matrix in figure 8.6 is explained as follows. The first row contains all transition probabilities from representation 1 to another representation. In the running example (figure 8.2) we see that representation 1 allows three transitions: promotion to representation 2 or 3 and grafting to representation 5. Each transition has the same probability which then becomes $\frac{1}{3}$, because otherwise the resulting matrix would not be a stochastic matrix. Note that $p_{ii} = 0$ for all $i \in \mathcal{S}$.

8.4.2 Hill climbing

Hill climbing exists in several variants (see also section 6.5.5 and 7.4.1). In this section the following variant is used. For a given representation i a neighbour j is generated at random. Then i is replaced by j if j has a fitness not lower than the fitness of i. The corresponding transition matrix P_h is based on the set of better (or equally good) neighbours defined by:

$$B_i = \{j \in \mathsf{Neighbours}(i) \mid \mathsf{Fitness}(j) \geq \mathsf{Fitness}(i)\}$$

Now the transition matrix is defined as follows. If B_i is empty then p_{ij} is defined by the Kronecker delta function δ_{ij} = (if $i = j$ then 1 else 0). For nonempty B_i it is defined by:

$$p_{ij} = \begin{cases} \frac{1}{\mathsf{NNb}(i)} & \text{if } j \in B_i \\ 1 - \frac{|B_i|}{\mathsf{NNb}(i)} & \text{if } j = i \\ 0 & \text{otherwise} \end{cases}$$

It is easily verified that P_h is a stochastic matrix. Note that hill climbing has a local optimum problem: if a representation i only has neighbours with a lower fitness (i is a strong local optimum, $B_i = \phi$) the chain will remain in this state forever (i is absorbing) without finding any new representations. This is a well-known disadvantage of hill climbing. The following lemma expresses this problem.

Lemma 8.4.2 Local optima during climbing: $\mathsf{AllStrongDead}(\mathcal{S}, P_h)$.

Proof:

All strong local optima are absorbing under hill climbing, since the only way to leave a strong local optimum would be to accept a worsening. Hence all strong local optima, not being a global optimum as well, are dead. □

Some weak local optima may be found on dead 0-plateaus, while others will be located on a living 0-plateau. So in general $\mathsf{WeakPartialDead}(\mathcal{S}, P_h)$.

The transition matrix for the running example is given in the right part of figure 8.6. This matrix is explained as follows. In the running example (figure 8.2) we see that representation 1 allows three transitions, and only one transition results in fitness improvement. So, the other two transitions have probability 0 and the improving transition has probability $\frac{1}{3}$. Then, p_{11} has to be $\frac{2}{3}$, because otherwise the resulting matrix would not be a stochastic matrix. Note that representations 3 (local optimum) and 7 (global optimum) correspond to absorbing states ($p_{ii} = 1$).

When hill climbing is applied, an efficiency problem occurs. The basis of this problem is expressed in the following lemma:

Lemma 8.4.3 Let T_i be the stochastic variable representing the number of consecutive transitions from i to i, where i is a non absorbing state. Then:

$$E[T_i] = \frac{\mathsf{NNb}(i)}{|B_i|} - 1$$

Proof:

The probability for exactly n consecutive transitions from i to i is given by:

$$Prob(T_i = n) = (p_{ii})^n \times (1 - p_{ii})$$

As a consequence:

$$E[T_i] = \sum_{n=0}^{\infty} n \times (p_{ii})^n \times (1 - p_{ii}) = \frac{p_{ii}}{1 - p_{ii}}$$

since $\sum_{n=0}^{\infty} n \times x^n = x/(1-x)^2$ for $0 \leq x < 1$.

For hill climbing $p_{ii} = 1 - \frac{|B_i|}{\mathsf{NNb}(i)}$. Now the lemma follows. □

From this lemma it can be concluded that if representation i has relatively many worse neighbours, there will be many superfluous transitions from i to i. Note that if i is a strong local optimum (i is absorbing) the lemma does not apply. However, in that case $E[T_i] = \infty$: the evolution process will remain in the strong local optimum forever.

8.4.3 Steepest ascent

Steepest ascent always aims at a maximal improvement (see also section 6.5.5 and 7.4.1). It does not accept any neighbour other than the best. Such a neighbour is not necessarily unique. Let the set of 'maximal better' neighbours MB_i of representation i be defined as follows:

$$\mathsf{MB}_i = \{j \in \mathsf{Neighbours}(i) \mid \forall_{k \in \mathsf{Neighbours}(i)}[\mathsf{Fitness}(j) \geq \mathsf{Fitness}(k)]\}$$

Now the corresponding transition matrix P_a is defined as follows. If no better neighbours exist ($B_i = \varnothing$) then p_{ij} is defined by the Kronecker delta function δ_{ij}. For nonempty B_i it is defined by:

$$p_{ij} = \begin{cases} \frac{1}{|\,\mathsf{MB}_i\,|} & \text{if } j \in \mathsf{MB}_i \\ 0 & \text{otherwise} \end{cases}$$

The following lemma expresses that steepest ascent is indeed a climber:

Lemma 8.4.4 $\mathsf{AllStrongDead}(\mathcal{S}, P_a)$.

Some weak local optima may be found on dead 0-plateaus, while others will be located on a living 0-plateau. Hence in general $\mathsf{WeakPartialDead}(\mathcal{S}, P_a)$.

The matrix for the running example is given in the left part of figure 8.7. From this matrix it can be concluded that in the running example each representation has at most one 'maximal better' neighbour ($|\,\mathsf{MB}_i\,| \leq 1$ for all i): evolution becomes *deterministic*. In other cases this situation may occur as well, since the probability that a representation has two or more neighbours with exactly the same and maximal fitness is expected to be low (but depends on the definition of the fitness function). This property is typical for steepest ascent, and has consequences w.r.t. the choice of initial representations (see also section 8.5.4).

The matrix in figure 8.7 also shows that the running example forces steepest ascent to arrive in an absorbing state in at most one step. This absorbing state is either the local optimum (representation 3) or the global optimum (representation 7), depending on the initial state.

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \qquad \begin{bmatrix} \frac{5}{12} & \frac{1}{12} & \frac{1}{4} & 0 & \frac{1}{4} & 0 & 0 \\ \frac{1}{4} & \frac{1}{6} & 0 & \frac{1}{4} & \frac{1}{3} & 0 & 0 \\ \frac{1}{8} & 0 & \frac{3}{4} & \frac{1}{8} & 0 & 0 & 0 \\ 0 & \frac{1}{12} & \frac{1}{4} & \frac{5}{12} & 0 & \frac{1}{4} & 0 \\ \frac{1}{4} & 0 & 0 & 0 & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ 0 & 0 & 0 & \frac{1}{4} & \frac{1}{12} & \frac{5}{12} & \frac{1}{4} \\ 0 & \frac{1}{12} & 0 & 0 & \frac{1}{12} & \frac{1}{12} & \frac{3}{4} \end{bmatrix}$$

Figure 8.7: Example matrices for steepest ascent and S/M improvement

8.4.4 Single/multiple improvement

Next we consider evolution under single/multiple improvement (see also section 6.2.7). Single/multiple improvement is not based on a combined fitness function, but on the separate Time and Space components. It is a general approach, based on optimization in two directions (time and space). For example, it is possible to use one algorithm (e.g. hill climbing) to find representations with time improvement, another algorithm to do the same for space improvement. Then the results are combined. If there are any neighbours found by both 'child' algorithms, one of these is selected as a next representation (since these representations provide multiple improvement, both on time and space). If there are no neighbours in common, then one neighbour from the union is selected (such a neighbour provides a single improvement, either on time or on space). Finally if no improvement whatsoever can be made, either a disimprovement is accepted, or an absorbing state is entered (depending on the exact variant of single/multiple improvement).

The basic idea is displayed in figure 8.8. Define MI_i as the set of neighbours of i leading to multiple improvement, and define NI_i as the set of neighbours which do not lead to any improvement:

$$\begin{aligned} \mathsf{MI}_i &= \{j \in \mathsf{Neighbours}(i) | \mathsf{Time}(j) < \mathsf{Time}(i) \wedge \mathsf{Space}(j) < \mathsf{Space}(i)\} \\ \mathsf{NI}_i &= \{j \in \mathsf{Neighbours}(i) | \mathsf{Time}(j) > \mathsf{Time}(i) \wedge \mathsf{Space}(j) > \mathsf{Space}(i)\} \end{aligned}$$

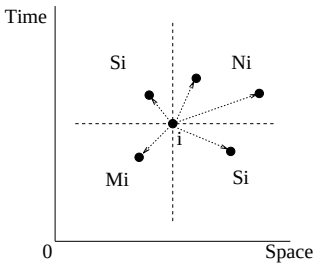


Figure 8.8: Areas for single/multiple improvement

Define the neighbours leading to a *single* improvement by: $\mathsf{SI}_i = \mathsf{Neighbours}(i) - \mathsf{MI}_i - \mathsf{NI}_i$. Furthermore define $m_i = |\,\mathsf{MI}_i\,|$, $s_i = |\,\mathsf{SI}_i\,|$ and $n_i = |\,\mathsf{NI}_i\,|$. Then the corresponding transition matrix is defined as follows:

$$m_i > 0 \Rightarrow p_{ij} = \begin{cases} \frac{1}{m_i} & \text{if } j \in \mathsf{MI}_i \\ 0 & \text{otherwise} \end{cases}$$

$$m_i = 0 \wedge s_i > 0 \Rightarrow p_{ij} = \begin{cases} \frac{1}{s_i} & \text{if } j \in \mathsf{SI}_i \\ 0 & \text{otherwise} \end{cases}$$

$$m_i = 0 \wedge s_i = 0 \Rightarrow p_{ij} = \begin{cases} \frac{1}{n_i} & \text{if } j \in \mathsf{NI}_i \\ 0 & \text{otherwise} \end{cases}$$

This corresponds with an algorithm that checks all neighbours, computes the sets MI_i, SI_i and NI_i, and makes transitions according to the general single/multiple improvement idea. A disadvantage of this algorithm is that all neighbours and their fitness values must be computed. We therefore prefer the approach presented in figure 8.9.

```
Single/multiple improvement:
    Generate y IN Neighbours(i);
    if y ∈ MI_i
    then Replace(i, y)
    else if y ∈ SI_i
            then Replace(i, y) WITH Probability λ_S
            else Replace(i, y) WITH Probability λ_N
            end
    end
```

Figure 8.9: Alternative single/multiple improvement algorithm

The corresponding transition matrix P_{sm} is defined by:

$$p_{ij} = \begin{cases} \frac{1}{\mathsf{NNb}(i)} & \text{if } j \in \mathsf{MI}_i \\ \frac{\lambda_S}{\mathsf{NNb}(i)} & \text{if } j \in \mathsf{SI}_i \\ \frac{\lambda_N}{\mathsf{NNb}(i)} & \text{if } j \in \mathsf{NI}_i \\ 1 - \frac{m_i + \lambda_S \cdot s_i + \lambda_N \cdot n_i}{\mathsf{NNb}(i)} & \text{if } j = i \\ 0 & \text{otherwise} \end{cases}$$

where λ_S is the probability a neighbour will be selected if it provides a single improvement, and λ_N is a similar probability for neighbours that provide only disimprovement. The probabilities λ_S and λ_N are configuration parameters for this algorithm.

If single/multiple improvement is used, then in general (for $\lambda_S > 0, \lambda_N > 0$):

Lemma 8.4.5 $\mathsf{AllStrongAlive}(\mathcal{S}, P_{sm}) \wedge \mathsf{AllWeakAlive}(\mathcal{S}, P_{sm})$

In case $\lambda_S = \lambda_N = 0$, then $\mathsf{AllStrongDead}(\mathcal{S}, P_{sm}) \wedge \mathsf{AllWeakDead}(\mathcal{S}, P_{sm})$.

The matrix for the running example is given in the right part of figure 8.7. In the example λ_S has been set to $\frac{3}{4}$ and λ_N to $\frac{1}{4}$. Of course λ_S and λ_N should be chosen carefully as p_{ii} increases linear with a decrease of λ_S or λ_N. Note that the values in the P_{sm} matrix in figure 8.7 cannot be computed directly from the running example in figure 8.2, since no time/space values are shown.

8.4.5 Fitness proportional walk

Fitness proportional algorithms attempt to guide the evolution process by making transition probabilities proportional to the fitness of neighbours. The idea is to prefer transitions to better neighbours while still allowing a decreasing fitness, for example in order to leave a local optimum. The corresponding transition matrix P_{f1} can be defined as follows:

$$p_{ij} = \begin{cases} \frac{\mathsf{Fitness}(j)}{\sum_{\mathsf{Neighbour}(i,k)} \mathsf{Fitness}(k)} & \text{if } \mathsf{Neighbour}(i,j) \\ 0 & \text{otherwise} \end{cases}$$

The matrix for the running example is shown in the left part of figure 8.10. Note that $p_{ii} = 0$ for all i.

A disadvantage of the definition above is that each step requires a considerable amount of computation, since all neighbours and their fitness have to be computed in order to determine the transition probability.

$$\begin{bmatrix} 0 & \frac{18}{41} & \frac{237}{533} & 0 & \frac{62}{533} & 0 & 0 \\ \frac{118}{355} & 0 & 0 & \frac{118}{355} & 0 & 0 & \frac{119}{355} \\ \frac{1}{2} & 0 & 0 & \frac{1}{2} & 0 & 0 & 0 \\ 0 & \frac{39}{89} & \frac{79}{178} & 0 & 0 & \frac{21}{178} & 0 \\ \frac{236}{537} & 0 & 0 & 0 & 0 & \frac{21}{179} & \frac{238}{537} \\ 0 & 0 & 0 & \frac{59}{134} & \frac{31}{268} & 0 & \frac{119}{268} \\ 0 & \frac{234}{359} & 0 & 0 & \frac{62}{359} & \frac{63}{359} & 0 \end{bmatrix} \qquad \begin{bmatrix} \frac{47}{70} & \frac{11}{70} & \frac{6}{35} & 0 & 0 & 0 & 0 \\ \frac{37}{210} & \frac{7}{15} & 0 & \frac{37}{210} & 0 & 0 & \frac{19}{105} \\ \frac{17}{70} & 0 & \frac{18}{35} & \frac{17}{70} & 0 & 0 & 0 \\ 0 & \frac{11}{70} & \frac{6}{35} & \frac{47}{70} & 0 & 0 & 0 \\ \frac{1}{3} & 0 & 0 & 0 & \frac{17}{105} & \frac{6}{35} & \frac{1}{3} \\ 0 & 0 & 0 & \frac{1}{3} & \frac{17}{105} & \frac{6}{35} & \frac{1}{3} \\ 0 & \frac{31}{210} & 0 & 0 & 0 & 0 & \frac{179}{210} \end{bmatrix}$$

Figure 8.10: Example matrices for both variants of fitness proportional walk

Similarly to random walk (lemma 8.4.1), fitness proportional walk is fully alive:

Lemma 8.4.6 $\mathsf{AllStrongAlive}(\mathcal{S}, P_{f1}) \wedge \mathsf{AllWeakAlive}(\mathcal{S}, P_{f1})$.

The exhaustive computation in the definition of P_{f1} can be avoided by using a *bounded* fitness proportional selection mechanism. Then it is not necessary to compute all neighbours. The corresponding transition matrix P_{f2} is based on a function $fprop$ capturing the fitness proportional part:

$$p_{ij} = \begin{cases} \frac{fprop(i,j)}{\mathsf{NNb}(i)} & \text{if } \mathsf{Neighbour}(i,j) \\ 1 - \frac{1}{\mathsf{NNb}(i)} \sum_{k \in \mathsf{Neighbours}(i)} fprop(i,k) & \text{if } j = i \\ 0 & \text{otherwise} \end{cases}$$

Function $fprop$ can be further parameterized with threshold constants $T_{min}, T_{max} \geq 0$. Here T_{min} is an upperbound for the fitness worsening that will be accepted proportionally, and T_{max} is an upperbound for the fitness improvement that will be accepted proportionally:

$$fprop(i,j) = \begin{cases} 0 & \text{if } \mathsf{Fitness}(j) \leq \mathsf{Fitness}(i) - T_{min} \\ \frac{\mathsf{Fitness}(j) - \mathsf{Fitness}(i) + T_{min}}{T_{min} + T_{max}} & \text{if } \mathsf{Fitness}(i) - T_{min} < \mathsf{Fitness}(j) < \mathsf{Fitness}(i) + T_{max} \\ 1 & \text{otherwise} \end{cases}$$

So, any neighbour which results in a fitness improvement of at least T_{max} will be selected with probability one. Neighbours which result in a smaller improvement, or even in a worsening (restricted by T_{min}), will be selected with a probability proportional to the improvement. Neighbours with a worsening of T_{min} or more will not be selected at all. Note that if $T_{max} = 0$ and $T_{min} \approx 0$ the algorithm becomes identical to hill climbing[1].

The matrix for the running example is shown in the right part of figure 8.10. From this matrix it is clear that $p_{ii} \neq 0$. So, although the exhaustive computation in the first definition of fitness proportional walk (see above) is avoided, the bounded definition has the problem of doing superfluous transitions from i to i. Furthermore it is clear that the two definitions result in quite different transition probabilities (see figure 8.10). Of course this also depends on the seting of parameters T_{min} and T_{max}. In the example T_{min} and T_{max} have been set to $35 \approx \frac{238}{7}$.

For 'average' T_{min} the second variant of fitness proportional walk is partially alive:

[1] Actually, take $T_{min} > 0$, but smaller than the smallest fitness difference between neighbours with a different fitness.

Lemma 8.4.7

$$\mathsf{StrongPartialDead}(\mathcal{S}, P_{f2}) \wedge \mathsf{WeakPartialDead}(\mathcal{S}, P_{f2})$$

For sufficiently large T_{min} all local optima will be alive, whereas for very small values of T_{min} all local optima will be dead.

The above lemma indicates the strong influence of the parameter T_{min} on the presence of dead points. Hence this parameter should be chosen with care. Small values for T_{min} result in higher p_{ii} values, because the worst neighbours are no longer accepted. However, using a large T_{min} might lead to the situation where too many bad neighbours are accepted.

For T_{max} a similar property holds. For large T_{max} relatively many neighbours are accepted proportionally (instead of acceptance with probability one). This again leads to a higher p_{ii}, which might result in an efficiency problem during evolution (see also the proof of lemma 8.4.3).

In figure 8.11 the average value of p_{ii} (w.r.t. all $i \in \mathcal{S}$) is plotted as a function of T_{max} for various values of T_{min}. This plot relates to the running example. Note that for values of T_{max} which correspond to a fitness difference between two neighbours, a 'break' occurs in the plot (e.g. 1, 2 and 4). This is the result of changing proportional acceptance to acceptance with probability one, or vice versa (2nd and 3rd case of *fprop*). Similarly, if $T_{min}, T_{max} \leq 1$ there is no proportional behaviour, resulting in a horizontal plot (see figure 8.11).

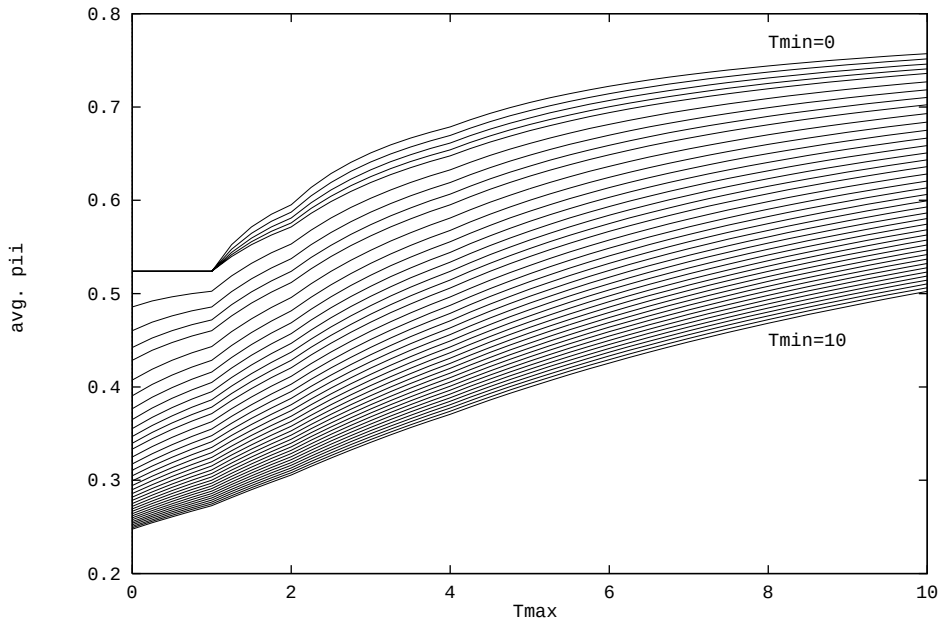


Figure 8.11: Average value of p_{ii} as a function of T_{max} for various values of T_{min}

8.4.6 Fitness proportional hill climbing

Analogous with fitness proportional walk we now consider fitness proportional hill climbing. We give the corresponding fitness matrices P_{h1} and P_{h2}.

First P_{h1} based on exhaustive computation of all neighbours is considered. Let B_i be the set of better (or equally good) neighbours of representation i. If $B_i = \varnothing$ then $p_{ij} = \delta_{ij}$. For nonempty B_i the transition probability is given by:

$$p_{ij} = \begin{cases} \frac{\mathsf{Fitness}(j)}{\sum_{k \in B_i} \mathsf{Fitness}(k)} & \text{if } j \in B_i \\ 0 & \text{otherwise} \end{cases}$$

In this case $\mathsf{AllStrongDead}(\mathcal{S}, P_{h1})$. In general some weak local optima can be found on dead 0-plateaus, while others will be located on a living 0-plateau. Hence in general $\mathsf{WeakPartialDead}(\mathcal{S}, P_{h1})$. Note that this definition of hill climbing does not have the efficiency problems the hill climber from section 8.4.2 has.

The corresponding transition matrix P_{h2} is based on a function *hprop* capturing the fitness proportional part:

$$p_{ij} = \begin{cases} \frac{hprop(i,j)}{\mathsf{NNb}(i)} & \text{if } j \in B_i \\ 1 - \frac{1}{\mathsf{NNb}(i)} \sum_{k \in B_i} hprop(i,k) & \text{if } j = i \\ 0 & \text{otherwise} \end{cases}$$

Similarly to the function *fprop* from the previous section, the function *hprop* can be parameterized with $T_{max} \geq 0$ specifing the maximum fitness improvement that is accepted fitness proportionally. Furthermore, we use parameter λ_P ($0 \leq \lambda_P \leq 1$ specifying the probability of accepting a neighbour in the current 0-plateau (once this neighbour has been generated). The function *hprop* then is defined as:

$$hprop(i,j) = \begin{cases} 0 & \text{if } \mathsf{Fitness}(j) < \mathsf{Fitness}(i) \\ \lambda_P & \text{if } \mathsf{Fitness}(j) = \mathsf{Fitness}(i) \\ \frac{\mathsf{Fitness}(j) - \mathsf{Fitness}(i)}{T_{max}} & \text{if } 0 < \mathsf{Fitness}(j) - \mathsf{Fitness}(i) < T_{max} \\ 1 & \text{otherwise} \end{cases}$$

Parameter λ_P can be set to a value above 0 to assure that weak local optima do not become absorbing (in contrast with strong local optima). Note that if T_{min} of P_{f2} is set to 0, the definition of P_{h2} is obtained (with λ_P set to 0).

Here we have $\mathsf{AllStrongDead}(\mathcal{S}, P_{h2})$. If $\lambda_P = 0$, then $\mathsf{AllWeakDead}(\mathcal{S}, P_{h2})$. In general ($\lambda_P > 0$) some weak local optima can be found on dead 0-plateaus, while others will be located on a living 0-plateau. Hence in general $\mathsf{WeakPartialDead}(\mathcal{S}, P_{h2})$.

8.5 Basic properties of evolutionary algorithms

In this section we consider basic properties of evolutionary algorithms: success probability, first success probability, local optimum probability, and success entropy. An important role is played by the *reduced* transition matrix, which is defined as follows:

$$P_{red}(i,j) = \begin{cases} 0 & \text{if } \mathsf{GlobOpt}(i) \vee \mathsf{GlobOpt}(j) \\ P(i,j) & \text{otherwise} \end{cases}$$

So the reduced matrix P_{red} resembles P with the exception of global optima. Besides the reduced matrix we use the abbreviations given in figure 8.12 for initialization, success and first success.

In [153] several other properties are considered, including the expected time to find a global optimum and the expected fitness after a given number of evolution steps.

notation	explanation	definition
I_i	initial representation is i	$X_0 = i$
S_k	success within k steps	$\exists_{n \leq k}[\mathsf{GlobOpt}(X_n)]$
FS_k	first success after k steps	$\mathsf{GlobOpt}(X_k) \wedge \neg S_{k-1}$

Figure 8.12: Abbreviations for initialization and success

8.5.1 Success probability

In this section we consider the probability that at least one global optimum is found within a given (predetermined) number of steps $k \geq 1$. This probability is given by:

Theorem 8.5.1 $Prob(S_k \mid I_i) = 1 - \sum_{j \in \mathcal{S}} P^k_{red}(i,j)$

Proof:

Observe that $Prob(S_k \mid I_i\ ) = 1 - Prob(\neg S_k \mid I_i\ )$. The latter probability can be interpreted as the probability of a walk from i to some j, without ever visiting a global optimum. For a given j this probability is $P^k_{red}(i,j)$, since the reduced matrix excludes exactly those paths with a global optimum. Now summarize over all possible j to obtain the theorem. □

The success graphs for the running example are given in figure 8.13.

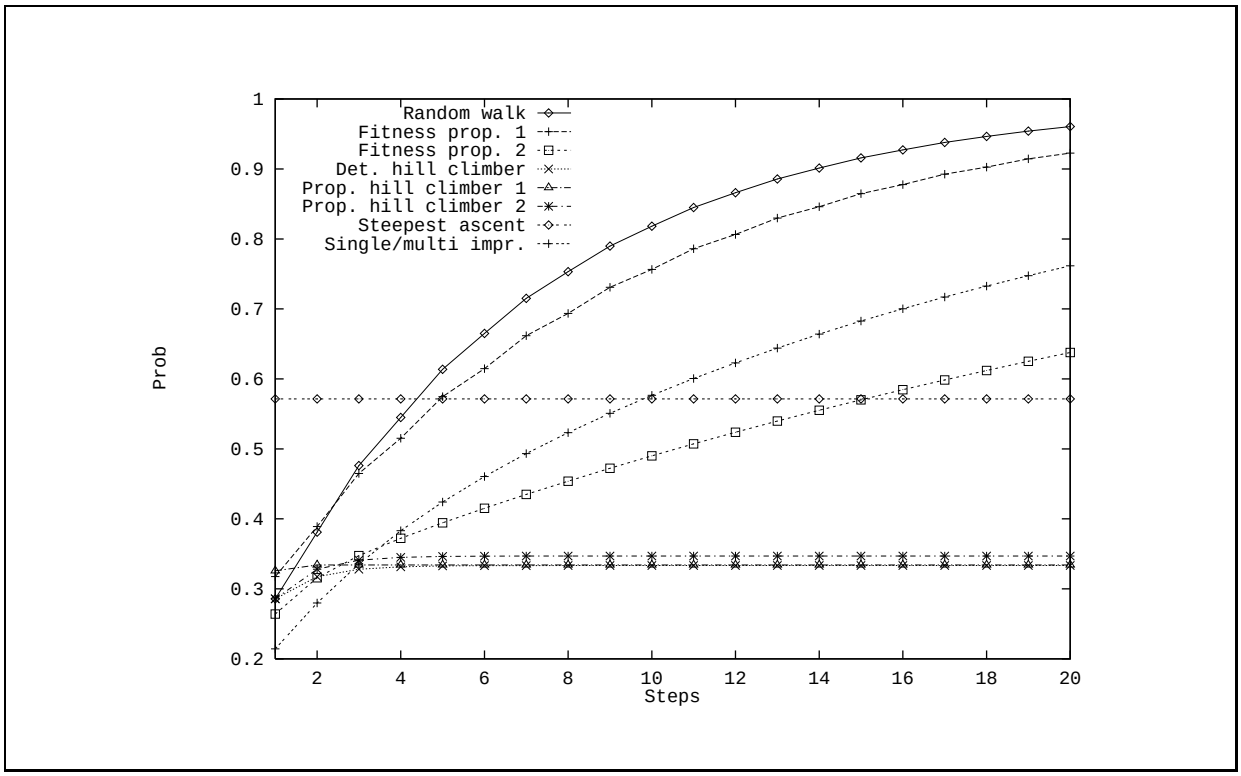


Figure 8.13: Success graphs for running example

From the graphs in figure 8.13 it can be concluded that hill climbing (including the proportional variants) performs rather poor for the running example. This can be explained by the fact that the local optimum

(representation 3) is quite tempting for hill climbers, but leads them into an undesired absorbing state. Another remarkable thing is the fact that hill climbing with deterministic replacement and hill climbing with fitness proportional replacement perform almost exactly the same. This suggests that, at least for the running example, proportional hill climbing is not worth the extra work. The hill climbers show approximately the same probabilities for small k, and since absorbtion (either at representation 3 or 7) becomes quite certain for larger values of k, the plots are approximately identical in the long run as well.

The fitness proportional walk (both variants) and single/multiple improvement perform somewhat worse than random walk. Although random walk is known to perform very poor in general, in the running example it does not. An explanation again relates to representation 3. Although fitness proportional walk and single/multiple improvement do not have a local optimum problem, they are still attracted to representation 3 since its fitness value is very close to the global optimum. However, its distance to the actual global optimum (7) is 3 (equal to the diameter), which causes a more difficult reachability of representation 7. Furthermore, parameters T_{min} and T_{max} of the second fitness proportional walk, and parameters λ_S and λ_N of single/multiple improvement have a substantial influence on the results (see e.g. the analysis of p_{ii} values in figure 8.11). For single/multiple improvement there is only one transition yielding a multiple improvement: representation 2 to 7 (the time/space values in the running example were omitted). So, more single improvement or no improvement steps will be made, being more dependent on the setting of parameters λ_S and λ_N.

Only for algorithms without dead point problem ultimate success seems certain, as the plots for these algorithms approach one for large k. This is expressed by the following lemma:

Lemma 8.5.1 $\forall_{i\in\mathcal{S}}[\lim_{k\to\infty} Prob(S_k \mid I_i) = 1] \Longleftrightarrow \forall_{i\in\mathcal{S}}[\neg\mathsf{Dead}(i)]$

Proof:

For algorithms without dead representations, all non global optimal representations communicate with a global optimum. Consequently, in the long run a global optimum will be found with probability 1. For algorithms with dead points however, ultimate succes is not certain. □

8.5.2 First success probability

In this section we consider the probability that an evolutionary algorithm finds its *first* global optimum at the k-th step ($k \geq 1$), given the initial state is i. This probability is given by:

Theorem 8.5.2 $Prob(FS_k \mid I_i) = \sum_{\mathsf{GlobOpt}(g)} \sum_{j\in\mathsf{Neighbours}(g)} P_{red}^{k-1}(i,j)p_{jg}$

Proof:

Let i be the initial state and let g be a global optimum. The probability that g is the first global optimum found at the k-th step is equal to the probability of 'traversing' a path from i of length $k-1$ steps, without visiting a global optimum, to some neighbour j of g and then making the transition from j to g. For a given j this probability equals $P_{red}^{k-1}(i,j)p_{jg}$, since the reduced matrix excludes exactly all global optima. Summarize over all j and g to obtain the theorem. □

Note that if $\mathsf{GlobOpt}(i)$ all probabilities for first success at the k-th step ($k \geq 1$) are zero. The first success graphs for the running example are given in figure 8.14.

From the graphs in figure 8.14 it is clear that the first success probability descends with the number of steps. This is an obvious consequence of the fact that the success probability grows with the number of steps (see figure 8.13). However, the relation between success probability (theorem 8.5.1) and first success probability (theorem 8.5.2) is not immediately clear. We therefore examine this relation in more detail.

Lemma 8.5.2 Let $\neg\mathsf{GlobOpt}(i)$. Then: $Prob(S_k|I_i) = \sum_{n=1}^{k} Prob(FS_n|I_i)$

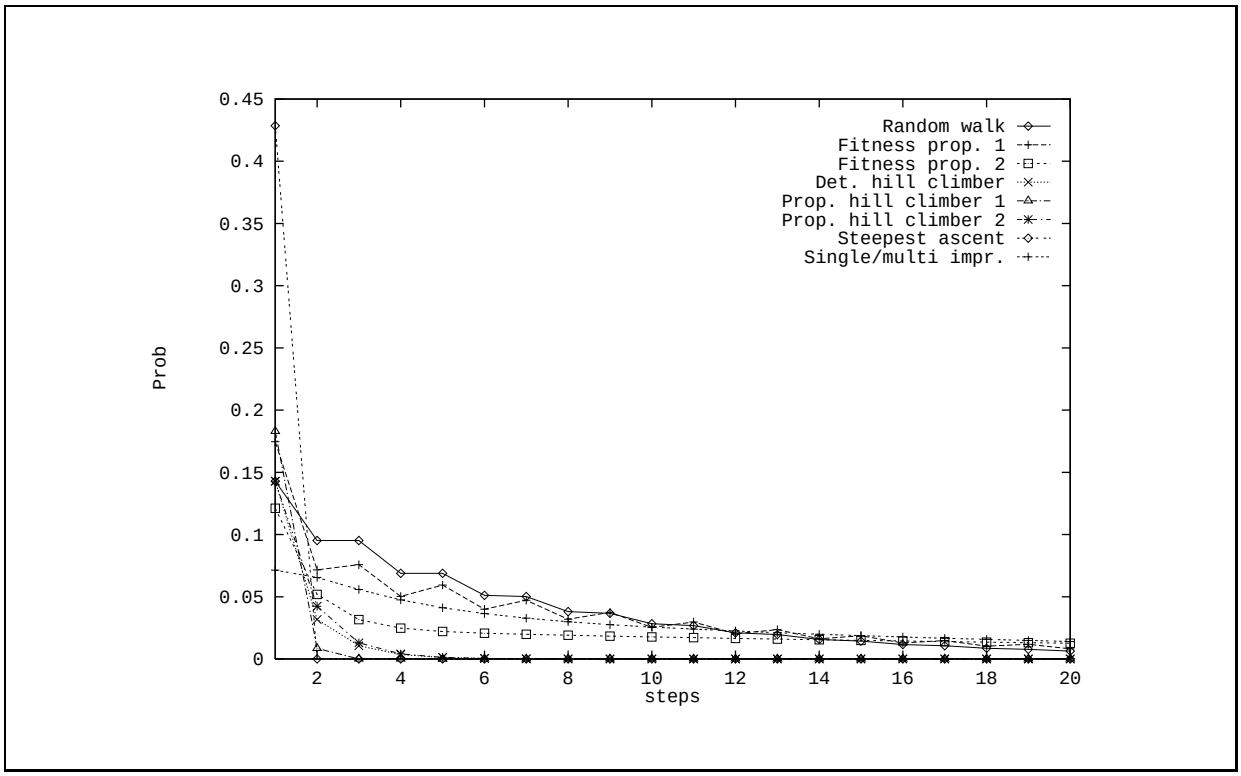


Figure 8.14: First success graphs for running example

Proof:

Success within $k \geq 1$ steps is possible by first success at the $n = 1$st, ..., kth step. Summarize over n to obtain the lemma. □

First success probabilities provide an indication of the steepness of the corresponding success probabilities: they can be considered as a sort of 'first derivative'. As an example, the steepness of the success graph of fitness proportional walk (2) in figure 8.13 is almost constant. Consequently, the corresponding first success graph in figure 8.14 is almost horizontal.

8.5.3 Local optimum probability

For certain algorithms (e.g. hill climbing) strong local optima (and sometimes even weak local optima) become absorbing states. When an absorbing state is entered, the algorithm will be caught there and no further evolution is performed. The following theorem calculates the probability of ending up in an absorbing state, without ever visiting a global optimum, after a fixed number of steps $k \geq 1$.

Theorem 8.5.3 Let Abs be a predicate indicating whether a state is absorbing: $\mathsf{Abs}(j) \Longleftrightarrow p_{jj} = 1$. We then have:

$$Prob(\mathsf{Abs}(X_k) \wedge \neg S_k \mid I_i) = \sum_{\mathsf{Abs}(a)} P^k_{red}(i,a)$$

Proof:

For a given absorbing state a, the probability equals the probability of a walk from i to a in exactly k steps, without visiting a global optimum. This probability is equal to $P^k_{red}(i,a)$ by definition of P_{red}. Now summarize over all possible a. □

The local optimum graphs for the running example are given in figure 8.15. Note that no graphs are given for random walk, single/multiple improvement and fitness proportional walk, since they have no local optimum problem.

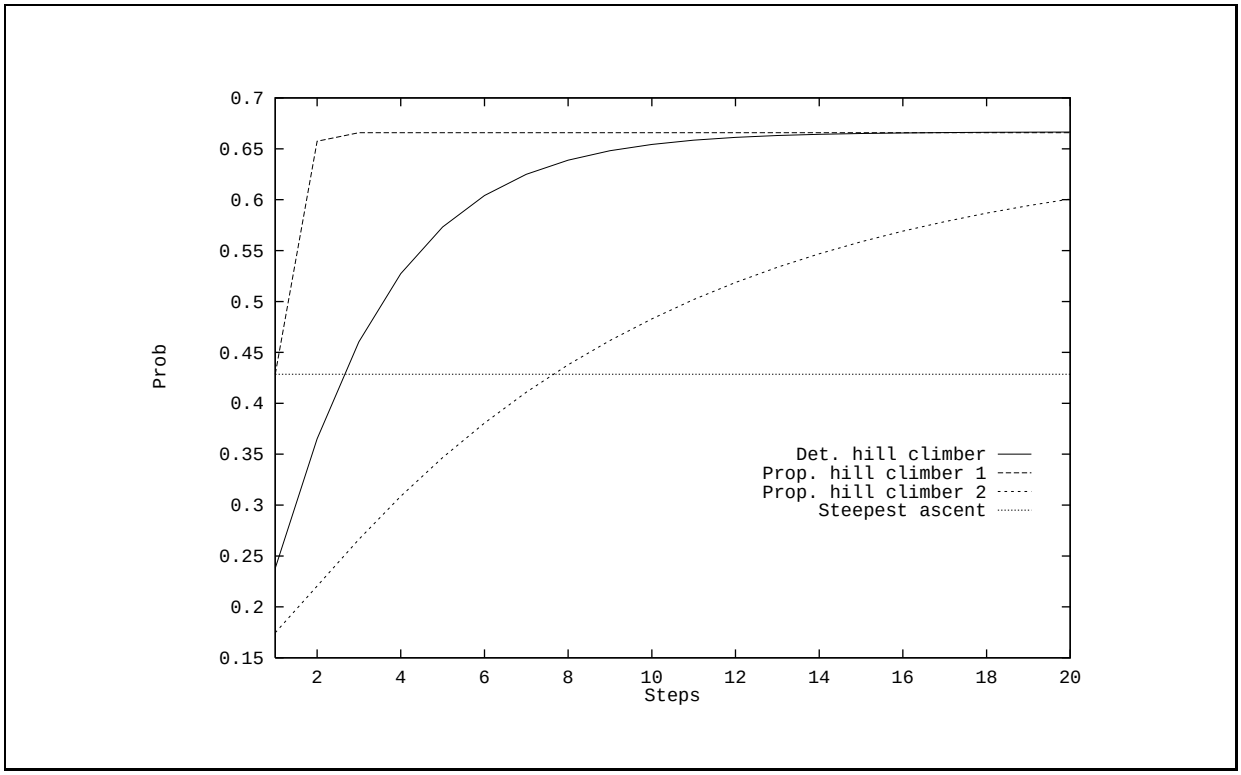


Figure 8.15: Local optimum graphs for running example

From the graphs in figure 8.15 it is clear that the local optimum probability grows with the number of steps. This is natural because more steps have more possibilities to reach a local optimum. However, this is not the case for steepest ascent, which can be explained as follows. In the running example, steepest ascent has three possibilities to be in a local optimum after $k = 1$ steps ($3/7 \approx 0.43$). For larger k this is also the case and as a result the corresponding graph is horizontal. This is typical for the running example. So, in general steepest ascent is not expected to have a horizontal local optimum graph. Note furthermore that local optimum graphs *never* descend, since it is impossible that more steps have less possibilities to reach a local optimum (provided the algorithm used has a local optimum problem).

8.5.4 Success entropy

The choice for an initial representation may be important for the entire evolution process. In this section the importance of initial representations is considered using a well-known concept from information theory, called *entropy*.

For a random variable X taking values in A, the entropy H is defined by (see e.g. [65]):

$$H = -\sum_{x \in A} Prob(X = x) \times \log(Prob(X = x))$$

where $0 \times \log(0) =_{\text{def}} 0$. We use this definition to define the influence of initial representations on the k-step success probabilities (theorem 8.5.1).

However, first a correct probability distribution has to be defined. The values taken by $Prob(S_k|I_i)$ for different $i \in \mathcal{S}$ cannot be used directly, as their sum is not necessarily 1. Let I_k be the random variable indicating which initial representation is chosen for an evolution process of k steps. Use C_k as an abbreviation for $\sum_{i \in \mathcal{S}} Prob(S_k|I_i)$. Now define the probability distribution function f_k (for I_k) by:

$$f_k(i) = Prob(I_k = i) = \frac{Prob(S_k|I_i)}{C_k}$$

Here representations with a higher success probability have a higher probability of becoming the initial state. The entropy for k-step success H_k can now be defined by:

$$H_k = -\sum_{i \in \mathcal{S}} \frac{Prob(S_k|I_i)}{C_k} \times \log(\frac{Prob(S_k|I_i)}{C_k})$$

Lower entropy values usually indicate a higher degree of certainty, making the choice of an initial representation of greater importance. As an example, for steepest ascent certain initial representations (almost) certainly result in success, while other initial representations (almost) certainly result in failure. So for steepest ascent, the degree of certainty is high, indicating that the choice for the initial representation is of great importance. Figure 8.16 presents the success entropy graphs for the running example.

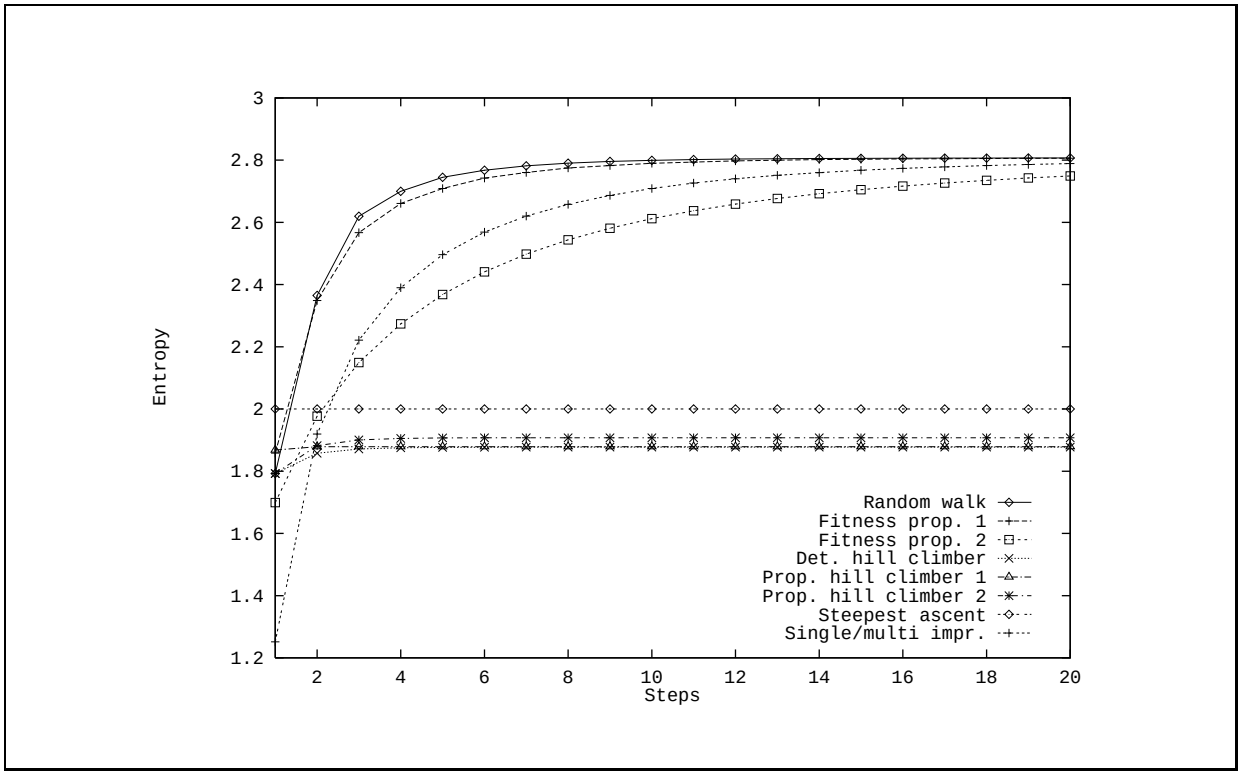


Figure 8.16: Success entropy graphs for running example

As expected the entropy for steepest ascent is low, indicating that the choice of the initial representation is of importance for its success. For random walk, fitness proportional walks and single/multiple improvement, the choice for an initial pool seems less important than for hill climbing and steepest ascent. This is what we would expect.

Note that for hill climbing the choice of an initial pool is even more important than for steepest ascent. The reason the various hill climbers are similar is that they have approximately the same success probabilities (see figure 8.13).

When the choice for an initial representation is important, so is the choice for an initial pool: a pool with many different representations then will yield better success probabilities. If on the other hand the initial representation has little influence on the performance of an evolutionary algorithm, one could use a simple initial pool consisting of only a few different representations or even all indentical representations.

8.6 Discussion: exploration versus exploitation

During an evolution process, *exploration* indicates the force that increases diversity (see e.g. [105]). An algorithm with high exploration rate is considered a relatively blind search. On the other hand, *exploitation* indicates the force that decreases diversity, trying to direct the evolution process towards desired solutions, and cutting off dead ends. Usually there is a trade-off between exploration and exploitation. They should be balanced to make a good evolutionary algorithm: too much exploration will make the algorithm blind, but too much exploitation increases the risk of missing a desired representation, e.g. when ending in a local optimum.

However, this balance depends on the specific solution space and fitness function. Consider for example a fitness function resulting in a single mountain in the solution space. Of course exploration is still required, but exploitation should form the strongest force, making advantage of the easy structure for example by using steepest ascent. If on the other hand a solution space with a very rugged structure is used (for example many representations leading to local optima), exploration becomes much more important: an exploiting algorithm will soon find itself in a local optimum. In general, an algorithm with more exploitation implies faster success but also a higher risk of failure. For algorithms with more exploration the converse holds: the risk of failure is limited, but it takes longer before a desired representation is found.

As a measure for the trade-off between exploration and exploitation, we use the following ratio ϑ:

$$\vartheta = \frac{1}{|\mathcal{S}|} \times \sum_{i \in \mathcal{S}} \frac{|\mathsf{RNb}(i)|}{\mathsf{NNb}(i)}$$

where RNb is the set of reachable neighbours:

$$\mathsf{RNb}(i) = \{j \in \mathcal{S} | p_{ij} > 0 \wedge i \neq j\}$$

In fact ϑ is the average amount of reachable neighbours as a fraction of the total number of neighbours. It varies between 0 and 1. Algorithms with $\vartheta \approx 0$ have maximum exploitation, at the cost of a minimal exploration. For algorithms with $\vartheta = 1$ the opposite holds.

We consider ϑ in more detail for the evolutionary algorithms discussed in previous sections. Random walk has $\vartheta_r = 1$, since it can explore all neighbours. So random walk does little exploitation: it is a blind search, as one would expect. Steepest ascent has $\vartheta_s \approx 0$, since it only accepts neighbours with maximum improvement. So steepest ascent exhibits maximum exploitation and minimum exploration.

For hill climbing ϑ directly depends on the (average) number of better neighbours. For example, $\vartheta_h = \frac{1}{2}$ if on the average half of the neighbours have a better fitness (and are thus reachable). In figure 8.17 these values of ϑ are illustrated in a so-called spectrum of algorithms.

Fitness proportional walk has an exploration between the exploration of random walk and hill climbing. So ϑ_f will be in the range from ϑ_r to ϑ_h, depending on the setting of the parameters. For fitness proportional hill climbing a similar argument can be used: then ϑ will be in the range from ϑ_h to ϑ_s, again dependent on the parameters. The ϑ value of single/multiple improvement depends on the setting of paramters λ_S

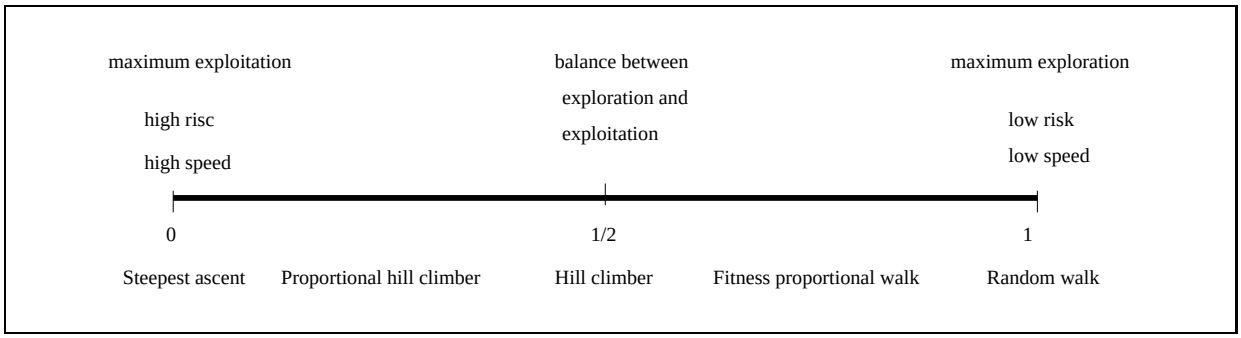


Figure 8.17: Spectrum of evolutionary algorithms

and λ_N. If both parameters are close to 1, then the ϑ value will approach 1 as well. If on the other hand both parameters approach 0, single/multiple improvement starts behaving like a hill climber.

Concluding, the ϑ ratio can be used to choose between different evolutionary algorithms. For a rugged solution space, exploration should be preferred over exploitation, and consequently algorithms with a larger ϑ should be used. If on the other hand the ruggedness is found to be quite low, exploitation becomes much more important, and an algorithm with a lower ϑ should be applied. Techniques for examining the ruggedness of solution spaces can be based on a statistical analysis of the fitness values found by a random walk. This is elaborated in more detail in e.g. [104].

8.7 Summary

In this chapter Markov chain fundamentals for evolutionary database optimization were introduced. After examining additional properties of the internal solution space, transition matrices for basic optimization algorithms were defined. Then success probability, first success probability, local optimum probability and success entropy were defined, and illustrated with the given transition matrices. Note that the probabilities were defined for all possible transition matrices. Consequently other (more advanced) optimization algorithms can be easily incorporated.

Chapter 9

Object-oriented transformations

When designing underlying databases of information systems, data are first modelled on conceptual level and then the obtained conceptual data model is transformed to a database. The focus of this chapter is the transformation of conceptual models into object-oriented database systems. See also [97], [98], [95] and [96].

For a conceptual schema, consisting of an information structure and a set of integrity constraints, both the structure and the constraints have to be translated into the target environment. In our approach this mapping is captured within the framework of a two level architecture. Conceptual models are first mapped to abstract intermediate specifications, which are then transformed to database schemas in a given object-oriented target database environment (e.g. SQL3, ODMG).

To express intermediate representations of conceptual models we use F-logic, a logic-based abstract specification language for object-oriented systems. In the present paper we focus on the first step of the overall transformation, i.e. the mapping of conceptual models into F-logic. Several transformation alternatives are discussed, and a corresponding graphical notation for specifying transformation alternatives is provided.

9.1 Introduction

It has been generally agreed on that conceptual data modelling is very important when building information systems. This means that data must be modelled first on conceptual level, and then the obtained conceptual model (conceptual schema) must be translated to the external and internal level, according to the three level architecture for information systems modelling ([66]). By doing so, the issues of correctness and efficiency are well-separated, which is quite desirable. In this paper we deal with the transformation of conceptual data models into internal (implementation oriented) database models.

During the past decades a number of conceptual data modelling techniques have been developed, e.g. (extended) ER ([36], [53], [52]), NIAM ([124]) and PSM ([82], [78]). PSM (Predicator Set Model) is an extension of NIAM and is fully formalized. It introduces advanced modelling constructs, e.g. generalisation (besides specialisation), power types, and sequence types. All constructs have graphical representations for easy understanding, and at the same time the precise formalism guarantees unambiguous semantics. Furthermore, in PSM a number of simple and complex integrity constraints can be specified and can be represented in a graphical style. Also, a conceptual language called LISA-D has been developed based on PSM ([81], [78]). Since it has proved to be a very expressive data modelling technique, we use PSM to express data models on the conceptual level. However, our approach is easily applicable to other conceptual modelling techniques (e.g. ER) due to the usage of similar constructs.

Internal models are considered to be database models that are supported by database management systems (DBMS) running on computers. It can also be assumed that some database language is provided for such

a model. Until now several database models and database languages have been defined and are supported by DBMSs. The well-known relational model ([39]) is the underlying database model of today's RDBMSs and the related database language is typically SQL (see e.g. [44]). The relational model and SQL have gained great popularity and have become standards. However, the relational technology is not always appropriate for some real-life applications, e.g. multimedia or geographical applications, where usually highly structured, complex objects must be stored and manipulated. To overcome the limitations of the relational model, advanced database models have been developed. The nested relational model (see e.g. [138], [10]) omits the assumption that relations are always in first normal form.

It has been recognized that powerful database models and languages can be obtained by incorporating object-oriented features into database technology. Such systems serve as candidates for next (or current) generation database systems. Although a number of such models and query languages have been proposed (see e.g. [3], [142], [9], [139], [89]), there is no object-oriented database model and language yet, that has been commonly accepted. This fact is known and inspired people attempting to define the requirements for next generation DBMSs ([5], [149], [90], [42]). At the same time the standardisation of database management with object-orientation is already in progress. Currently two standard proposals exist, ODMG-93 ([34]) and SQL3 ([114]).

This paper addresses the problem of how to transform conceptual data models into implementation environments. As targets of the transformation we consider next generation database systems, that is database systems with object-oriented features. A conceptual data model (conceptual schema) consists of an information structure and a set of integrity constraints. Both the information structure and the constraints have to be translated. The expected end product of the transformation is something that can be run on a computer with the given target environment (DBMS) for creating a database. That is, at the end a sequence of statements in the database language of the assumed DBMS has to be generated to create the database schema as well as to specify the integrity constraints corresponding to the conceptual model at hand.

At the level of statement generation many technical details have to be dealt with. These details are not relevant for the essence of the overall transformation and are different in different concrete systems. Therefore, in our approach the transformation is captured within the framework of a two level architecture. Conceptual schemas are first transformed to abstract intermediate specifications (design step). Then the obtained intermediate specifications are translated into the final implementation environment (implementation step). For intermediate specifications F-logic ([89]), a logic-based abstract specification language for object-oriented systems, is used.

The problem of database design and implementation is not new. Much valuable previous work has been done to map semantic models to relational systems (e.g. [152], [124]), and the results of this big amount of work have also been exploited in practice for long. The transformation of conceptual models to object-oriented databases has been addressed by only few number of proposals ([121], [94], [12]). Although these proposals provide some step toward designing OO databases based on conceptual modelling, they are not general enough in all aspects. Clearly, more work has to be done in this area in order to define a general transformation mechanism. On one hand, advanced modelling constructs (set types, list types, generalisation) should be considered as well. On the other hand, tranformation alternatives not covered by those proposals can be considered to improve generality. Also, the translation of more complex (general) conceptual constraints should be addressed. Our work has been motivated by these considerations.

In this paper we present the framework of a general transformation mechanism consisting of two steps. The focus of the paper is on the design (first) step, the implementation (second) step is discussed in general terms. We describe a number of alternatives for the translation of information structures into F-logic. Since the mapping of structures is influenced by simple uniqueness constraints, they are also covered. Although it belongs to the whole picture, the transformation of complex conceptual constraints in general is beyond the scope of the present paper.

The rest of the paper is organized as follows. In section 9.2 our approach is presented and related work is discussed. In section 9.3 the conceptual data modelling technique PSM is summarized to an extent

needed for the purpose of the paper. Section 9.4 gives an overview on F-logic, that is used for expressing intermediate specifications. Alternatives for the transformation of conceptual information structures into F-logic (the design step) are discussed in section 9.5. In section 9.6 an example for the transformation of PSM information structures into F-logic is presented in detail. The implementation step is discussed in general terms in section 9.7, where also an example schema definition in ODMG-93 is provided. Section 9.8 contains the conclusions and topics for further research.

9.2 Approach and related work

9.2.1 The general architecture

As it was already mentioned in the Introduction, in our approach the transformation is captured in the framework of a two level architecture, i.e. it is performed in two steps as shown in figure 9.1. Conceptual schemas are first transformed to abstract intermediate specifications (design step). Then the obtained intermediate specifications are translated into the final implementation environment (implementation step). The task of the second step is the generation of statements in a concrete database language. In [12] a similar approach is taken.

Figure 9.1: Two level architecture for transformation

There are several advantages of a two level architecture approach, e.g.:

- Provided that the intermediate specification language is general enough to cover all the final target models that are intended to be considered, the design step, which is the more complex and essential part of the whole transformation, becomes the same single task for different target environments and is independent of the choice of the actual target system. The different system specific details must be dealt with in the implementation step only.
- Provided that conceptual models and intermediate specifications have underlying precise formalism, the transformation from the conceptual to the intermediate level can be given algorithmically in a formal framework. This is very fundamental in order to have an automated transformation mechanism.
- Since a given conceptual model may have a number of correct representations on the internal level and possibly the best candidate should be chosen, optimization is important. In a two level transformation optimization can be incorporated at both levels.

To express data models on the conceptual level, we will use the Predicator Set Model ([82], [78]), an extension of NIAM ([124]). PSM is a fully formalized expressive modelling technique. It is briefly summarized in section 9.3. Some important issues concerning final target environments and intermediate specifications are discussed below.

9.2.2 Implementation environments

Under the term "implementation environments" (final target environments) we mean database management systems running on computers and serving as the underlying database engines of information systems to be built. A DBMS is always provided with some data model and database language it supports. A number of

database models and database languages (relational and OO) have been defined and implemented in commercial DBMSs. The question arises: What implementation environments to consider when transforming conceptual models?

The transformation of (extended) ER and NIAM models to relational structures in certain normal forms has been defined (see e.g. [152], [124]) and has also been implemented in some CASE-tools. In [24] and [21] a formalized variant of NIAM (an early simpler version of PSM) is mapped to nested (or flat) relational structures (see e.g. [138]). In these papers internal models are captured in terms of a tree-like intermediate representation.

In [121], [94] and [12] semantic models are transformed into object-oriented databases. Our work belongs to this category inspired by our conclusion that more research has to be done in this area. Although the mentioned approaches provide valuable steps, they are not general enough. This comes basicly from the fact that they deal with the mapping of conceptual models defined by modelling techniques that are less expressive than PSM, such as ER and BR (binary-relationship) models. On the other hand, more transformation alternatives can be recognized than the ones covered by the above papers. Our aim is providing a general translation for PSM (including advanced modelling constructs and complex constraints). This paper provides the basis for that.

We consider next generation database systems as target environments of the transformation. It is commonly agreed that a next generation DBMS shall be somehow object-oriented. However, it is not commonly accepted in the database community what this exactly means, to what extent and in what sense such a DBMS should be object-oriented. Since this problem has already been recognized, people have tried to define the requirements for next generation database systems. Until now three database system manifestos have been published ([5], [149], [42]) for that purpose. At the same time the standardisation of OO database management is in progress. Currently two candidate standards have been worked on, SQL3 ([114]) and ODMG-93 ([34]).

The most fundamental difference in different approaches is the starting point for a next generation system. Some people say that it should rely on the relational philosophy, thus exploiting the results of the huge amount of work for relational systems. In this approach the main structures are still relations and OO concepts are added to support advanced features, e.g. user-defined data types, operations, inheritance, etc. Since classes correspond to domains of relations, in [42] this is called the "class = domain" approach. Database systems of this kind are also known as object-relational systems (ORDBMS). Other groups of researchers and implementors ignore the relational model and propose pure object-oriented database models, where the counterpart of a relation is the extent of a corresponding class. According to [42] this is the "class = relation" approach.

The basic difference just discussed is reflected at different levels of database management, such as in different formal models, in the mentioned manifestos, in concrete DBMS implementations, and also in standard proposals. A representative of the "class = domain" approach is e.g. the formal data model of [3]. Concrete DBMS implementations of this category are e.g. the ORDBMSs of UniSQL and Montage. Formal foundations of truly object-oriented database systems (OODBMS, "class = relation") can be found e.g. in [139] and [89]. O_2 and Ontos are examples of such OODBMS implementations. In the first database system manifesto ([5]) this latter approach has been taken and proposed. However, it turned out that such systems suffer from many unsolved significant problems. The two succeeding manifestos, the second ([149]) and the third ([42]), propose going back to the relational data management as basis. The same is argued in [90]. In addition, [42] emphasizes that the basis should not be SQL, but the pure relational model invented by Codd ([39]).

Anyhow, the situation is that there exist and there will exist ORDBMSs as well as OODBMSs, and we have to take this fact into account when transforming conceptual data models into implementation environments. Due to our two level architecture, however, we do not have to deal with this distinction in the design step. Instead, the abstract intermediate specifications can be produced in a uniform way, and the necessary additional mappings to ORDBMSs and OODBMSs are defined in the implementation step.

Since there are also significant differences within ORDBMSs and within OODBMSs, we also have to make some choice what systems to directly support in our transformation (in the second step). As mentioned earlier, there are two candidates for next generation database system standards currently, namely SQL3 and ODMG-93. The starting point of SQL3 ([114], [111], [60]) is the database model and language SQL/92 and OO features are incorporated. Therefore it can be viewed as a standard proposal for ORDBMSs. ODMG-93 ([34]) took the object model of OMG ([147]) as starting point and incorporates database funcionalities. The Object Database Management Group (ODMG) was formed by OODBMS vendors, and thus ODMG-93 is intended to be a standard for OODBMSs. (We note that despite the different approaches to their object models, it seems that SQL3 and ODMG will converge in their query languages to some extent in the future, see e.g. [113].)

As final environments for the implementation step of the transformation we consider SQL3 as well as ODMG-93, since they are supposed to be supported in the future by ORDBMSs and OODBMSs, respectively. However, it is clear that from intermediate specifications one can generate databases in other specific ORDBMSs and OODBMSs with minor additional effort.

9.2.3 Intermediate specifications

According to the two level architecture of figure 9.1, fixing an appropriate specification language for specifying intermediate models is a basic task. When doing so, the following requirements are fundamental to be taken into account:

- The chosen specification language should be implementation oriented, i.e. it should be able to deal with concepts of implementation environments. At the same time it should provide a high level of abstraction to make it possible for us not to deal with the irrelevant aspects in the design step.
- It must be general enough and support object-oriented concepts to cover object-relational and object-oriented target systems.
- It must be provided with sufficient support for constraint specifications. Since a conceptual schema consists of an information structure and a set of integrity constraints, beside the transformation of *structures* also the constraints have to be mapped, this is a very essential requirement.
- It must have formal syntax and semantics. This makes it possible to define our transformation in a formal framework.

To sum it up, we need an abstract and formal database model with object-oriented facilities, that also allows to specify integrity constraints. After investigating a number of proposals (e.g. [3], [142], [139], [89]) we have concluded that F-logic ([89]) is the one that fulfils our needs the best. Models of other proposals are not general enough and/or are not provided with precise formal syntax and semantics and/or do not deal with constraints at all.

Therefore, we use F-logic for the formalization of schemas and constraints on the abstract intermediate level. F-logic is a logic-based powerful language with formal syntax and semantics, which combines the power of object-oriented and deductive databases, and provides high level abstraction. It is a general formalism for object-oriented and frame-based languages and can be used effectively for OO data modelling purposes. Due to its logical foundations, arbitrary integrity constraints can also be expressed. F-logic was presented in [89]. In section 9.4 an overview of F-logic is given based on [89] augmented with considerations on certain aspects relevant for the transformation.

We note that F-logic has a pure object-oriented view. However, it can serve as an abstract intermediate specification language in case of object-relational database sytems as well.

9.2.4 The concrete architecture

After discussing some important aspects of our approach, the picture of figure 9.1 can now be refined to be more concrete. Figure 9.2 illustrates our approach more concretely. As it shows, the input for the transformation is a conceptual model defined in term of PSM. In the first step (design step) the PSM model is translated into F-logic. In the second step (implementation step) the obtained F-logic specification is translated into a database language, such as SQL3, ODMG-93, or possibly other. The figure also shows that for different target environments the same design step can be applied and only the implementation step will differ. That is, choosing a new target system requires only the implementation step to be adapted.

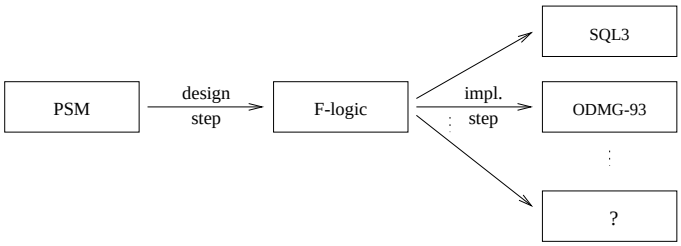


Figure 9.2: Framework of transformation

9.3 Example conceptual data model

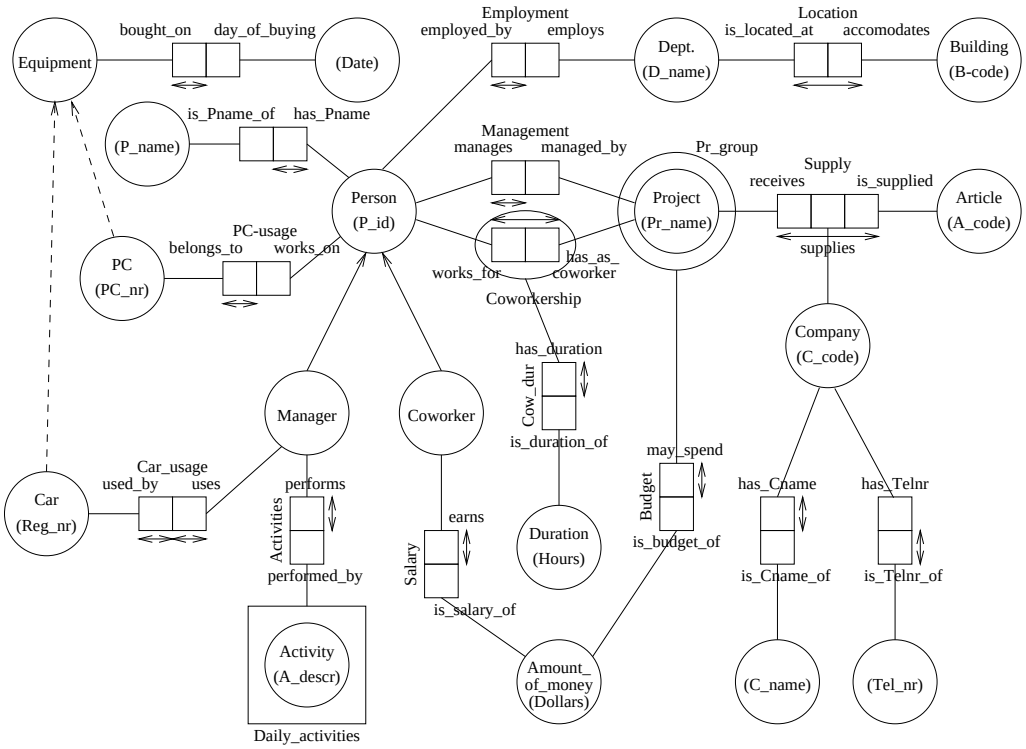


Figure 9.3: Information structure with uniqueness constraints

Figure 9.3 shows an example information structure. It will be used for giving examples when discussing transformation alternatives for information structures. In this figure we have a number of entity types, e.g.

Person and *Project*, represented by circles. Label types, also represented by circles, appear in parentheses, examples are *Date* and *Tel_nr*. The convention is used that if a label type is an identifier for an entity type, then the label type is represented within the same circle, see e.g. entity type *Person* and its identifying label type *P_id*. Fact types consist of predicators represented by boxes connected with circles for their base object types. For example fact type *Employment* is a binary fact type consisting of two predicators, *employed_by* and *employs*. Fact type *Supply* is an example of a ternary fact type.

Figure 9.3 also contains representatives of advanced modelling constructs. We have power type *Pr_group* with element type *Project* and sequence type *Daily_activities* with element type *Activity*. Power types and sequence types are represented by circles and boxes, respectively, around their element type. Entity types *Manager* and *Coworker* are specialised object types (subtypes) represented by solid arrows from subtypes to supertypes. Their common supertype is entity type *Person*. Entity type *Equipment* is a generalised object type with entity types *Car* and *PC* as its specifiers. Generalisations are represented by dashed arrows from specifiers to generalised types. As distinguished fact types, for bridge types many examples can be seen, e.g. $\{has\_Cname, is\_Cname\_of\}$ connecting entity type *Company* with label type *C_name*. As an example of objectified fact types we have fact type *Coworkership*.

9.4 F-logic

F-logic, presented in [89], is a powerful language with formal syntax and semantics, which combines the power of object-oriented and deductive databases. All the well-known object-oriented features, such as object identity, complex objects, object classes, (multiple) inheritance, methods, polymorphism, etc., are incorporated in F-logic. Due to its extensibility, some concepts coming from object-orientation and having no direct F-logic representation (e.g. lists) can be modelled in F-logic as well. F-logic has a first order model-theoretic semantics.

As it was said earlier, we use F-logic for expressing intermediate specifications when transforming conceptual models into OO target database systems. In this section we give a summary of the basics of F-logic based on [89] augmented with considerations on aspects relevant for the transformation. For a comprehensive description of F-logic we refer to [89].

9.4.1 Syntax and semantics

The basic syntactic elements of F-logic are a set of *function symbols* (*object constructors*) denoted by $\mathcal{F}$, a set of *variables* denoted by $\mathcal{V}$, some auxiliary symbols, and the usual logical connectives and quantifiers. With each function symbol an *arity* , the number of its arguments, is associated. *Constants* are function symbols with arity 0. An *id-term* is a usual first order term built out of function symbols and variables as in classical logic. They denote objects, classes and methods. When an id-term is variable-free, it is called a *ground* id-term. Ground id-terms (denoted by $U(\mathcal{F})$) play the role of *logical* object identifiers (*oid*). By means of id-terms we can construct *F-molecules*, and from F-molecules complex formulas (*F-formulae*) can be constructed. An *F-molecule* is one of the following (C, D, O, M, R, A_i-s, R_i-s, AT_i-s and RT_i-s are id-terms):

1. An *is-a assertion* of the form $C :: D$ or of the form $O : C$.
2. An *object mulecule* of the form O[a ';'-separated list of *method expressions*].

   The method expressions are either *data expressions*, or *signature expressions*, which in turn are further classified.

   (a) Data expressions are either

   - *scalar* data expressions of the form ($k \geq 0$)
     $M \,@\, A_1, ..., A_k \rightarrow R$; or

- *set-valued* data expressions of the form $(k, l \geq 0)$
  $M @ A_1, ..., A_k \twoheadrightarrow \{R_1, ..., R_l\}$.

(b) Signature expressions are either

- *scalar* signature expressions of the form $(k, l \geq 0)$
  $M @ AT_1, ..., AT_k \Rightarrow (RT_1, ..., RT_l)$; or
- *set-valued* signature expressions of the form $(k, l \geq 0)$
  $M @ AT_1, ..., AT_k \Rightarrow\!\!> (RT_1, ..., RT_l)$.

Complex formulas (*F-formulae*) are built from these simpler formulas by means of logical connectives $(\wedge, \vee, \neg)$ and quantifiers $(\forall, \exists)$ (and are interpreted) in the usual way. The implication connective " $\longleftarrow$ " can also be used. It is defined in the usual way, that is $\varphi \longleftarrow \psi$ is a shorthand for $\varphi \vee \neg\psi$.

F-logic is provided with a model-theoretic semantics defined by means of semantic structures called *F-structures*. An F-structure is a tuple consisting of a domain U, a partial order on U for the interpretation of subclass relationships, a binary relationship to model class mebership, and mappings for the interpretation of function symbols, data expressions and signature expressions. F-structures and the satisfaction of formulas are defined in such a way that the commonly know object-oriented features are incorporated. Instead of recalling the complex definitions, here we only summarize the most important aspects of the semantics of F-molecules. They are obtained in terms of properties following from the definition of F-structures and satisfaction. For formal details see [89].

The is-a assertion $C :: D$ states that class C is a subclass of class D (D is a superclass of C). The is-a assertion $O : C$ states that object O is a member of class C. Each class is subclass and superclass of itself (is-a reflexivity). The subclass relation is transitive, and subclass hierarchies are acyclic. The interaction between the subclass relation and class membership relation is covered by the subclass inclusion property stating that objects belonging to a class also belong to any superclass of that class. The type inheritence property says that structure propagates down from classes to subclasses, that is signature expressions (explained below) are inherited from superclasses.

In the definition of object molecules O denotes an object or a class. M corresponds to a method. In data expressions it denotes method invocation, while in signature expressions it denotes the signature of some method. The syntactic context of M indicates that the corresponding method is a scalar function (single-headed arrows, $\rightarrow$, $\Rightarrow$) or a set-valued function(double-headed arrows, $\twoheadrightarrow$, $\Rightarrow\!\!>$).

In scalar data expressions R represents the output of the method M when invoked on object O with arguments (id-terms) $A_1, ..., A_k$. Similarly, in set-valued data expressions R_i-s represent elements of the set resulting from the invokation of M on O with arguments $A_1, ..., A_k$.

We note that in [89] *data* expressions are more general than what we call data expressions here. They distinguish between *non-inheritable* and *inheritable* data expressions. Since inheritable data expressions are irrelevant for us, they are not discussed here. In our context data expressions mean *non-inheritable* data expressions. For details on inheritable data expressions we refer to [89].

In signature expressions RT_i-s represent the *types* (classes) of the result (scalar case) or the types of the elements of the result (set-valued case) of the method M when invoked on an object of class O with arguments of types $AT_1, ..., AT_k$. The output (or the elements of the output, resp.) of the method must belong to *all* the RT_i classes. In case there is only one result type specified (which is the most typical case in general), the parentheses may be omitted. (In our transformation this will always be the case.) The definition of F-structures does not provide the link between data expressions and signature expressions. Well-typing conditions are defined on a meta level. This will be discussed in a later section of this paper.

It is important to see that in F-logic different kinds of objects are treated in a uniform way. For example, a class is also considered to be an object. This is not surprising, since in OO programming languages such a uniformity is often a fundamental principle (e.g. in Smalltalk) and F-logic has been developed to serve as a general abstract model for OO and frame-based languages. However, in database models a clear

separation is usually made between schemas and instances, therefore the possibility of mixing schemas (classes) and data (objects) can be confusing. Since we use F-logic as an abstract specification language to express intermediate data models in our transformation process, we will always keep the separation between schemas and data, which is quite desirable in this context. This means that in the transformation either object molecules with signature expressions only, or object molecules with signature expressions only will be generated (see example 9.4.1).

There is another thing here that has to be noted. In object-oriented database models and systems a clear distinction is made between attributes and methods (operations). Since on an abstract level there is no essential difference between them, in F-logic attributes are considered to be methods without arguments. When denoting such methods the '@' symbol may be omitted. During the transformation of conceptual models into F-logic the distinction between attributes and methods will be treated on a notational level. By doing so their uniform treatment in F-logic is still exploited, and at the same time it enables to generate attributes as well as methods in final target systems (see section 9.5).

Example 9.4.1

Let's suppose that we have a simple database about persons, employees and projects. Such a database can be defined in F-logic as follows.

Schema (declarations)

$employee :: person$

$person\,[\,name \Rightarrow string;$
$\quad age \Rightarrow integer\,]$

$employee\,[\,salary \Rightarrow integer;$
$\quad works\_for \Rightarrow\!\!\Rightarrow project\,]$

$project\,[\,title \Rightarrow string;$
$\quad budget \Rightarrow integer;$
$\quad is\_involved\,@\,employee \Rightarrow boolean\,]$

Object base

$smith : person$

$jane : employee$

$natlang : project$

$dbopt : project$

$smith\,[\,name \rightarrow "John\ Smith";$
$\quad age \rightarrow 38\,]$

$jane\,[\,name \rightarrow "Eva\ Jane";$
$\quad age \rightarrow 22;$
$\quad salary \rightarrow 5100;$
$\quad works\_for \twoheadrightarrow \{dbopt, natlang\}\,]$

$natlang\,[\,title \rightarrow "Natural\ Language\ Processing";$
$\quad budget \rightarrow 50000\,]$

$dbopt\,[\,title \rightarrow "Database\ Optimization";$
$\quad budget \rightarrow 65000;$
$\quad is\_involved\,@\,jane \rightarrow true\,]$

□

In the schema part of example 9.4.1 we defined the classes *person*, *employee* and *project*. The is-a assertion states that *employee* is a subclass of *person*, and thus (as it will become clear from the semantics of F-logic

discussed in the next section) the methods defined for *person* objects can be automatically applied to *employee* objects. *name* is a scalar method without arguments (i.e. an attribute) which results a single *string*. $works\_for$ is a set-valued attribute of *employee* objects returning a set of *project* objects for an employee. $is\_involved$ represents a scalar method with one argument of type *employee*. When it is applied to a project with a given employee it returns *true* or $false$.

The data part of example 9.4.1 describes the actual content of the database. Objects are identified by logical object identifiers. Is-a assertions represent class memberships. For instance, the example shows that Eva Jane identified by *jane* is an employee. For individual objects the values of attributes, or more generally the results of method invokations with some arguments, are given by data expressions , e.g. $name \rightarrow$"$Eva\ Jane$" for employee *jane*, and $is\_involved\,@\,smith \rightarrow false$ for project *dbopt*.

Finally, we would like to discuss inheritance in more detail. As said earlier, structure propagates down from classes to subclasses. Any additional structure for a subclass is added to the inherited structure. From the interpretation of signature expressions it also follows that if a class has several (directly specified or inherited) signature expressions for the same method, then that method belongs to the intersection of all types of all signature expressions that the method has in the given class. In other words, the result types (classes) of the "overall" signature expression for that method is the union of all the types specified in separate (directly specified or inherited) signature expressions for that method. This means that the type restrictions given by the "overall" signature expression are obtained in such a way that the type restrictions specified by separate signature expressions restrict one another. This is the way F-logic interprets structural inheritance. However, in practice inheritance is often interpreted differently. In many systems it is not even possible to modify the signatures of inherited methods in subclasses, but only the implementation of methods can be redefined.

Fortunately, in our context this particular treatment of structural inheritance of F-logic does not cause any problems. In the transformation of conceptual structures into F-logic, methods and their signatures can only be results of *predicator* translation. However, within a conceptual schema predicators are unique (predicators are not identical to role *names*) and one predicator always has a single base object type. The consequence of this is that in the resulting F-logic specification we will never have the situation that more than one (directly specified or inherited) signature expression belongs to one method. Moreover, this also means that no signature expression will restrict the result type of a method to more than one class (except the superclasses, which is covered by this case by definition, and is common in object-oriented systems). That is, our F-logic specifications will not be affected by the F-logic specific interpretation of structural inheritance.

9.4.2 F-logic databases (F-programs)

In section 9.4.1 we summarized the most important aspects of the syntax and semantics of F-logic. We covered the basics of F-logic in general, which is important for general understanding. Since we use F-logic as an abstract specification language to express implementation oriented data models, we are particularly interested in how F-logic is used for data modelling. In this and the subsequent two sections we discuss the facilities of F-logic that are more specific to database management purposes.

An F-logic *database* is a logic program in F-logic, also called an *F-program*. Basicly, an F-program is an arbitrary set of F-formulae. However, this definition is too general, therefore restrictions on the form of the allowed formulas are applied. In the sequel F-programs will mean logic programs in F-logic consisting of a set of *rules*, that are statements of the form

$$head \longleftarrow body$$

where *head* is an F-molecule and *body* is a conjunction of literals (F-molecules or negated F-molecules). When no negated molecules occur in rule bodies (i.e. the rules are *Horn rules*), we get *Horn F-programs* (the counterpart of Horn programs in classical logic).

The semantics of F-programs is given by Herbrand interpretation, e.i. in terms of *Herbrand models* (H-models). We do not present the definitions here, see [89] for details. For Horn F-programs we have the nice properties with respect to fixpoints, least models and logical entailment known from the theory of classical Horn logic programs. With *normal* F-programs (or simply F-programs), where negation in rule bodies is allowed, we gain more expressive power, but the connection between fixpoints, minimal models, and logical entailment is lost. Such programs may have several minimal models, and their semantics is given by some *canonic* models. For a stratified program there exists a unique canonic H-model, called the *perfect model*. Since the details of these models do not matter, it is simply assumed that *some* canonic H-model exists for each F-program and determines the semantics of the program. Anyway, each F-program (F-logic database) obtained during our transformation will be a Horn F-program, for which its canonic H-model coincides with its unique minimal (least) H-model.

9.4.2.1 The structure of F-programs

F-programs specify different kinds of information. They organize objects along class hierarchies, define method signatures, and specify what each method is supposed to do. As mentioned earlier, in the context of databases the separation of schema and data is important. According to the type of information they specify, every F-program can be split into the following disjoint parts:

- *Schema definition (declarations).*
  - *Class hierarchy declaration* consisting of the rules whose head molecule is an is-a assertion for subclass relationships (::).
  - *Signature declaration* consisting of the rules whose head molecule is an object-molecule that contains signature expressionsi only.
- *Object-base definition.*
  - *Class membership declaration* consisting of the rules whose head molecule is an is-a assertion for class membership relationships (:).
  - *Data definiton* consisting of the rules whose head molecule is an object-molecule that contains data expressions only.

The above classification considers rule-heads only. For rules in one subset it is legal to have body literals defined in another subset. However, we note that during our transformation of conceptual databases into F-logic we will obtain F-programs with rules having only heads in most cases. Note furthermore that the F-logic database (F-program) of example 9.4.1 has been defined according to the above classification.

9.4.3 Type-correctness

The role of signature expressions is applying type restrictions on arguments and results of methods. However, the intended connection between data expressions and signature expressions is not captured by the defintion of F-structures. This missing link is provided at a meta level by means of well-typing conditions.

An F-program **P** is *well-typed* if every canonic H-model of **P** is a typed canonic H-model. A canonic H-model of **P** is said to be a *typed canonic model* of **P** if it obeys the type restrictions given by signature expressions occuring in **P**. This means that (1) for any data expression on any object in **P** there exists a covering signature expression (i.e. the method can be invoked on the object with the given arguments), and (2) the result of such a data expression is of type prescribed by the covering signature expression. For formal treatment of type-correctness we refer to [89].

9.4.4 Constraints

In database systems usually a set of integrity constraints is associated with a particular database to disallow invalid database states. Since integrity constraints can be specified on conceptual level, when transforming conceptual schemas into F-logic (beside the transformation of structures) we have to provide translation for the conceptual constraints. Although the transformation of constraints in general is out of the scope of the present paper, some basic (e.g. inverse) constraints have to be generated already during structure transformation. Therefore, constraint mechanism in the target model of the transformation is essential for us. F-logic itself does not define the concept of integrity (semantic) constraints. However, due to the logic-based approach of F-logic it is not difficult to extend it in order to introduce constraints explicitly. In [12] this has been done, and we use the extension presented there.

An *integrity constraint* in F-logic is an arbitrary F-formula. An *F-program with constraints* is a tuple $(\mathbf{P}, \mathbf{IC})$, where $\mathbf{P}$ is an F-program and $\mathbf{IC}$ is a set of integrity constraints (F-formulae). As a syntactic convention, integrity constraints are preceded by the symbol "!– ". The semantics of an F-program with constraints $(\mathbf{P}, \mathbf{IC})$ is defined by some canonic H-model of $\mathbf{P}$ that is also a model of $\mathbf{IC}$. In order to be a *well-typed F-program with constraints* $(\mathbf{P}, \mathbf{IC})$, $\mathbf{P}$ must be a well-typed F-program.

Example 9.4.2

Let's suppose that in the F-logic database of example 9.4.1 project titles must be unique. This can be specified by the following F-logic integrity constraint:

$$!\!-\ \ X_1 \doteq X_2 \ \longleftarrow\ \ X_1 : project\,[\,title \rightarrow Y\,]\ \wedge\ X_2 : project\,[\,title \rightarrow Y\,]$$

□

In the above example we used $X_1 : project\,[\,title \rightarrow Y\,]$ as a shorthand for $X_1 : project \wedge X_1 : [\,title \rightarrow Y\,]$ (with X_2 analogously). This kind of shorthand is often used in F-logic. The equality predicate (defined in [89]) expresses that the two objects are identical.

9.4.5 Modelling lists and sets

For the transformation of PSM sequence types we need an F-logic construct that is usually known as *list*. F-logic itself is not provided with list data types, but lists can be modelled. In [89] lists are defined as a parametric family of classes the following clauses:

$$nil : list(T)$$
$$cons(X, L) : list(T) \ \longleftarrow\ \ X : T \wedge L : list(T)$$

For example, $list(integer)$ is the class of lists of integers. *cons* is the list constructor, e.g. $cons(3, cons(1, cons(5, nil)))$ represents the list of integers $(5, 1, 3)$.

Although the concept of set-valued attributes (methods) enable to manage sets, its applicability is limited. Defining attributes of nested sets (sets of sets) is not easy and natural. However, to transform PSM power types we need such a mechanism. Therefore, similarly to lists above, we define a parametric family of classes to model sets (taken from [89] and refined):

$$\varnothing : set(T)$$
$$add(X, S) : set(T) \ \longleftarrow\ \ X : T \wedge S : set(T)$$
$$add(X, add(Y, S)) \doteq add(Y, add(X, S))$$
$$add(X, add(X, S)) \doteq add(X, S)$$

For example, $set(integer)$ is the class of sets of integers, and $add(3, add(1, add(5, \varnothing)))$ denotes the set $\{3, 1, 5\}$. The two equations are needed to ensure that the order of elements is irrelevant and multiple elements do not occur in sets.

From now on we assume that the above clauses are implicitly defined in each F-program, and $list(t)$ and $set(t)$, where t is a ground id-term denoting a class, can be used in F-programs whenever needed, e.g. in signature expressions to specify the result type of some method.

9.5 From PSM to F-logic: the design step

9.5.1 Framework

In this section we outline the essence of our approach to the transformation of conceptual models into F-logic (design step, see figure 9.2) by means of activity graphs with different decomposition levels. States denoted by ellipses are input and/or output of activities denoted by rectangles.

Basicly, since the final goal is database schema generation, we are interested in schema (data structure + integrity constraints) transformation. However, the semantics of conceptual structures, and more particularly, the constraints are defined in terms of populations. This means that we have to deal with population transformation too. In fact, populations must be transformed anyway when our approach is integrated in an executable transformation mechanism concerning also operations and their transformation. Such an execution model is essential e.g. for optimization.

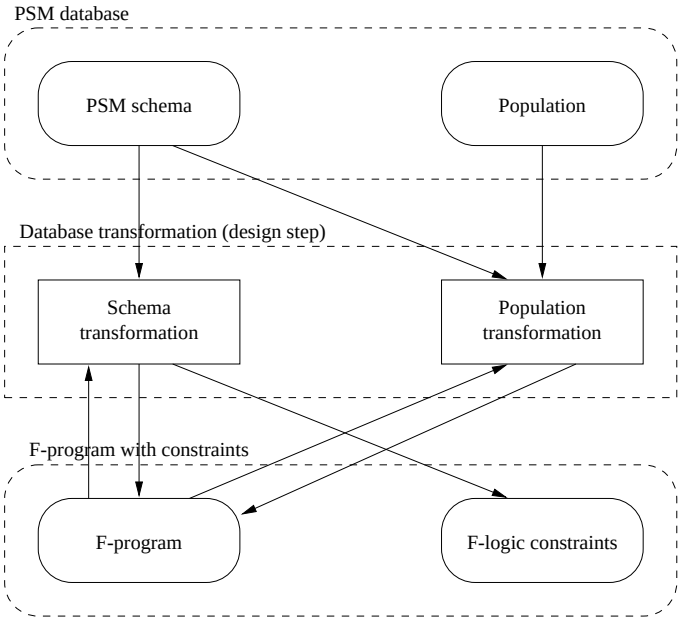


Figure 9.4: The design step

As depicted in figure 9.4, a PSM schema $\Sigma = \langle \mathcal{I}, \mathcal{C} \rangle$ together with a population $\mathsf{Pop}_{\mathcal{I}}$ (PSM database) is translated to an F-program with constraints $(\mathbf{P}, \mathbf{IC})$. The activity graph of this figure already shows a first level decomposition to separate the schema from the population and their transformations. The population is represented in $\mathbf{P}$ (object base part), while the schema (structure + constraints) is represented in $\mathbf{P}$ (declaration part) as well as in $\mathbf{IC}$. Obviously, for the transformation of populations and constraints (part of the schema) the result of the structure transformation is needed. That is the reason for the upward-directed arrows.

To illustrate the transformation of schemas in more detail, the corresponding part of the diagram of figure 9.4 is decomposed as depicted in figure 9.5. The information structure $\mathcal{I}$ becomes (the declaration) part of **P** on one hand, and some basic constraints (e.g. inverse constraints) are also generated. The conceptual constraints in $\mathcal{C}$ are translated to F-logic constraints in **IC**. The transformation of constraints also takes the conceptual structure and its internal counterpart as its input, which implies that first information structures must be transformed and then constraints. Or, at least that parts of the structure which are involved in a certain constraint must be transformed before the constraint translation. The transformation of independent parts can be performed in parallel. (Similar considerations are valid for structures and populations.) Futhermore, the figure also shows that the transformation of structures is influenced by so-called uniqueness constraints.

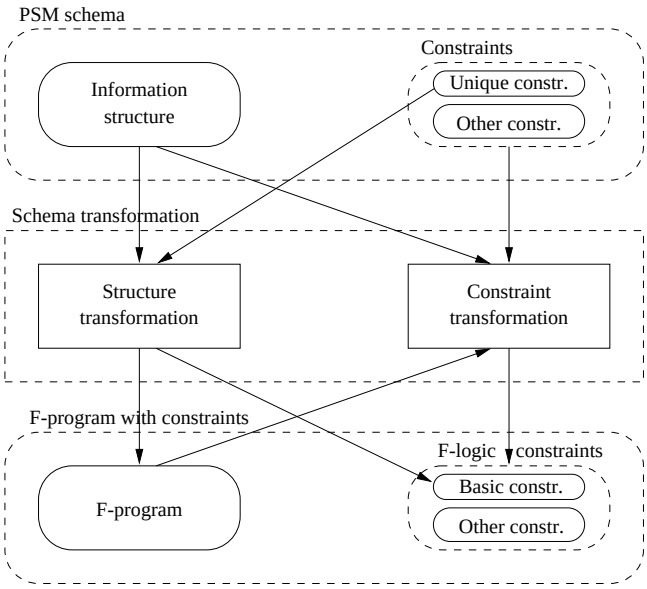


Figure 9.5: Decomposition of schema transformation

We do not further decompose the design step by means of activity graphs, because it would lead to too complex a figure and would require detailed explanation. In the rest of section 9.5 the transformation of information structures is discussed.

9.5.2 General issues

When transforming information structures, for all possible constructs of PSM we have to define their counterparts in F-logic. Basicly, an information structure is translated into F-logic class definitions, i.e. a set of object molecules with signature expressions only. For simplicity, we do not deal with assigning concrete names (ground id-terms) to the obtained F-logic components. Intead, an abstract naming convention (notation) is used in general indicating the kind of an F-logic component and the components of the information structure it resulted from. For example, a class is denoted by C_X, where X contains the PSM component(s) to which the class corresponds. Concrete names can be assigned, e.g. based on the names of the PSM components, via naming functions. This, however, will be part of the implementation (second) step of the transformation only.

Remember that in F-logic no difference is made between attributes and methods of classes. An attribute is considered to be a method without arguments. This enables the uniform treatment of (stored or derived) attributes and methods. However, in object-oriented database systems a clear distinction between them is essential. Therefore, we will define our transformation in such a way that we take this distinction into account. This is important also because, due to the elimination of certain kinds of redundancy during

information analysis, a conceptual (PSM) schema represents data to be stored. The distinction between (stored) attributes and general methods, however, will be done only on syntactic (notational) level, thus preserving their uniform treatment in F-logic. From now on, by attributes we mean stored attributes. An attribute and a method is denoted by $Attr_X$ and $Meth_X$, respectively,

For certain kinds of PSM components the translation is quite straightforward, but for others several alternatives are possible. The basic PSM components to be transformed are: object types, predicators, specialisation and generalisation hierarchies. The notion of object types in PSM is very general, it covers a number of more specific concepts, such as entity types, label types, fact types, set types, sequence types. Below we discuss the transformation of different PSM constructs separately by means of abstract figures. Figure 9.6 shows how arrows in such figures are interpreted.

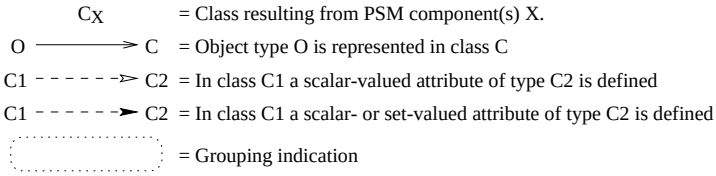


Figure 9.6: Graphical notation for specifying transformation alternatives

As we will see, some of the transformation alternatives will clearly reflect relational view in a sense. In truly OO systems this does not matter, since there everything becomes a class. However, in object-relational systems (what we do want to consider as potential final target systems) those obtained structures should result in relations. A possible way to support this already in the design step could be applying the notational convention that relation classes are denoted by RC_X instead of C_X. This does not mean, however, that "normal" classes cannot become relations in object-relational systems as result of the implementation (second) step of the overall transformation. We note that, in general, OR database design is more complex than pure OO database design, because the decision whether something should be a relation or a class (abstract data type) is not obvious and the right decision strongly depends on the application domain. Despite the fact that in our general approach we consider OR targets, in the present paper we do not apply the aforementioned notational distinction for the two kinds of classes, since the mapping of PSM into F-logic is in the focus and our goal here is the discussion of transformation alternatives. Such a distinction will be important when a transformation algorithm in a formal framework is defined.

9.5.3 Entity types

By default, entity types are translated to F-logic classes. The corresponding class *definitions* are object molecules with signature expressions only. As an initial step, for $E \in \mathcal{E}$ the empty object molecule $C_E[\,]$ is introduced representing the counterpart class.

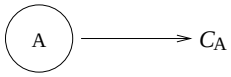


Figure 9.7: Entity types are mapped to classes

The structure of the class corresponding to an entity type depends on how the fact types in which the entity type is involved are transformed.

In some cases an entity type simply models values of a label type connected with it (see e.g. entity type *Duration* in figure 9.3). In such cases the elimination of the counterpart class of the entity type is reasonable. Then the corresponding class is substituted with the concrete domain of the connected label type.

9.5.4 Label types and bridge types

A label type represents a set of values from a concrete (simple) domain. For simplicity, we assume that an appropriate (built-in) F-logic class exists for each concrete domain. These values of these domains (represented by label types) do not have independent existence. Therefore, for a label type L, in contrast with entity types, no separate F-logic class is created, but it is mapped to the (assumed) built-in class for its concrete domain $\mathsf{Dom}(L)$.

A bridge type is a special binary fact type connecting a non-label type with a label type. Assuming that the involved non-label type is mapped to a class, a bridge type is incorporated as an attribute in that class. If a uniqueness constraint is specified on the predicator with the non-label type as its base, then the attribute is scalar-valued. Otherwise, it is defined to be set-valued. Situations when our assumption does not hold will be mentioned and treated places where they may arise. The translation of bridge types and the related label types is illustrated in figure 9.8.

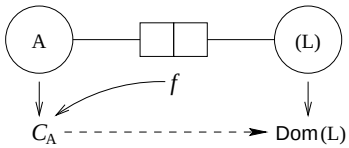


Figure 9.8: Mapping of bridge types and label types

Let $f = \{p, q\} \in \mathcal{B}$ be a bridge type with $\mathsf{Base}(p) \notin \mathcal{L}$ and $\mathsf{Base}(q) \in \mathcal{L}$. The translation of f results in the object molecule

$$C_{\mathsf{Base}(p)}\,[\,Attr_p\{\Rightarrow\,|\,\twoheadrightarrow\}\,\mathsf{Dom}(\mathsf{Base}(q))\,]$$

Here $C\,[\,Attr_p\{\Rightarrow\,|\,\twoheadrightarrow\}...\,]$ stands for $C\,[\,Attr_p \Rightarrow ...\,]$ if $\mathsf{unique}(p)$ is specified, and for $C\,[\,Attr_p \twoheadrightarrow ...\,]$ otherwise. This notation will be used often in the sequel.

9.5.5 Fact types and predicators

In this section we consider non-bridge fact types, when discussing the mapping of fact types into F-logic. The transformation of fact types is not straightforward, there are several possibilities how to transform them and their predicators. The translation of a fact type has strong effect on the final F-logic counterparts of the object types involved in the fact type via predicators. Alternatives are discussed below.

9.5.5.1 Trivial mapping

The simplest solution is generating a separate F-logic (relation) class for each fact type. Then each predicator of a fact type results in a scalar-valued attribute in the corresponding class. Figure 9.9 illustrates this trivial translation.

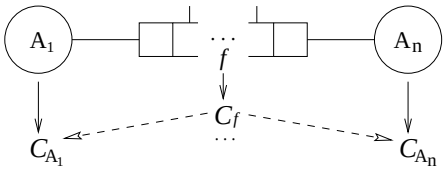


Figure 9.9: Illustration of trivial mapping

Let $f = \{p_1, ..., p_n\} \in \mathcal{F} - \mathcal{B}$ be a (non-bridge) fact type. For each $i \in \{1, n\}$ the following object molecule is produced:

$$C_f\,[\,Attr_{p_i} \Rightarrow C_{\mathsf{Base}(p_i)}\,]$$

The above transformation is valid only if the base object type of each predicator in f has a corresponding F-logic class. However, this is not necessarily the case, e.g. if a base object type is a power type. Such cases will be discussed and treated later at the appropriate places.

Since the population of a fact type is a set of tuples, that are in turn *total* functions, it has to be ensured that each attribute of a fact (relationship) object carries some value. This can (has to) be forced by introducing appropriate constraints. On the other hand, fact objects are value-based and are identified by the participating objects. Therefore, a constraint has to be generated requiring that no two different fact objects may correspond to the same combination of participating objects (see also [12]). As a consequence, a mandatory constraint $!\!-\ \mathsf{Mandatory}(C_f, \{Attr_{p_1}, ..., Attr_{p_n}\})$ and a key constraint $!\!-\ \mathsf{Unique}(C_f, \{Attr_{p_1}, ..., Attr_{p_n}\})$ are generated.

Although the trivial mapping is sufficient to store all data, in order to improve query performance additional inverse attributes can be defined in any/all of the classes that correspond to the participating object types. Then such an inverse attribute stores references to relationship objects in which they participate (see also [94]). The type of inverse attributes is influenced by uniqueness constraints. Moreover, if inverse attributes are introduced, then also inverse constraints have to be generated to guarantee integrity. For $i \in \{1, n\}$ an (inverse) attribute in $C_{\mathsf{Base}(p_i)}$ can be defined yielding: $C_{\mathsf{Base}(p_i)}\,[\,Attr_{p_i}\{\Rightarrow \mid \twoheadrightarrow\}C_f\,]$. An inverse constraint is also needed, namely $!\!-\ \mathsf{InverseOneToOne}(C_{\mathsf{Base}(p_i)}, Attr_{p_i}, C_f, Attr_{p_i})$ if $\mathsf{unique}(p_i)$ and $!\!-\ \mathsf{InverseOneToMany}(C_{\mathsf{Base}(p_i)}, Attr_{p_i}, C_f, Attr_{p_i})$ otherwise.

Introducing inverse attributes with apprpriate inverse constraints is a general option in the transformation. In principle, they can be generated in all alternatives that will be discussed for the transformation of fact types. In the sequel we will not explicitly mention the possibility again and again.

9.5.5.2 Incorporation of fact types

Fact types can be translated in such a way that they are incorporated in classes obtained for their object types, e.g. classes for entity types (see also [94]). In this case fact types are represented as reference attributes in such classes. More precisely, those attributes correspond to predicators constituting fact types.

On F-logic level all fact types of a PSM schema have to be represented as stored components somehow, i.e. some attributes must be defined that correspond to them. In case of incorporation this can always be done for binary fact types. However, fact types of higher degree can be encoded in classes for some of its base object types as attributes only in special situations, namely when this is enabled by uniqueness constraints. Otherwise, facts cannot be stored unambigously. Therefore, in the latter situation the incorporation of such fact types can be complementary only, yielding general methods. This will be discussed under the term *methodization*.

Binary fact types

First we consider binary fact types as subjects of incorporation. As shown infigure 9.10, a binary fact type f can be incorporated in any/both of the two classes corresponding to its two base object types. The actual kind of reference attributes (scalar-valued or set-valued) is guided by uniqueness constraints. If f is incorporated in both classes, then an inverse constraint is needed as well.

Let $f = \{p, q\} \in \mathcal{F} - \mathcal{B}$ be a binary fact type. f can be incorporated as an attribute in either $C_{\mathsf{Base}(p)}$, or in $C_{\mathsf{Base}(q)}$, or in both. The incorporation of f in $C_{\mathsf{Base}(p)}$ results in $C_{\mathsf{Base}(p)}\,[\,Attr_p\{\Rightarrow \mid \twoheadrightarrow\}C_{\mathsf{Base}(q)}\,]$. The result of incorporating f in $C_{\mathsf{Base}(q)}$ is obtained analogously. If f is incorporated in both classes, then an inverse constraint is also produced. The type of that constraint is driven by the scalar nature of the defined reference attributes.

Note that this way of transforming binary fact types looks very much like the transformation of bridge types (see section 9.5.4), which is not surprising, since bridge types are always incorporated in the classes for the non-label types involved.

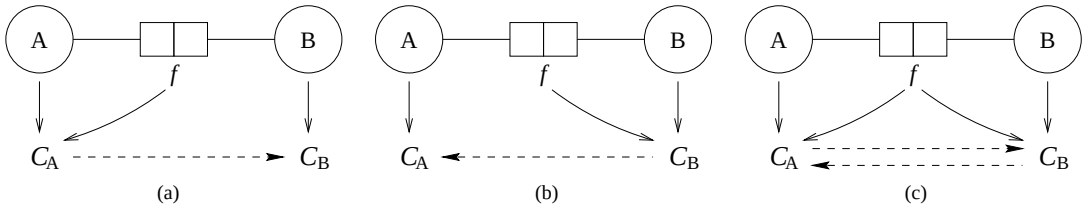


Figure 9.10: Incorporation of binary fact types

Relational view of incorporation

For binary fact types the incorporation mechanism can be combined with the trivial mapping as follows. A class is introduced for the fact type f similarly to the case of trivial mapping. At the same time, however, one of the two base object types is chosen, around which the related objects are arranged, similarly to the case of incorporation. Then the predicator with the "central" base object type becomes a scalar-valued attribute in the class corresponding to the fact type. The other predicator becomes either a set-valued or a scalar-valued attribute. This new alternative is depicted in figure 9.11.

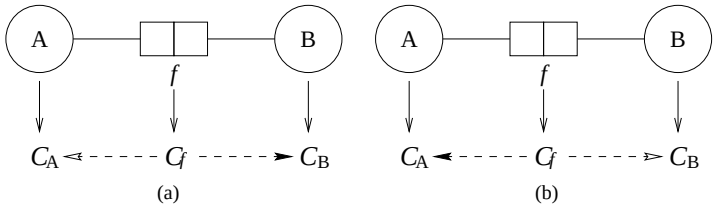


Figure 9.11: Relational incoporation of binary fact types

If a uniqueness constraint is specified on the predicator with the "central" base object type, then the result is indentical to that of the trivial mapping. The combination of trivial mapping with incorporation can be viewed as a nest operation performed on the result of the trivial mapping. The basic constraints mentioned there are also needed here, but they have to be adapted according to the different structure.

We note that basicly this alternative has relational nature. Since constructs from object-orientation are involved as well (set-valued attributes) it fits in the object-relational approach.

Fact types of higher degree

An n-ary fact type f ($n > 2$) can be incorporated in a class corresponding to a base object type of a predicator in f, for which a uniqueness constraint is specified (provided that each object type involved in f has a counterpart class, for now it is assumed). The situation is depicted in firgure 9.12.

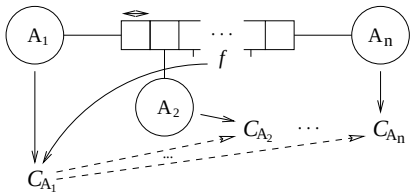


Figure 9.12: Incorporation of n-ary fact types

Here the uniqueness constraint is required, because without that the concrete relationships between objects cannot be stored unambigously. The result of this transformation is that a class in which f is incorporated will have scalar-valued attributes for referencing related objects. The basic constraints discussed for trivial

mapping have to be defined conforming this structure. If f is incorporated in more than one class, then inverse constraints are needed as well.

Let $f = \{p_1, ..., p_n\}$ $(n > 2)$ be a fact type. If for some i $(1 \leq i \leq n)$ we have $\mathsf{unique}(p_i)$, then f can be incorporated in $C_{\mathsf{Base}(p_i)}$. This results in C_{A_1} in figure 9.12. Then for each $j \in \{1, n\}$ with $j \neq i$ the object molecule $C_{\mathsf{Base}(p_i)}\,[\,Attr_{\{p_i, p_j\}} \Rightarrow C_{\mathsf{Base}(p_j)}\,]$ is yielded. Again, $\mathsf{unique}(p_i)$ is required, because without that the concrete relationships between objects cannot be stored unambigously. This results in $C_{A_2}, ..., C_{A_n}$ in figure 9.12. For unique representation of relationships the key constraint $!\!-\ \mathsf{Unique}(C_{\mathsf{Base}(p_i)}, \{Attr_{\{p_i, p_j\}} \mid 1 \leq j \leq n, j \neq i\})$ is also produced. Moreover, if f is incorporated in more than one class, then appropriate inverse-like constraints are needed as well, which is not further defined here.

We note that this situation is not typical in practice. Due to the uniqueness constraint, in principle, f can be split into a number of binary fact types. On the other hand, however, fact types represent elementary recording types. The situation might occur that e.g. the latter principle requires a fact type to be ternary, but the application domain implies uniqueness constraint on a single role. Therefore, we do not exclude such situations in general.

For the transformation of binary fact types we considered the combination of trivial mapping and incorporation. Since to incorporate fact types of higher degree uniqueness constraints over single predicators are required, we would get back exactly the trivial mapping when applying such a combination. Therefore, this alternative is not considered at all.

Quasi-incorporation

Although our latest note is valid, for n-ary fact types, where $n > 2$, a way to combine incorporation with relational view can be recognized. The mechanism is slightly different from what we had for binary fact types. It is shown in figure 9.13. Fact type f is represented in a class for one of its base object types as well as in a subsidiary class denoted by $C_{f'}$. The class C_{A_1} in wich f is incorporated has an attribute (scalar- or set-valued) for referencing objects of class $C_{f'}$. Those objects represent combinations of other objects that are related to a given C_{A_1} object via f. The attribute in C_{A_1} is scalar-valued if uniqueness constraint is on the predicator with base A_1, otherwise set-valued. As usual, structure conforming basic constraints are needed.

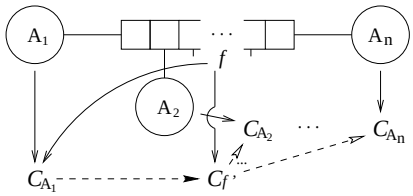


Figure 9.13: Illustration of quasi-incorporation

Quasi-incorporation of fact type $f = \{p_1, ..., p_n\} \in \mathcal{F} - \mathcal{B}$ in class $C_{\mathsf{Base}(p_i)}$, $i \in \{1, n\}$, introduces a subsidiary class C_{f^i}. This results in C_{A_1} and $C_{f'}$ in figure 9.13. For each $j \in \{1, n\}$, $j \neq i$, the object molecule $C_{f^i}\,[\,Attr_{p_j} \Rightarrow C_{\mathsf{Base}(p_j)}\,]$ is produced. This results in $C_{A_2}, ..., C_{A_n}$ in figure 9.13. In class $C_{\mathsf{Base}(p_i)}$ a reference attribute is defined: $C_{\mathsf{Base}(p_i)}\,[\,Attr_{p_i}\{\Rightarrow \mid \twoheadrightarrow\}C_{f^i}\,]$. As usual, basic constraints are needed as well, not further detailed.

Note that, in contrast with pure incorporation of n-ary fact types, here no uniqueness constraint has been required. That is, this kind of transformation is generally applicable. Note furthermore that quasi-incorporation of binary fact does not make sense, since it would produce a subsidiary class, as an unnecessary extra shell, with a single attribute.

9.5.5.3 Grouping

Since during information analysis fact types are determined such that they represent elementary recording types ([124]), most of them are binary or ternary in practice. That is, applying e.g. the trivial transformation would result in small but many object classes. Instead of transforming each fact type to a separate class, another altenative is to perform some grouping of fact types that are joinable via common object types before generating F-logic class definitions. Then for fact types in the same group a single class is created.

The grouping mechanism is implemented by means of a so-called *grouping profile*. A grouping *profile* is a set of groupings, where a *grouping* is a set of sets of predicators satisfying certain wellformedness conditions, e.g. conditions that require the joinability of fact types to be grouped. The conditions we require are in the style of [24] and [21].

A grouping *profile* $\mathcal{GP}$ is a set of groupings, where a *grouping* G is a set of sets of predicators ($G \subseteq 2^{\mathcal{P}}$) satisfying the following wellformedness conditions:

(G_1) The grouping profile specifies only real groupings: $\forall_{G \in \mathcal{GP}} \forall_{A \in G} [|A| > 1]$.

(G_2) No multiple occurence of predicators: $\forall_{G_1,G_2 \in \mathcal{GP}} \forall_{A_1 \in G_1, A_2 \in G_2, A_1 \neq A_2} [A_1 \cap A_2 = \varnothing]$.

(G_3) *Local* (via common object type) joinability: $\forall_{G \in \mathcal{GP}} \forall_{A \in G} \forall_{p,q \in A} [\mathsf{Base}(p) = \mathsf{Base}(q)]$.
For $A \in G$ this common base object type is denoted by $\mathsf{Base}(G, A)$. A specifies a particular way of grouping around $\mathsf{Base}(G, A)$.

(G_4) Grouping a fact type with itself is excluded: $\forall_{G \in \mathcal{GP}} \forall_{A \in G} \forall_{p,q \in A, p \neq q} [\mathsf{Fact}(p) \neq \mathsf{Fact}(q)]$.

(G_5) Bridge types are not considered for grouping: $\forall_{G \in \mathcal{GP}} [\mathsf{GrFacts}(G) \cap \mathcal{B} = \varnothing]$.

(G_6) *Global* joinability (interconnectivity): $\forall_{G \in \mathcal{GP}} \forall_{A_1,A_2 \in G, A_1 \neq A_2} \exists_{p \in A_1, q \in A_2} [\mathsf{Conn}(G, p, q)]$, where $\mathsf{Conn}(G, p, q) \stackrel{\text{def}}{=}$ ($\mathsf{Fact}(p) = \mathsf{Fact}(q) \ \vee \ \exists_{A \in G, r,s \in A} [\mathsf{Conn}(G, p, r) \wedge \mathsf{Conn}(G, s, q)]$).

(G_7) No overlap between different groupings: $\forall_{G_1,G_2 \in \mathcal{GP}, G_1 \neq G_2} [\mathsf{GrFacts}(G_1) \cap \mathsf{GrFacts}(G_2) = \varnothing]$.

One particular grouping in the grouping profile implicitly specifies a set of fact types to be grouped together. The result of grouping is that one class is generated for the fact types in one group, and all those fact types are represented in that class. The structure of the class is obtained in such a way that one attribute is defined for each set of predicators in the grouping and one attribute is defined for each non-grouping predicator occuring in the grouped fact types.

The grouping meachnism is illustrated in figure 9.14. According to the figure, the grouping profile consists of a single (maximal) grouping. That grouping contains two sets of predicators (with common object types).

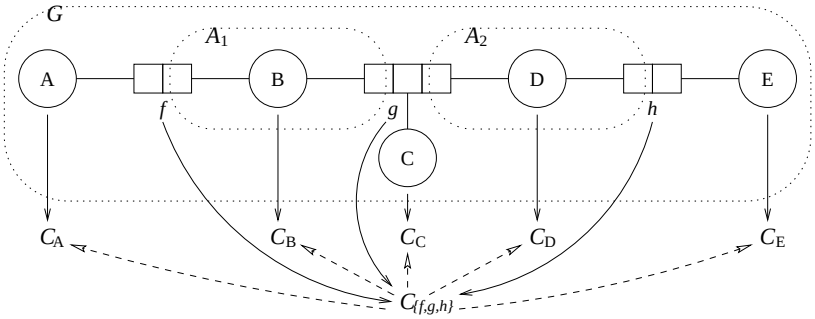


Figure 9.14: Illustration of the grouping mechanism

As the result of grouping, one class is generated for each grouping in the grouping profile. Let $\mathcal{GP}$ be a grouping profile, and let $G \in \mathcal{GP}$ be a particular grouping. The operator $\mathsf{GrFacts}(G)$ returns the fact types that are in the group specified by G: $\mathsf{GrFacts}(G) = \{f \in \mathcal{F} \mid \exists_{A \in G} \exists_{p \in A} [\mathsf{Fact}(p) = f]\}$. For G the class $C_{\mathsf{GrFacts}(G)}$ is generated. For each set of predicators $A \in G$ one attribute is defined yielding the object molecule $C_{\mathsf{GrFacts}(G)}[\, Attr_A \Rightarrow C_{\mathsf{Base}(G,A)}\,]$. Moreover, for each non-grouping predicator $p \in \bigcup \mathsf{GrFacts}(G) - \bigcup G$ we obtain one attribute: $C_{\mathsf{GrFacts}(G)}[\, Attr_p \Rightarrow C_{\mathsf{Base}(p)}\,]$.

Guided grouping

The grouping mechanism described so far is very general and clearly has relational style. Moreover, it produces flat structures. When designing object-oriented or object-relational databases this general mechanism may lead to very unnatural and unlucky database schemes. Even when relational databases are designed this can be the case, because no normal forms are taken into account, therefore redundancy and anomalies may arise. On the other hand, such a grouping can also be reasonable, since in certain cases uniqueness constraints may guarantee the flat characteristic. To ensure that grouping is performed only in such cases an additional grouping condition could be introduced. By doing so, however, we would lose the general applicability of the mechanism. Indeed, for performance reasons a database designer may want to store relationships between objects in a flat manner, even if it induces redundancy and update anomalies. It is not unusual in practice. Therefore, we do not restrict our grouping mechanism, thus providing a wider range of possibilities for the transformation. Instead, in an algoritmic tranformation a guiding parameter can be introduced, so that a designer can avoid flat-like grouping of inherently non-flat structures, if intended.

9.5.5.4 OO-like grouping

In section 9.5.5.2 we discussed the incorporation of fact types in classes corresponding to base object types. For binary fact types the relational variant was also considered resulting from the combination of trivial mapping and incorporation. In this section we show how this combined alternative can be further integrated with a restricted version of the grouping mechanism.

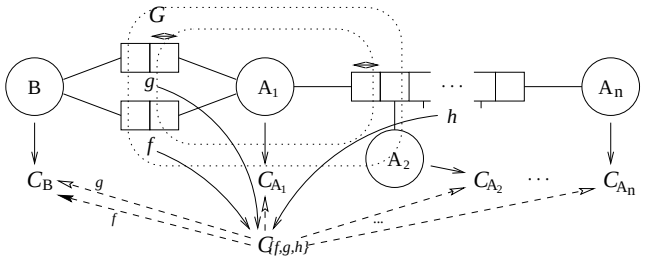


Figure 9.15: OR-like grouping

The situation is depicted in figure 9.15. The restrictions on the general grouping (with flat behavior) means that a particular grouping in the grouping profile may consist of only a single set of predicators with common base object type. (Note the difference with figure 9.14 with respect to the grouping indication.) Similarly to the relational incorporation for binary fact types (see figure 9.11), a central object type is chosen around which related data coming from several (grouped) fact types is arranged. This gives some OO characteristic to this transformation, while keeping also the relational view of grouping. From the alternatives discussed so far the general idea illustrated by figure 9.15 can be seen.

Grouping with quasi-incorporation

In figure 9.13 an alternative way to incorporate fact types of higher degree (called quasi-incorporation) with some relational characteristic was illustrated. There subsidiary classes have been introduced. That mechanism can be combined with (restricted) grouping analogously with the combination presented in the

previos section, as shown in figure 9.16. Since only alternatives discussed earlier are combined in a way that has also been described, we do not explain this combination in more detail.

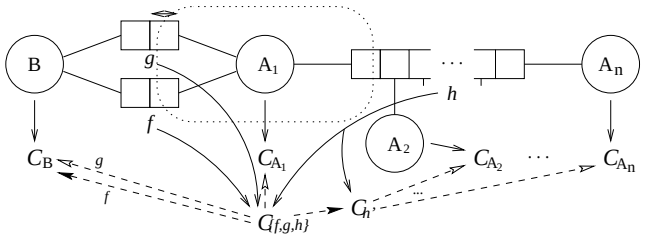


Figure 9.16: Grouping with quasi-incorporation

9.5.5.5 Objectification

As said in section 9.3, a fact type is objectified if it is the base object type of some predicator. The transformation of objectified fact types must be examined with some special care, since it has to be ensured that data attached to facts (and not e.g. to entities) are storable as well. Now we consider how the alternatives for the transformation of fact types are applicable to objectified fact types.

Objectification and trivial mapping

Obviously, the trivial mapping method can be applied to an objectified fact type as to any fact type without problems, which is the most natural solution. During the transformation of the rest of the information structure any of the alternatives can be chosen. Then the (objectified) fact type behaves as if it was an entity type.

Objectification and grouping

In principle, an objectified fact type can be considered for grouping, but not as it stands. In order to be able to transform the other fact types it is involved in, first it has to be unnested with respect to all those fact types. In many cases, however, the same final result can be obtained by applying different transformation alternatives. Therefore, we do not deal with providing an unnest operation for objectifications and do not consider objectified fact types to be grouped in any way. An additional grouping condition is introduced:

(G_8) Objectified fact types may not be grouped: $\forall_{G \in \mathcal{GP}} [\mathsf{GrFacts}(G) \cap \mathcal{H} = \varnothing]$.

Incorporation of objectified fact types

In section 9.5.5.2 the alternative of incorporating fact types in classes for their base object types was discussed. Binary fact types can always be incorporated, but fact types of higher degree can be considered for incorporation at presence of uniquness constraints over single predicators. Incorporation of objectified fact types is a meaningful option in even more restrictive situations only. The rationale behind this is that beside representation of (objectified) facts, we also have to deal with the representability of facts attached to (objectified) facts.

The mechanism of incorporating objectified fact types is illustrated in figure 9.17. It shows that if an objectified fact type f is incorporated in the counterpart class of one of its base object types, then the fact types in which f participates are also incorporated in the same class. The unambigous representability is ensured by the required uniqueness constraint.

Let $f = \{p_1, ..., p_n\} \in \mathcal{F}$ be an objectified fact type. f can be considered for incorporation in class $C_{\mathsf{Base}(p_i)}$ ($i \in \{1, n\}$) if $\mathsf{unique}(p_i)$ is specified. The result of incorporating f in $C_{\mathsf{Base}(p_i)}$ is obtained by applying the following steps:

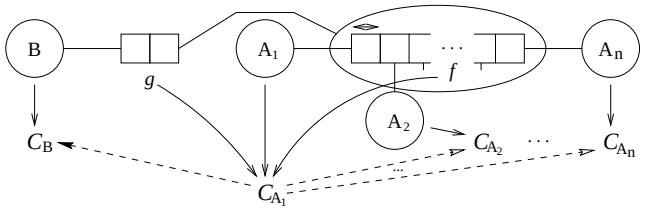


Figure 9.17: Incorporation of objectified fact type

1. Incorporate f in $C_{\mathsf{Base}(p_i)}$ as described in section 9.5.5.2.
2. Incorporate all fact types involving f in class $C_{\mathsf{Base}(p_i)}$. For each $p \in \mathcal{P}$ with $\mathsf{Base}(p) = f$ incorporate $\mathsf{Fact}(p)$ in $C_{\mathsf{Base}(p_i)}$ according to section 9.5.5.2 such that f is substituted with $\mathsf{Base}(p_i)$.

Obviously, this transformation requires the fact types with base object type f to fulfill the condition for incorporation. If for any of those fact types this precondition is not satisfied, then f cannot be incorporated either. This leads to a situation that can be captured by recursive checking and evaluation.

At some earlier points of this paper we assumed the existence of classes for base object types when describing transformation alternatives for fact types (also bridge types). We also promised to highlight and treat situations when it's not the case. Now we reached such a situation, because the incorporation mechanism is applied to fact types with base f, but no class for f exists. The special treatment here is that we explicitly prescribed the class in wich the incorporation has to be performed instead of the class for f. Note, however, that during the incorporation of f the existence of classes for its base object types is still an assumption (precondition).

The relational-like incorporation for binary fact types (see figure 9.11) is generally not applicaple to objectified binary fact types, only when the trivial mapping is obtained back due to uniqueness constraints. Otherwise the problem of unambigous representability arises.

Quasi-incorporation of objectified fact types is only possible at the presence of appropriate uniqueness constraints. The illustrating figure is obtained as a combination of figures 9.13 and 9.17 and is presented in figure 9.18 without further explanation.

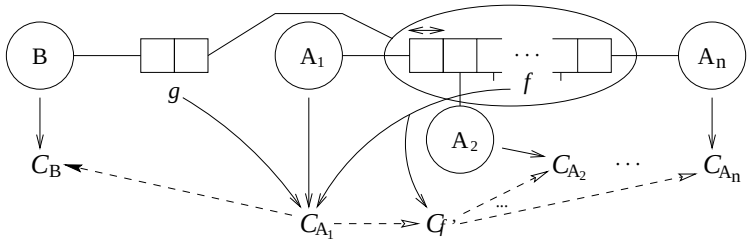


Figure 9.18: Quasi-incorporation of objectified fact type

9.5.6 Power types

Power types in PSM represent sets. An instance of a power type is a (non-empty) set of instances of its element type and is identified by its elements. Since in F-logic we have the concept of set-valued attribute, at first sight it might seem that power types can be represented simply by means of set-valued attributes.

However, the general situation is not that simple. The first complicating factor is that set-valued attributes have limited applicability. Nested power types (power types of power types) cannot be expressed in F-logic

in terms of set-valued attributes. For intance, in our example if budgets belong to groups of groups of projects, then that situation cannot be represented in the way shown above.

In section 9.4.5 we defined the parametric class $set(C)$, where the parameter C can be an arbitrary class, to model sets in F-logic more generally. This parametric class will be used for the transformation of power types. Set-valued attributes are used only for representing set-valued references (see e.g. section 9.5.5.2).

The second complicating factor is that, like other kinds of object types (except label types), power types can be subject of specialisation and generalisation. As we will see in section 9.5.8, the transformation of specialisation and generalisation hierarchies in PSM schemas implies inheritance hierarchies between F-logic classes. The solution for this is that a power type results in a class if it is specialised or generalised.

Even if a power type is neither specialised nor generalised, having a separate class corresponding to it provides wider range for the tranformation of the whole conceptual schema at hand. Then, for example, fact types involving a power type can be considered for incorporation in the counterpart class of the power type. In more general, all alternatives for the transformation of fact types where the existence of classes for base object types was assumed (see section 9.5.5) can be applied.

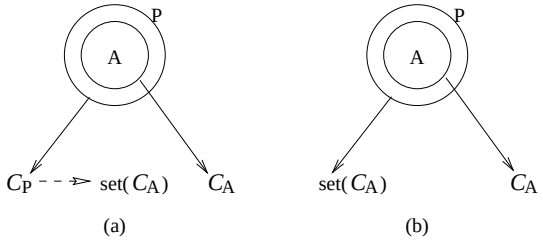


Figure 9.19: Alternatives for power types

The considerations above suggest that the transformation of power types with or without defining counterpart classes can be both reasonable solutions, see figure 9.19. The right decision depends on the application domain. Our manner of transforming a power type P covers both alternatives defined in a uniform way, consisting of three steps as follows:

1. First, as an initial step, a class C_P is introduced with a single (implicit) scalar-valued attribute $Attr_{elements}$ of type $set(C_{\mathsf{Elt}(P)})$ for containing the set elements:

$$C_P\,[\,Attr_{elements} \Rightarrow set(C_{\mathsf{Elt}(P)})\,]$$

2. During the transformation of the overall information structure consider P as if it was an entity type (remember that entity types are always mapped to classes). All possible alternatives for transformation of fact types involving P can now be applied.

3. When the whole information structure has been translated C_P will or will not contain attributes other than $Attr_{elements}$. If C_P contains no attribute other than $Attr_{elements}$, then optionally the class C_P can be eliminated by substituting it with $set(C_{\mathsf{Elt}(P)})$ where it occurs.

   Here the following design matter has to be noted. This optional step should be considered as potential option only when the sets represented by power type P are considered to be immutable (they cannot be modified, only replaced). Otherwise, collection update anomalies may arise. For example, adding a new element to a set may require the same modification at several places.

As it can be seen, the translation of a power type requires its element type (not necessarily entity type, it can be e.g. a fact type) to be mapped to a class. This has to be taken into account during the transformation.

In concrete terms it means that if a fact type is the element type of a power type, then the fact type has to result in a class, i.e. it has to be transformed according to the trivial mapping.

Furthermore, note that although at the end power types do not necessarily result in classes, the assumption, made at several places before, that corresponding classes for base object types exist when fact types are mapped (see section 9.5.5) did not become invalid, because the elimination of the counterpart class for a power type may be performed as a final step only.

9.5.7 Sequence types

Sequence types in PSM represent lists (sequences of values). Although such a construct is not part of the definition, it can be modelled in F-logic easily. In [89] the parametric class $list(C)$ has been defined (modelled), where the parameter C can be an arbitrary class. That definition was recalled and refined in section 9.4.5.

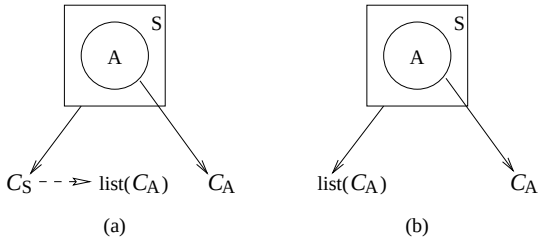


Figure 9.20: Alternatives for sequence types

In principle, the transformation of sequence types into F-logic is analogous to that of power types, see figure 9.20. The only differences are that (1) instead of the parametric class $set(C)$ we use the parametric class $list(C)$, and (2) the implicit attribute is denoted by $Attr_{sequence}$ instead of $Attr_{elements}$. In other aspects the procedure is identical, therefore is not further detailed here.

Furthermore, similar considerations and notes are valid for the mapping of sequence types what we had for the transformation of power types.

9.5.8 Specialisation and generalisation

Specialisation and generalisation hierarchies of PSM models are translated to subclass hierarchies in F-logic. Both A Spec B and B Gen A result in $C_A :: C_B$ (C_A is subclass of C_B). Therefore, object types that are involved in such hierarchies have to be mapped to classes. In section 9.3 we said that only entity types can act as subtype or generalised object type. They always result in classes. It was also said that label types are prohibited from being specialised or generalised, i.e. we do not have to deal with them here. As a consequence, if an object type O is either a supertype (the upper object type in specialisation) or a specifier (the lower object type in generalisation), then:

- If O is a fact type, then it has to be translated according to the trivial mapping.
- If O is either a power type or a sequence type, then the elimination of its corresponding class is not allowed.

Up to now, specialisation and generalisation were treated in the same way. However, they are different concepts. As mentioned in section 9.3, the difference is reflected in the way subtypes and generalised object types get their identification, and in the way their population is derived. A generalised object type inherits identification from its specifiers and its population is the union of the populations of its specifiers.

A subtype inherits identificaton from its supertype and its population is a subset of the population of its subtype and is derived by means of a subtype defining rule.

Despite the fact that both kinds of hierarchies result in F-logic subclass hierarchies, the aforementioned differences are reflected on F-logic level as well. In object-oriented systems it is usual to have the concept of abstract classes. An abstract class is a class that is never instantiated, i.e. its only purpose is acting as a superclass. Clearly, generalisation corresponds to introducing abstract classes. This means that what has to be ensured is that in an F-logic class hierarchy that corresponds to a generalisation hierarchy non-leaf level classes do not have object instances that do not belong to some leaf level class. This is, not surprisingly, in line with the way generalised object types are populated and can be achieved by appropriate constraints.

For a subtype relationship the subtype defining rule has be taken into account. To achieve this, a rule can be defined in the F-program. The body of the rule is the subtype defining rule translated into F-logic, while the head is an is-a assertion for class membership. We note however, that in concrete systems specialisation via constraints (subtype defining rules) is often not allowed, because it cannot be combined with other important principles. For example, in [111] it is explained that substitutability, static type checking, mutability and specialisation via constraints cannot be present together, therefore one of them has to be left out. According to [111], SQL3 has chosen to leave out the facility of specialisation via constraints.

9.5.9 Methodization

In general, an attribute of a class carries direct reference(s) to related object(s), which means that by means of attributes (methods without arguments) only relationships between two objects can be captured. However, it would often be useful to have a device to enable that objects related to a given combination of other objects can be obtained. This can be achived by defining general methods (method with arguments). A mechanism, called *methodization*, can be introduced for this purpose as a complementary device in order to provide efficient access (querying) of data stored in attributes of classes.

For example, provided that a fact type is mapped to a class, facts (relationships) become stored objects. The attributes of a relationship object contain references to objects that are in relationship. If now we want to know what objects of a given class are in relationship with some combination of objects of some classes, we have to perform an appropriate query. Clearly, it can be very useful to define access *methods* to make querying (traversing relationships) easier and more efficient.

The scale of choices for methods is quite wide. The signatures of methods are strongly influenced by uniqueness constraints. The behavior of methods can be defined in terms of F-logic rules. In [12] a similar technique is introduced, which is called *pivoting*.

Inverse attributes vs methodization

It was said earlier that, in principle, inverse attributes can be optionally defined beside attributes needed for sufficient storage of data. It improves query performance, but makes update more complex. To ensure integrity, introducing inverse attributes must be done along with generating inverse constraints.

Alternatively, stored inverse attributes can be substituted with *methods* without arguments. In this case the inverse constraints needed for the inverse attributes are converted to rules in the F-program. Those rules define the behavior of (inverse) methods.

9.6 Example

Until now we discussed possible ways to transform conceptual structures to OO database schemas expressed in terms of F-logic. To illustrate some alternatives, in this section we transform the PSM schema in figure 9.3 into F-logic. During this mapping object molecules defining only parts of classes are obtained often.

Since, according to the semantics of F-logic, they can be unified to get equivalent more complex object molecules, at the end we will also present the unified result.

9.6.1 Declarations

Entity types are mapped to classes. As an initial step, for each entity type a an empty object molecule is generated, i.e. we obtain:

$$\begin{array}{ccccc} C_{Equipment}\,[\,] & C_{Car}\,[\,] & C_{PC}\,[\,] & C_{Person}\,[\,] & C_{Manager}\,[\,] \\ C_{Coworker}\,[\,] & C_{Dept.}\,[\,] & C_{Building}\,[\,] & C_{Project}\,[\,] & C_{Article}\,[\,] \\ C_{Company}\,[\,] & C_{Activity}\,[\,] & C_{Duration}\,[\,] & C_{Amount\_of\_money}\,[\,] & \end{array}$$

Furthermore, in the initial step a class is created for each power type and sequence type with a single attribute for containing the set and list elements, respectively:

$$C_{Pr\_group}\,[\,Attr_{elements} \Rightarrow set(C_{Project})\,] \quad C_{Daily\_activities}\,[\,Attr_{sequence} \Rightarrow list(C_{Activity})\,]$$

Specialisation and generalisation relationship result in subclass relationships:

$$\begin{array}{cc} C_{Manager} :: C_{Person} & C_{Coworker} :: C_{Person} \\ C_{Car} :: C_{Equipment} & C_{PC} :: C_{Equipment} \end{array}$$

Bridge types are incorporated in classes obtained for the involved non-label type. Suppose that with the occuring label types the following concrete domains, that are assumed to be built-in classes in F-logic, are associated:

$$\begin{array}{l} \mathsf{Dom}(Date) = Date \\ \mathsf{Dom}(Dollars) = \mathsf{Dom}(Hours) = Integer \\ \mathsf{Dom}(Reg\_nr) = \mathsf{Dom}(PC\_nr) = \mathsf{Dom}(P\_id) = \mathsf{Dom}(P\_name) = String \\ \mathsf{Dom}(Pr\_name) = \mathsf{Dom}(D\_name) = \mathsf{Dom}(B\_code) = \mathsf{Dom}(A\_code) = String \\ \mathsf{Dom}(C\_code) = \mathsf{Dom}(C\_name) = \mathsf{Dom}(Tel\_nr) = \mathsf{Dom}(A\_descr) = String \end{array}$$

Furthermore, for identifying bridge types the predicator with non-label base object type X is referred to as has_X. Then the mapping of bridge types yields the following object molecules:

$$\begin{array}{cc} C_{Person}\,[\,Attr_{has\_P\_id} \Rightarrow String\,] & C_{Person}\,[\,Attr_{has\_Pname} \Rightarrow String\,] \\ C_{Equipment}\,[\,Attr_{bought\_on} \Rightarrow Date\,] & C_{Car}\,[\,Attr_{has\_Reg\_no} \Rightarrow String\,] \\ C_{PC}\,[\,Attr_{has\_PC\_nr} \Rightarrow String\,] & C_{Project}\,[\,Attr_{has\_Pr\_name} \Rightarrow String\,] \\ C_{Dept.}\,[\,Attr_{has\_D\_name} \Rightarrow String\,] & C_{Building}\,[\,Attr_{has\_B\_code} \Rightarrow String\,] \\ C_{Article}\,[\,Attr_{has\_A\_code} \Rightarrow String\,] & C_{Activity}\,[\,Attr_{has\_A\_descr} \Rightarrow String\,] \\ C_{Duration}\,[\,Attr_{has\_Hour} \Rightarrow Integer\,] & C_{Amount\_of\_money}\,[\,Attr_{has\_Dollars} \Rightarrow Integer\,] \\ C_{Company}\,[\,Attr_{has\_C\_code} \Rightarrow String\,] & C_{Company}\,[\,Attr_{has\_Cname} \Rightarrow String\,] \\ C_{Company}\,[\,Attr_{has\_Telnr} \Rightarrow\!\!\!\!\Rightarrow String\,] & \end{array}$$

Finally, we transform (non-bridge) fact types. In section 9.5.5 a number of alternatives were discussed. We transform the fact types of figure 9.3 such that we cover as many alternatives as reasonable according to the complexity of the input schema. In order to identify chosen alternatives unambigously we always refer to the corresponding figures. The specified uniqueness constraints, of course, are taken into account.

The objectified fact type *Coworkership* is mapped according to the trivial mapping (figure 9.9), which results in:

$$C_{Coworkership}\,[\,Attr_{works\_for} \Rightarrow C_{Person};\;\; Attr_{has\_as\_coworker} \Rightarrow C_{Project}\,]$$

The binary fact type *Cow_dur* is incorporated (figure 9.10) in the class obtained for the base object type of its predicator *has_as_duration* yielding:

$$C_{Coworkership}\,[\,Attr_{has\_as\_duration} \Rightarrow C_{Duration}\,]$$

Incorporation of binary fact types in one of the two classes corresponding to their base object types is applied to other fact types as well. Fact types *Salary*, *Activities* and *Budget* are incorporated in classes $C_{Coworker}$, $C_{Manager}$ and C_{Pr_group}, respectively. The following object molecules are generated:

$$C_{Coworker}\,[\,Attr_{earns} \Rightarrow C_{Amount\_of\_money}\,]$$
$$C_{Manager}\,[\,Attr_{performes} \Rightarrow C_{Daily\_activities}\,]$$
$$C_{Pr\_group}\,[\,Attr_{may\_spend} \Rightarrow C_{Amount\_of\_money}\,]$$

Fact type *Car_usage* is incorporated in both classes corresponding to the involved object types (figure 9.10, alternative (c)), namely in C_{Car} and $C_{Manager}$. Two object molecules are obtained:

$$C_{Car}\,[\,Attr_{used\_by} \Rightarrow C_{Manager}\,]$$
$$C_{Manager}\,[\,Attr_{uses} \Rightarrow C_{Car}\,]$$

In figure 9.11 a relational-like way of incorporating binary fact types, as the combination of trivial mapping and incorporation (alternatively seen as nesting on the result of trivial mapping), has been shown. Fact type *Location* is mapped according to this way of transformation. Object type *Dept.* is chosen as central object type. This results in:

$$C_{Location}\,[\,Attr_{is\_located\_at} \Rightarrow C_{Dept.};\;\; Attr_{accomodates} \twoheadrightarrow C_{Building}\,]$$

To illustrate the alternative coming from the combination of incorporation (OO nature) with restricted grouping (relational nature), fact types *Employment* and *PC_usage* with common object type *Person* are grouped according to the OR-like grouping shown in figure 9.15. The following single class (object molecule) is generated:

$$C_{\{Empl.,\,PC\_usage\}}\,[\,Attr_{\{empl.\_by,\,works\_on\}} \Rightarrow C_{Person};\, Attr_{employs} \Rightarrow C_{Dept.};\, Attr_{belongs\_to} \twoheadrightarrow C_{PC}\,]$$

The combination of OR-like grouping and quasi-incorporation has been depicted in Figure 9.16. To set an example for this mechanism, fact types *Management* and *Supply* are grouped together via object type *Project* such that for fact type *Supply* a subsidiary class $C_{Supply'}$ is introduced. Beside this subsidiary class, a relation class is also defined representing fact type *Management* and partially fact type *Supply*. The obtained object molecules are the following:

$$C_{Supply'}\,[\,Attr_{supplies} \Rightarrow C_{Company};\;\; Attr_{is\_supplied} \Rightarrow C_{Article}\,]$$
$$C_{\{Man.,\,Supply\}}\,[\,Attr_{\{man.\_by,\,rec.\}} \Rightarrow C_{Project};\;\; Attr_{manages} \twoheadrightarrow C_{Person};\;\; Attr_{rec.} \twoheadrightarrow C_{Supply'}\,]$$

By now, each fact type of figure 9.3 has been translated. We did not exploit every particular alternative discussed in section 9.5.5 coming from different kinds of combinations. However, all the basic building blocks used in some combination, such as trivial mapping, incorporation, quasi-incorporation, grouping, have been covered.

Class elimination

After completing the transformation of the PSM structure of figure 9.3, the elimination of classes for power types and sequence types can be considered (see sections 9.5.6 and 9.5.7). Since the class C_{Pr_group} contains an attribute other than $Attr_{elements}$, it cannot be eliminated. The elimination of class $C_{Daily\_activities}$, however, is reasonable. The class is substituted with $list(C_{Activity})$ and is removed.

As mentioned in section 9.5.3, classes for entity types with label type nature can also be eliminated. In our example such classes are $C_{Duration}$ and $C_{Amount\_of\_money}$ at least. They are, therefore, eliminated. Occurences are replaced with (the built-in classes for) the concrete domains of their corresponding label types.

Unified declarations

Due to the fact that in some cases above different parts of class definitions were obtained at different points of the transformation, some classes are defined by means of more than one object molecule. The separate part can now be put together as presented below. The class eliminations above are taken into account. Clearly, the input schema does not cover all the details of the application domain, which lead to simple (entity) classes in many cases. Those classes likely have additional attributes, which is also indicated below.

During the above transformation we used denotations rather than names for classes and attributes. Although in our approach assigning meaningful names to obtained F-logic components by means of a naming function belongs to the implementation (second) step (see section 9.5.2), in order to make the example more readable we already give names to classes and attributes now.

$Person\,[\,P\_id \Rightarrow String;\ name \Rightarrow String; ...\,]$

$Manager :: Person$

$Manager\,[\,daily\_activities \Rightarrow list(Activity);\ uses\_car \Rightarrow Car; ...\,]$

$Coworker :: Person$

$Coworker\,[\,salary \Rightarrow Integer; ...\,]$

$Equipment\,[\,bought\_on \Rightarrow Date; ...\,]$

$Car :: Equipment$

$Car\,[\,reg\_nr \Rightarrow String;\ used\_by \Rightarrow Manager; ...\,]$

$PC :: Equipment$

$PC\,[\,pc\_nr \Rightarrow String; ...\,]$

$Project\,[\,title \Rightarrow String; ...\,]$

$Department\,[\,name \Rightarrow String; ...\,]$

$Building\,[\,B\_code \Rightarrow String; ...\,]$

$Article\,[\,A\_code \Rightarrow String; ...\,]$

$Activity\,[\,description \Rightarrow String; ...\,]$

$Company\,[\,C\_code \Rightarrow String;\ name \Rightarrow String;\ tel\_nrs \Rightarrow\!\!> String; ...\,]$

$Pr\_group\,[\,projects \Rightarrow set(Project);\ budget \Rightarrow Integer; ...\,]$

$Coworkership\,[\,employee \Rightarrow Person;\ project \Rightarrow Project;\ duration \Rightarrow Integer\,]$

$Dept\_loc\,[\,dept \Rightarrow Department;\ building \Rightarrow\!\!> Building\,]$

$Employment\,[\,employee \Rightarrow Person; dept \Rightarrow Department; works\_on \Rightarrow\!\!> PC\,]$

$Supply\,[\,supplier \Rightarrow Company;\ article \Rightarrow Article\,]$

$Project\_rel\,[\,project \Rightarrow Project;\ managers \Rightarrow\!\!> Person;\ receives \Rightarrow\!\!> Supply\,]$

9.6.2 Constraints

The result of the transformation of the information structure in figure 9.3 was influenced by simple conceptual uniqueness constraints. On the other hand, in addition to class definitions, the structure transformation results in some kinds of basic constraints in F-logic, such as key, mandatory and inverse constraints. In this section we provide these constraints for our example.

Uniqueness constraints

F-logic uniqueness (key) constraints are obtained in two ways. Firstly, from the unique representation of facts (relationships) (see also [12]). Secondly, the simple conceptual uniqueness constraints in figure 9.3 are translated as well. For uniqueness constraints the macro "!– Unique$(C, \{A_1, ..., A_n\})$" is defined as follows:

$$!-\ X \doteq Y \ \longleftarrow\ \begin{array}{l} X : C\,[\,A_1 \rightarrow(\twoheadrightarrow)\ R_1; ...; A_n \rightarrow(\twoheadrightarrow)\ R_n\,] \wedge \\ Y : C\,[\,A_1 \rightarrow(\twoheadrightarrow)\ R_1; ...; A_n \rightarrow(\twoheadrightarrow)\ R_n\,] \end{array}$$

The notation $A_i \rightarrow(\twoheadrightarrow)\ R_i$ means that if A_i is scalar-valued, then $\rightarrow$ is used, otherwise $\twoheadrightarrow$. In our example the following F-logic uniqueness constraints are generated:

!– Unique$(Person, \{P_id\})$
!– Unique$(Car, \{reg_nr\})$
!– Unique$(PC, \{pc_nr\})$
!– Unique$(Department, \{name\})$
!– Unique$(Article, \{A_code\})$
!– Unique$(Company, \{C_code\})$
!– Unique$(Coworkership, \{employee, project\})$
!– Unique$(Employment, \{employee\})$
!– Unique$(Supply, \{supplier, article\})$
!– Unique$(Manager, \{uses_car\})$
!– Unique$(Car, \{used_by\})$
!– Unique$(Project, \{title\})$
!– Unique$(Building, \{B_code\})$
!– Unique$(Activity, \{description\})$
!– Unique$(Pr_group, \{projects\})$
!– Unique$(Dept_loc, \{dept\})$
!– Unique$(Employment, \{works_on\})$
!– Unique$(Project_rel, \{project\})$

Mandatory constraints

Since the population of fact types consists of total functions, it has to be ensured that if a class corresponds to a fact type, then each attribute of a member of that class (fact object) carries some value. Again, a general macro "!– Mandatory$(C, \{A_1, ..., A_n\})$" is introduced to serve as a shorthand for the following:

$$\begin{array}{l} !-\ (\exists Y)X\,[\,A_1 \rightarrow(\twoheadrightarrow)\ Y\,] \ \longleftarrow\ X : C \\ \quad \ldots \\ !-\ (\exists Y)X\,[\,A_n \rightarrow(\twoheadrightarrow)\ Y\,] \ \longleftarrow\ X : C \end{array}$$

In the case of our example the following mandatory constraints are necessary:

$$!\!- \ \mathsf{Mandatory}(Coworkership, \{employee, project\})$$
$$!\!- \ \mathsf{Mandatory}(Dept\_loc, \{dept, building\})$$
$$!\!- \ \mathsf{Mandatory}(Employment, \{employee\})$$
$$!\!- \ \mathsf{Mandatory}(Supply, \{supplier, article\})$$
$$!\!- \ \mathsf{Mandatory}(Project\_rel, \{project\})$$

Inverse constraints

When two attributes are considered to be inverses of each other, inverse constraints have to be defined. According to the possible kinds of relationship types between two classes we introduce the following three macros:

$$!\!- \ \mathsf{InverseOneToOne}(C_1, A_1, C_2, A_2) \stackrel{\text{def}}{=} \quad !\!- \ Y : C_2\,[\,A_2 \rightarrow X\,] \ \longleftarrow \ X : C_1\,[\,A_1 \rightarrow Y\,]$$
$$!\!- \ Y : C_1\,[\,A_1 \rightarrow X\,] \ \longleftarrow \ X : C_2\,[\,A_2 \rightarrow Y\,]$$

$$!\!- \ \mathsf{InverseOneToMany}(C_1, A_1, C_2, A_2) \stackrel{\text{def}}{=} \quad !\!- \ Y : C_2\,[\,A_2 \rightarrow X\,] \ \longleftarrow \ X : C_1\,[\,A_1 \twoheadrightarrow Y\,]$$
$$!\!- \ Y : C_1\,[\,A_1 \twoheadrightarrow X\,] \ \longleftarrow \ X : C_2\,[\,A_2 \rightarrow Y\,]$$

$$!\!- \ \mathsf{InverseManyToMany}(C_1, A_1, C_2, A_2) \stackrel{\text{def}}{=} \quad !\!- \ Y : C_2\,[\,A_2 \twoheadrightarrow X\,] \ \longleftarrow \ X : C_1\,[\,A_1 \twoheadrightarrow Y\,]$$
$$!\!- \ Y : C_1\,[\,A_1 \twoheadrightarrow X\,] \ \longleftarrow \ X : C_2\,[\,A_2 \twoheadrightarrow Y\,]$$

In our example fact type Car_usage has been incorporated in both classes obtained for the two involved entity types, namely in classes $Manager$ and Car, yielding one attribute in each being inverses of each other. Therefore, an inverse constraint is also required as follows:

$$!\!- \ \mathsf{InverseOneToOne}(Manager, uses\_car, Car, used\_by)$$

Effects of specialisation and generalisation

The population of subtypes (specialisation) is defined by subtype defining rule. This mechanism has to be translated into F-logic. Since we focus on structures in the present paper, we do not deal with the translation of subtype defining rules in general. In our example we $Manager\ \mathsf{Spec}\ Person$ and $Coworker\ \mathsf{Spec}\ Person$. Let $\Psi_{Manager}$ and $\Psi_{Coworker}$ denote the F-logic counterpart of the subtype defining rules for subtypes $Manager$ and $Coworker$, respectively. Then we introduce the following two constraints:

$$!\!- \ X : Manager \ \longleftarrow \ X : Person \ \wedge \ \Psi_{Manager}(X)$$
$$!\!- \ X : Coworker \ \longleftarrow \ X : Person \ \wedge \ \Psi_{Coworker}(X)$$

The subtype defining rules are: A person is a manager if he/she plays the role $manages$. A person is a coworker if he/she plays the role $works\_for$. It has effect on the final result of the translation of fact types $Management$ and $Coworkership$. In class $Project\_rel$ the result type $Person$ is replaced with $Manager$:

$$Project\_rel\,[\,project \Rightarrow Project;\ managers \twoheadrightarrow Manager;\ receives \twoheadrightarrow Supply\,]$$

In class $Coworkership$ the result type $Person$ is replaced with $Coworker$:

$$Coworkership\,[\,employee \Rightarrow Coworker;\ project \Rightarrow Project;\ duration \Rightarrow Integer\,]$$

In case of generalisation it has to be ensured that every instance in the population of a generalised type belongs to the population of one of its specifiers. This can be expressed in terms of constraints on F-logic level. In our example we obtain:

$$!\!- \ (X : Car \ \vee \ X : PC) \ \longleftarrow \ X : Equipment$$

9.6.3 Inverse attributes and methods

As said in section 9.5.9, a wide range of introducing inverse attributes with inverse constraints and methods with the definition of their behavior is possible. To illustrate these general mechanisms, now we introduce an additional inverse attribute as well as a method. Class *Coworker* is augmented with attribute *involved_in* containing references to *Coworkership* objects in which a given person is involved. The following additional object molecule, that can be unified with the exisiting one for class *Coworker*, and inverse constrain are defined:

$$Coworker\,[\,involved\_in \twoheadrightarrow Coworkership\,]$$
$$!\!-\ \mathsf{InverseOneToMany}(Coworker, involved\_in, Coworkership, employee)$$

As an example of an access method, the method *suppliers* is introduced in class *Project* returning the companies that supply a given article for the project on which the method is invoked. The object molecule defining the signature of the method is the following:

$$Project\,[\,suppliers\,@\,Article \twoheadrightarrow Company\,]$$

The semantics of the method is given by means of the following F-logic rule:

$$Y\,[\,suppliers\,@\,W \twoheadrightarrow V\,] \longleftarrow X : Project\_rel\,[\,project \rightarrow Y;\ receives \twoheadrightarrow Z\,]\ \wedge\ Z : Supply\,[\,supplier \rightarrow V;\ article \rightarrow W\,]$$

9.7 Database generation: example in ODMG-93

The second step of the transformation (the implementation step, see figure 9.1) is the translation of intermediate models defined in terms of F-logic into a OO final target environment, e.g. SQL3 or ODMG-93. This means that a sequence of DDL statements in the corresponding database language has to be generated from F-logic specifications. Unlike in the design step, in the implementation step separate translations have to be defined for different target environments, where all the system specific details must be dealt with. This can be implemented by means of translation tables. They contain all the system specific information (e.g. supported data types, correspondece between F-logic "built-in" classes and system specific data types) needed for the generation of DDL statements.

Since in the paper the main focus is on the design step, we do not further discuss the implementation step in general. Instead, below we show how a part of the F-logic schema in section 9.6 obtained for the PSM schema of figure 9.3 is translated into ODMG-93. Beside class declarations (including the additional inverse attribute and method), uniqueness and inverse constraints are translated. Specifying other constraints is not supported directly in ODMG-93. For the syntax and semantics of ODMG-93 we refer to [34].

```
interface Person
(   extent Persons
    keys P_id   ) : persistent
{   attribute string P_id ;
    attribute string name ; ...  } ;

interface Manager : Person
(   extent Managers
    keys uses_car   ) : persistent
{   relationship List<Activity> daily_activities ;
    relationship Car used_car inverse Car:: used_by ; ...  } ;
```

```
interface Coworker : Person
(   extent Coworkers   ) : persistent
{   attribute integer salary ;
    relationship Set<Coworkership> involved_in inverse Coworkership:: employee ; ...  } ;

interface Activity
(   extent Activities
    keys description   ) : persistent
{   attribute string description ; ...  } ;

interface Project
(   extent Projects
    keys title   ) : persistent
{   attribute string title ;
    ...
    Set<Company> suppliers( in Article art ) ;   } ;

interface Article
(   extent Articles
    keys A_code   ) : persistent
{   attribute string A_code ; ...  } ;

interface Company
(   extent Companies
    keys C_code   ) : persistent
{   attribute string C_code ;
    attribute string name ;
    attribute Set<string> tel_nrs ; ...  } ;

interface Coworkership
(   extent Coworkerships
    keys (employee, project)   ) : persistent
{   attribute integer duration ;
    relationship Coworker employee inverse Coworker:: involved_in ;
    relationship Project project ;   } ;

interface Supply
(   extent Supplies
    keys (supplier, article)   ) : persistent
{   relationship Company supplier ;
    relationship Article article ;   } ;

interface Project_rel
```

```
(   extent Project_rels
    keys project   ) : persistent
{   relationship Project project ;
    relationship Set<Person> managers ;
    relationship Set<Supply> receives ;   } ;
```

9.8 Summary

In this paper we dealt with the transformation of conceptual data models into next generation (in some way object-oriented) database environments. In our approach this transformation is captured within the framework of a two level architecture (see figure 9.1). Conceptual models are first mapped to intermediate specifications (design step). Then the obtained intermediate specifications are translated into the database language of a given target database system (implementation step). For expressing conceptual models we use the object-role modelling technique PSM (Predicator Set Model), a formalized extension of NIAM. As final target environments we consider OO (truly object-oriented) and OR (object-relational) database systems in general, and ODMG-93 and SQL3 in particular. Intermediate specifications are expressed in terms of F-logic, a logic-based abstract specification language for object-oriented systems. (The concrete architecture is shown in figure 9.2.)

We presented the general framework on one hand, and discussed a number of alternatives for the transformation of conceptual schemas on the other hand. The focus was on the design step. A corresponding graphical notation for specifying transformation alternatives has also been provided. We dealt with the mapping of information structures, which is influenced by simple uniqueness constraints. Therefore, such constraints have been covered too. A general mechanism for translating complex constraints is under development.

Our work was inspired by the fact that only few proposals exist for the mapping of conceptual (semantic) models to OO databases ([121], [94], [12]), and no transformation method has been defined yet, in our opinion, which is general enough. Here we provided the kernel of a more general transformation mechanism. On one hand, we considered advanced modelling constructs not covered by the mentioned papers, such as power (set) types, sequence (list) types and generalisation. On the other hand, we discussed more possibilities for transforming structures. We also provided several alternatives having relational view combined with OO view. This is important in order to cover object-relational target systems beside truly OO systems. In the aforementioned proposals the transformation into truly OO database systems is aimed at.

For further research the following directions have been recognized:

- In section 9.5 transformation alternatives from PSM to F-logic were discussed and were illustrated by means of our graphical notation. Since both PSM and F-logic have underlying formalism, the formal treatment of the discussed alternatives is possible and needed. We are currently working on it.

- In order to provide an automated transformation method an algorithmic translation may be defined based on the formal definition of different transformation alternatives. Such an algorithm can be provided with a number of parameters for guiding the transformation process according to the wishes of the database designer.

- In the present paper we mainly dealt with the tranformation of structures. Based on the result of structure transformation, the population of information structures may also be mapped in a formal (algorithmic) way. The mapping of populations can be defined along with the mapping of the corresponding structures.

- Conceptual integrity constraints are part of conceptual schemas and are important. Constraints have to be translated as well. In [94] graphical constraints are considered, but those constraints are not very general. [12] considers several kinds of dependency constraints not exceeding the boundaries of single relationship types. PSM is rich in expressing integrity constraints on the conceptual level. The graphical PSM constraints are similar to the ones treated in [94], but are much more general. Beside graphical constraints, more complex constraints can be specified in the conceptual language LISA-D ([81]) developed for PSM.

Chapter 10

Conclusions

The approach discussed in this book allows us to describe, execute, and analyze data transformation algorithms. The database structures that are produced are specified in terms of an underlying conceptual data model. We summarize our results and identify directions for further research.

10.1 General

First we consider general requirements with respect to conceptual data models and their internal representations.

formal description of conceptual models: *Conceptual data models are input to the transformation process and should therefore be defined in a formal way.*

The Predicator Model described in chapter 2 is a formalization of object-role models. The verification of object-role models has also been considered (see [20]).

Further research should consider the question whether and how other kinds of specifications can serve as input to the optimization process. As an example, object-oriented models also need translation into efficient database structures. Moreover, the Predicator Model has been extended to the Predicator Set Model (PSM [82]), [78]). The use of the advanced modelling constructs from PSM in the context of database optimization was considered in section 4.5. This however should be examined in more detail.

correctness of internal representations: *In order to guarantee correctness of internal representations, the representation mechanism used must present a clear relation with an underlying conceptual model.*

The representation mechanism introduced in chapter 3 is defined in terms of the predicators from the conceptual model. In chapter 4 it was shown how conceptual populations are transformed into populations of internal representations and how conceptual updates affect internal populations.

However, equivalence of conceptual model and internal representations was not formally proved. For this formal proof, it is suggested to use homomorphisms between the set of possible conceptual populations and the set of possible internal populations. A first step in this direction was taken in [79] and [13]. This approach is in line with [126], where semantic equivalence is also based on sets of possible populations.

Another point to be mentioned here concerns objectified fact types. The representation mechanism from chapter 3 allows fact types to be incorported in internal representations explicitly (see e.g. figure 3.2). This is not always the most natural way of representing object-role information structures, for example in the case of a fact type containing two or more predicators having the same non-atomic base. Several alternative ways of treating objectifications were discussed in section 4.5.3.

generality of approach: *The approach should support different types of target Database Management Systems.*

The internal representations introduced in chapter 3 can be implemented in terms of different data models, including the relational model, the hierarchical model and the network model. Nested relations were considered in section 3.2.5, and were further illustrated in chapter 4. Flat relations can be obtained by observing certain integrity rules (see e.g. parameter NoNesting in section 3.3.2) or by flattening nested relations. The flattening of nested relations corresponds to ignoring the edges and labels in internal representations (see also section 5.2). In section 4.5.2 the use of CODASYL network models was considered.

Although nested relations are not exactly the same as conventional hierarchical data models (e.g. IMS data structures [43]), they are quite similar (see also the hierarchical record structures in section 3.2.5).

However, the representation mechanism introduced in this book is *not* the most general representation mechanism. A more general mechanism may be obtained by relaxing the condition that an internal representation should consist of *trees* of predicator-nodes. For example, we can also allow networks of nodes. The resulting mechanism then covers the current representations (a tree is a special case of a network), but also other kinds of representations such as insertion order graphs ([31], [30]).

Another generalization of the representation mechanism concerns the fact that each predicator is allowed to occur in a representation at most once. Relaxing this requirement will result in more possibilities to introduce redundancy.

Note that generalization of the representation mechanism will lead to more possibilities for optimization. In certain cases this may be desirable, but it will also increase the size of the solution space (which may cause problems w.r.t. the performance of optimization algorithms).

10.2 Requirements concerning evolution

Next we consider the requirements concerning the evolution mechanism. This includes the parameterized initialization, the correctness of mutations, and the completeness of mutations.

parameterized initialization: *There should be a parameterized generator, producing candidate internal representations having specific desirable properties.*

The guidance conditions discussed in section 3.3.2 show how the initialization process can be forced to yield representations with specific properties, such as absence of redundancy and optional database fields, or on other structural properties (e.g. database table size). These properties are guaranteed on the basis of guidance conditions.

However, several aspects of conceptual data models were not embedded in the generation mechanism. For example, more advanced integrity constraints (such as set constraints) can also be used to guide the generation process. This would result in a more complete parameterization. Advanced integrity constraints and their mapping are considered in e.g. [135] and [47]. Other extensions of the generation mechanism may deal with design decisions concerning the implementation of subtype hierarchies and identification structures.

Another direction for future research is a more elaborate comparison of different starting-points for the transformation process. So far our experiments did not show important inequalities between different initial representations, even if the distance between these representations is big (see e.g. figure 7.19). This however has to be examined in more detail and the guidance conditions from section 3.3.2 will play a central role here. The relation with success entropy indicating the importance of initial representations provides an additional possibility to examine this aspect (see section 8.5.4).

correctness of mutations: *The mutation operators must result in correct internal representations for the same conceptual data model.*

The operators Prune, Graft and Promote introduced in chapter 5 were shown to be correct (theorem 5.4.1, 5.4.2 and 5.4.4). This correctness could be guaranteed as a result of the fact that the mutation operators use knowledge of the underlying conceptual model.

However, it may be desirable to incorporate additional knowledge in the mutation operators. As an example, the operators can be further parameterized in the style of the generation algorithm from section 3.3.2.

Further parameterization may help in solving performance problems, such as the consecutive $i - i$ transition problem addressed in lemma 8.4.3. This lemma expresses that, if a representation i has many worse neighbours, hill climbing will perform many superfluous transitions from i to i. Note that fitness proportional walk may also be used to deal with this problem, although the problem mentioned above can also occur there (see figure 8.11).

completeness of mutations: *The mutation operators must be complete in the sense that for each representation, any other representation can be produced by applying mutation operators.*

The mutation operators introduced in chapter 5 were shown to be complete. The completeness of Join and Split is a direct consequence of solution space $\mathcal{S}_r$ being inductively defined (theorem 5.5.1). The completeness of Prune, Graft, Promote is guaranteed by constructing a mutation path from an arbitrary representation to another arbitrary representation (theorem 5.5.2).

However, this completeness depends on the representation mechanism used for describing database structures. For a more general representation mechanism (see e.g. requirement generality discussed before), additional operators would be necessary. As an example, a representation mechanism allowing horizontal fragmentation (or partitioning, [35]) requires corresponding fragmentation operators. Schema transformations given in [47] and [159] can be applied here.

10.3 Requirements concerning optimizer transformations

Next we consider the requirements concerning the optimization process. This includes the optimization aspect, the executability of the approach, and the formalization of optimization dynamics.

optimization of internal representations: *There should be an instrument for optimization w.r.t. several criteria.*

The evolutionary approach described in chapter 6 allows for the optimization of different (conflicting) objectives, including response time and storage space. Existing approaches to database design (e.g. ONF) were embedded in our framework (section 6.2).

Future research should address further integration with existing approaches. As an example, in [70] and [72] a conceptual model is first optimized before the ONF algorithm is applied. From our point of view, each conceptual model has a corresponding internal solution space. So it would be interesting to compare the internal solution space for the optimized conceptual model with the internal solution space for the original conceptual model.

executability of approach: *There should be a prototype in which data transformations are implemented.*

The prototype EDO introduced in chapter 7 is an interactive tool for evolutionary database optimization. EDO has been used to obtain basic experimental results. The aim of our experimental results was to illustrate the approach. Consequently, convergence and configuration of evolutionary algorithms were not thoroughly examined.

It is obvious that this has to be examined in more detail. This is particularly the case when the approach is enhanced, which may result in problems with respect to the performance of optimization

algorithms (see also requirement generality considered before). Another enhancement leading to performance problems concerns the time/space cost models. As an example, these cost models will become much more complex when considering (a) integrity constraint enforcement after updates, (b) alternative implementation structures for subtype hierarchies and complex naming references, (c) more physically oriented aspects, and (d) different query optimization strategies.

Then, more advanced evolutionary algorithms and alternative ways of fitness computation (e.g. based on partial fitness) are likely to become necessary. Integration with other optimization approaches may also be of help here (see requirement optimization considered before).

formalization of transformation dynamics: *There should be a formal basis for describing and predicting the behaviour of transformation algorithms.*

The Markov chain fundamentals introduced in chapter 8 include transition matrices for basic evolutionary algorithms, along with formulae for success probability, first success probability, local optimum probability and success entropy. These formulae were illustrated in graphs for the running example (e.g. success graphs, figure 8.13).

However, when considering these graphs only some aspects were highlighted. Future research should examine this in more detail. Moreover, in future research a framework for comparing experimental results (chapter 7) with theoretical results (chapter 8) should be set up. This will also require a further elaboration of ratio ϑ, used as a measure for the trade-off between exploration and exploitation (section 8.6).

Appendix A

Legend of symbols

chapter	symbol	explanation
2	$\Sigma = \langle \mathcal{I}, \mathcal{C} \rangle$	conceptual data schema
	$\mathcal{C}$	set of integrity constraints
	$\mathcal{I} = \langle \mathcal{P}, \mathcal{O}, \mathcal{L}, \mathcal{E}, \mathcal{F}, \mathsf{Base}, \mathsf{Spec} \rangle$	information structure
	$\mathcal{P}$	set of predicators
	$\mathcal{O}$	set of object types
	$\mathcal{L}$	set of label types
	$\mathcal{E}$	set of entity types
	$\mathcal{F}$	set of fact types (a fact type is a set of predicators)
	$\mathsf{Base} : \mathcal{P} \longrightarrow \mathcal{O}$	function specifying the object type in a predicator
	$\mathsf{Spec} \subseteq \mathcal{E} \times \mathcal{E}$	binary relation defining specialization hierarchy
	$\mathcal{H} \subseteq \mathcal{P}$	set of predicators with non-atomic object type
	$\mathsf{Fact} : \mathcal{P} \to \mathcal{F}$	function specifying the fact type of a predicator
	$\mathsf{Pop}_{\mathcal{I}} : \mathcal{O} \to \wp^{\mathrm{fin}}(\Omega)$	function specifying the population of an object type
	Pop	(subscript is omitted when no confusion is likely to occur)
	Ω	universe of instances used for populating object types
	$\mathsf{dom}(f)$	domain of function
	$\mathsf{total}(\tau)$	total role constraint on set of predicators
	$\mathsf{unique}(\tau)$	uniqueness constraint on set of predicators
3	$T = \langle \mathcal{N}, E, \ell \rangle$	internal (tree) representation
	$\mathcal{N}$	set of nodes (a node is a set of predicators)
	$E \subseteq \mathcal{N} \times \mathcal{N}$	set of edges
	$\ell : E \to \mathcal{F}$	function specifying the fact type associated with an edge
	$\mathsf{Ext}(f) \subseteq \mathcal{N}$	set of nodes being children associated with a fact type
	$\mathsf{Forest}(T, \mathcal{I})$	predicate expressing partial correctness
	$\mathsf{CompleteForest}(T, \mathcal{I})$	predicate expressing complete correctness
	$\mathsf{Node}: \mathcal{P} \to \mathcal{N}$	function specifying the node a predicator is contained in
	$\mathsf{Anchor}: \mathcal{N} \to \mathcal{P}$	partial function specifying the anchor of a node
	$\mathsf{Hook}: \mathcal{F} \to \mathcal{P}$	function specifying the hook of a fact type
	$\mathcal{R} \subseteq \mathcal{N}$	set of nodes being the root of some tree
	$\mathcal{S}$	set of candidate representations (internal solution space)
	$\mathcal{S}_s$	set of candidate singleton internal representations
	$\mathcal{S}_r$	set of candidate relational representations

chapter	symbol	explanation
3	$\phi(n)$	operator for writing a tree as a nested record type
	$\overline{m}$	primary key of record type
	$[n_1, ..., n_k]$op	optional attribute group within record type
	$[n_1, ..., n_k]$rep	repeating attribute group within record type
	GenerateForest($\mathcal{I}$)	operator generating an internal representation
	CanExtend(m, n)	operator checking extension of an incomplete representation
	ProcessFactType(m, n)	operator extending a representation with a fact type
	ProcessObjectifications(m, n)	operator extending a representation with objectifications
	CanExtendReduced(m, n)	operator for extension under guidance conditions
	NoNesting	guidance condition for representations without nesting
	NoRedundancy	condition for representations without redundancy
	NoOptionals	condition for representations without optional values
	SizePreference	condition for representations with a specific size
	DepthPreference	condition for representations with a specific depth
4	$\mathsf{Pop}_T : \mathcal{N} \to \wp^{\text{fin}}(\Omega)$	function specifying the population of a node
	Pop	(subscript is omitted when no confusion is likely to occur)
	Insert$_{\mathcal{I}}$	operator inserting tuples into object type populations
	Insert$_T$	operator inserting tuples into node populations
	Insert	(subscript is omitted when no confusion is likely to occur)
	Delete	operator for deleting a tuple
	Modify	operator for modifying a tuple
	TransformInsertion	operator for conceptual-to-internal mapping of insertions
	TransformDeletion	operator for conceptual-to-internal mapping of deletions
	TransformModification	operator for conceptual-to-internal mapping of modifications
	InsertAtomicTuples	operator for inserting atomic tuples to a node poulation
	DeleteAtomicTuples	operator for deleting atomic tuples from a node population
	ConnectTuples	operator for connecting tuples in a node population
	DisconnectTuples	operator for disconnecting tuples in a node population
5	Join	mutation operator for joining relation types
	Split	mutation operator for splitting relation types
	Prune	mutation operator for pruning a subtree from a tree
	Graft	mutation operator for grafting a cutting to another tree
	Promote	mutation operator promoting an anchor to become a hook
	DescNodes(p) $\subseteq \mathcal{N}$	set of nodes being the 'descendant' of a predicator
	DescEdges(p) $\subseteq \mathcal{E}$	set of edges being source of 'descendant nodes'
	M $= \langle$S$, \mu_1, ..., \mu_n\rangle$	mutation system over given solution space (general not.)
	$\mathcal{M} = \langle \mathcal{S}$, Prune, Graft, Promote$\rangle$	mutation system for internal solution space
	$\mathcal{M}_r = \langle \mathcal{S}_r$, Join, Split$\rangle$	mutation system for relational representations
	Dist_a	anchor distance between internal representations
	Dist_m	mutation distance between internal representations
	Crossover	operator for combining internal representations

chapter	symbol	explanation
6	$\mathsf{Fitness} : \mathcal{S} \to \Re$	function specifying the fitness of an internal representation
	$\mho_f$	fitness parameters
	$\mathsf{Height} : \mathcal{S} \to \mathbb{N}$	function specifying the height of an internal representation
	$\mathsf{Breadth} : \mathcal{S} \to \mathbb{N}$	function specifying the breadth of an internal representation
	$\mathsf{NrTrees} : \mathcal{S} \to \mathbb{N}$	function specifying the nr of trees in an internal representation
	K	threshold in optimization
	$\mathsf{Anomaly} : \mathcal{S} \to \mathsf{Bool}$	function indicating whether a given type of anomaly occurs
	$\mathsf{NrDep} : \mathcal{S} \to \mathbb{N}$	function specifying the nr of dependencies in a representation
	$\mathsf{Time} : \mathcal{S} \to \Re$	function specifying the expected response time of a representation
	$\mathsf{Space} : \mathcal{S} \to \Re$	function specifying the expected storage space for a representation
	β	weight coefficient for time optimization
	γ_t	average cost of response time
	γ_s	average cost of storage space
	$T_2 < T_1$	representation T_2 is better than representation T_1
	$T_2 \leq T_1$	representation T_2 is at least as good as representation T_1
	$T_2 <> T_1$	representation T_2 is an alternative for representation T_1
	$T_2 \equiv T_1$	representation T_2 has the same properties as representation T_1
	$T_2 \prec T_1$	representation T_2 dominates representation T_1
	$\mathbf{PO} \subseteq \mathcal{S}$	set of Pareto optimal representations
	$\langle \mathsf{Paths}, \mathsf{Freq} \rangle$	path profile (access profile)
	Paths	set of paths through information structure
	$\mathsf{Freq} : \mathsf{Paths} \to \mathbb{N}$	function specifying the frequency of a path
	R	matrix specifying an affinity profile (access profile)
	$\langle \mathsf{NrInst}, \mathsf{LabelSize} \rangle$	size profile (data profile)
	$\mathsf{NrInst} : \mathcal{O} \to \mathbb{N}$	function specifying the expected number of object instances
	$\mathsf{LabelSize} : \mathcal{L} \to \mathbb{N}$	function specifying the expected size of the labels of a given type
	Pool_t	multiset of internal representations at a given point in time
	μ	number of representations in parent pool
	λ	number of representations in offspring pool
	$\mathsf{Terminate}$	operator checking whether evolution process should be ended
	Mutate	operator transforming a pool into a new pool using mutation operators
	$\mathsf{Recombine}$	operator transforming a pool into a new pool using crossover operators
	Select	operator for selecting the new parent pool from the old offspring pool
	$\mho_t$	termination parameters
	$\mho_m$	mutation parameters
	$\mho_r$	recombination parameters
	$\mho_s$	selection parameters
	$Q_t = \{\varnothing, \mathsf{Pool}_t\}$	additional parameter for selection operator

chapter	symbol	explanation
8	P	one step transition matrix for Markov chain
	P^k	transition matrix for given number of steps
	P_{red}	reduced transition matrix
	$\mathsf{Diam}(\mathcal{S})$	diameter of solution space
	$\mathsf{Neighbours}(i)$	set of neighbours for given representation
	$\mathsf{NNb}(i)$	number of neighbour representations
	B_i	set of better neighbours
	MB_i	set of maximal better neighbours
	Dist_f	fitness distance between representations
	$\mathsf{Plat} \subseteq \mathcal{S}$	plateau of representations
	δ_{ij}	Kronecker delta function
	λ_S, λ_N	configuration parameters for S/M improvement
	T_{min}, T_{max}	configuration parameters for fitness proportional walk
	T_{min}, λ_P	configuration parameters for fitness proportional hill climbing
	I_i	predicate indicating initial representation
	S_k	predicate indicating success after given number of steps
	FS_k	predicate indicating first success after given number of steps
	H_k	entropy for success within given number of steps
	ϑ	ratio for exploration versus exploitation

Appendix B

Typical fitness development graphs

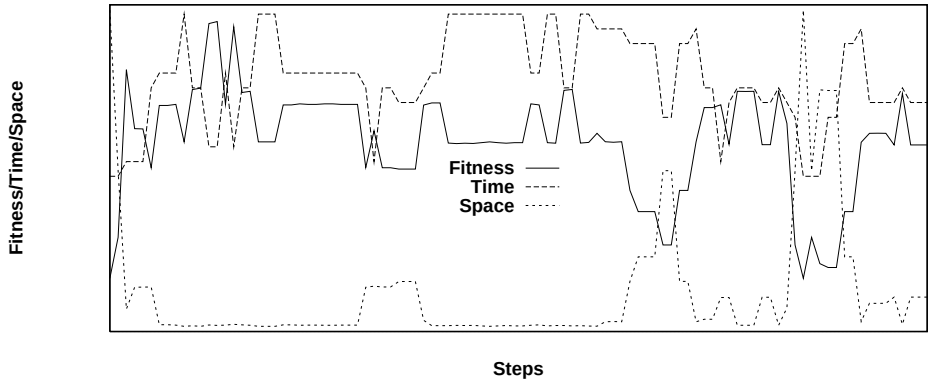


Figure B.1: Fitness, time and space development for random walk

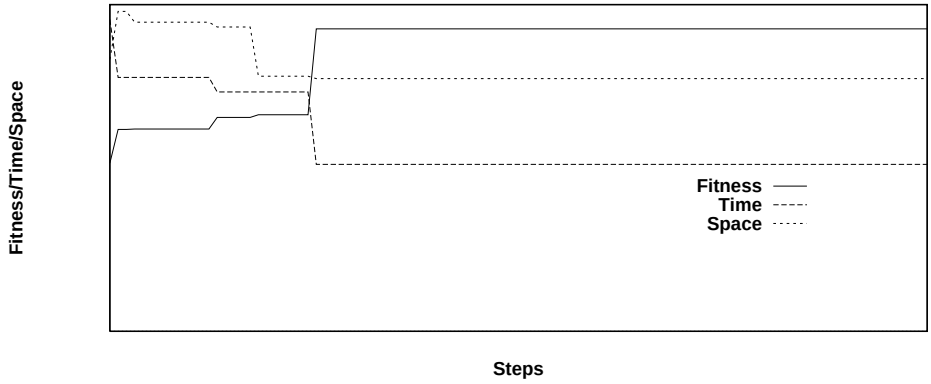


Figure B.2: Fitness, time and space development for hill climbing

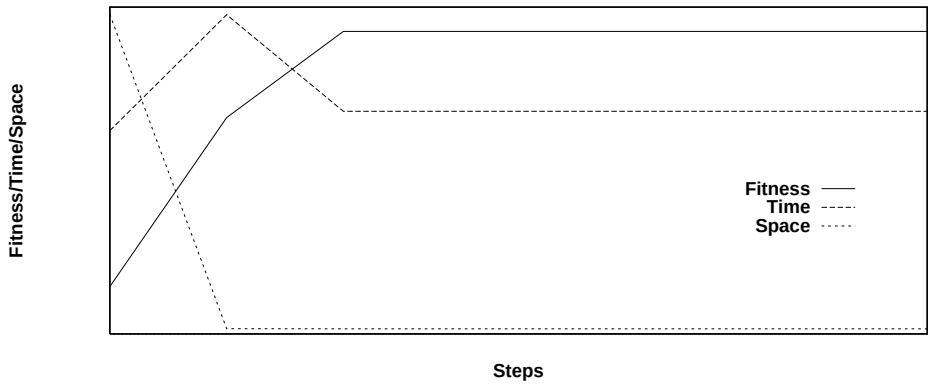


Figure B.3: Fitness, time and space development for steepest ascent

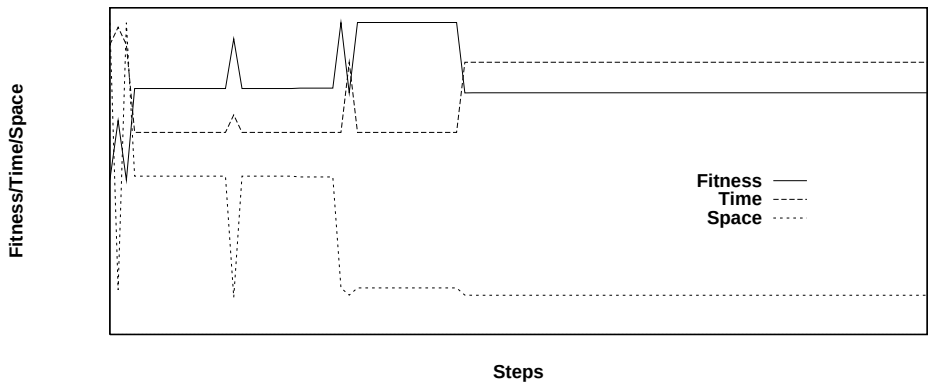


Figure B.4: Fitness, time and space development for single improvement

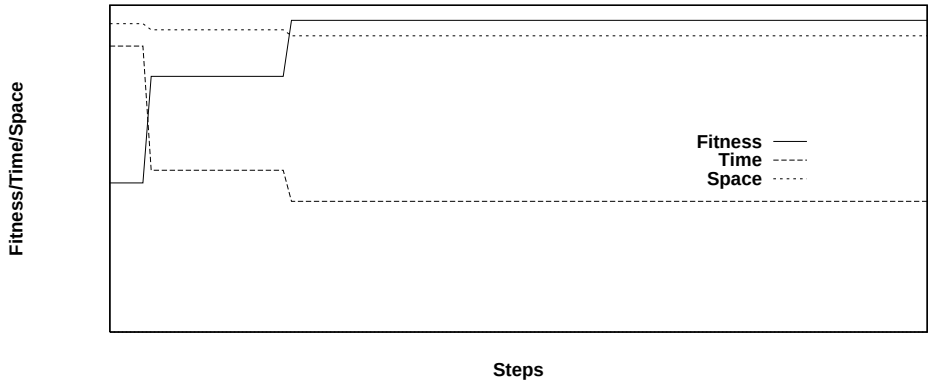


Figure B.5: Fitness, time and space development for multiple improvement

Appendix C

Further experimental results

C.1 Response time and storage space

C.1.1 Random database transformations

Figure C.1 displays the fitness (quality) of database structures found by random transformations. Three possibilities can be recognized here: increasing, equal, and descreasing fitness. Equal fitness means that the transformation process fails to produce a correct (new) structure.

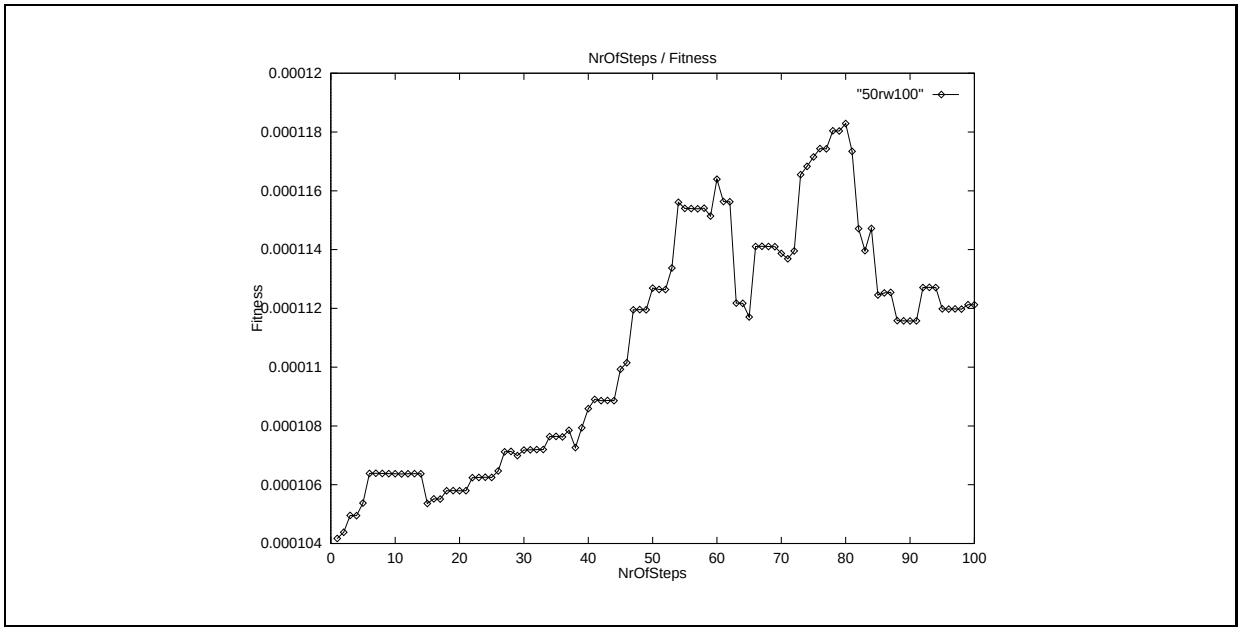


Figure C.1: A Random Walk of 100 steps

The fitness values in figure C.1 are computed by a weighted fitness function:

$$\mathsf{Fitness}(T) = \frac{1}{k_1 \times \mathsf{Time}(T) + k_2 \times \mathsf{Space}(T)}$$

The corresponding values for response time and storage space are given in figure C.2.

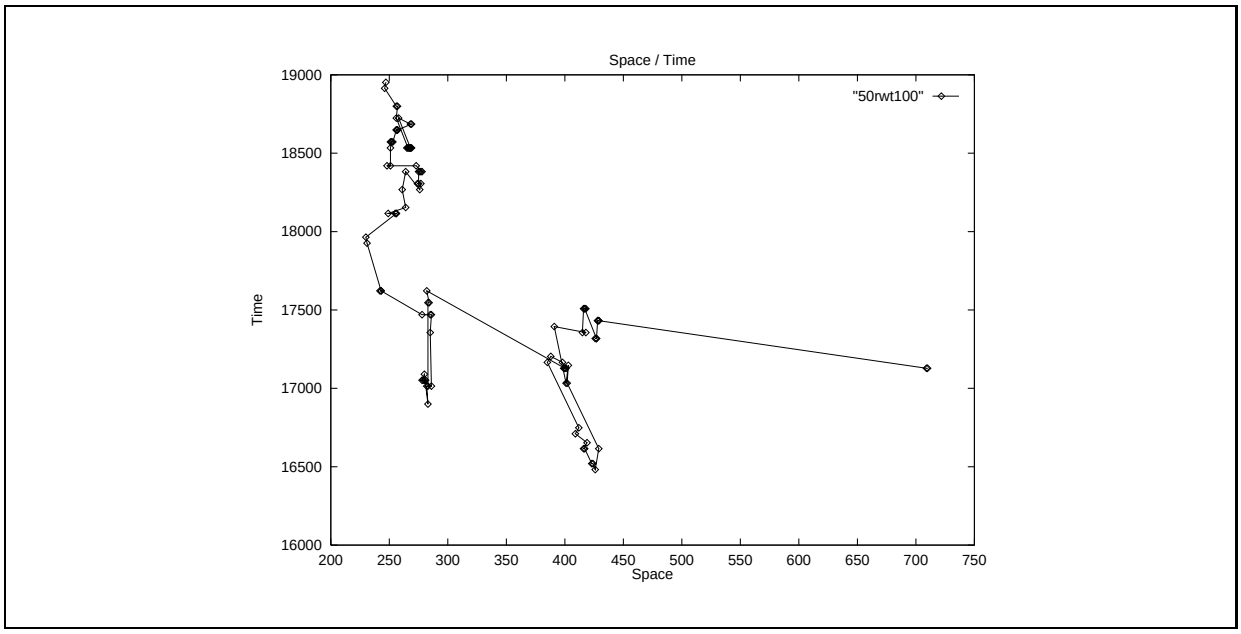


Figure C.2: The corresponding time/space graph

From figure C.2 it is clear that random transformations may result in decreasing both time and space, increasing both time and space, decreasing time in combination with increasing space, increasing time in combination with decreasing space, or equal time and space.

C.1.2 Hill Climbing database transformations

Figure C.3 displays the fitness of database structures found by hill climbing transformations. Two possibilities can be recognized here: increasing and equal fitness. Equal fitness means that the transformation process fails to produce an improved (correct) structure.

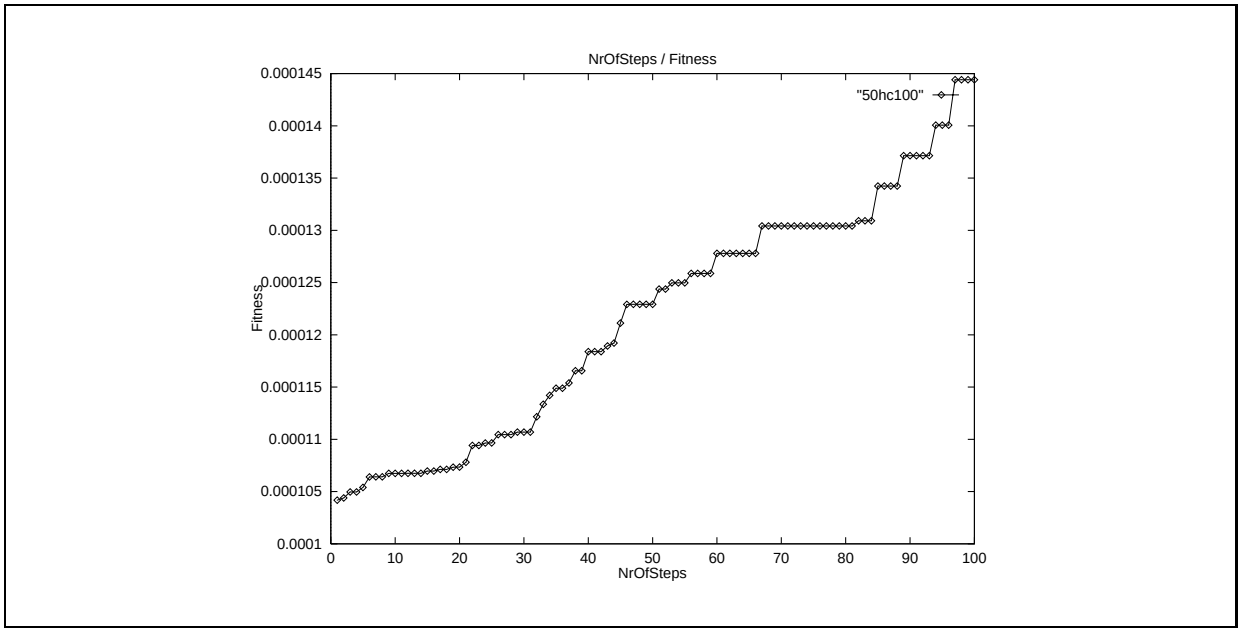


Figure C.3: 100 steps Hill Climber

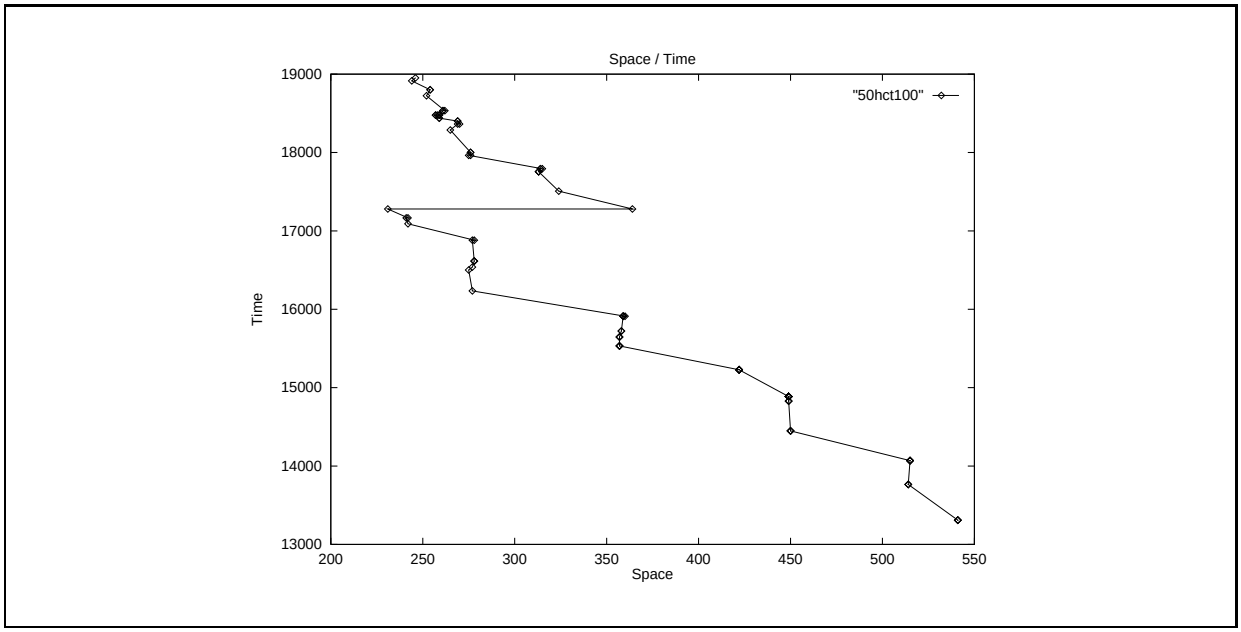


Figure C.4: The corresponding time/space graph

C.1.3 Improving time or space

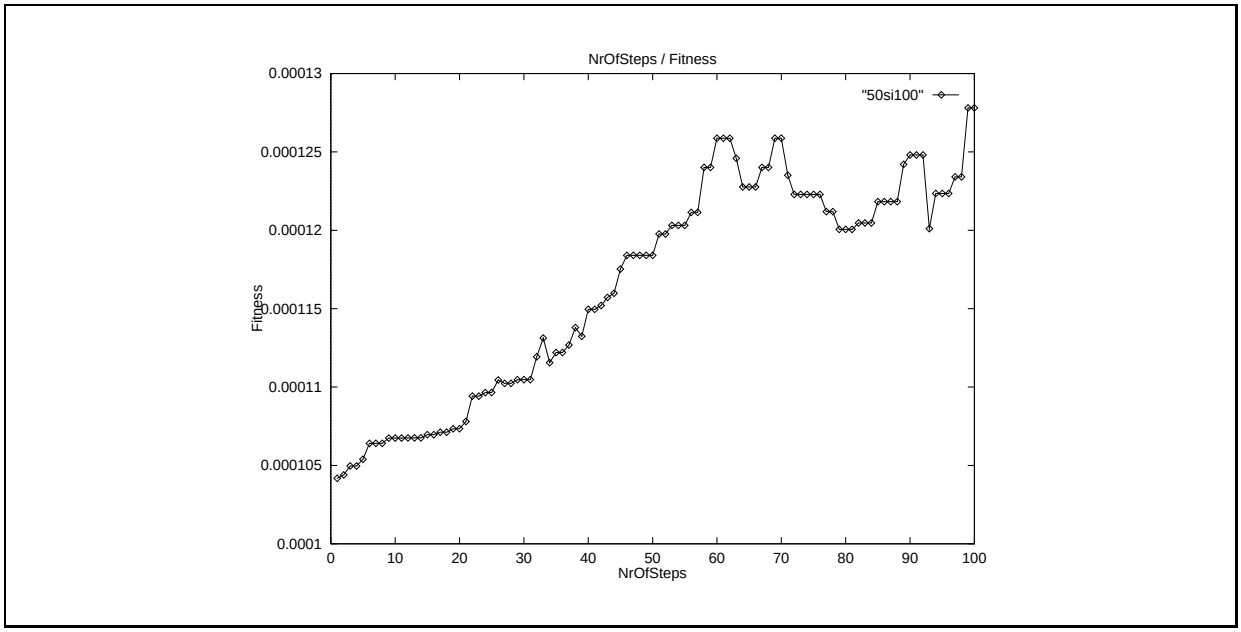


Figure C.5: 100 steps Singleton Improvement

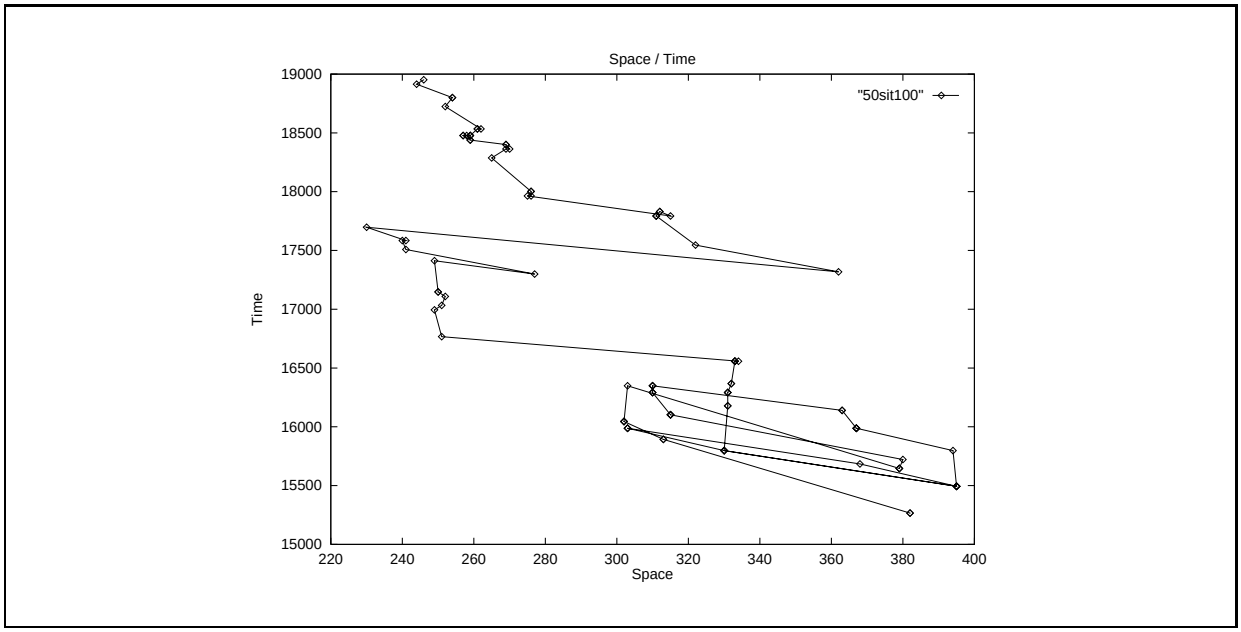


Figure C.6: The corresponding time/space graph

C.1.4 Improving time and space

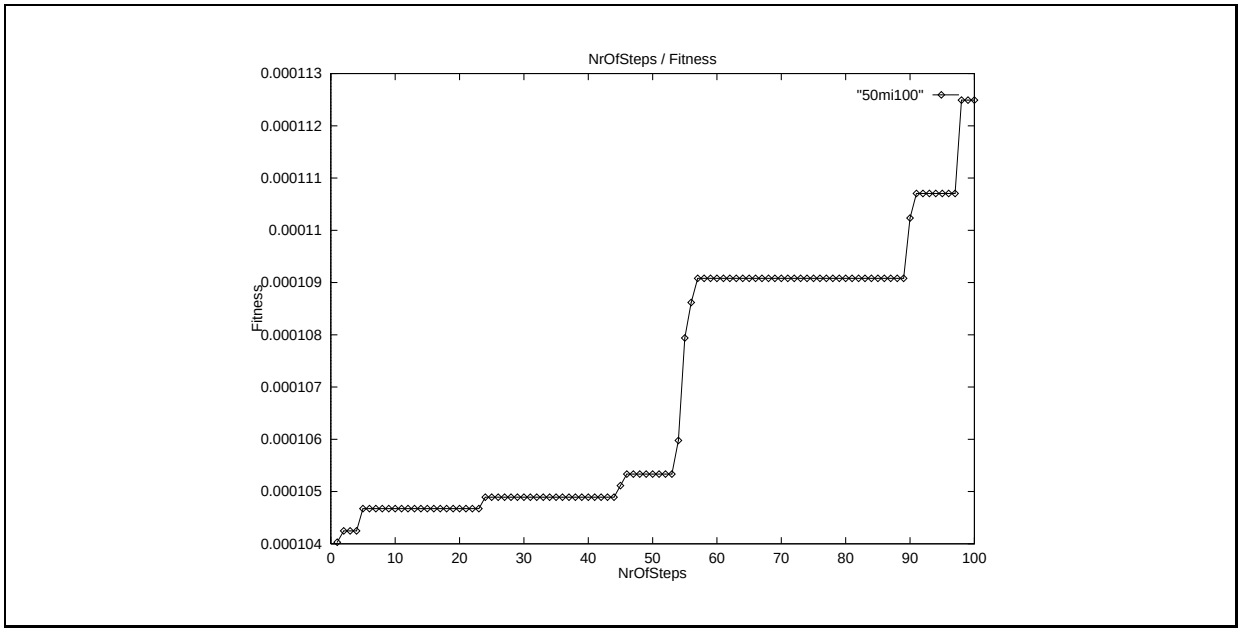


Figure C.7: 100 steps Multiple Improvement

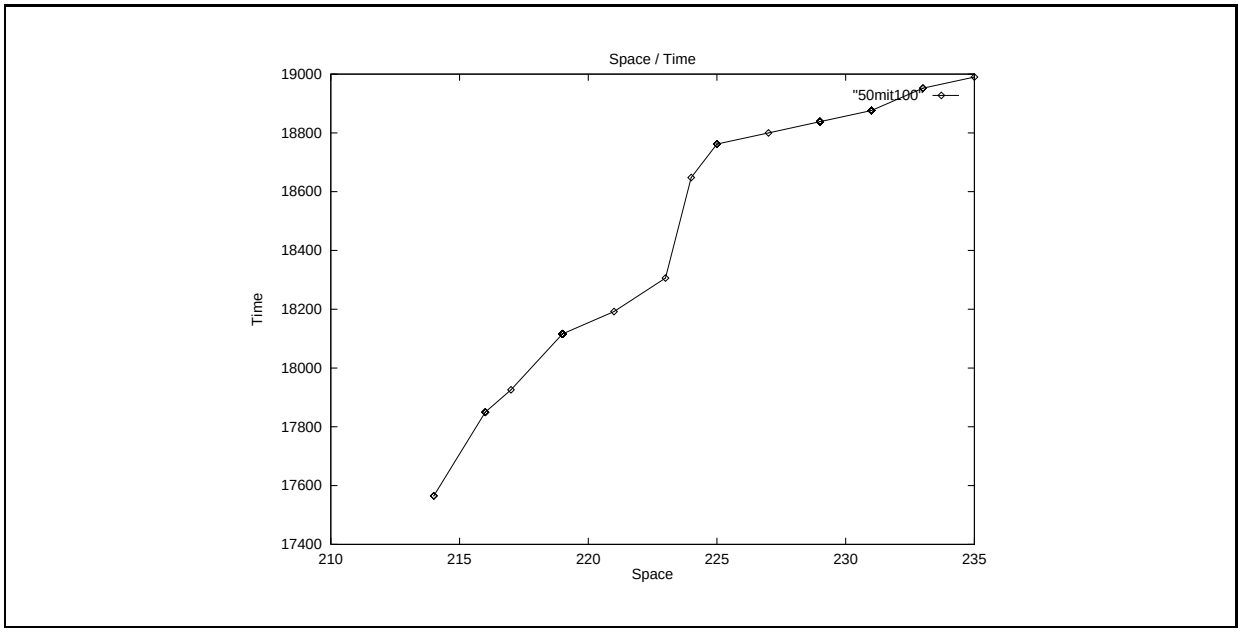


Figure C.8: The corresponding time/space graph

C.2 Number of tables

In figure C.9 we see a transformation process where the number of tables is reduced. This means that tables are *joined* rather than split.

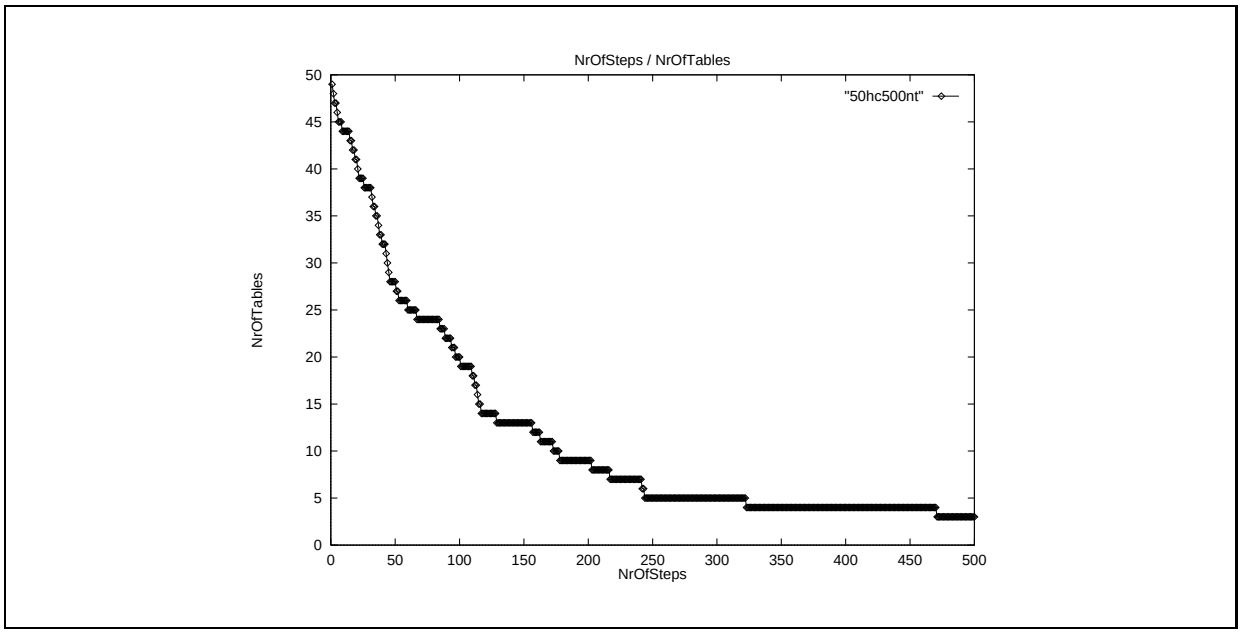


Figure C.9: Number of tables

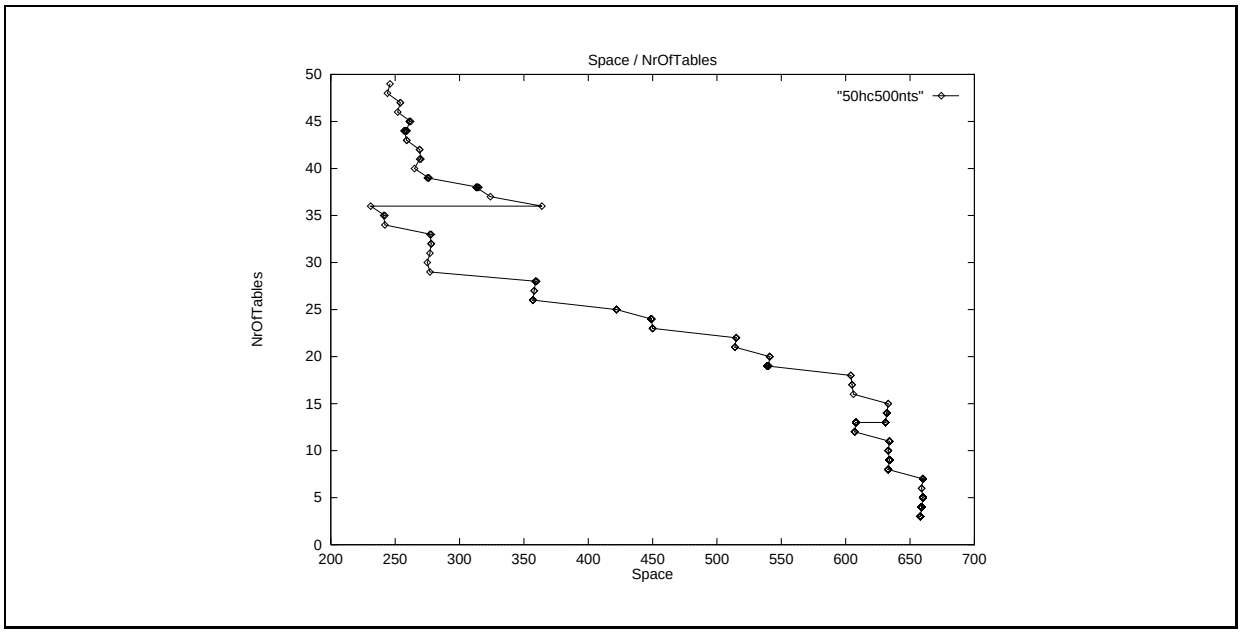


Figure C.10: Storage space versus number of tables

C.3 Example scenario

User interface: Examine Main Menu and Status Report.

Input: View SQL code for simple schema (Lister: SQL, Files: Output).

Random transformations:

- Random walk with parameters Steps = 100, Schema = 50.

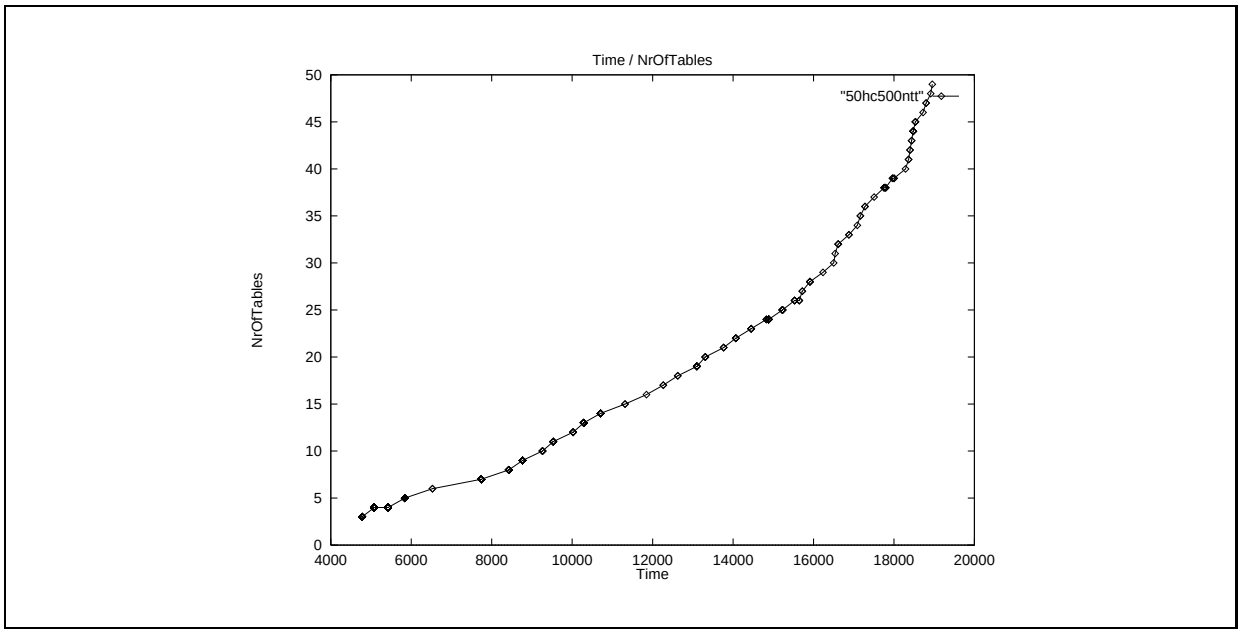


Figure C.11: Response time versus number of tables

- Examine Evolution Screen (Fitness Development Graph, Time/Space Graph).
- Random transformations find maximum fitness of approx 1.2.
- View SQL code for result.

Hill climbing: Hill climber 100 steps: hill climbing finds fitness of 1.4.

Output: View SQL code for result.

Other interesting parameters are:

- schema 20, 100 steps: hill climber is better than random walk.
- schema 10, 100 steps: random walk is slightly better than hill climbing.
- schema 1, 100 steps: random walk is better than hill climbing.
- schema 50, 500 steps: random walk yields approx 1.3, while for hill climbing the sky is the limit.

C.4 Scenario for time/space trade-off

C.4.1 Singleton improvement

Input: View SQL code for result.

Examine trade off:

- *Single improvement* with parameters Steps = 100, Schema = 50.
- Explain Evolution Screen.

Output: View SQL code for result.

Other interesting parameters are:

- schema 20, 100 steps.
- schema 10, 100 steps.
- schema 1, 100 steps.
- schema 50, 500 steps.

C.4.2 Multiple improvement

Try the Multiple Improvement strategy:

- schema 50, 100 steps.
- if you have time: schema 50, 500 steps.
- if you have more time: schema 50, 500 steps.

C.5 Some input data models

The schema in figure C.14 has at least $2^{50} \approx 10^{15}$ correct internal representations.

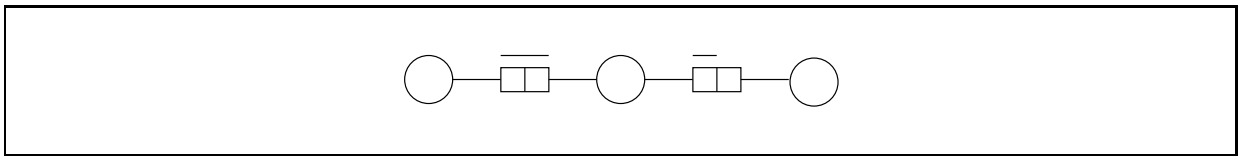


Figure C.12: A simple schema (Schema 1)

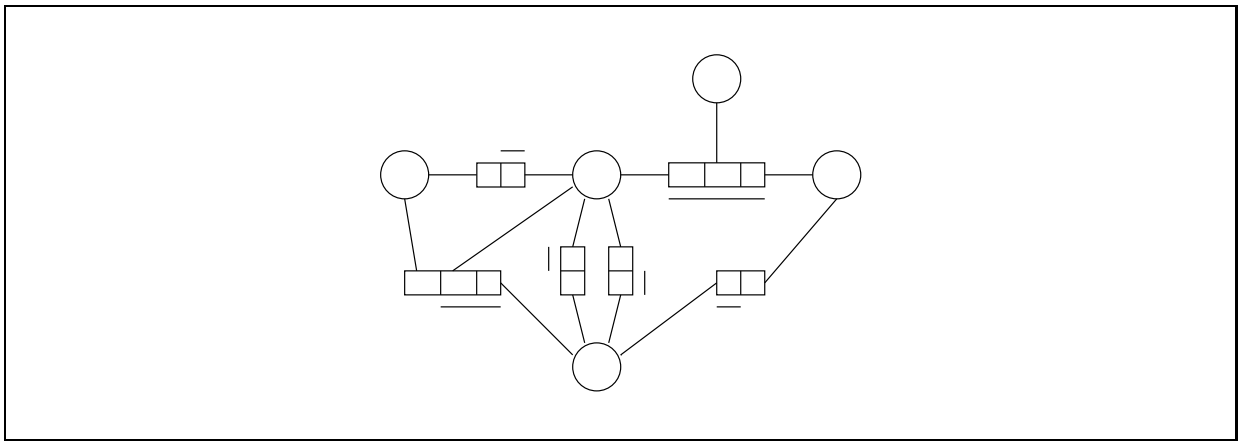


Figure C.13: A more complex but still simple schema (Schema 10)

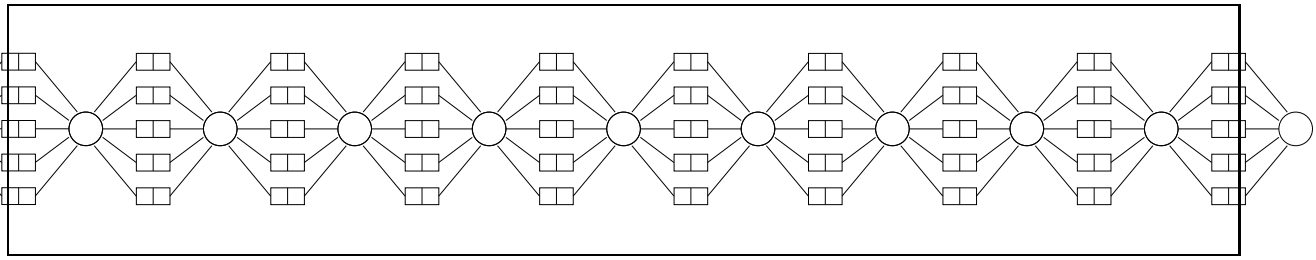


Figure C.14: Larger but still not very large schema (Schema 50)

Bibliography

[1] S. Abiteboul, P.C. Fischer, and H.J. Schek. *Nested Relations and Complex Objects in Databases*, volume 361 of *Lecture Notes in Computer Science*. Springer-Verlag, Berlin, Germany, 1987.

[2] S. Abiteboul and R. Hull. IFO: A Formal Semantic Database Model. *ACM Transactions on Database Systems*, 12(4):525–565, December 1987.

[3] S. Abiteboul and P.C. Kanellakis. Object Identity as a Query Language Primitive. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 159–173, 1989.

[4] A. Amikam. On the automatic generation of optimal internal schemata. *Information Systems*, 10(1):37–45, 1985.

[5] M. Atkinson, D. DeWitt, D. Maier, F. Bancilhon, K. Dittrich, and S.B. Zdonik. The object-oriented database system manifesto. In W. Kim, J.-M. Nicolas, and S. Nishio, editors, *Proceedings of the First International Conference on Deductive and Object-Oriented Databases (DOOD-89)*, pages 40–57, Kyoto, Japan, 1989. Elsevier Science Publishers.

[6] A.D. Atri and D. Sacca. Equivalence and mapping of database schemes. In U. Dayal, G. Schlageter, and L.H. Seng, editors, *Proceedings of the Tenth International Conference on Very Large Data Bases*, pages 187–195, Singapore, August 1984.

[7] Th. Bäck and H.-P. Schwefel. An Overview of Evolutionary Algorithms for Parameter Optimization. *Evolutionary Computation*, 1(1):1–23, 1993.

[8] G.P. Bakema, J.P.C. Zwart, and H. van der Lek. Fully Communication Oriented NIAM. In *Proceedings of NIAM-ISDM 2*, pages 1–35, Albuquerque, New Mexico, August 1994.

[9] C. Beeri. A formal approach to object-oriented databases. *Data & Knowledge Engineering*, 5:353–382, 1990.

[10] A. Benczúr, Cs. Hajas, and Gy. Kovács. Datalog Extension for Nested Relations. *Computers Math. Applic.*, 30(12):51–79, 1995.

[11] K. Bennett, M.C. Ferris, and Y.E. Ioannidis. A Genetic Algorithm for Database Query Optimization. In R.K. Belew and L.B. Booker, editors, *Proceedings of the Fourth International Conference on Genetic Algorithms*, pages 400–407, San Mateo, CA, 1991. Morgan Kaufmann.

[12] J. Biskup, R. Menzel, and T. Polle. Transforming an Entity-Relationship Schema into Object-Oriented Database Schemas. In *Proceedings of the International Workshop on Advances in Databases and Information Systems*, pages 67–78, Moscow, June 1995.

[13] P. van Bommel. Designing a Workbench Kernel. Master's thesis, University of Nijmegen, December 1989.

[14] P. van Bommel. A Randomised Schema Mutator for Evolutionary Database Optimisation. *The Australian Computer Journal*, 25(2):61–69, May 1993.

[15] P. van Bommel. Database Design Modifications based on Conceptual Modelling. In H. Jaakkola, H. Kangassalo, T. Kitahashi, and A. Márkus, editors, *Information Modelling and Knowledge Bases V: Principles and Formal Techniques*, pages 275–286, Amsterdam, The Netherlands, 1994. IOS Press.

[16] P. van Bommel. Experiences with EDO: an Evolutionary Database Optimizer. *Data & Knowledge Engineering*, 13:243–263, 1994.

[17] P. van Bommel. Implementation Selection for Object-Role Models. In T.A. Halpin and R. Meersman, editors, *Proceedings of the First International Conference on Object-Role Modelling (ORM-1)*, pages 103–112, Magnetic Island, Australia, July 1994.

[18] P. van Bommel. Database Design by Computer Aided Schema Transformations. *Software Engineering Journal*, 10(4):125–132, 1995.

[19] P. van Bommel. *Database Optimization: An Evolutionary Approach.* PhD thesis, University of Nijmegen, Nijmegen, The Netherlands, 1995.

[20] P. van Bommel, A.H.M. ter Hofstede, and Th.P. van der Weide. Semantics and verification of object-role models. *Information Systems*, 16(5):471–495, October 1991.

[21] P. van Bommel, Gy. Kovács, and A. Micsik. Transformation of database populations and operations from the conceptual to the internal level. *Information Systems*, 19(2):175–191, 1994.

[22] P. van Bommel, C.B. Lucasius, and Th.P. van der Weide. Genetic Algorithms for Optimal Logical Database Design. *Information and Software Technology*, 36(12):725–732, 1994.

[23] P. van Bommel, C.B. Lucasius, and Th.P. van der Weide. Pool Heuristics in Evolutionary Database Optimization. In N. Prakash, editor, *Proceedings of the International Conference on Information Systems and Management of Data (CISMOD 94)*, pages 76–90, Madras, India, October 1994.

[24] P. van Bommel and Th.P. van der Weide. Reducing the search space for conceptual schema transformation. *Data & Knowledge Engineering*, 8:269–292, 1992.

[25] P. van Bommel and Th.P. van der Weide. Towards Database Optimization by Evolution. In A.K. Majumdar and N. Prakash, editors, *Proceedings of the International Conference on Information Systems and Management of Data (CISMOD 92)*, pages 273–287, Bangalore, India, July 1992.

[26] P. van Bommel (editor). *Information modeling for internet applications.* Idea group publishing, USA, 2003.

[27] P. van Bommel (editor). *Transformation of knowledge, information and data: theory and applications.* Information science publishing, USA, 2005.

[28] S.A. Borkin. Data Model Equivalence. In *Proceedings of the Fourth International Conference on Very Large Data Bases*, pages 526–534, 1978.

[29] S.J. Brouwer, C.L.J. Martens, G.H.W.M. Bronts, and H.A. Proper. Towards a Unifying Object Role Modelling Approach. In T.A. Halpin and R. Meersman, editors, *Proceedings of the First International Conference on Object-Role Modelling (ORM-1)*, pages 259–273, Magnetic Island, Australia, July 1994.

[30] L. Campbell. Adding a New Dimension to Flat Conceptual Modelling. In T.A. Halpin and R. Meersman, editors, *Proceedings of the First International Conference on Object-Role Modelling (ORM-1)*, pages 294–309, Magnetic Island, Australia, July 1994.

[31] L. Campbell and T.A. Halpin. Abstraction Techniques for Conceptual Schemas. In R. Sacks-Davis, editor, *Proceedings of the 5th Australasian Database Conference*, volume 16, pages 374–388, Christchurch, New Zealand, January 1994. Global Publications Services.

[32] M.A. Casanova, L. Tucherman, A.L. Furtado, and A.P. Braga. Optimization of relational schemas containing inclusion dependencies. In *Proceedings of the Fifteenth VLDB Conference*, pages 317–325, 1989.

[33] M.A. Casanova, L. Tucherman, and A.H.F. Laender. On the design and maintenance of optimized relational representations of entity-relationship schemas. *Data & Knowledge Engineering*, 11:1–20, 1993.

[34] R.G.G. Catell. *The Object Database Standard: ODMG-93*. Morgan Kaufmann, San Francisco, California, 1994.

[35] S. Ceri and G. Pelagate. *Distributed Databases: Principles and Systems*. McGraw-Hill, New York, 1984.

[36] P.P. Chen. The Entity-Relationship Model: Toward a Unified View of Data. *ACM Transactions on Database Systems*, 1(1):9–36, March 1976.

[37] S. Christodoulakis. Implications of Certain Assumptions in Database Performance Evaluation. *ACM Transactions on Database Systems*, 9(2):163–186, June 1984.

[38] S. Christodoulakis and D.A. Ford. File Organizations and Access Methods for CLV Optical Discs. In N.J. Belkin and C.J. van Rijsbergen, editors, *Proceedings of the Twelfth ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 152–159, Massachusetts, USA, June 1989. ACM Press.

[39] E.F. Codd. A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, 13(6):377–387, 1970.

[40] E.F. Codd. Extending the database relational model to capture more meaning. *ACM Transactions on Database Systems*, 4(4):397–434, 1979.

[41] L.S. Colby. A recursive algebra for nested relations. *Information Systems*, 15(5):567–582, 1990.

[42] H. Darwen and C.J. Date. The Third Manifesto. *SIGMOD Record*, 24(1), March 1995.

[43] C.J. Date. *An Introduction to Data Base Systems*. Addison-Wesley, Reading, Massachusetts, 1986.

[44] C.J. Date and H. Darwen. *A Guide to the SQL Standard*. Addison-Wesley, Reading, Massachusetts, 1992.

[45] L. Davis, editor. *Handbook of genetic algorithms*. Van Nostrand Reinhold, New York, 1991.

[46] O.M.F. De Troyer. RIDL*: A Tool for the Computer-Assisted Engineering of Large Databases in the Presence of Integrity Constraints. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 418–429, Portland, Oregon, 1989.

[47] O.M.F. De Troyer. *On Data Schema Transformations*. PhD thesis, Tilburg University, Tilburg, The Netherlands, 1993.

[48] O.M.F. De Troyer. A Logical Formalization of the Binary Object-Role Model. In T.A. Halpin and R. Meersman, editors, *Proceedings of the First International Conference on Object-Role Modelling (ORM-1)*, pages 28–44, Magnetic Island, Australia, July 1994.

[49] O.M.F. De Troyer, R. Meersman, and F. Ponsaert. RIDL User Guide. Research report, International Centre for Information Analysis Services, Control Data Belgium, Inc., Brussels, Belgium, 1984.

[50] O.M.F. De Troyer, R. Meersman, and P. Verlinden. RIDL* on the CRIS Case: A Workbench for NIAM. In T.W. Olle, A.A. Verrijn-Stuart, and L. Bhabuta, editors, *Computerized Assistance during the Information Systems Life Cycle*, pages 375–459, Amsterdam, The Netherlands, 1988. North-Holland/IFIP.

[51] D. De Witt. Implementation Techniques for Main Memory Databases. In B. Yormark, editor, *Proceedings of the ACM SIGMOD International Conference on the Management of Data*, Boston, Massachusetts, June 1984.

[52] R. Elmasri and S.B. Navathe. Advanced data models and emerging trends. In *Fundamentals of Database Systems*, chapter 21. Benjamin Cummings, Redwood City, California, 1994. Second Edition.

[53] G. Engels, M. Gogolla, U. Hohenstein, K. Hülsmann, P. Löhr-Richter, G. Saake, and H-D. Ehrich. Conceptual modelling of database applications using an extended ER model. *Data & Knowledge Engineering*, 9(4):157–204, 1992.

[54] E.D. Falkenberg. Data Bases and Information Systems I. Lecture Notes, University of Nijmegen, 1987.

[55] E.D. Falkenberg. An Approach to Deterministic Event-Tuned Information Analysis. In G.M. Nijssen, editor, *Proceedings of NIAM-ISDM*. NIAM-GUIDE, September 1993.

[56] E.D. Falkenberg. DETERM: Deterministic Event-Tuned Entity-Relationship Modeling. In R. Elmasri and V. Kouramajian, editors, *Proceedings of the 12th International Conference on the Entity-Relationship Approach*, Arlington, Texas, USA, December 1993.

[57] D.B. Fogel. *System identification through simulated evolution: A machine learning approach to modeling*. Ginn, Needham Heights, MA, 1991.

[58] L.J. Fogel, A.J. Owens, and M.J. Walsh. *Artificial intelligence through simulated evolution*. John Wiley, New York, 1966.

[59] O. Frieder and H.T. Siegelmann. On the Allocation of Documents in Multiprocessor Information Retrieval Systems. In *Proceedings of the Fourteenth International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 230–239, October 1991.

[60] L. Gallagher. Influencing Database Language Standards. *SIGMOD Record*, 23(1), March 1994.

[61] R.L. Glass, editor. *Real-Time Software*. Prentice-Hall, Englewood Cliffs, New Jersey, 1983.

[62] D.E. Goldberg. *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley, Reading, Massachusetts, 1989.

[63] D.E. Goldberg and P. Segrest. Finite Markov chain analysis of genetic algorithms. In *Proceedings of the Second International Conference on Genetic Algorithms*, pages 1–8, 1987.

[64] J. Gray, editor. *The Benchmark Handbook for Database and Transaction Processing Systems*. Morgan Kaufmann, California, 1991.

[65] R.M. Gray. *Entropy and Information Theory*. Springer-Verlag, New York, New York, 1990.

[66] J.J. van Griethuysen, editor. *Concepts and Terminology for the Conceptual Schema and the Information Base*. Publ. nr. ISO/TC97/SC5-N695, 1982.

[67] G.R. Grimmett and D.R. Stirzaker. *Probability and Random Processes*. Oxford University Press, Oxford, UK, 1992.

[68] J.-L. Hainaut, M. Cadelli, B. Decuyper, and O. Marchand. Database CASE Tool Architecture: Principles for Flexible Design Strategies. In P. Loucopoulos, editor, *Proceedings of the Fourth International Conference CAiSE'92 on Advanced Information Systems Engineering*, volume 593 of *Lecture Notes in Computer Science*, pages 187–207, Manchester, United Kingdom, 1992. Springer-Verlag.

[69] T.A. Halpin. *A logical analysis of information systems: static aspects of the data-oriented perspective*. PhD thesis, University of Queensland, Brisbane, Australia, 1989.

[70] T.A. Halpin. Conceptual schema optimization. *Australian Computer Science Communications*, 12(1):136–145, 1990.

[71] T.A. Halpin. A Fact-Oriented Approach to Schema Transformation. In B. Thalheim, J. Demetrovics, and H.-D. Gerhardt, editors, *MFDBS 91*, volume 495 of *Lecture Notes in Computer Science*, pages 342–356, Rostock, Germany, 1991. Springer-Verlag.

[72] T.A. Halpin. WISE: a Workbench for Information System Engineering. In V.-P. Tahvanainen and K. Lyytinen, editors, *Next Generation CASE Tools*, volume 3 of *Studies in Computer and Communication Systems*, pages 38–49. IOS Press, 1992.

[73] T.A. Halpin and M.E. Orlowska. Fact-oriented modelling for data analysis. *Journal of Information Systems*, 2(2):97–119, April 1992.

[74] M. Hammer and D. McLeod. Database Description with SDM: A Semantic Database Model. *ACM Transactions on Database Systems*, 6(3):351–386, September 1981.

[75] R.W. Hamming. *Coding and information theory.* Prentice-Hall, Englewood Cliffs, New Jersey, 1980.

[76] K.M. van Hee and P.A.C. Verkoulen. Integration of a data model and high-level petri nets. In *Proceedings of the Twelfth International Conference on Applications and Theory of Petri Nets*, pages 410–431, Gjern, Denmark, 1991.

[77] C.A. Heuser, E.M. Peres, and G. Richter. Towards a complete conceptual model: Petri nets and entity-relationship diagrams. *Information Systems*, 18(5):275–298, 1993.

[78] A.H.M. ter Hofstede. *Information Modelling in Data Intensive Domains.* PhD thesis, University of Nijmegen, Nijmegen, The Netherlands, 1993.

[79] A.H.M. ter Hofstede, H.A. Proper, and Th.P. van der Weide. A Note on Schema Equivalence. Technical Report 92-30, Department of Information Systems, University of Nijmegen, Nijmegen, The Netherlands, 1992.

[80] A.H.M. ter Hofstede, H.A. Proper, and Th.P. van der Weide. Data Modelling in Complex Application Domains. In P. Loucopoulos, editor, *Proceedings of the Fourth International Conference CAiSE'92 on Advanced Information Systems Engineering*, volume 593 of *Lecture Notes in Computer Science*, pages 364–377, Manchester, United Kingdom, May 1992. Springer-Verlag.

[81] A.H.M. ter Hofstede, H.A. Proper, and Th.P. van der Weide. Formal definition of a conceptual language for the description and manipulation of information models. *Information Systems*, 18(7):489–523, 1993.

[82] A.H.M. ter Hofstede and Th.P. van der Weide. Expressiveness in conceptual data modelling. *Data & Knowledge Engineering*, 10(1):65–100, February 1993.

[83] A.H.M. ter Hofstede and Th.P. van der Weide. Fact Orientation in Complex Object Role Modelling Techniques. In T.A. Halpin and R. Meersman, editors, *Proceedings of the First International Conference on Object-Role Modelling (ORM-1)*, pages 45–59, Townsville, Australia, July 1994.

[84] J.H. Holland. *Adaptation in Natural and Artificial Systems.* University of Michigan Press, Ann Arbor, 1975.

[85] J. Horn. Finite Markov Chain Analysis of Genetic Algorithms with Niching. In S. Forrest, editor, *Proceedings of the Fifth International Conference on Genetic Algorithms*, pages 110–117, San Mateo, California, 1993. Morgan Kaufmann Publishers.

[86] Y.E. Ioannidis and Y. Kang. Randomized Algorithms for Optimizing Large Join Queries. In *Proceedings of the 1990 ACM SIGMOD Conference on the Management of Data*, pages 312–321, May 1990.

[87] Y.E. Ioannidis and E. Wong. Query Optimization by Simulated Annealing. In *Proceedings of the 1987 ACM SIGMOD Conference on the Management of Data*, pages 9–22, May 1987.

[88] S. Jajodia, P.E. Ng, and F.N. Springsteel. The problem of equivalence for entity-relationship diagrams. *IEEE Transactions on Software Engineering*, 9(5):617–630, 1983.

[89] M. Kifer, G. Lausen, and J. Wu. Logical Foundations of Object-Oriented and Frame-Based Languages. *Journal of the ACM*, 42(4):741–843, 1995.

[90] W. Kim. Object-Oriented Database Systems: Promises, Reality, and Future. In *Proceedings of the 19th VLDB Conference*, pages 676–687, Dublin, Ireland, 1993.

[91] S.T. Klein, A. Bookstein, and S. Deerwester. Storing Text Retrieval Systems on CD-ROM: Compression and Encryption Considerations. In N.J. Belkin and C.J. van Rijsbergen, editors, *Proceedings of the Twelfth ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 160–167, Massachusetts, USA, June 1989. ACM Press.

[92] I. Kobayashi. Classification and transformations of binary relationship relation schemata. *Information Systems*, 11(2):109–122, 1986.

[93] I. Kobayashi. Losslessness and semantic correctness of database schema transformation: another look of schema equivalence. *Information Systems*, 11(1):41–59, 1986.

[94] Y. Kornatzky and P. Shoval. Conceptual design of object-oriented database schemas using the binary-relationship model. *Data & Knowledge Engineering*, 14:265–288, 1994.

[95] Gy. Kovács and P. van Bommel. Overview of F-logic from Database Transformation Perspective. Technical Note CSI-N9607, Computing Science Institute, University of Nijmegen, Nijmegen, The Netherlands, 1996.

[96] Gy. Kovács and P. van Bommel. Designing Advanced Databases Using Dependency Graph Conducted Schema Transformation Algorithm. Technical Note CSI-N9703, Computing Science Institute, University of Nijmegen, Nijmegen, The Netherlands, 1997.

[97] Gy. Kovács and P. van Bommel. From Conceptual Model to OO Database via Intermediate Specification. *Acta Cybernetica*, 13, 1997.

[98] Gy. Kovács and P. van Bommel. Conceptual Modelling Based Design of Object-Oriented Databases. *Information and Software Technology*, 40(1), 1998.

[99] P.J.M. van Laarhoven and E.H.L. Aarts. *Simulated Annealing: Theory and Applications*. Kluwer, Dordrecht, The Netherlands, 1988.

[100] H. van der Lek. Note on the structure of joins. *Information Systems*, 17(4):343–346, 1992.

[101] C.M.R. Leung and G.M. Nijssen. From a NIAM Conceptual Schema into the Optimal SQL Relational Database Schema. *The Australian Computer Journal*, 19(2):69–75, May 1987.

[102] C.M.R. Leung and G.M. Nijssen. Relational database design using the NIAM conceptual schema. *Information Systems*, 13(2):219–227, 1988.

[103] X. Lin and Y. Zhang. A New Graphical Method for Vertical Partitioning in Database Design. In M.E. Orlowska and M. Papazoglou, editors, *Proceedings of the Fourth Australian Database Conference*, Advances in Database Research, pages 131–144. World Scientific, Brisbane, Australia, February 1993.

[104] C.B. Lucasius. *Towards Genetic Algorithm Methodology in Chemometrics*. PhD thesis, University of Nijmegen, Nijmegen, The Netherlands, 1993.

[105] C.B. Lucasius and G. Kateman. Understanding and Using Genetic Algorithms, Part 1. Concepts, properties and context. *Chemometrics and Intelligent Laboratory Systems*, 19:1–33, 1993.

[106] P. Lyngbaek and V. Vianu. Mapping a semantic database model to the relational model. In *SIGMOD Record*, pages 132–142, 1987.

[107] S.W. Mahfoud. Finite Markov Chain Models of an Alternative Selection Strategy for the Genetic Algorithm. *Complex Systems*, 7(2):155–170, April 1993.

[108] D. Maier. *The Theory of Relational Databases*. Computer Science Press, Rockville, Maryland, 1988.

[109] M.V. Mannino, P. Chu, and T. Sager. Statistical Profile Estimation in Database Systems. *ACM Computing Surveys*, 20(3):191–221, 1988.

[110] S.T. March. A mathematical programming approach to the selection of access paths for large multiuser data bases. *Decision Sciences*, 14(4):564–587, 1983.

[111] N. Mattos and L.G. DeMichiel. Recent Design Trade-offs in SQL3. *SIGMOD Record*, 23(4), December 1994.

[112] F.J. McErlean, D.A. Bell, and S.I. McClean. The use of simulated annealing for clustering data in databases. *Information Systems*, 15(2):233–245, 1990.

[113] J. Melton. Accomodating SQL3 and ODMG. ISO DBL YOU-032 and ANSI X3H2-95-161, April 1995.

[114] J. Melton. (ISO/ANSI Working Draft) Database Language SQL/Foundation (SQL3). ISO DBL MCI-004 and ANSI X3H2-96-059, March 1996.

[115] N. Mfourga. Extracting Entity-Relationship Schemas from Relational Databases: A Form-Driven Approach. In C. Verhoef I. Baxter, A. Quilici, editor, *Proceedings of the 4th Working Conference on Reverse Engineering*, pages 184–193, Amsterdam, The Netherlands, October 1997.

[116] Z. Michalewicz. *Genetic Algorithms + Data Structures = Evolution Programs*. Springer-Verlag, Berlin, Germany, 1992.

[117] T. Mostardi. Estimating the size of relational SPΘJ operation results: an analytical approach. *Information Systems*, 15(5):591–601, 1990.

[118] D. Motzkin. The design of optimal access paths for relational databases. *Information Systems*, 12(2):203–213, 1987.

[119] J.D. Musa. Operational Profiles in Software-Reliability Engineering. *IEEE Software*, pages 14–32, March 1993.

[120] J. Mylopoulos, P.A. Bernstein, and H.K.T. Wong. A Language Facility for Designing Database-Intensive Applications. *ACM Transactions on Database Systems*, 5(2):185–207, June 1980.

[121] J. Nachouki, M.P. Chastang, and H. Briand. From Entity-Relationship Diagram to an Object-Oriented Database. In *Proceedings of the 10th International Conference on the Entity-Relationship Approach*, pages 459–482, San Mateo, California, 1991.

[122] S. Navathe, S. Ceri, G. Wiederhold, and J. Dou. Vertical Partitioning Algorithms for Database Design. *ACM Transactions on Database Systems*, 9(4):680–710, December 1984.

[123] S.H. Nienhuys-Cheng. Classification and syntax of constraints in binary semantical networks. *Information Systems*, 15(5):497–513, 1990.

[124] G.M. Nijssen and T.A. Halpin. *Conceptual Schema and Relational Database Design: a fact oriented approach*. Prentice-Hall, Sydney, Australia, 1989.

[125] V. Pareto. *Cours d'Economie Politique*. Rouge, Lausanne, 1896.

[126] H. Partsch. *Specification and Transformation of Programs - a Formal Approach to Software Development*. Springer-Verlag, Berlin, Germany, 1990.

[127] R.S. Pressman. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, New York, 1992.

[128] G.J. Ramackers. *Integrated Object Modelling, an Executable Specification Framework for Business Analysis and Information System Design*. PhD thesis, University of Leiden, Leiden, The Netherlands, 1994.

[129] G.J.E. Rawlins, editor. *Foundations of genetic algorithms*. Morgan Kaufmann Publishers, San Mateo, California, 1991.

[130] I. Rechenberg. *Evolutionsstrategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution*. Frommann-Holzboog, Stuttgart, Germany, 1973.

[131] C.R. Reeves, editor. *Modern heuristic techniques for combinatorial problems*. Blackwell Scientific Publications, Oxford, United Kingdom, 1993.

[132] N. Rishe. A file structure for semantic databases. *Information Systems*, 16(4):375–385, 1991.

[133] P.R. Ritson. Relational Mapping and Optimal Normal Form. In T.A. Halpin and R. Meersman, editors, *Proceedings of the First International Conference on Object-Role Modelling (ORM-1)*, pages 70–88, Magnetic Island, Australia, July 1994.

[134] P.R. Ritson. *Use of Conceptual Schemas for a Relational Implementation*. PhD thesis, University of Queensland, Brisbane, Australia, 1994.

[135] P.R. Ritson and T.A. Halpin. Mapping Integrity Constraints to a Relational Schema. In M.E. Orlowska and M. Papazoglou, editors, *Proceedings of the Fourth Australian Conference on Information Systems*, pages 381–400. World Scientific, Brisbane, Australia, September 1993.

[136] M.A. Roth, H.F. Korth, and A. Silberschatz. Extended algebra and calculus for nested relational databases. *ACM Transactions on Database Systems*, 13(4):389–417, December 1988.

[137] J.B.M. Runneboom. Database Optimization by Evolution: Experiences with singular and multiple strategy searching. Master's thesis, University of Nijmegen, December 1994. No. 336.

[138] H.J. Schek and M.H. Scholl. The relational model with relation-valued attributes. *Information Systems*, 11(2):137–147, 1986.

[139] K.-D. Schewe and B. Thalheim. Fundamental Concepts of Object Oriented Databases. *Acta Cybernetica*, 11(1-2):49–83, 1993.

[140] H.-P. Schwefel. *Evolution and Optimum Seeking*. John Wiley and Sons, New York, 1995.

[141] D.E. Shasha. *Database Tuning: A Principled Approach*. Prentice-Hall, Englewood Cliffs, New Jersey, 1992.

[142] G.M. Shaw and S.B. Zdonik. A Query Algebra for Object-Oriented Databases. In *Proceedings of the 6th International Conference on Data Engineering*, pages 154–162, 1990.

[143] D.W. Shipman. The Functional Data Model and the Data Language DAPLEX. *ACM Transactions on Database Systems*, 6(1):140–173, March 1981.

[144] P. Shoval and M. Even-Chaime. ADDS: A system for automatic database schema design based on the binary-relationship model. *Data & Knowledge Engineering*, 3(2):123–144, 1987.

[145] P. Shoval and N. Shreiber. Database reverse engineering: From the Relational to the Binary Relationship model. *Data & Knowledge Engineering*, 10:293–315, 1993.

[146] H.T. Siegelmann and O. Frieder. The Allocation of Documents in Multiprocessor Information retrieval: An Application of Genetic Algorithms. In *1991 IEEE International Computer Conference on Systems, Man and Cybernetics*, Charlottesville, Virginia, October 1991.

[147] R.M. Soley and W. Kent. The OMG Object Model. In W. Kim, editor, *Modern Database Systems*, chapter 2, pages 18–41. ACM Press, Addision-Wesley, New York, New York, 1995.

[148] I. Sommerville. *Software Engineering*. Addison-Wesley, Reading, Massachusetts, 1989.

[149] M. Stonebreaker, L.A. Rowe, B. Lindsay, J. Gray, M. Carey, M. Brodie, P. Bernstein, and D. Beech. Third-Generation Database System Manifesto. *SIGMOD Record*, 19(3):31–44, September 1990.

[150] J. Suzuki. A Markov Chain Analysis on A Genetic Algorithm. In S. Forrest, editor, *Proceedings of the Fifth International Conference on Genetic Algorithms*, pages 146–153, San Mateo, California, 1993. Morgan Kaufmann Publishers.

[151] T.J. Teorey, D. Yang, and J.P. Fry. A logical design methodology for relational databases using the extended entity-relationship model. *ACM Computing Surveys*, 18(2):197–222, 1986.

[152] T.J. Teorey, D. Yang, and J.P. Fry. A logical design methodology for relational databases using the extended entity-relationship model. *ACM Computing Surveys*, 18(2):197–222, 1986.

[153] P.G.H. Toonen. Markov Chain Analysis of Evolutionary Database Optimization. Master's thesis, University of Nijmegen, July 1994. No. 315.

[154] P.G.H. Toonen and P. van Bommel. Markov Chain Fundamentals for Data Schema Transformations. Technical Report CSI-R9505, Department of Information Systems, University of Nijmegen, Nijmegen, The Netherlands, 1995.

[155] L. Tucherman, M.A. Casanova, and A.L. Furtado. The CHRIS consultant - A tool for database design and rapid prototyping. *Information Systems*, 15(2):187–195, 1990.

[156] S. Twine. Mapping between a NIAM conceptual schema and KEE frames. *Data & Knowledge Engineering*, 2(4):125–155, 1989.

[157] J.D. Ullman. *Principles of Database and Knowledge-base Systems*, volume I. Computer Science Press, Rockville, Maryland, 1989.

[158] G.M.A. Verheijen and J. van Bekkum. NIAM: an Information Analysis Method. In T.W. Olle, H.G. Sol, and A.A. Verrijn-Stuart, editors, *Information Systems Design Methodologies: A Comparative Review*, pages 537–590. North-Holland/IFIP, Amsterdam, The Netherlands, 1982.

[159] T.F. Verhoef. *Effective Information Modelling Support*. PhD thesis, Delft University of Technology, Delft, The Netherlands, 1993.

[160] T.L. Vincent and W.J. Grantham. *Optimality in Parametric Systems*. John Wiley and Sons, New York, 1981.

[161] I.M. Walter, P.C. Lockemann, and H.H. Nagel. Database Support for Knowledge-Based Image Evaluation. In *Proceedings of the Thirteenth VLDB Conference*, pages 3–11, 1987.

[162] Th.P. van der Weide, A.H.M. ter Hofstede, and P. van Bommel. Uniquest: Determining the Semantics of Complex Uniqueness Constraints. *The Computer Journal*, 35(2):148–156, April 1992.

[163] T.A. Wiggerts. Using Clustering Algorithms in Legacy Systems Remodularization. In C. Verhoef I. Baxter, A. Quilici, editor, *Proceedings of the 4th Working Conference on Reverse Engineering*, pages 33–43, Amsterdam, The Netherlands, October 1997.

[164] J.J.V.R. Wintraecken. *The NIAM Information Analysis Method: Theory and Practice*. Kluwer, Deventer, The Netherlands, 1990.

[165] P.L. Yu and G. Leitmann. Compromise Solutions, Domination Structures, and Salukvadze's Solution. In G. Leitmann, editor, *Multicriteria Decision Making and Differential Games*, New York, 1976. Plenum.

[166] Y. Zhang and M.E. Orlowska. A new polynomial time algorithm for BCNF relational database design. *Information Systems*, 17(2):185–193, 1992.